Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: **Informatyka Techniczna**

MODELOWANIE I ANALIZA SYSTEMÓW INFORMATYCZNYCH PROJEKT

System do udostępniania zasobów biurowych

Dawid Chudzicki 272715 - Kierownik Projektu

Krzysztof Nowicki 294721 - Kierownik Strategii

Jakub Czajkowski 272709 - Kierownik Taktyki

Piotr Szczerbiak 294722 - Kierownik Testowania

Szymon Kowaliński 272695 - Kierownik Implementacji

Opiekun pracy
Dr inż. Paweł Głuchowski

WROCŁAW 2026

Contents

1 Opis dziedziny biznesowej **1**
- 1.1 Przeglądanie oferty i wnioskowanie o zasoby . . . 1
- 1.2 Procesowanie wniosków i nadzór nad wypożyczeniami . . . 1
- 1.3 Cykl życia udostępnienia i obsługa zdarzeń . . . 2
- 1.4 Zarządzanie bazą zasobów biurowych . . . 2
- 1.5 Zarządzanie konfiguracją i polityką dostępu . . . 2

2 Modelowanie Strategiczne **3**
- 2.1 Cel Etapu . . . 3
- 2.2 Określenie podziedzin . . . 3
  - 2.2.1 Dziedzina główna . . . 3
  - 2.2.2 Dziedziny wspierające . . . 4
  - 2.2.3 Dziedziny generyczne . . . 4
- 2.3 Konteksty ograniczone . . . 7
- 2.4 Język wszechobecny . . . 8
- 2.5 Bounded context canvas . . . 10
  - 2.5.1 Udostępnianie zasobów . . . 10
  - 2.5.2 Zarządzanie zasobami biurowymi . . . 11
  - 2.5.3 Administracja wnioskami . . . 12
- 2.6 Przypadki użycia . . . 13
  - 2.6.1 Diagram przypadków użycia . . . 13
  - 2.6.2 PU-01: Przeglądanie oferty zasobów biurowych . . . 13
  - 2.6.3 PU-02: Składanie wniosku o udostępnienie zasobu biurowego . . . 14
  - 2.6.4 PU-03: Anulowanie wniosku o udostępnienie zasobu biurowego . . . 15
  - 2.6.5 PU-04: Przeglądanie historii udostępnień . . . 15
  - 2.6.6 PU-05: Analiza wniosku o udostępnienie . . . 15
  - 2.6.7 PU-06: Zatwierdzenie wniosku o udostępnienie . . . 16
  - 2.6.8 PU-07: Odrzucenie wniosku o udostępnienie . . . 16
  - 2.6.9 UC-08: Wydanie zasobu biurowego . . . 16
  - 2.6.10 PU-09: Zwrot zasobu biurowego . . . 17
  - 2.6.11 PU-10: Obsługa przeterminowanego udostępnienia . . . 17
  - 2.6.12 PU-11: Monitorowanie udostępnień dla pracownika przez Kierownika udostępnień . . . 17
  - 2.6.13 PU-12: Zarządzanie dostępnością zasobów biurowych . . . 18
  - 2.6.14 PU-13: Zarządzanie katalogiem zasobów biurowych . . . 18
  - 2.6.15 PU-14: Konfiguracja limitu udostępnień . . . 19

3 Modelowanie Taktyczne **21**
3.1 Cel etapu . . . 21
3.2 Styl architektury . . . 22
3.2.1 Zasady stylu . . . 22
3.2.2 Heksagon dziedziny głównej — kontekst `allocation` . . . 23
3.2.3 Heksagon dziedziny pomocniczej — kontekst `resources` . . . 24
3.2.4 Heksagon dziedziny pomocniczej — kontekst `administration` . . . 25
3.2.5 Modułowy monolit jako kontener heksagonów . . . 25
3.3 Mapowanie kontekstów . . . 26
3.3.1 Diagram dziedzin i kontekstów ograniczonych . . . 26
3.3.2 Mapowanie między warstwami w ramach jednego kontekstu . . . 26
3.3.3 Mapowanie między kontekstami ograniczonymi . . . 27
3.3.4 Warstwa antykorupcyjna (Anti-Corruption Layer) . . . 28
3.3.5 Mapowanie zdarzeń dziedziny między kontekstami . . . 28
3.4 Model struktury i zachowania całego systemu . . . 30
3.4.1 Diagram komponentów . . . 30
3.4.2 Diagram wdrożenia . . . 31
3.4.3 Zachowanie całego systemu . . . 32
3.5 Model struktury backendu — warstwa dziedziny . . . 34
3.5.1 Diagram pakietów backendu . . . 34
3.5.2 Kontekst `allocation` — dziedzina główna . . . 35
3.5.3 Kanwa agregatu: WniosekOUdostępnienie . . . 37
3.5.4 Kontekst `resources` — dziedzina pomocnicza . . . 40
3.5.5 Kanwa agregatu: ZasóbBiurowy . . . 42
3.5.6 Kontekst `administration` — dziedzina pomocnicza . . . 45
3.5.7 Konteksty generyczne: `identity`, `notifications`, `employees` . . . 46
3.6 Przypadki użycia — realizacja . . . 47
3.6.1 PU-02: Składanie wniosku o udostępnienie zasobu biurowego . . . 47
3.6.2 PU-03: Anulowanie wniosku o udostępnienie zasobu biurowego . . . 48
3.6.3 PU-05: Analiza wniosku o udostępnienie . . . 49
3.6.4 PU-06: Zatwierdzenie wniosku o udostępnienie . . . 50
3.6.5 PU-07: Odrzucenie wniosku o udostępnienie . . . 50
3.6.6 PU-08: Wydanie zasobu biurowego . . . 52
3.6.7 PU-09: Zwrot zasobu biurowego . . . 53
3.6.8 PU-10: Obsługa przeterminowanego udostępnienia . . . 54
3.6.9 PU-12: Zarządzanie dostępnością zasobów biurowych . . . 55
3.6.10 PU-13: Zarządzanie katalogiem zasobów biurowych (wariant: dezaktywacja) . . . 56
3.6.11 PU-14: Konfiguracja limitu udostępnień . . . 58

4 Testowanie projektowanego systemu **61**
4.1 Cel etapu . . . 61
4.2 Zastosowane narzędzia . . . 61
4.3 Organizacja testów w projekcie . . . 62
4.4 Testy jednostkowe modelu domenowego . . . 62

4.5 Testy usług dziedzinowych . . . 63
4.6 Testy usług aplikacyjnych . . . 64
4.7 Testy integracyjne adapterów . . . 64
4.8 Testy adapterów REST i walidacji . . . 65
4.9 Testy systemowe . . . 65
4.10 Testy zdarzeń domenowych . . . 66
4.11 Testy architektury . . . 66
4.12 Uruchamianie testów . . . 67
4.13 Wnioski z etapu testowania . . . 67

5 Implementowanie projektowanego systemu 69
5.1 Cel etapu . . . 69
5.2 Ogólna charakterystyka rozwiązania . . . 69
5.3 Technologie użyte w implementacji . . . 69
5.4 Struktura backendu . . . 70
5.5 Model domenowy . . . 71
5.6 Globalna obsługa wyjątków . . . 72
5.7 Usługi dziedzinowe . . . 73
5.8 Usługi aplikacyjne i przypadki użycia . . . 73
5.9 Porty i adaptery . . . 74
5.10 Zdarzenia domenowe i EDA . . . 75
5.11 Warstwa persystencji . . . 76
5.12 Uwierzytelnianie i autoryzacja . . . 76
5.13 Frontend . . . 77
5.14 Integracja komponentów . . . 77
5.15 Wdrożenie . . . 78
5.16 Spełnienie zasad DDD, SOLID i czystej architektury . . . 79
5.17 Wnioski z etapu implementacji . . . 80

6 Listing kodu źródłowego, testów i skryptów wdrożeniowych 81
6.1 Kod źródłowy backendu . . . 81
DeskflowApplication.java . . . 81
6.1.1 Kontekst Allocation - alokacja zasobów . . . 81
AllocationRequest . . . 81
EmployeeAllocationProfile . . . 83
ResourceAllocationProfile . . . 83
AllocationOverdue . . . 84
AllocationRequestApproved . . . 84
AllocationRequestCancelled . . . 84
AllocationRequestRejected . . . 85
AllocationRequestSubmitted . . . 85
ResourceIssued . . . 86
ResourceReturned . . . 86
AllocationRequestId . . . 87
DateRange . . . 87

RequestStatus . . . 88
AllocationRequestFactory . . . 88
ApproveAllocationRequestDomainService . . . 89
CancelAllocationRequestDomainService . . . 89
EnsureEmployeeWithinLimitDomainService . . . 89
EnsureResourceAvailableInPeriodDomainService . . . 90
EnsureResourceEligibleForAllocationDomainService . . . 90
IssueAllocationResourceDomainService . . . 90
MarkAllocationRequestOverdueDomainService . . . 90
RejectAllocationRequestDomainService . . . 91
ReturnAllocationResourceDomainService . . . 91
AllocationRequestView . . . 91
ResourceCatalogItem . . . 92
AllocationAdministrationInputPort . . . 92
AllocationRequestInputPort . . . 93
AllocationRequestRepositoryPort . . . 93
EmployeeProfilePort . . . 94
ResourceCatalogPort . . . 94
AllocationAdministrationAppService . . . 94
AllocationRequestAppService . . . 96
AllocationRequestController . . . 98
AllocationRequestRestMapper . . . 99
CurrentUserResolver . . . 100
AllocationRequestDTO . . . 100
AllocationRequestResponseDTO . . . 101
EmployeeProfileAdapter . . . 101
AllocationRequestEntity . . . 102
AllocationRequestMapper . . . 102
HibernateAllocationRequestRepositoryAdapter . . . 103
SpringDataAllocationRequestRepository . . . 104
ResourceCatalogAdapter . . . 105
6.1.2 Kontekst Resources - zasoby biurowe . . . 106
OfficeResource . . . 106
ResourceDeactivated . . . 107
ResourceMadeAvailable . . . 108
ResourceMadeUnavailable . . . 108
ResourceUpdated . . . 108
AllocationUsageStatus . . . 109
Location . . . 109
DeactivateOfficeResourceDomainService . . . 110
MarkOfficeResourceAvailableDomainService . . . 110
MarkOfficeResourceUnavailableDomainService . . . 110
OfficeResourceFactory . . . 111
UpdateOfficeResourceDomainService . . . 111
ResourceView . . . 111

ResourceManagementInputPort . . . 112
OfficeResourceRepositoryPort . . . 112
ResourceAllocationRequestPort . . . 113
ResourceAppService . . . 113
ResourceAvailabilityEventListener . . . 115
ResourceController . . . 115
ResourceRequestDTO . . . 116
AllocationRequestAdapter . . . 117
HibernateOfficeResourceRepositoryAdapter . . . 117
OfficeResourceEntity . . . 118
OfficeResourceMapper . . . 119
SpringDataOfficeResourceRepository . . . 119
6.1.3 Kontekst Administration - administracja . . . 120
AdministeredAllocationRequest . . . 120
AdministeredAllocationRequestId . . . 120
AdministeredAllocationStatus . . . 121
AdministeredAllocationView . . . 121
AdministrationInputPort . . . 121
AdministrationAllocationPort . . . 122
AdministrationAppService . . . 122
OverdueAllocationScheduler . . . 124
AdministrationController . . . 124
RejectRequestDTO . . . 126
AllocationAdministrationAdapter . . . 126
6.1.4 Kontekst Employees - pracownicy . . . 127
Employee . . . 127
Role . . . 128
EmployeeCredentials . . . 128
EmployeeManagementInputPort . . . 128
EmployeeRepositoryPort . . . 129
EmployeeAppService . . . 129
EmployeeAdminController . . . 130
AllocationLimitEntity . . . 131
EmployeeEntity . . . 131
EmployeeMapper . . . 132
HibernateEmployeeRepositoryAdapter . . . 133
SpringDataAllocationLimitRepository . . . 134
SpringDataEmployeeRepository . . . 134
6.1.5 Kontekst Identity - tożsamość i uwierzytelnianie . . . 134
AuthenticatedIdentity . . . 134
IdentityProfile . . . 135
UserCredentials . . . 135
IdentityAccessInputPort . . . 135
AccessTokenIssuerPort . . . 135
CredentialsAuthenticationPort . . . 135

IdentityEmployeeProfilePort . . . 136
UserCredentialsPort . . . 136
IdentityAccessAppService . . . 136
AuthController . . . 137
LoginRequest . . . 138
LoginResponse . . . 138
EmployeeIdentityProfileAdapter . . . 138
EmployeeUserCredentialsAdapter . . . 138
SpringCredentialsAuthenticationAdapter . . . 139
DeskflowUserDetailsService . . . 139
JwtAuthenticationFilter . . . 140
JwtTokenProvider . . . 141
SecurityConfig . . . 142
6.1.6 Kontekst Notifications - powiadomienia . . . 143
NotificationEventInputPort . . . 143
NotificationPort . . . 143
NotificationAppService . . . 144
NotificationEventListener . . . 145
SmtpNotificationAdapter . . . 146
6.1.7 Kontekst Common - kod współdzielony . . . 146
EmployeeId . . . 146
ResourceId . . . 147
ResourceType . . . 148
DomainEventPublisherPort . . . 148
DataInitializer . . . 148
DomainBeanConfig . . . 150
GlobalExceptionHandler . . . 151
SpringDomainEventPublisherAdapter . . . 152
DomainEvent . . . 152
6.2 Kod źródłowy testów . . . 154
CleanArchitectureTest . . . 154
DomainIsolationTest . . . 155
6.2.1 Testy - Allocation . . . 155
AllocationRequestTest . . . 155
DateRangeTest . . . 158
AllocationLifecycleDomainServicesTest . . . 159
EmployeeLimitDomainServiceTest . . . 161
ResourceAvailabilityDomainServicesTest . . . 161
AllocationRequestViewTest . . . 163
AllocationAdministrationAppServiceTest . . . 164
AllocationRequestAppServiceMissingDataTest . . . 166
AllocationRequestAppServiceTest . . . 167
AllocationRequestRestMapperTest . . . 170
EmployeeProfileAdapterTest . . . 171
AllocationRequestRepositoryDateEdgeCasesTest . . . 172

AllocationRequestRepositoryIntegrationTest . . . 173
ResourceCatalogAdapterTest . . . 174
6.2.2 Testy - Resources . . . 176
LocationTest . . . 176
OfficeResourceTest . . . 176
ResourceDeactivationDomainServicesTest . . . 178
ResourceLifecycleDomainServicesTest . . . 179
ResourceAppServiceMissingResourceTest . . . 180
ResourceAppServiceTest . . . 181
ResourceAvailabilityEventListenerTest . . . 182
AllocationRequestAdapterTest . . . 183
OfficeResourceRepositoryIntegrationTest . . . 185
6.2.3 Testy - Administration . . . 186
AdministeredAllocationRequestIdTest . . . 186
AdministeredAllocationRequestTest . . . 187
AdministeredAllocationStatusTest . . . 189
AdministrationAppServiceTest . . . 189
OverdueAllocationSchedulerTest . . . 191
AllocationAdministrationAdapterTest . . . 191
6.2.4 Testy - Employees . . . 193
EmployeeTest . . . 193
EmployeeAppServiceTest . . . 195
EmployeeRepositoryIntegrationTest . . . 196
6.2.5 Testy - Identity . . . 198
IdentityAccessAppServiceTest . . . 198
EmployeeIdentityProfileAdapterTest . . . 199
EmployeeUserCredentialsAdapterTest . . . 200
DeskflowUserDetailsServiceTest . . . 201
JwtTokenProviderTest . . . 201
6.2.6 Testy - Notifications . . . 202
NotificationAppServiceTest . . . 202
NotificationEventListenerTest . . . 204
SmtpNotificationAdapterTest . . . 206
6.2.7 Testy - Common . . . 207
EmployeeIdTest . . . 207
ResourceIdTest . . . 208
ResourceTypeTest . . . 208
AuthorizationSystemTest . . . 209
FullAllocationCycleSystemTest . . . 210
GlobalExceptionHandlerTest . . . 213
InputValidationSystemTest . . . 214
RestControllerWebMvcTest . . . 216
SpringDomainEventPublisherAdapterTest . . . 222
6.3 Pliki konfiguracyjne i migracje . . . 223
application-test.properties . . . 223

application.properties . . . 223
V1__init_schema.sql . . . 223
6.4 Pliki konfiguracji budowania (pom.xml) . . . 225
deskflow-backend . . . 225
6.5 Skrypty wdrożeniowe i konfiguracja Docker . . . 227
.dockerignore . . . 227
Dockerfile.prod . . . 227
deskflow-backend/Dockerfile.dev . . . 228
docker-compose.dev.yml . . . 228
docker-compose.prod.yml . . . 229
.env.example . . . 230
6.6 Kod źródłowy frontendu . . . 230
deskflow-frontend/Dockerfile.dev . . . 230
deskflow-frontend/Dockerfile.frontend . . . 231
deskflow-frontend/docker-compose.yml . . . 231
deskflow-frontend/.gitignore . . . 232
deskflow-frontend/.dockerignore . . . 232
deskflow-frontend/package.json . . . 232
deskflow-frontend/tsconfig.json . . . 233
deskflow-frontend/next.config.mjs . . . 234
deskflow-frontend/eslint.config.mjs . . . 234
deskflow-frontend/postcss.config.mjs . . . 234
deskflow-frontend/components.json . . . 234
deskflow-frontend/next-env.d.ts . . . 235
deskflowfrontend/app/layout.tsx . . . 235
deskflowfrontend/app/globals.css . . . 235
deskflowfrontend/app/(auth)/login/page.tsx . . . 238
deskflowfrontend/app/(auth)/register/page.tsx . . . 240
deskflowfrontend/app/(dashboard)/layout.tsx . . . 242
deskflowfrontend/app/(dashboard)/page.tsx . . . 242
deskflowfrontend/app/(dashboard)/biurka/page.tsx . . . 245
deskflowfrontend/app/(dashboard)/rezerwacje/page.tsx . . . 248
deskflowfrontend/app/(dashboard)/sale/page.tsx . . . 250
deskflowfrontend/app/(dashboard)/sprzet/page.tsx . . . 250
deskflowfrontend/app/(dashboard)/zasoby/page.tsx . . . 251
deskflowfrontend/app/(dashboard)/zasobyadmin/page.tsx . . . 251
deskflowfrontend/app/(dashboard)/wnioski/page.tsx . . . 251
deskflowfrontend/app/(dashboard)/powiadomienia/page.tsx . . . 251
deskflowfrontend/app/(dashboard)/administracja/page.tsx . . . 251
deskflowfrontend/app/(dashboard)/admin/page.tsx . . . 251
deskflowfrontend/app/(dashboard)/admin/layout.tsx . . . 255
deskflowfrontend/app/(dashboard)/admin/it/page.tsx . . . 255
deskflowfrontend/lib/api.ts . . . 260
deskflowfrontend/lib/utils.ts . . . 263
deskflowfrontend/lib/authcontext.tsx . . . 263

deskflowfrontend/hooks/usetoast.ts 264
deskflowfrontend/hooks/usemobile.ts 266
deskflowfrontend/styles/globals.css 267
deskflowfrontend/components/authlanding.tsx 269
deskflowfrontend/components/themeprovider.tsx 270
deskflowfrontend/components/ui/accordion.tsx 270
deskflowfrontend/components/ui/alertdialog.tsx 271
deskflowfrontend/components/ui/alert.tsx 274
deskflowfrontend/components/ui/aspectratio.tsx 275
deskflowfrontend/components/ui/avatar.tsx 275
deskflowfrontend/components/ui/badge.tsx 276
deskflowfrontend/components/ui/breadcrumb.tsx 277
deskflowfrontend/components/ui/buttongroup.tsx 278
deskflowfrontend/components/ui/button.tsx 280
deskflowfrontend/components/ui/calendar.tsx 281
deskflowfrontend/components/ui/card.tsx 284
deskflowfrontend/components/ui/carousel.tsx 285
deskflowfrontend/components/ui/chart.tsx 289
deskflowfrontend/components/ui/checkbox.tsx 294
deskflowfrontend/components/ui/collapsible.tsx 295
deskflowfrontend/components/ui/command.tsx 295
deskflowfrontend/components/ui/contextmenu.tsx 298
deskflowfrontend/components/ui/dialog.tsx 302
deskflowfrontend/components/ui/drawer.tsx 304
deskflowfrontend/components/ui/dropdownmenu.tsx 307
deskflowfrontend/components/ui/empty.tsx 311
deskflowfrontend/components/ui/field.tsx 312
deskflowfrontend/components/ui/form.tsx 316
deskflowfrontend/components/ui/hovercard.tsx 319
deskflowfrontend/components/ui/inputgroup.tsx 319
deskflowfrontend/components/ui/inputotp.tsx 322
deskflowfrontend/components/ui/input.tsx 323
deskflowfrontend/components/ui/item.tsx 324
deskflowfrontend/components/ui/kbd.tsx 327
deskflowfrontend/components/ui/label.tsx 327
deskflowfrontend/components/ui/menubar.tsx 328
deskflowfrontend/components/ui/navigationmenu.tsx 332
deskflowfrontend/components/ui/pagination.tsx 335
deskflowfrontend/components/ui/popover.tsx 337
deskflowfrontend/components/ui/progress.tsx 338
deskflowfrontend/components/ui/radiogroup.tsx 338
deskflowfrontend/components/ui/resizable.tsx 339
deskflowfrontend/components/ui/scrollarea.tsx 340
deskflowfrontend/components/ui/select.tsx 341
deskflowfrontend/components/ui/separator.tsx 344

deskflowfrontend/components/ui/sheet.tsx 344
deskflowfrontend/components/ui/sidebar.tsx 347
deskflowfrontend/components/ui/skeleton.tsx 358
deskflowfrontend/components/ui/slider.tsx 358
deskflowfrontend/components/ui/sonner.tsx 359
deskflowfrontend/components/ui/spinner.tsx 359
deskflowfrontend/components/ui/switch.tsx 360
deskflowfrontend/components/ui/table.tsx 360
deskflowfrontend/components/ui/tabs.tsx 362
deskflowfrontend/components/ui/textarea.tsx 363
deskflowfrontend/components/ui/toast.tsx 364
deskflowfrontend/components/ui/toaster.tsx 366
deskflowfrontend/components/ui/togglegroup.tsx 366
deskflowfrontend/components/ui/toggle.tsx 367
deskflowfrontend/components/ui/tooltip.tsx 368
deskflowfrontend/components/ui/usemobile.tsx 369
deskflowfrontend/components/ui/usetoast.ts 370

1. Opis dziedziny biznesowej

System informatyczny „DeskFlow” to rozwiązanie biznesowe zaprojektowane w celu optymalizacji procesu rezerwacji i wypożyczania zasobów biurowych wewnątrz organizacji. System integruje potrzeby zwykłych pracowników z wymaganiami kadry zarządzającej oraz administracyjnej, zapewniając pełną kontrolę nad cyklem życia zasobów – od momentu dodania ich do katalogu, przez rezerwację, aż po wydanie i zwrot.

Dziedzina biznesowa systemu opiera się na kilku kluczowych obszarach tematycznych, z których wynikają określone procesy operacyjne i zachowania użytkowników (przypadki użycia). Wśród nich wyróżniamy:

1.1. Przeglądanie oferty i wnioskowanie o zasoby

Z punktu widzenia pracownika (użytkownika końcowego), głównym zadaniem jest zaspokojenie zapotrzebowania na narzędzia pracy.

- Pracownicy mogą na bieżąco przeglądać dostępną ofertę zasobów biurowych, do których zalicza się sprzęt komputerowy, biurka czy sale spotkań.
- Użytkownik ma możliwość złożenia wniosku o udostępnienie wybranego zasobu w określonym przedziale czasowym.
- Pracownik może śledzić własną historię udostępnień, a w przypadku zmiany planów ma prawo do samodzielnego anulowania złożonego wcześniej wniosku o rezerwację.

1.2. Procesowanie wniosków i nadzór nad wypożyczeniami

Z perspektywy kadry zarządzającej (np. Kierownika udostępnień), dziedzina obejmuje procesy weryfikacyjne i decyzyjne.

- Każdy poprawnie złożony wniosek podlega analizie, na podstawie której Kierownik udostępnień podejmuje decyzję o jego zatwierdzeniu bądź odrzuceniu (np. z powodu konfliktu terminów lub braku adekwatnego uzasadnienia).
- Kierownik zajmuje się również monitorowaniem historii udostępnień poszczególnych pracowników, co pozwala na identyfikację nadużyć oraz ocenę realnych potrzeb sprzętowych w zespołach.

1.3. Cykl życia udostępnienia i obsługa zdarzeń

Działalność biznesowa w tym obszarze sprowadza się do fizycznego i wirtualnego śledzenia losów sprzętu.

- Po pozytywnym rozpatrzeniu wniosku następuje moment operacyjnego wydania zasobu biurowego pracownikowi.
- Po upływie terminu rezerwacji oczekiwany jest fizyczny zwrot zasobu do bazy przedsiębiorstwa.
- W przypadkach, gdy sprzęt nie wróci do puli w wyznaczonym terminie, system wspiera obsługę i zarządzanie przeterminowanymi udostępnieniami, co w powiązaniu z procesami powiadomień przypomina stronom o konieczności zwrotu sprzętu.

1.4. Zarządzanie bazą zasobów biurowych

Funkcjonowanie rezerwacji wymaga stale aktualnej bazy danych sprzętu i przestrzeni biurowych, którymi dysponuje firma. Proces ten prowadzony jest przez administratorów zasobów (Kierowników zasobów).

- Dziedzina obejmuje pełne zarządzanie katalogiem zasobów (dodawanie nowych pozycji, edycję ich właściwości czy ostateczną dezaktywację).
- Z procesem tym łączy się również dynamiczne zarządzanie dostępnością – wyłączanie z użycia zasobów uszkodzonych lub w naprawie, uniemożliwiające pracownikom ich niecelowe rezerwowanie.

1.5. Zarządzanie konfiguracją i polityką dostępu

Ostatnim, lecz równie istotnym aspektem dziedziny jest definiowanie ogólnofirmowych reguł wypożyczania zasobów, którymi zarządzają administratorzy (np. dział IT).

- Określane są globalne parametry i limity udostępnień, zapobiegające monopolizowaniu zasobów (takich jak deficytowy sprzęt czy biurka premium) przez wąską grupę pracowników, gwarantując sprawiedliwą alokację i zgodność z polityką firmy.

2. Modelowanie Strategiczne

2.1. Cel Etapu

Celem etapu strategicznego w projekcie DeskFlow jest zrozumienie dziedziny biznesowej udostępniania zasobów biurowych oraz wyznaczenie wysokopoziomowej struktury systemu, zanim podjęte zostaną decyzje implementacyjne. Na tym etapie identyfikujemy kluczowe obszary problemowe, oddzielamy to, co stanowi rdzeń wartości biznesowej, od funkcji wspierających i generycznych, oraz ustalamy granice, w obrębie których obowiązuje spójny model i język. W ramach etapu strategicznego realizujemy następujące cele szczegółowe. Wyznaczamy dziedzinę główną — proces udostępniania zasobów biurowych — oraz dziedziny wspierające i generyczne, określając ich znaczenie dla całości systemu. Każdą z wyróżnionych dziedzin otaczamy kontekstem ograniczonym, w obrębie którego obowiązuje jednoznaczny Język Wszechobecny. Definiujemy relacje strategiczne między kontekstami, opisując sposób ich komunikacji oraz zależności, a także utrwalamy je w postaci diagramu Context Map oraz kart Bounded Context Canvas. Efektem tego etapu jest spójny obraz architektury dziedzinowej, który stanowi fundament dla dalszego, taktycznego modelowania poszczególnych kontekstów oraz ich implementacji.

2.2. Określenie podziedzin

2.2.1. Dziedzina główna

Nazwa: Udostępnianie zasobów biurowych
Opis: Obsługa procesu udostępniania zasobów biurowych z perspektywy pracownika. Udostępnia informacje o oferowanych zasobach oraz ich bieżącej dostępności, umożliwia przeglądanie dostępnych terminów, a także obsługuje tworzenie i anulowanie wniosków o udostępnienie zasobu biurowego.

Wymagania funkcjonalne

- Pracownik wyszukuje zasób na podstawie wybranych kryteriów.
- Pracownik przegląda informacje o swoich wnioskach o udostępnienie.
- Pracownik przegląda informacje o zasobach udostępnianych przez firmę.
- Pracownik przegląda dostępne terminy udostępnienia mu zasobów biurowych.
- Pracownik składa wniosek o udostępnienie zasobu biurowego.
- Pracownik anuluje wniosek o udostępnienie zasobu biurowego.

2.2.2. Dziedziny wspierające

Nazwa: Zarządzanie zasobami biurowymi
Opis: Obszar logiki biznesowej odpowiedzialny za zarządzanie katalogiem zasobów biurowych oraz informacjami o biurze. Obejmuje dodawanie, edytowanie i dezaktywowanie zasobów oraz określanie ich dostępności.

Wymagania funkcjonalne

- Kierownik zasobów dodaje nowe zasoby biurowe oraz wprowadza informacje o nich.
- Kierownik zasobów edytuje informacje o zasobach biurowych.
- Kierownik zasobów dezaktywuje zasoby biurowe.
- Kierownik zasobów oznacza zasób biurowy jako niedostępny.
- Kierownik zasobów przegląda obłożenie zasobów biurowych.

Nazwa: Administracja wnioskami
Opis: Procedowanie wniosków o udostępnienie zasobów biurowych oraz wydawanie i odbieranie udostępnionych zasobów.

Wymagania funkcjonalne

- Kierownik udostępnień przegląda wszystkie aktywne udostępnienia zasobów biurowych.
- Kierownik udostępnień zarządza wnioskami o udostępnienie zasobów biurowych oraz akceptuje bądź odrzuca wnioski.
- Kierownik udostępnień wydaje oraz odbiera udostępnione zasoby.

2.2.3. Dziedziny generyczne

Nazwa: Powiadomienia
Opis: Przekazuje powiadomienia do użytkowników systemu o zdarzeniach związanych z udostępnianiem zasobów biurowych.

Wymagania funkcjonalne

- Pracownik otrzymuje powiadomienie o utworzeniu wniosku o zasób.
- Pracownik otrzymuje powiadomienie o zatwierdzeniu wniosku o zasób.
- Pracownik otrzymuje powiadomienie o anulowaniu wniosku o zasób.
- Pracownik otrzymuje powiadomienie przypominające o oddaniu zasobu.
- Pracownik oraz Kierownik udostępnień otrzymują powiadomienie o przekroczeniu terminu oddania zasobu.

- Kierownik udostępnień otrzymuje powiadomienie o nowym wniosku o zasób.

Nazwa: Zarządzanie systemem
Opis: Zarządzanie globalnymi ustawieniami oraz konfiguracją działania systemu.

Wymagania funkcjonalne

- Informatyk zarządza globalnymi ustawieniami systemu.
- Informatyk konfiguruje limity udostępnień.
- Informatyk przegląda historię zdarzeń systemowych.
- Informatyk wprowadza informacje o firmie — nazwa, lokalizacja, logo.
- Informatyk wprowadza dane o pracownikach biura.

Nazwa: Uwierzytelnianie i autoryzacja
Opis: Odpowiedzialna za proces logowania użytkowników, zarządzanie sesją oraz kontrolę uprawnień dostępu do systemu.

Wymagania funkcjonalne

- Użytkownik może się zalogować i przeglądać system na poziomie związanym z jego uprawnieniami.

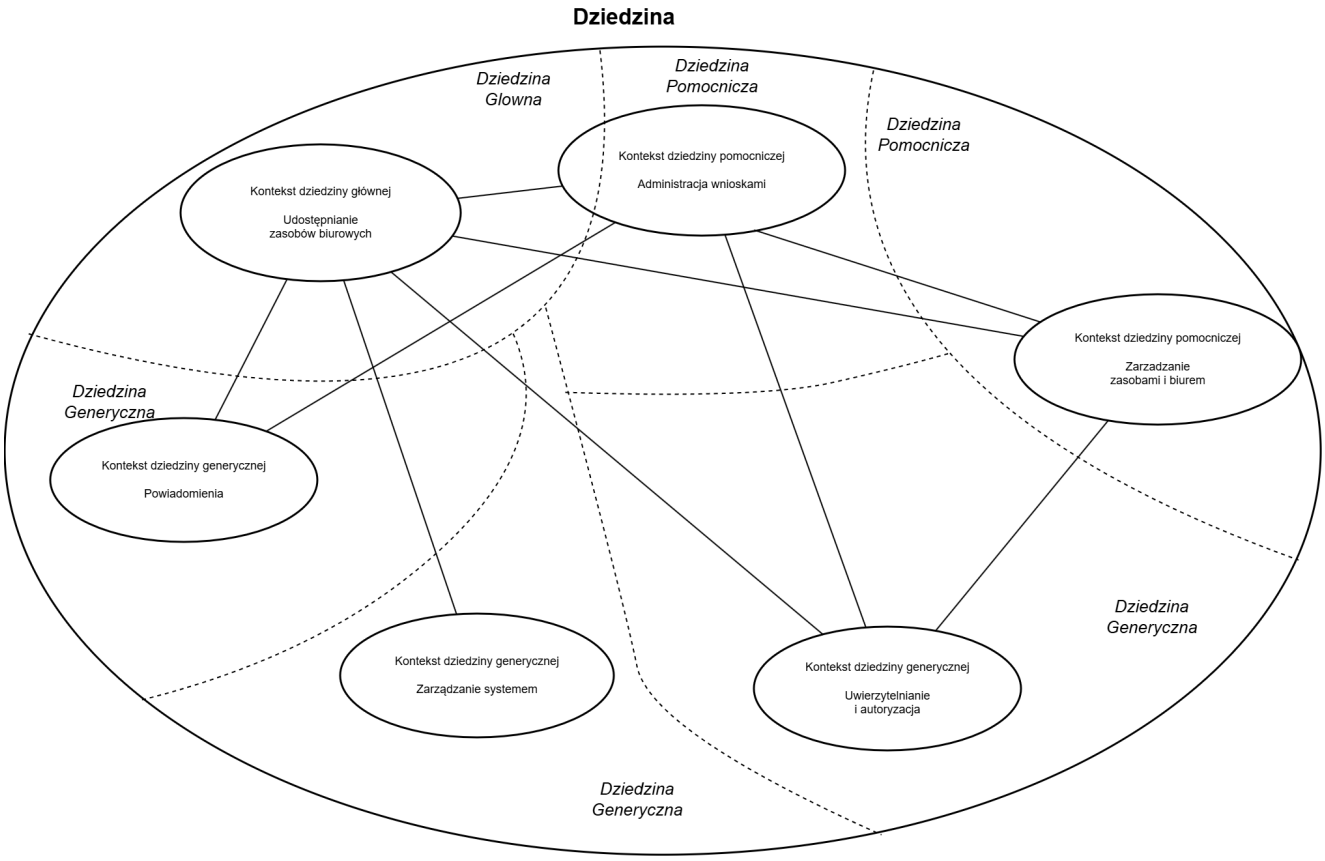


Figure 2.1: Model dziedziny.

2.3. Konteksty ograniczone

Każdą wyróżnioną dziedzinę otoczyliśmy kontekstem ograniczonym — udało się przy tym przypisać każdej dziedzinie dokładnie jeden taki kontekst. Granice każdego z nich wyznaczają ścisły zakres obowiązywania Języka Wszechobecnego.

Dziedzina	**Typ**	**Kontekst Ograniczony**	**Uzasadnienie**
Udostępnianie zasobów biurowych	Główna	Udostępnianie zasobów biurowych	Serce systemu. Obejmuje proces składania i anulowania wniosków o udostępnienie zasobu z perspektywy pracownika oraz przeglądanie oferty i dostępności.
Zarządzanie zasobami biurowymi	Wspierająca	Zarządzanie zasobami biurowymi	Odpowiada za katalog zasobów, ich atrybuty, oraz określanie dostępności.
Administracja wnioskami	Wspierająca	Administracja wnioskami	Procedowanie wniosków o udostępnienie oraz wydawanie i odbieranie udostępnionych zasobów.
Powiadomienia	Generyczna	Powiadomienia	Przekazywanie powiadomień do użytkowników o zdarzeniach związanych z udostępnianiem zasobów.
Uwierzytelnianie i autoryzacja	Generyczna	Uwierzytelnianie i autoryzacja	Logowanie użytkowników, zarządzanie sesją oraz kontrola uprawnień dostępu.
Zarządzanie systemem	Generyczna	Zarządzanie systemem	Globalne ustawienia, konfiguracja limitów, dane firmy oraz dane pracowników biura.

Table 2.1: Mapowanie dziedzin na konteksty ograniczone

2.4. Język wszechobecny

Nazwa (PL)	Odpowiednik w kodzie / (EN)	Opis
Zasób biurowy	`OfficeResource`	Zasób dostępny w katalogu firmy, który może zostać udostępniony pracownikowi.
Wniosek o udostępnienie	`AllocationRequest`	Prośba pracownika o możliwość korzystania z wybranego zasobu w określonym terminie.
Termin udostępnienia	`DateRange`	Okres, na który pracownik chce skorzystać z zasobu.
Dostępność zasobu	`available`	Informacja, czy zasób może zostać udostępniony w wybranym terminie.
Katalog zasobów	`ResourceCatalogPort`	Lista zasobów możliwych do przeglądania przez pracownika.
Limit udostępnień	`AllocationLimit`	Maksymalna liczba aktywnych udostępnień danego typu dla pracownika.
Historia wniosków	`browseHistory()`	Lista wcześniejszych i aktualnych wniosków pracownika.

Table 2.2: Język Wszechobecny — kontekst Udostępnianie zasobów biurowych

Nazwa (PL)	Odpowiednik w kodzie / (EN)	Opis
Zasób biurowy	`OfficeResource`	Element katalogu zasobów firmy, który może zostać udostępniony pracownikowi.
Typ zasobu	`ResourceType`	Rodzaj zasobu, np. biurko, sala konferencyjna, laptop, monitor, projektor albo pojazd.
Lokalizacja zasobu	`Location`	Informacja, gdzie znajduje się zasób, np. budynek, piętro i numer.
Dostępność zasobu	`available`	Informacja, czy zasób może być brany pod uwagę przy udostępnianiu.
Katalog zasobów	`ResourceCatalogPort`	Zbiór zasobów widocznych dla procesu udostępniania.
Dezaktywacja zasobu	`deactivate()`	Wycofanie zasobu z katalogu, aby nie mógł być dalej udostępniany.
Kierownik zasobów	`Role.MANAGER`	Osoba odpowiedzialna za zarządzanie katalogiem zasobów biurowych, ich dostępnością oraz informacjami o biurze.

Table 2.3: Język Wszechobecny — kontekst Zarządzanie zasobami biurowymi

Nazwa (PL)	**Odpowiednik w kodzie / (EN)**	**Opis**
Wniosek	`AllocationRequest`	Zgłoszenie pracownika przekazane do analizy przez Kierownika udostępnień. W tym kontekście jest oceniane i prowadzone przez kolejne etapy od decyzji po zwrot zasobu.
Analizowanie wniosku	`approve()` / `reject()`	Ocena wniosku przez Kierownika udostępnień, zakończona akceptacją albo odrzuceniem.
Wydanie zasobu	`issueResource()`	Przekazanie zasobu pracownikowi po zatwierdzeniu wniosku, rozpoczynające aktywne udostępnienie.
Odbiór zasobu	`returnResource()`	Przyjęcie zwracanego zasobu od pracownika i zakończenie udostępnienia.
Aktywne udostępnienie	`RequestStatus.ACTIVE`	Udostępnienie, w którym zasób został wydany pracownikowi i oczekuje na zwrot.
Przeterminowane udostępnienie	`RequestStatus.OVERDUE`	Aktywne udostępnienie, dla którego minął termin zwrotu zasobu.
Kierownik udostępnień	`Role.MANAGER`	Osoba odpowiedzialna za analizowanie wniosków oraz wydawanie i odbieranie zasobów.

Table 2.4: Język Wszechobecny — kontekst Administracja wnioskami

2.5. Bounded context canvas

2.5.1. Udostępnianie zasobów

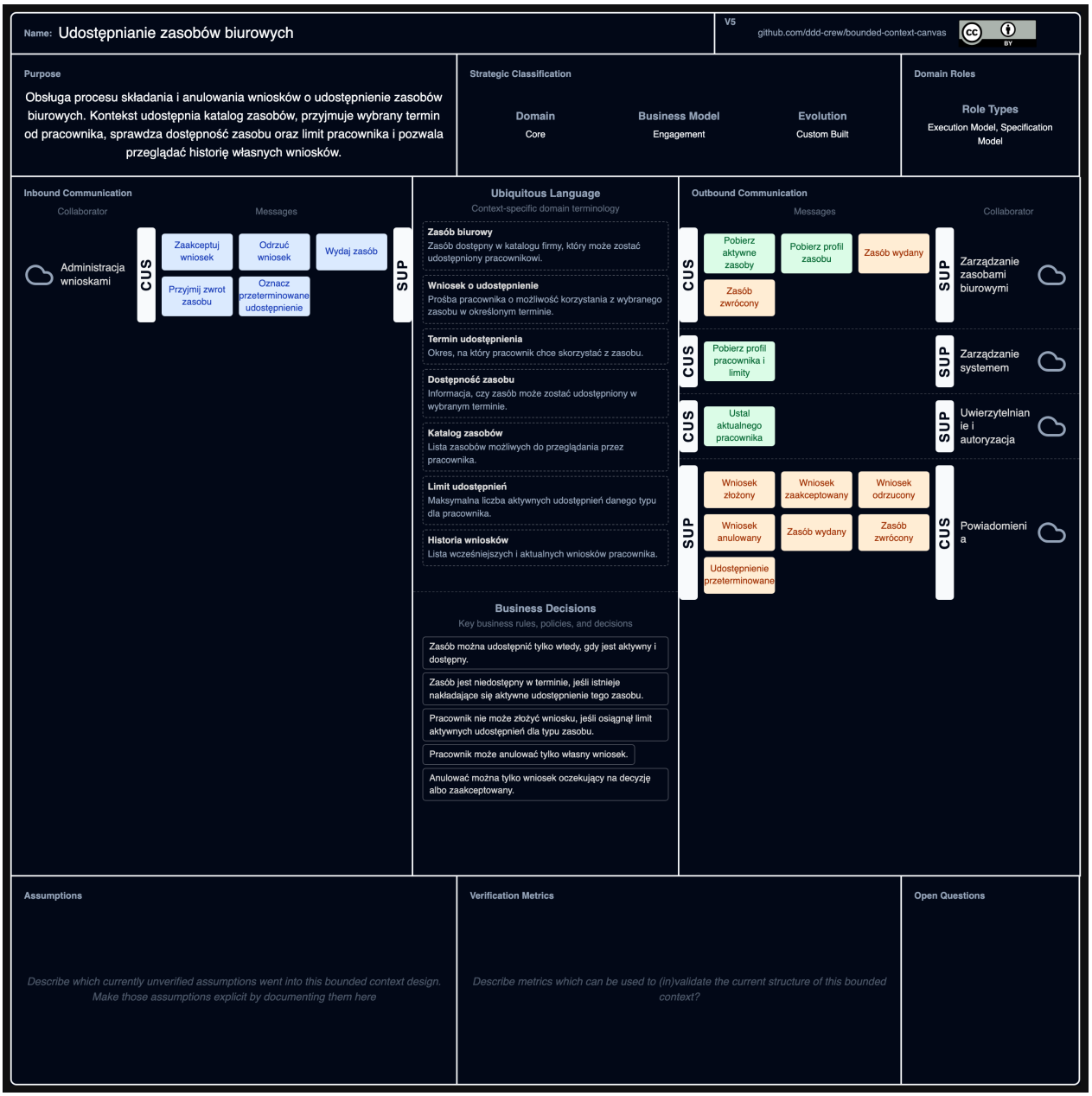


Figure 2.2: Bounded Context Canvas: Udostępnianie zasobów

2.5.2. Zarządzanie zasobami biurowymi

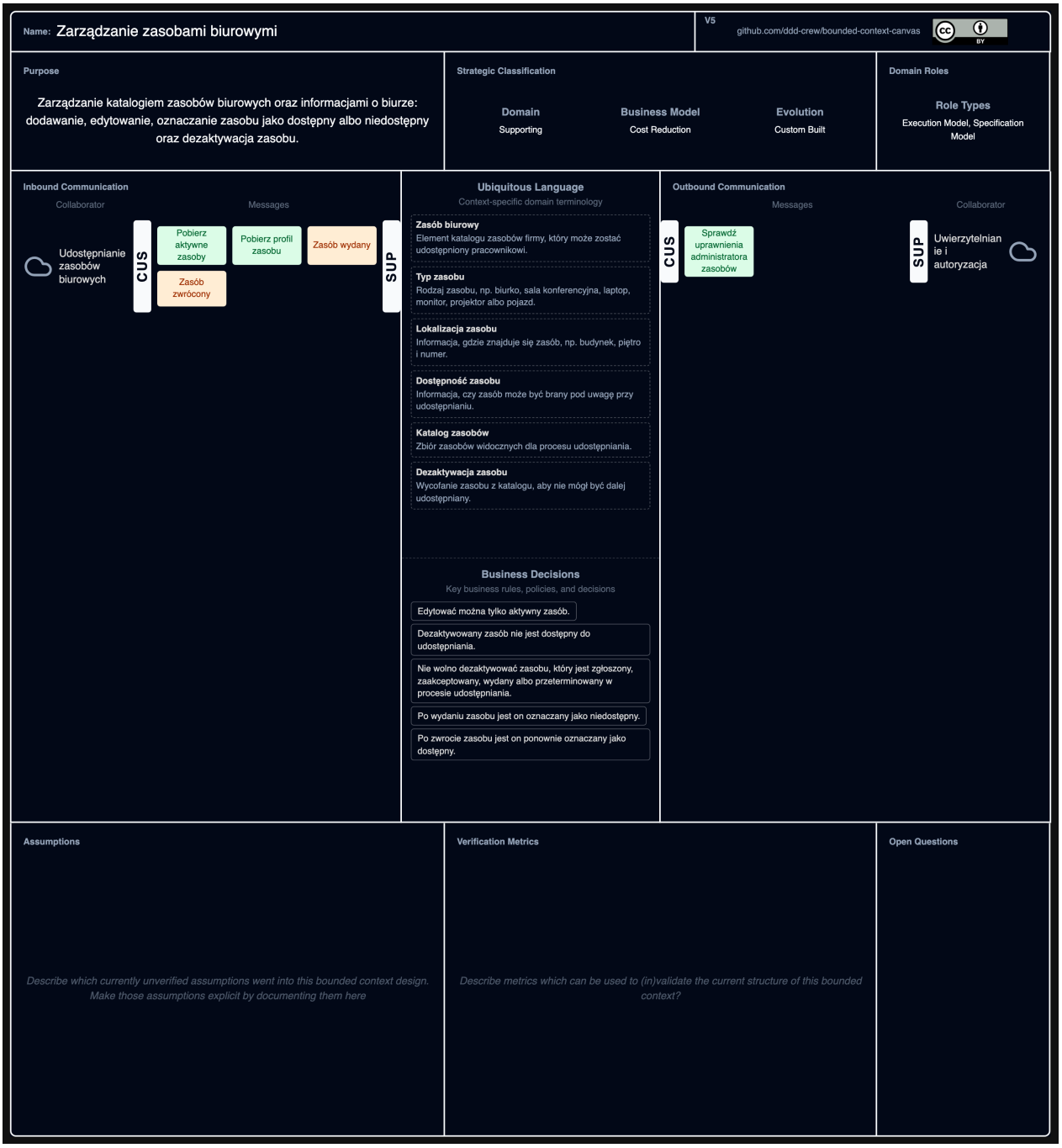


Figure 2.3: Bounded Context Canvas: Zarządzanie zasobami

2.5.3. Administracja wnioskami

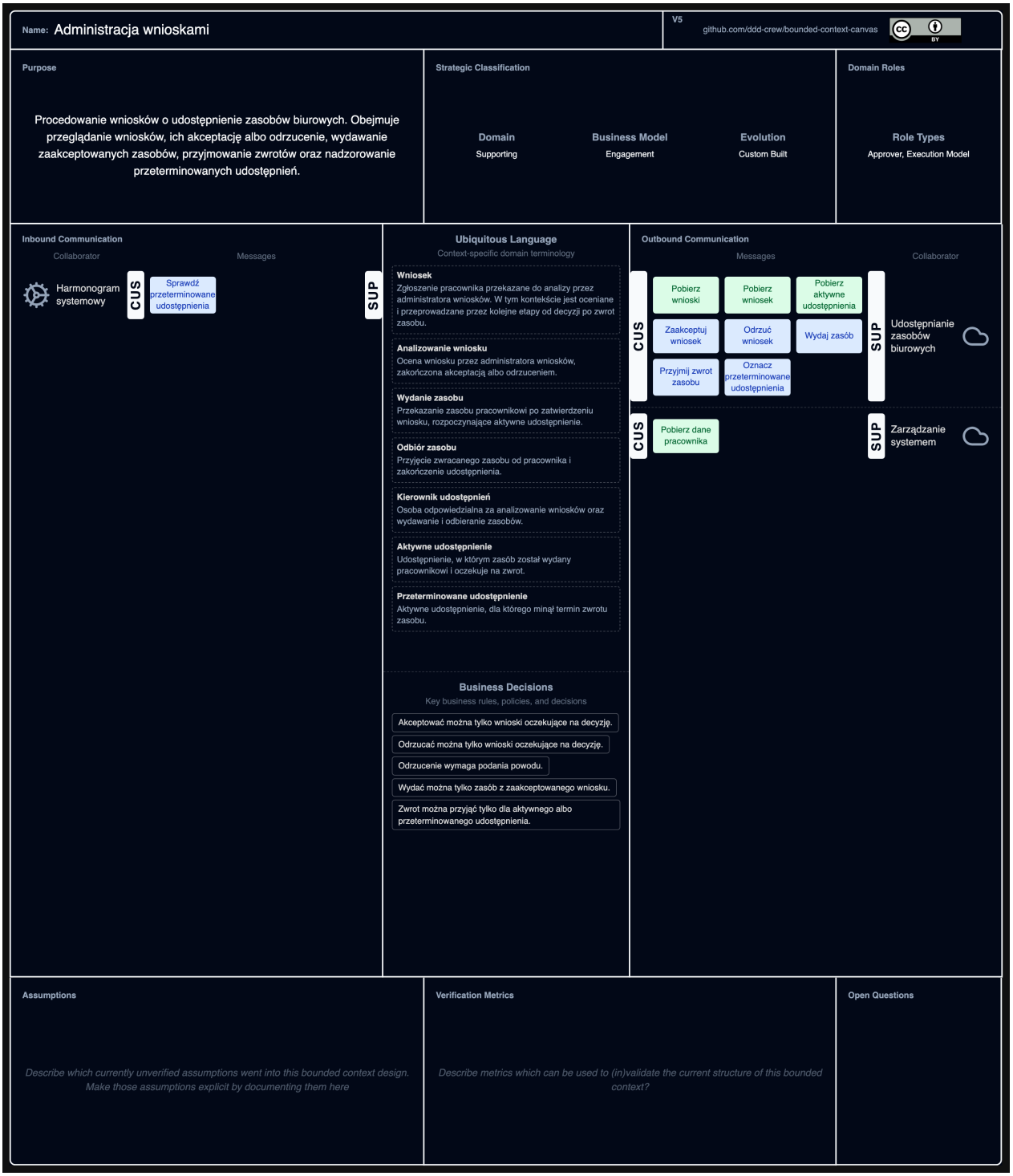


Figure 2.4: Bounded Context Canvas: Administracja wnioskami

2.6. Przypadki użycia

2.6.1. Diagram przypadków użycia

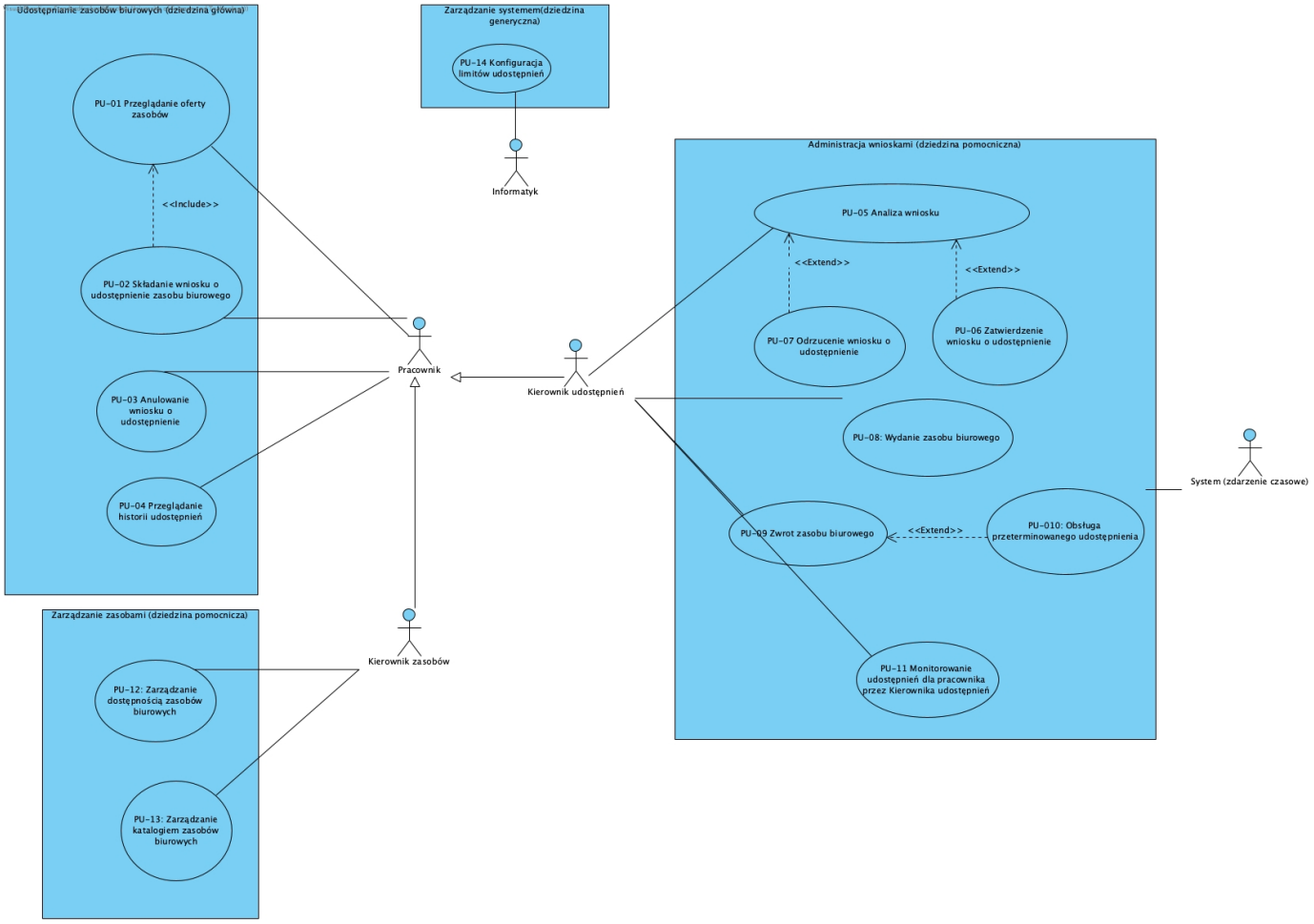


Figure 2.5: Diagram przypadków użycia.

2.6.2. PU-01: Przeglądanie oferty zasobów biurowych

Domena: Udostępnianie zasobów biurowych
Aktor: Pracownik
Warunek wstępny: Pracownik jest zalogowany
Kroki:

- Pracownik wyświetla katalog zasobów biurowych.
- Pracownik widzi zasoby, ich typ oraz dostępności.

Wynik: Pracownik widzi oferowane zasoby biurowe, według wprowadzonych kryteriów

Scenariusz alternatywny: Brak dostępnych zasobów w wybranym terminie — system informuje pracownika.

2.6.3. PU-02: Składanie wniosku o udostępnienie zasobu biurowego

Domena: Udostępnianie zasobów biurowych
Aktor: Pracownik
Cel aktora: Pracownik chce zarezerwować zasób biurowy na wybrany termin, aby móc z niego skorzystać w pracy.
Warunek wstępny: Pracownik jest zalogowany, zasób jest dostępny w wybranym terminie
Kroki:

- Pracownik wybiera zasób biurowy z katalogu.
- Pracownik wybiera termin udostępnienia.
- Limit wypożyczeń pracownika jest weryfikowany.
- Dostępność zasobu biurowego jest sprawdzana.
- Pracownik składa wniosek o udostępnienie.
- Pracownik jest powiadamiany o złożeniu wniosku.
- Kierownik jest powiadamiany udostępnień o złożeniu nowego wniosku.

Wynik: Wniosek o udostępnienie zasobu biurowego złożony.

Reguły biznesowe:

- O udstępnienie zasobu można wnioskować tylko wtedy gdy jest aktywny i dostępny w danym terminie
- Zasób jest niedostępny, jeśli istnieje zatwierdzony wniosek w nakładającym się terminie
- Pracownik nie może złożyć wniosku, jeśli osiągnął limit aktywnych udostępnień zasobu.
- Zasób jest aktywny, jeśli można o niego wnioskować.

Scenariusz alternatywny A: Pracownik przekroczył limit udostępnień - system uniemożliwia złożenie wniosku.
Scenariusz alternatywny B: Zasób niedostępny w terminie — system uniemożliwia złożenie wniosku.

2.6.4. PU-03: Anulowanie wniosku o udostępnienie zasobu biurowego

Domena: Udostępnianie zasobów biurowych
Aktor: Pracownik
Cel aktora: Pracownik chce wycofać swój wniosek o udostępnienie, ponieważ jego plany uległy zmianie.
Warunek wstępny: Pracownik jest zalogowany, jego wniosek oczekuje na decyzję lub został zaakceptowany
Kroki:

- Pracownik przegląda swoje wnioski o udostępnienie.
- Pracownik wybiera własny wniosek, który chce anulować.
- Pracownik anuluje wybrany wniosek.
- System nadaje wnioskowi status *Anulowany.*
- System zwalnia zasób biurowy w terminie objętym wnioskiem.

Wynik: Wniosek anulowany, zasób biurowy ponownie dostępny. **Reguły biznesowe:**

- Pracownik może anulować wyłącznie własny wniosek.
- Anulować można tylko wniosek nieprzeanalizowany lub zaakcpetownany przez Kierownika zasobów

Scenariusz alternatywny: Pracownik próbuje anulować wniosek, który został już wydany lub zakończony - anulowanie nie jest możliwe.

2.6.5. PU-04: Przeglądanie historii udostępnień

Domena: Udostępnianie zasobów biurowych
Aktor: Pracownik
Warunek wstępny: Pracownik jest zalogowany
Kroki:

- Pracownik przegląda swoje aktywne udostępnienia.
- Pracownik przegląda historię zakończonych udostępnień.

Wynik: Pracownik widzi swoje aktywne i historyczne udostępnienia.

2.6.6. PU-05: Analiza wniosku o udostępnienie

Domena: Administracja wnioskami
Aktor: Kierownik udostępnień
Cel aktora: Kierownik udostępnień chce ocenić, czy wniosek pracownika może zostać zrealizowany.
Warunek wstępny: Kierownik udostępnień jest zalogowany, wniosek ma status *Złożony*
Kroki:

- Kierownik udostępnień otrzymuje powiadomienie o nowym wniosku.
- Kierownik udostępnień przegląda listę wniosków o udostępnienie.
- Kierownik udostępnień analizuje wniosek — sprawdza zasób, termin, pracownika.
- Kierownik udostępnień zatwierdza lub odrzuca wniosek.

Wynik: Wniosek przeanalizowany — rozszerzenie o PU-05 lub PU-06.

2.6.7. PU-06: Zatwierdzenie wniosku o udostępnienie

Domena: Administracja wnioskami
Aktor: Kierownik udostępnień
Warunek wstępny: Wniosek jest złożony **Kroki:**

- Kierownik udostępnień zatwierdza wniosek.
- Dostępności zasobu biurowego jego aktualizowana.
- Pracownik jest powiadamiany o zatwierdzeniu wniosku.

Wynik: Wniosek zatwierdzony, dostępność zasobu zaktualizowana

2.6.8. PU-07: Odrzucenie wniosku o udostępnienie

Domena: Administracja wnioskami
Aktor: Kierownik udostępnień
Warunek wstępny: Wniosek jest złożony. **Kroki:**

- Kierownik udostępnień odrzuca wniosek.
- Pracownik jest powiadamiany o odrzuceniu wniosku.

Wynik: Wniosek odrzucony, dostępność zasobu biurowego nie zaktualizowana.

2.6.9. UC-08: Wydanie zasobu biurowego

Domena: Administracja wnioskami
Aktor: Kierownik udostępnień
Cel aktora: Kierownik udostępnień chce przekazać zasób pracownikowi, który ma zatwierdzony wniosek.
Warunek wstępny: Wniosek jest zatwierdzony, pracownik zgłosił się po odbiór
Kroki:

- Kierownik udostępnień przegląda zatwierdzone wnioski.
- Kierownik udostępnień wydaje zasób biurowy pracownikowi.
- Datę i godzinę wydania zasobu biurowego jest rejestrowana.
- Pracownik jest powiadamiany o wydaniu zasobu biurowego.

Wynik: Zasób biurowy wydany, udostępnienie aktywne.

2.6.10. PU-09: Zwrot zasobu biurowego

Domena: Administracja wnioskami
Aktor: Kierownik udostępnień
Warunek wstępny: Udostępnienie ma status *Aktywne*
Kroki:

- Pracownik zwraca zasób biurowy.
- Kierownik udostępnień odbiera zasób biurowy.
- Pracownik jest powiadamiany o zakończeniu udostępnienia.

Wynik: Zasób biurowy zwrócony, udostępnienie zakończone.
Scenariusz alternatywny: Termin zwrotu przekroczony.

2.6.11. PU-10: Obsługa przeterminowanego udostępnienia

Domena: Udostępnianie zasobów biurowych
Aktor: System (zdarzenie czasowe)
Warunek wstępny: Udostępnienie ma status *Aktywne*, termin zwrotu minął
Kroki:

- System wykrywa przekroczenie terminu zwrotu zasobu biurowego.
- System stwierdza przeterminownie udostępnienia
- Pracownik oraz Kierownik udostępnień jest powiadamiany o przekroczeniu terminu zwrotu zasobu

Wynik: Pracownik i Kierownik udostępnień poinformowani o przekroczeniu terminu.

2.6.12. PU-11: Monitorowanie udostępnień dla pracownika przez Kierownika udostępnień

Domena: Administracja wnioskami
Aktor: Kierownik udostępnień
Warunek wstępny: Kierownik udostępnień jest zalogowany
Kroki:

- Kierownik udostępnień przegląda wszystkie aktywne udostępnienia zasobów biurowych.
- Kierownik udostępnień monitoruje terminy zwrotu zasobów biurowych.
- Kierownik udostępnień przegląda historię wniosków o udostępnienie pracowników.

Wynik: Kierownik udostępnień ma pełny widok udostępnień.

2.6.13. PU-12: Zarządzanie dostępnością zasobów biurowych

Domena: Zarządzanie zasobami biurowymi
Aktor: Kierownik zasobów
Cel aktora: Kierownik zasobów chce utrzymać aktualną informację o dostępności zasobów biurowych.
Warunek wstępny: Kierownik zasobów jest zalogowany
Kroki:da

- Kierownik zasobów przegląda obłożenie zasobów biurowych.
- Kierownik zasobów identyfikuje przeciążone strefy lub niedostępne zasoby.
- Kierownik zasobów oznacza zasób biurowy jako niedostępny.

Wynik: Dostępność zasobów biurowych zaktualizowana.
Scenariusz alternatywny: Zasób ma aktywne udostępnienia

2.6.14. PU-13: Zarządzanie katalogiem zasobów biurowych

Domena: Zarządzanie zasobami biurowymi
Aktor: Kierownik zasobów
Cel aktora: Kierownik zasobów chce utrzymać aktualny katalog zasobów biurowych dostępnych dla pracowników.
Warunek wstępny: Kierownik zasobów jest zalogowany
Kroki — Dodanie zasobu:

- Kierownik zasobów dodaje nowy zasób biurowy.
- Kierownik zasobów wprowadza dane zasobu — typ, nazwa, lokalizacja, atrybuty.
- Zasób jest dodawany do katalogu zasobów biurowych.

Kroki — Modyfikacja zasobu:

- Kierownik zasobów wybiera zasób biurowy do modyfikacji.
- Możliwości modyfikacji zasobu jest weryfikowna -jeśli zasób ma aktywne udostępnienia. System uniemożliwia modyfikację.
- Kierownik zasobów edytuje dane zasobu biurowego.
- System aktualizuje katalog zasobów biurowych.

Kroki — Dezaktywacja zasobu:

- Kierownik zasobów wybiera zasób biurowy do dezaktywacji.
- Możliwości modyfikacji jest weryfikowana - jeśli zasób ma aktywne udostępnienia. System uniemożliwia dezaktywację.

- Zasób biurowy zostaje dezaktywowany i nie jest już dostępny do wypożyczenia.

Wynik: Katalog zasobów biurowych zaktualizowany.

Reguły biznesowe:

- Modyfikacja jest niemożliwa, jeśli jest aktualnie udostępniany. *Aktywne.*
- Modyfikacja jest niemożliwa, jeśli istnieją zaakceptowane wnioski w przyszłości

2.6.15. PU-14: Konfiguracja limitu udostępnień

Domena: Zarządzanie systemem
Aktor: Informatyk
Cel aktora: Informatyk chce określić maksymalną liczbę zasobów, o które pracownik może jednocześnie wnioskować, aby kontrolować wykorzystanie zasobów biurowych.
Warunek wstępny: Informatyk jest zalogowany
Kroki:

- Informatyk przechodzi do ustawień limitów udostępnień.
- Informatyk wprowadza nowy limit wypożczyeń indywidualny dla wybranego pracownika lub dla wszytkich

Wynik: Limit udostępnień zaktualizowany.
Scenariusz alternatywny: Informatyk wprowadza nieprawidłową wartość (np. ujemną) - system odrzuca zmianę i informuje o błędzie.

3. Modelowanie Taktyczne

3.1. Cel etapu

Celem etapu modelowania taktycznego było przełożenie modelu strategicznego systemu *DeskFlow* — dziedzin, kontekstów ograniczonych oraz przypadków użycia opisanych w rozdziale poprzednim — na model struktury i zachowania oprogramowania, zgodny z zasadami *Domain-Driven Design* oraz stylem czystej architektury. Modelowanie taktyczne obejmuje wnętrze każdego kontekstu ograniczonego: model dziedziny (agregaty, encje, obiekty wartości, fabryki, repozytoria, usługi dziedziny i zdarzenia dziedziny), a także sposób, w jaki ten model jest udostępniany na zewnątrz przez warstwę aplikacji i infrastruktury.

Zakres prac wykonanych na tym etapie obejmuje:

- wybór i zastosowanie stylu czystej architektury — architektury heksagonalnej (portów i adapterów) — jako wspólnego szkieletu dla każdego kontekstu ograniczonego,
- wykonanie modelu struktury i zachowania całego systemu, obejmującego backend oraz dołączoną infrastrukturę (interfejs użytkownika, baza danych, powiadomienia e-mail),
- wykonanie modelu struktury i zachowania backendu, w szczególności warstwy dziedziny: usług dziedziny realizujących przypadki użycia, zdarzeń dziedziny, agregatów, encji i obiektów wartości,
- wykonanie modelu mapowania pomiędzy warstwami systemu oraz pomiędzy kontekstami ograniczonymi,
- wykonanie modelu integracji i wdrożenia komponentów systemu,

Model taktyczny obejmuje sześć kontekstów ograniczonych wprowadzonych na etapie strategicznym: `allocation` (Udostępnianie zasobów biurowych — dziedzina główna), `resources` (Zarządzanie zasobami biurowymi), `administration` (Administracja wnioskami), `employees` (Zarządzanie systemem), `identity` (Uwierzytelnianie i autoryzacja) oraz `notifications` (Powiadomienia). Każdy kontekst jest odrębnym pakietem Javy w ramach modułowego monolitu *DeskFlow* i posiada własny model dziedziny, własne porty i adaptery.

W dalszej części rozdziału opisano kolejno: wybrany styl architektury, model całego systemu, model backendu skoncentrowany na warstwie dziedziny, opis najważniejszych przypadków użycia wraz z diagramami sekwencji ilustrującymi ich realizację, mapowanie między kontekstami, model integracji i wdrożenia, a także zastosowane wzorce projektowe i zasady SOLID.

3.2. Styl architektury

Zgodnie z wymaganiami etapu modelowania taktycznego, dla systemu *DeskFlow* wybrano styl **czystej architektury** w wariancie **architektury heksagonalnej** (*ports and adapters*). Wybór ten był naturalną konsekwencją modelu strategicznego: każdy z siedmiu kontekstów ograniczonych (`allocation`, `resources`, `administration`, `employees`, `identity`, `notifications`) jest modelowany jako odrębny, samodzielny „sześciokąt" — z własnym modelem dziedziny w centrum, własnymi portami wejściowymi i wyjściowymi oraz własnymi adapterami.

3.2.1. Zasady stylu

W architekturze heksagonalnej zastosowanej w projekcie wyróżniono trzy warstwy, konsekwentnie powtórzone w każdym kontekście ograniczonym:

- **Warstwa dziedziny** (`application.domain`) — znajduje się w centrum heksagonu. Zawiera agregaty, encje, obiekty wartości, fabryki, zdarzenia dziedziny oraz usługi dziedziny. Warstwa ta nie zależy od żadnej technologii ani od warstw zewnętrznych — nie zawiera żadnych odwołań do Springa, JPA, HTTP ani SQL.

- **Warstwa aplikacji** (`application.port`, `application.service`) — otacza warstwę dziedziny. Definiuje porty wejściowe (*inbound ports*) — interfejsy opisujące przypadki użycia kontekstu — oraz porty wyjściowe (*outbound ports*) — interfejsy opisujące zależności od świata zewnętrznego (repozytoria, inne konteksty, infrastruktura powiadomień). Usługi aplikacyjne (*application services*) implementują porty wejściowe w roli fasady: orkiestrują wykonanie przypadku użycia, zarządzają transakcją i publikują zdarzenia dziedziny, nie zawierając przy tym logiki biznesowej.

- **Warstwa infrastruktury** (`infrastructure.in`, `infrastructure.out`) — znajduje się na zewnątrz heksagonu. Zawiera adaptery wejściowe (kontrolery REST, nasłuchy zdarzeń) oraz adaptery wyjściowe (repozytoria JPA/Hibernate, klient SMTP, adaptery integrujące z innymi kontekstami ograniczonymi).

Zależności pomiędzy warstwami są jednokierunkowe i skierowane do wewnątrz — zgodnie z zasadą odwrócenia zależności (*Dependency Inversion*): infrastruktura zależy od portów aplikacji, porty aplikacji zależą od modelu dziedziny, a model dziedziny nie zależy od niczego innego. Dzięki temu model dziedziny pozostaje testowalny w izolacji (patrz rozdział o testowaniu) i niezależny od zmian technologicznych w infrastrukturze.

Z punktu widzenia całego systemu, granice kontekstów ograniczonych pełnią rolę granic usług w stylu zorientowanym na usługi (SOA) — komunikacja między kontekstami odbywa się wyłącznie przez porty wejściowe drugiego kontekstu, a nie poprzez bezpośrednie odwołania do jego modelu dziedziny. Ponieważ wszystkie konteksty działają w jednym procesie Javy (modułowy monolit), wywołania te są wywołaniami metod w pamięci, a nie zapytaniami sieciowymi — co odróżnia to rozwiązanie od architektury mikroserwisowej, ograniczając złożoność wdrożenia bez utraty wyraźnych granic dziedzinowych.

3.2.2. Heksagon dziedziny głównej — kontekst `allocation`

Dziedzina główna systemu, *Udostępnianie zasobów biurowych*, jest reprezentowana przez kontekst ograniczony `allocation`. Diagram architektury heksagonalnej tego kontekstu przedstawia:

- w centrum — agregat `AllocationRequest` (WniosekOUdostępnienie) wraz z obiektami wartości oraz usługami dziedziny realizującymi reguły biznesowe,
- port wejściowy `AllocationRequestInputPort`, implementowany przez usługę aplikacyjną `AllocationRequestAppService`, zasilany od zewnątrz przez adapter wejściowy `AllocationRequestController` (REST),
- port wejściowy `AllocationAdministrationInputPort`, używany przez kontekst `administration` (jako klienta tego kontekstu) do operacji administracyjnych na wniosku,
- porty wyjściowe:
  - `AllocationRequestRepositoryPort` (persystencja),
  - `ResourceCatalogPort` i `EmployeeProfilePort` (warstwa antykorupcyjna w stronę kontekstów `resources` i `employees`),
  - `DomainEventPublisherPort` (publikacja zdarzeń dziedziny),
- adaptery wyjściowe realizujące te porty:
  - `HibernateAllocationRequestRepositoryAdapter`,
  - `ResourceCatalogAdapter`,
  - `EmployeeProfileAdapter`,
  - `SpringDomainEventPublisherAdapter`.

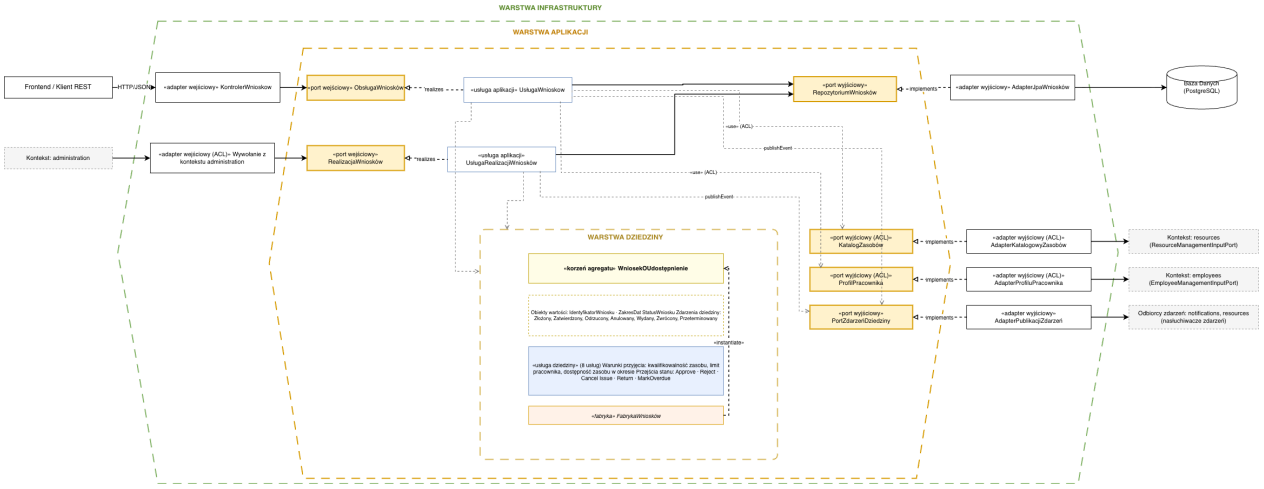


Figure 3.1: Architektura heksagonalna — dziedzina główna (kontekst `allocation`)

3.2.3. Heksagon dziedziny pomocniczej — kontekst `resources`

Jako reprezentatywny przykład dziedziny pomocniczej przyjęto kontekst `resources` (*Zarządzanie zasobami biurowymi*) — jest to dziedzina wspierająca w rozumieniu modelu strategicznego: modeluje istotny, ale nie centralny fragment systemu, i posiada własny, kompletny zestaw elementów taktycznych (agregat, fabryka, repozytorium, cztery usługi dziedziny, cztery zdarzenia dziedziny). Heksagon tego kontekstu jest zbudowany analogicznie do dziedziny głównej:

- w centrum — agregat `OfficeResource` (ZasóbBiurowy) wraz z obiektem wartości `Location` oraz usługami dziedziny odpowiedzialnymi za aktualizację, zmianę dostępności i dezaktywację zasobu,
- port wejściowy `ResourceManagementInputPort`, implementowany przez `ResourceAppService`, zasilany przez adapter wejściowy `ResourceController` (REST) oraz przez adapter wejściowy `ResourceAvailabilityEventListener` (reakcja na zdarzenia z innych kontekstów),
- porty wyjściowe:
  - `OfficeResourceRepositoryPort` (persystencja),
  - `ResourceAllocationRequestPort` (warstwa antykorupcyjna w stronę kontekstu `allocation` — sprawdzenie, czy zasób ma aktywne udostępnienia przed modyfikacją),
  - `DomainEventPublisherPort`,
- adaptery wyjściowe:
  - `HibernateOfficeResourceRepositoryAdapter`,
  - `AllocationRequestAdapter`.

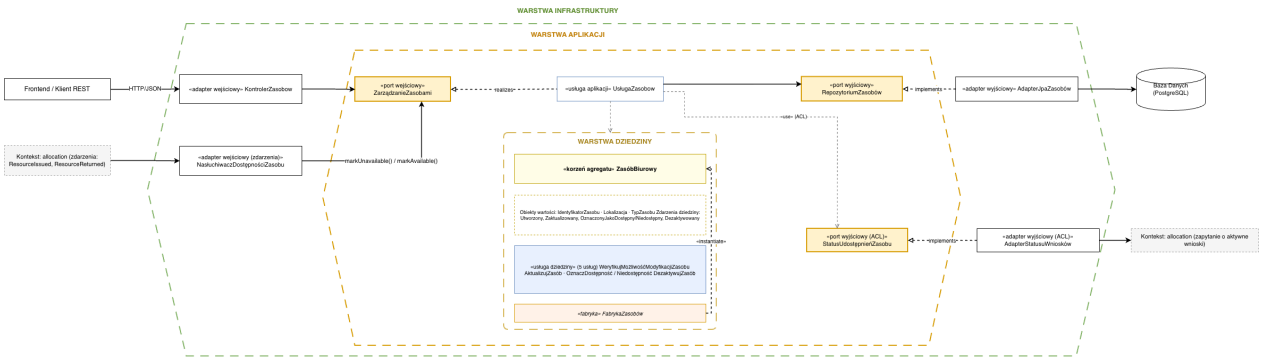


Figure 3.2: Architektura heksagonalna — dziedzina pomocnicza (kontekst `resources`)

3.2.4. Heksagon dziedziny pomocniczej — kontekst `administration`

Drugą dziedziną pomocniczą jest kontekst `administration` (*Administracja wnioskami*). W centrum jego heksagonu znajduje się obiekt wartości `AdministeredAllocationRequest`, który przed każdą operacją ponownie weryfikuje status wniosku na podstawie własnego, lokalnego rzutu danych. Po pozytywnej weryfikacji operacja jest delegowana przez port wyjściowy `AdministrationAllocationPort` (adapter `AllocationAdministrationAdapter`, warstwa antykorupcyjna) do kontekstu `allocation`, który pozostaje właścicielem danych wniosku i jego reguł biznesowych. Ten wariant opisano szczegółowo w podrozdziale 3.5.6.

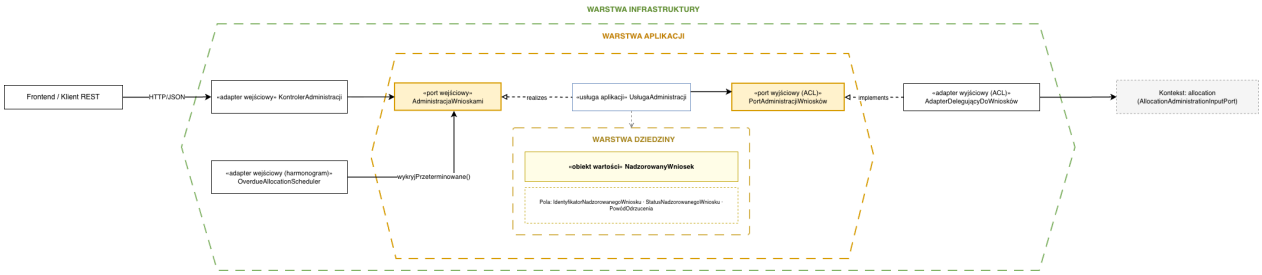


Figure 3.3: Architektura heksagonalna — dziedzina pomocnicza (kontekst `administration`)

3.2.5. Modułowy monolit jako kontener heksagonów

Wszystkie sześć heksagonów współistnieje w jednym procesie Spring Boot (*modułowy monolit*). Granice modułów Javy (pakiety `com.deskflow.<kontekst>`) odpowiadają granicom kontekstów ograniczonych, a struktura podpakietów w każdym module jednoznacznie odwzorowuje warstwy architektury (`application.domain`, `application.port`, `application.service`, `infrastructure.in`, `infrastructure.out`) — zostało to potwierdzone przez analizę rzeczywistego kodu repozytorium oraz przez testy architektury (ArchUnit), opisane w rozdziale dotyczącym testowania. Diagram pakietów backendu, ilustrujący ten podział dla wszystkich sześciu kontekstów jednocześnie, opisano w podrozdziale 3.5.1.

3.3. Mapowanie kontekstów

3.3.1. Diagram dziedzin i kontekstów ograniczonych

Rysunek 3.4 przedstawia mapę dziedzin systemu *DeskFlow* z podziałem na dziedzinę główną, pomocniczą i generyczną oraz przypisanymi do nich kontekstami ograniczonymi.

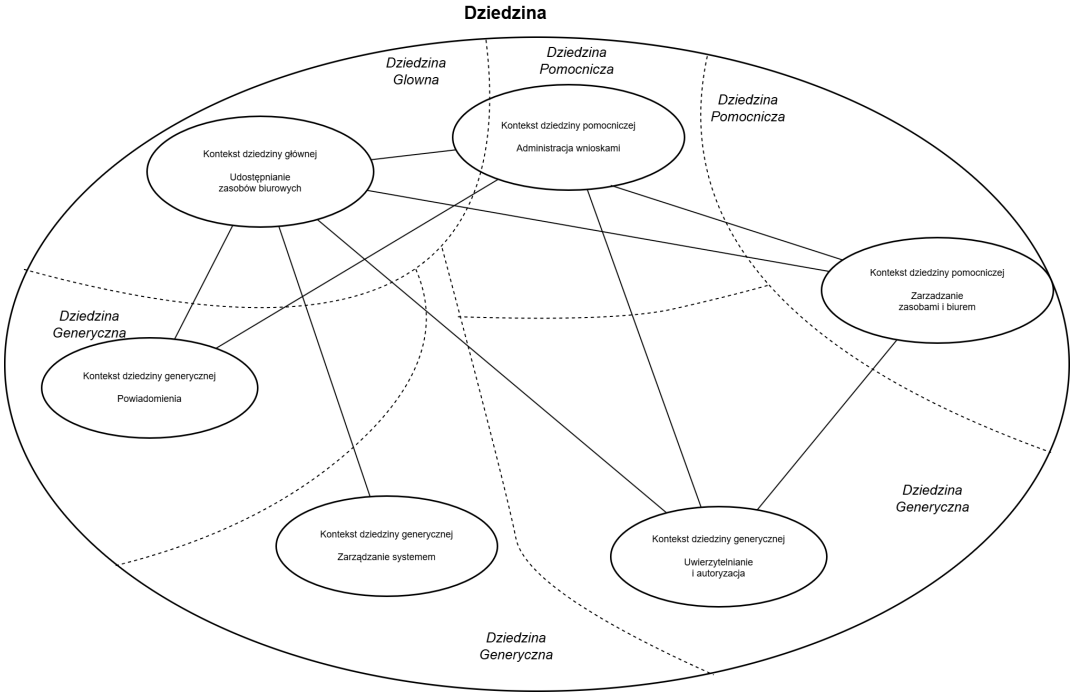


Figure 3.4: Diagram dziedzin i kontekstów ograniczonych systemu DeskFlow

Dziedzina główna obejmuje kontekst `allocation` (udostępnianie zasobów biurowych), który realizuje kluczową logikę biznesową systemu. Dziedziny pomocnicze skupiają konteksty `administration` (administracja wnioskami) oraz `resources` (zarządzanie zasobami i biurem). Dziedziny generyczne odpowiadają za przekrojowe aspekty techniczne: `notifications` (powiadomienia), `employees` (zarządzanie systemem) oraz `identity` (uwierzytelnianie i autoryzacja).

3.3.2. Mapowanie między warstwami w ramach jednego kontekstu

W każdym kontekście ograniczonym mapowanie między warstwami architektury heksagonalnej odbywa się w dwóch miejscach:

- **na granicy infrastruktura → aplikacja** — adaptery wejściowe (kontrolery REST) mapują obiekty DTO (np. `AllocationRequestDTO`) na typy portów wejściowych (identyfikatory domenowe, obiekty wartości), korzystając z dedykowanych klas mapujących (np. `AllocationRequestRestMapper`); analogicznie w drugą stronę — modele odpowiedzi (*Read Model*, np. `AllocationRequestView`) są mapowane na DTO odpowiedzi,

- **na granicy aplikacja → infrastruktura wyjściowa** — adaptery wyjściowe (repozytoria JPA) mapują encje persystencji (np. `AllocationRequestEntity`) na agregaty domeny i odwrotnie, korzystając z dedykowanych klas mapujących, tak aby encje JPA — z adnotacjami Hibernate — nigdy nie przenikały do warstwy dziedziny.

Ten podział zapewnia, że model dziedziny pozostaje całkowicie niezależny od technologii zarówno warstwy prezentacji (REST/JSON), jak i warstwy persystencji (JPA/SQL).

3.3.3. Mapowanie między kontekstami ograniczonymi

Modelowanie strategiczne zidentyfikowało sześć kontekstów ograniczonych. Na etapie taktycznym każda relacja między kontekstami została zrealizowana jednym z czterech wzorców mapowania kontekstów DDD, zgodnie z legendą zastosowaną na diagramie komponentów (rys. 3.5):

Table 3.1: Mapowanie kontekstów ograniczonych systemu DeskFlow

Kontekst górny	**Kontekst dolny**	**Wzorzec**	**Opis**
`allocation`	`resources`	Customer/Supplier + ACL	`allocation` jest klientem `resources` przy weryfikacji kwalifikowalności zasobu za pomocą dedykowanego portu zapytania, z warstwą antykorupcyjną tłumaczącą dane zasobu do lokalnego modelu używanego w module `allocation`.
`allocation`	`employees`	Customer/Supplier + ACL	`allocation` jest klientem danych o pracowniku (limity, profil) udostępnianych przez `employees`; dane te są tłumaczone przez warstwę antykorupcyjną do lokalnego, uproszczonego modelu używanego wyłącznie w module `allocation`.
`administration`	`allocation`	Customer/Supplier + ACL	`administration` jest klientem operacji administracyjnych wykonywanych na wniosku o alokację, z pełną warstwą antykorupcyjną tłumaczącą identyfikatory i statusy między modułami.
`administration`	`employees`	Customer/Supplier	zarządzanie danymi pracowników (przypadki użycia powiązane z rolą Informatyka) — relacja prostsza, bez odrębnego modelu antykorupcyjnego.

Table 3.1 — ciąg dalszy

Kontekst górny	**Kontekst dolny**	**Wzorzec**	**Opis**
`identity`	`employees`	Customer/Supplier + ACL	uwierzytelnianie odbywa się na podstawie danych pracownika udostępnianych przez moduł `employees`, bez ujawniania pełnego modelu pracownika warstwie bezpieczeństwa — dane są tłumaczone do węższego, dedykowanego modelu tożsamości.
`notifications`	`employees`	Customer/Supplier	`notifications` jest klientem danych kontaktowych pracownika — pobiera adres e-mail na potrzeby wysyłki powiadomienia.
`notifications`	`allocation`, `resources`	Publisher/Subscriber	`notifications` oraz częściowo `resources` nasłuchują zdarzeń dziedziny publikowanych przez `allocation` za pomocą wspólnej magistrali zdarzeń udostępnianej przez moduł `common`, bez znajomości wewnętrznego modelu dziedziny wydawcy.

3.3.4. Warstwa antykorupcyjna (Anti-Corruption Layer)

Najczęściej stosowanym wzorcem mapowania w projekcie jest **warstwa antykorupcyjna (ACL)**. W systemie *DeskFlow* każda relacja typu Customer/Supplier z ACL jest zrealizowana tym samym, powtarzalnym wzorcem implementacyjnym:

1. kontekst-klient definiuje własny port wyjściowy, wyrażony w pojęciach swojego własnego modelu dziedziny ,
2. adapter implementujący ten port jest jedynym miejscem w kodzie, które **zna oba modele** i tłumaczy typy jednego kontekstu na typy drugiego),
3. adapter wywołuje wyłącznie port wejściowy drugiego kontekstu — nigdy bezpośrednio jego agregatów, repozytoriów czy usług dziedziny.

Dzięki temu zmiana modelu dziedziny w kontekście dostawcy (np. dodanie nowego statusu wniosku) nie wymaga zmian w modelu dziedziny kontekstu-klienta — wystarczy zaktualizować mapowanie w jednym adapterze.

3.3.5. Mapowanie zdarzeń dziedziny między kontekstami

Mapowanie typu Publisher/Subscriber nie wymaga odrębnej warstwy antykorupcyjnej w tym samym sensie — zdarzenia dziedziny (`AllocationRequestSubmitted`, `ResourceIssued`

itd.) są same w sobie kontraktem między kontekstami: są to niewielkie, niemutowalne rekordy przenoszące tylko identyfikatory i dane elementarne (nigdy referencje do agregatów). Subskrybent (`NotificationEventListener`, `ResourceAvailabilityEventListener`) zna jedynie strukturę zdarzenia, a nie wewnętrzny model dziedziny wydawcy.

Publikacja jest dodatkowo odsunięta w czasie względem transakcji źródłowej — adnotacja `@TransactionalEventListener(phase = AFTER_COMMIT)` gwarantuje, że subskrybent zobaczy zdarzenie tylko wtedy, gdy transakcja zapisu agregatu zakończyła się powodzeniem, co odpowiada zasadzie z wykładu, że modyfikacja innego agregatu w reakcji na zdarzenie nigdy nie odbywa się w tej samej transakcji.

3.4. Model struktury i zachowania całego systemu

System *DeskFlow* składa się z backendu (modułowy monolit Java/Spring Boot) oraz dołączonej infrastruktury: frontendowego klienta webowego (Next.js), relacyjnej bazy danych (PostgreSQL) oraz usługi wysyłki powiadomień e-mail (SMTP). Poniższy podrozdział opisuje strukturę całego systemu na poziomie komponentów i węzłów wdrożeniowych, a następnie jego zachowanie — czyli sposób, w jaki żądanie użytkownika przepływa przez poszczególne komponenty.

3.4.1. Diagram komponentów

Diagram komponentów przedstawia sześć kontekstów ograniczonych backendu jako odrębne pakiety komponentów oraz sposób, w jaki komunikują się one ze sobą i ze światem zewnętrznym. Na diagramie wyróżniono: Diagram komponentów przedstawia sześć kontekstów ograniczonych backendu jako odrębne pakiety komponentów oraz sposób, w jaki komunikują się one ze sobą i ze światem zewnętrznym. Na diagramie wyróżniono:

- aktora zbiorczego *Employee / Manager / IT Admin*, reprezentującego role korzystające z systemu przez warstwę REST,
- kontrolery REST każdego kontekstu jako punkty wejścia:
  - `AuthController`,
  - `AllocationRequestController`,
  - `ResourceController`,
  - `AdministrationController`,
  - `EmployeeAdminController`,
- usługi aplikacyjne każdego kontekstu oraz bazy danych, z którymi się komunikują (`allocation_requests`, `office_resources`, `employees`),
- komponent `SpringDomainEventPublisherAdapter` (oznaczony na diagramie jako *Event-Bus*) — centralny, wewnątrzprocesowy mechanizm publikacji zdarzeń dziedziny, używany przez wiele kontekstów.

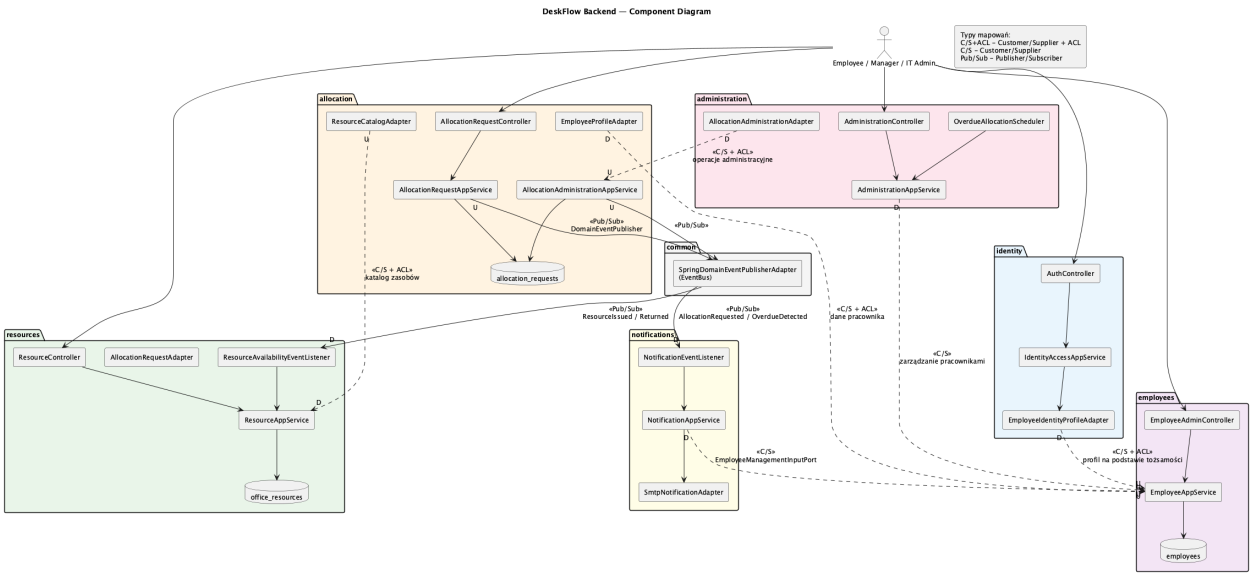


Figure 3.5: Diagram komponentów — backend systemu DeskFlow

3.4.2. Diagram wdrożenia

Diagram wdrożenia obejmuje całą infrastrukturę systemu — backend wraz z dołączonymi komponentami zewnętrznymi (interfejs użytkownika, repozytorium danych, serwer pocztowy). Środowisko deweloperskie systemu *DeskFlow* jest uruchamiane przy pomocy Docker Compose i obejmuje cztery kontenery:

- `deskflow-frontend` — aplikacja Next.js 15 serwująca interfejs użytkownika, nasłuchująca na porcie 3000,
- `deskflow-backend` — aplikacja Spring Boot 3 / Java 21 udostępniająca REST API, nasłuchująca na porcie 8080,
- `postgres` — baza danych PostgreSQL 17, dostępna dla backendu przez JDBC na porcie 5432,
- `mailhog` — serwer SMTP używany do przechwytywania i podglądu wiadomości e-mail w środowisku deweloperskim, dostępny na porcie 1025.

Urządzenie użytkownika (*User Device*) komunikuje się z frontendem przez przeglądarkę internetową protokołem HTTP. W środowisku produkcyjnym konfiguracja Docker Compose jest analogiczna, lecz przystosowana do wdrożenia na platformie Coolify, z parametrami przekazywanymi przez zmienne środowiskowe oraz docelowym serwerem SMTP w miejsce kontenera `mailhog`.

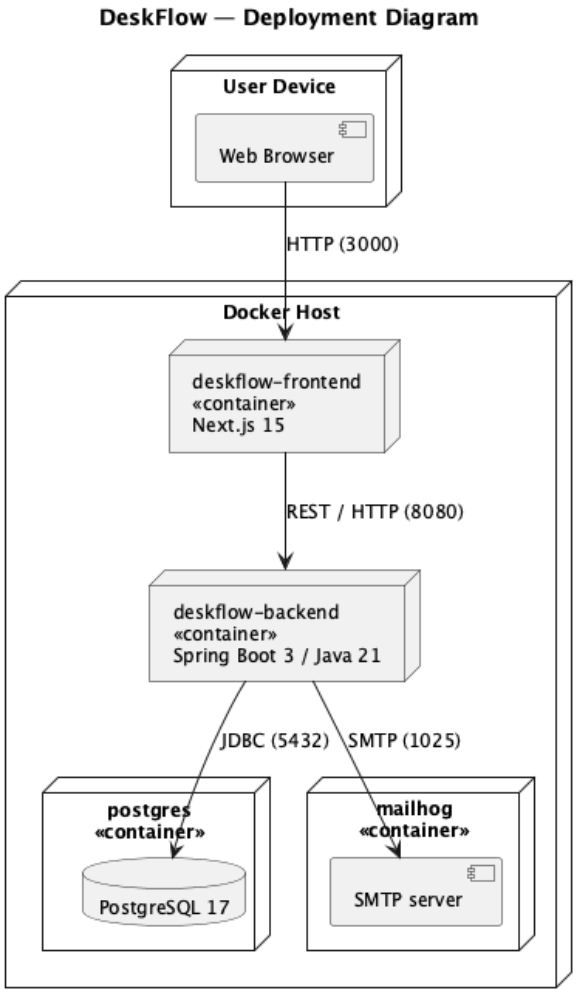


Figure 3.6: Diagram wdrożenia — system DeskFlow

3.4.3. Zachowanie całego systemu

Na poziomie całego systemu zachowanie składa się z trzech rodzajów przepływów, spójnych dla wszystkich przypadków użycia:

1. **Przepływ synchroniczny żądanie–odpowiedź.** Użytkownik wykonuje akcję w interfejsie webowym (np. składa wniosek, zatwierdza wniosek). Frontend Next.js wysyła żądanie REST/HTTP do backendu. Odpowiedni kontroler REST (adapter wejściowy) tłumaczy żądanie HTTP na wywołanie portu wejściowego odpowiedniego kontekstu, które jest realizowane synchronicznie w ramach jednej transakcji bazodanowej. Wynik operacji jest zwracany do frontendu jako odpowiedź HTTP.

2. **Przepływ wewnątrzprocesowy między kontekstami.** Jeżeli realizacja przypadku użycia wymaga danych lub operacji z innego kontekstu ograniczonego (np. `administration` potrzebuje zmienić status wniosku należącego do kontekstu `allocation`), wywołanie odbywa się przez port wyjściowy z warstwą antykorupcyjną — jako bezpośrednie

wywołanie metody Javy w tym samym procesie, nie jako żądanie sieciowe. Pozwala to zachować wyraźne granice modelu dziedziny bez kosztu komunikacji rozproszonej.

3. **Przepływ asynchroniczny — zdarzenia dziedziny.** Po zatwierdzeniu transakcji (`@TransactionalEventListener(phase = AFTER_COMMIT)`) opublikowane zdarzenia dziedziny (np. `AllocationRequestSubmitted`, `ResourceIssued`) są dostarczane przez wewnątrzprocesową magistralę zdarzeń Springa do zainteresowanych nasłuchów — w szczególności do kontekstu `notifications`, który wysyła powiadomienie e-mail przez serwer SMTP, oraz do kontekstu `resources` (`ResourceAvailabilityEventListener`), reagującego na zmiany stanu udostępnienia zasobu.

Te trzy przepływy są ilustrowane na poziomie poszczególnych przypadków użycia za pomocą diagramów sekwencji w podrozdziale **??**.

3.5. Model struktury backendu — warstwa dziedziny

Poniższy podrozdział opisuje model struktury backendu systemu *DeskFlow*, skoncentrowany przede wszystkim na warstwie dziedziny — agregatach, encjach, obiektach wartości, fabrykach, repozytoriach, usługach dziedziny i zdarzeniach dziedziny każdego kontekstu ograniczonego. Opis stosuje stereotypy DDD: «korzeń agregatu», «obiekt wartości», «fabryka», «repozytorium», «usługa dziedziny», «zdarzenie dziedziny». Nazwy klas podano w formie zgodnej z rzeczywistą implementacją (kod źródłowy backendu w języku angielskim), natomiast nazwy pojęciowe — zgodnie z językiem wszechobecnym projektu w języku polskim.

3.5.1. Diagram pakietów backendu

Diagram pakietów przedstawia podział kodu backendu (`com.deskflow`) na sześć pakietów odpowiadających kontekstom ograniczonym:

- `administration`, `allocation`, `employees`,
- `identity`, `notifications`, `resources`,

a w ramach każdego z nich — na podpakiety odpowiadające warstwom architektury heksagonalnej:

- `application.domain.model`, `application.domain.service`,
- `application.port.in`, `application.port.out`, `application.service`,
- `infrastructure.in`, `infrastructure.out`.

Strzałki zależności między pakietami najwyższego poziomu odzwierciedlają rzeczywiste zależności kompilacyjne w kodzie: `administration` zależy od `allocation`, `allocation` zależy od `employees` i `resources`, `identity` zależy od `employees`, a `notifications` zależy od `allocation`.

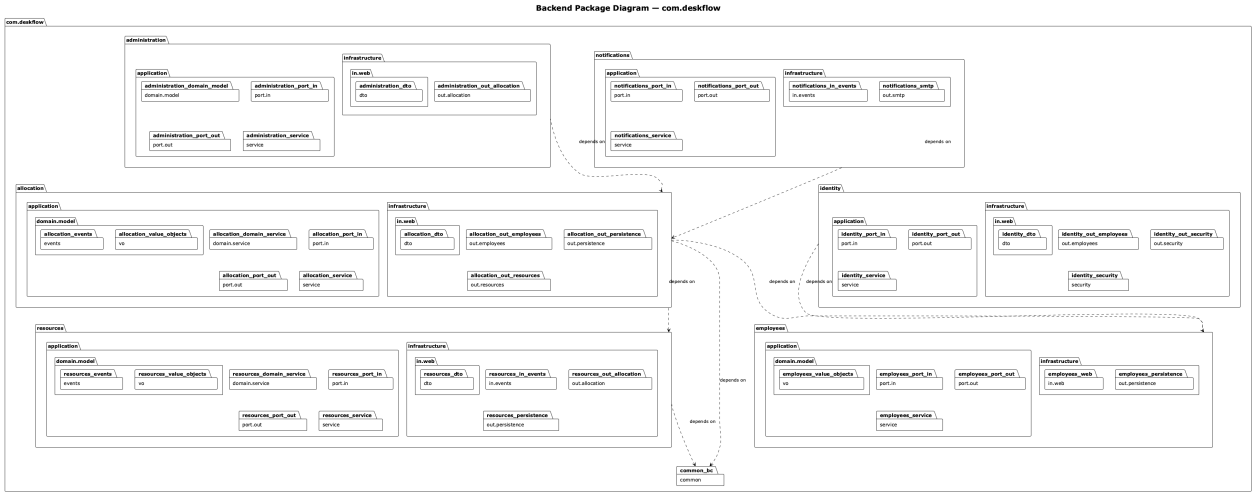


Figure 3.7: Diagram pakietów — backend systemu DeskFlow (`com.deskflow`)

3.5.2. Kontekst `allocation` — dziedzina główna

Kontekst `allocation` realizuje dziedzinę główną *Udostępnianie zasobów biurowych* i zawiera najbogatszy model dziedziny w systemie.

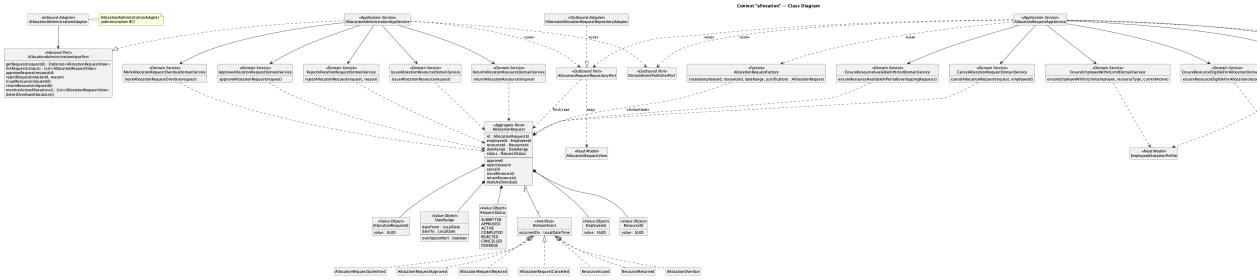


Figure 3.8: Diagram klas — kontekst `allocation` (agregat WniosekOUdostępnienie)

Agregat. `AllocationRequest` «korzeń agregatu» (pojęciowo: **WniosekOUdostępnienie**) reprezentuje wniosek pracownika o udostępnienie zasobu biurowego. Stan agregatu obejmuje identyfikator pracownika, identyfikator zasobu, zakres dat udostępnienia oraz status. Agregat samodzielnie pilnuje poprawności przejść między statusami — każda metoda biznesowa (`approve()`, `reject(reason)`, `cancel()`, `issueResource()`, `returnResource()`, `markAsOverdue()`) weryfikuje status wejściowy i zgłasza wyjątek, jeśli operacja nie jest dozwolona w danym stanie.

Obiekty wartości. «obiekt wartości»:

- `AllocationRequestId`, `EmployeeId`, `ResourceId` — identyfikatory oparte na `UUID`,
- `DateRange` — zakres dat udostępnienia, z metodą `overlaps(other)` używaną do wykrywania kolizji terminów,
- `RequestStatus` — wyliczenie siedmiu statusów cyklu życia wniosku: `SUBMITTED` (Złożony), `APPROVED` (Zatwierdzony), `ACTIVE` (Aktywny), `COMPLETED` (Zakończony), `REJECTED` (Odrzucony), `CANCELLED` (Anulowany), `OVERDUE` (Przeterminowany).

Modele `EmployeeAllocationProfile` i `ResourceAllocationProfile` są rzutowanymi (*Read Model*) reprezentacjami danych pracownika i zasobu, otrzymywanymi przez warstwę antykorupcyjną z kontekstów `employees` i `resources` — nie są częścią agregatu `AllocationRequest`, lecz danymi pomocniczymi do weryfikacji reguł biznesowych.

Fabryka. `AllocationRequestFactory` «fabryka» tworzy nowy egzemplarz `AllocationRequest` w statusie `SUBMITTED`, generując jednocześnie zdarzenie `AllocationRequestSubmitted`.

Repozytorium. `AllocationRequestRepositoryPort` «repozytorium» (port wyjściowy) definiuje operacje wyszukiwania i zapisu agregatu, w tym metody specyficzne dla reguł biznesowych: `findActiveForEmployee`, `findActiveForResource`, `findOverdue`. Implementacją jest adapter `HibernateAllocationRequestRepositoryAdapter`.

Usługi dziedziny. «usługa dziedziny» — dziewięć usług, każda odpowiadająca jednej regule biznesowej lub jednemu przejściu stanu wniosku:

- `EnsureResourceEligibleForAllocationDomainService` — sprawdza, czy zasób jest aktywny i dostępny,
- `EnsureEmployeeWithinLimitDomainService` (Weryfikacja limitu pracownika) — sprawdza, czy pracownik nie przekroczył limitu udostępnień danego typu zasobu,
- `EnsureResourceAvailableInPeriodDomainService` (Sprawdzanie dostępności zasobu biurowego) — sprawdza, czy w żądanym terminie nie istnieją kolidujące wnioski,
- sześć prostych usług dziedziny, każda delegująca do odpowiadającej operacji biznesowej agregatu — usługa odrzucenia dodatkowo waliduje, że podano niepustą przyczynę odrzucenia:
  - `CancelAllocationRequestDomainService` → `cancel()`,
  - `ApproveAllocationRequestDomainService` → `approve()`,
  - `RejectAllocationRequestDomainService` → `reject()`,
  - `IssueAllocationResourceDomainService` → `issueResource()`,
  - `ReturnAllocationResourceDomainService` → `returnResource()`,
  - `MarkAllocationRequestOverdueDomainService` → `markAsOverdue()`.

Zdarzenia dziedziny. «zdarzenie dziedziny» — siedem zdarzeń, jedno dla każdego istotnego przejścia stanu:

- `AllocationRequestSubmitted`, `AllocationRequestApproved`,
- `AllocationRequestRejected`, `AllocationRequestCancelled`,
- `ResourceIssued`, `ResourceReturned`, `AllocationOverdue`.

Wszystkie implementują wspólny interfejs `DomainEvent` (pole `occurredOn`). Agregat gromadzi zdarzenia wewnętrznie i udostępnia je przez metodę `flushEvents()`, wywoływaną przez usługę aplikacyjną tuż po zapisie agregatu — zgodnie z wzorcem publikacji zdarzeń dziedziny przedstawionym na wykładzie.

Porty wejściowe i usługi aplikacyjne. Kontekst udostępnia dwa porty wejściowe: `AllocationRequestInputPort` (przypadki użycia pracownika: PU-01, PU-02, PU-03, PU-04).

Implementowany jest on przez `AllocationAdministrationAppService` i wywoływany przez kontekst `administration`.

3.5.3. Kanwa agregatu: WniosekOUdostępnienie

Przejścia stanów. Stan agregatu przechowywany jest w polu `status` typu `RequestStatus`. Każde przejście jest strzeżone metodą `requireStatus(...)` lub równoważnym warunkiem — niedozwolone przejście kończy się wyjątkiem `IllegalStateException`. Stany `REJECTED`, `CANCELLED` i `COMPLETED` są stanami końcowymi — żadna z metod mutujących `AllocationRequest` nie udostępnia powrotu z nich do innego stanu.

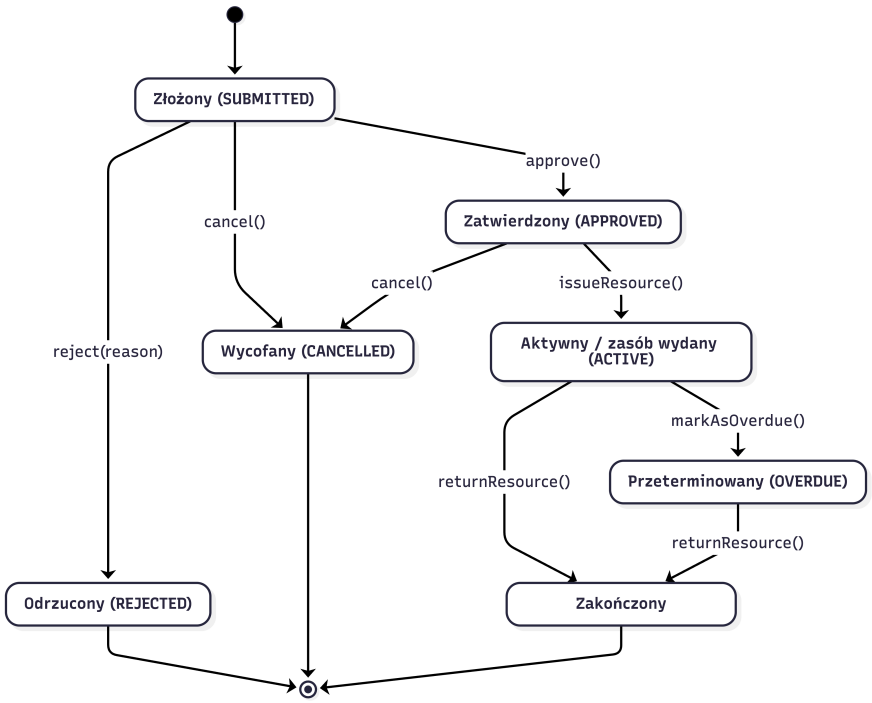


Figure 3.9: Diagram stanów — agregat WniosekOUdostępnienie

Stan	Znaczenie biznesowe	Metoda wejścia
`SUBMITTED`	wniosek złożony, czeka na decyzję	`create(...)`
`APPROVED`	wniosek zaakceptowany, czeka na wydanie zasobu	`approve()`
`ACTIVE`	zasób fizycznie wydany pracownikowi	`issueResource()`
`OVERDUE`	upłynął termin zwrotu, zasób nadal niewydany	`markAsOverdue()`
`COMPLETED`	zasób zwrócony — stan końcowy	`returnResource()`
`REJECTED`	wniosek odrzucony — stan końcowy	`reject(reason)`
`CANCELLED`	wniosek wycofany przez pracownika — stan końcowy	`cancel()`

Table 3.2: Stany agregatu WniosekOUdostępnienie

Niezmienniki wymuszane w agregacie.

1. Wniosek można zatwierdzić (`approve()`) tylko ze stanu `SUBMITTED`.

2. Wniosek można odrzucić (`reject()`) tylko ze stanu `SUBMITTED`, a treść `rejectionReason` nie może być puste/`blank` (weryfikowane przez `RejectAllocationRequestDomainService`).

3. Wniosek można wycofać (`cancel()`) tylko ze stanu `SUBMITTED` lub `APPROVED`.

4. Zasób można wydać (`issueResource()`) tylko ze stanu `APPROVED`.

5. Zasób można zwrócić (`returnResource()`) tylko ze stanu `ACTIVE` lub `OVERDUE`.

6. Wniosek można oznaczyć jako przeterminowany (`markAsOverdue()`) tylko ze stanu `ACTIVE`.

7. Wycofać wniosek może tylko pracownik, który go złożył.

   `CancelAllocationRequestDomainService` porównuje identyfikator pracownika żądającego z identyfikatorem pracownika przypisanym do wniosku.

8. Obiekt wartości `DateRange` wymusza, że data początkowa jest wcześniejsza niż data końcowa — niezmiennik weryfikowany w konstruktorze obiektu wartości, nie w samym agregacie.

Polityki korygujące. Trzy reguły biznesowe nie mogą być wymuszone atomowo przez sam agregat, ponieważ zależą od czasu, danych innych agregatów lub wielu instancji tego samego typu:

- **Przeterminowanie wniosku** jest korygowane asynchronicznie — agregat nie ma mechanizmu, by samoistnie „zauważyć" upływ czasu. `OverdueAllocationScheduler` (`@Scheduled`, domyślnie co 15 minut) okresowo odpytuje repozytorium o zaległe wnioski i wywołuje na każdym z nich przejście `markAsOverdue()`.

- **Brak kolizji terminów dla tego samego zasobu** jest polityką, nie niezmiennikiem agregatu — `EnsureResourceAvailableInPeriodDomainService` sprawdza nakładające się wnioski w momencie składania nowego, odpytując repozytorium o inne instancje tego samego agregatu. Ponieważ sprawdzenie i zapis nie są jedną operacją na jednym agregacie, przy dużej współbieżności teoretycznie możliwy jest wyścig (*race condition*) — kod nie stosuje blokady pesymistycznej ani unikalnego ograniczenia bazodanowego adresującego ten przypadek.

- **Limit przydziałów pracownika** (`EnsureEmployeeWithinLimitDomainService`) jest liczony przez zliczenie aktywnych wniosków danego pracownika dla danego typu zasobu — reguła rozpięta na wielu instancjach agregatu, wymuszana na poziomie usługi aplikacyjnej, a nie pojedynczej instancji.

Komenda	Metoda domenowa	Wymagany stan
`submitRequest(...)`	`create(...)`	— (nowa instancja)
`cancelRequest(requestId, employeeId)`	`cancel()`	`SUBMITTED`, `APPROVED`
`approveRequest(requestId)`	`approve()`	`SUBMITTED`
`rejectRequest(requestId, reason)`	`reject(reason)`	`SUBMITTED`
`issueResource(requestId)`	`issueResource()`	`APPROVED`
`returnResource(requestId)`	`returnResource()`	`ACTIVE`, `OVERDUE`
`detectOverdueAllocations()`[1]	`markAsOverdue()`	`ACTIVE`

Table 3.3: Komendy agregatu WniosekOUdostępnienie

Obsługiwane komendy.

Przepustowość. Cykl życia pojedynczej instancji jest krótki, a liczba klientów wydających komendy — mała (zwykle pracownik składający/wycofujący wniosek oraz jeden administrator zatwierdzający/wydający/zwracający zasób; maksymalnie do 3–4, jeśli różni administratorzy obsługują różne etapy, a scheduler dolicza przejście w przeterminowanie).

Metryka	Średnio (avg)	Maksymalnie (max)
Tempo obsługi komend na instancję	∼1 komenda / kilka godzin (3–4 komendy rozłożone na dni)	2 komendy w krótkim odstępie (np. złożenie i błyskawiczne zatwierdzenie)
Liczba klientów wydających komendy na instancję	2 (pracownik + administrator)	3–4 (różni administratorzy + scheduler dla `markAsOverdue`)

Table 3.4: Przepustowość agregatu WniosekOUdostępnienie

Szansa konfliktu współbieżności na pojedynczej instancji jest więc niska — realne ryzyko leży nie w samym agregacie, lecz w przekrojowej operacji sprawdzania kolizji terminów na wielu jego instancjach (patrz wyżej, polityka korygująca).

Rozmiar. Liczba zdarzeń na instancję jest ograniczona z góry przez skończony automat stanów — od 1 (sama rejestracja) do 5 w najdłuższej ścieżce (`Submitted → Approved → Issued → Overdue → Returned`); typowa „szczęśliwa ścieżka" bez przeterminowania to 4 zdarzenia. Czas życia biznesowy instancji jest krótki — od godzin do kilku tygodni, zależnie od zakresu dat i typu zasobu. Po wejściu w stan końcowy (`REJECTED`/`CANCELLED`/`COMPLETED`) instancja nie jest jednak usuwana — pozostaje w repozytorium jako historia, więc czas przechowywania jest praktycznie nieograniczony mimo krótkiego cyklu biznesowego. Agregat nie posiada nieograniczonych kolekcji wewnętrznych, więc nie wymaga mechanizmu snapshotów.

[1] Komenda wywoływana cyklicznie przez `OverdueAllocationScheduler`, nie przez użytkownika.

3.5.4. Kontekst `resources` — dziedzina pomocnicza

Kontekst `resources` realizuje dziedzinę pomocniczą *Zarządzanie zasobami biurowymi*.

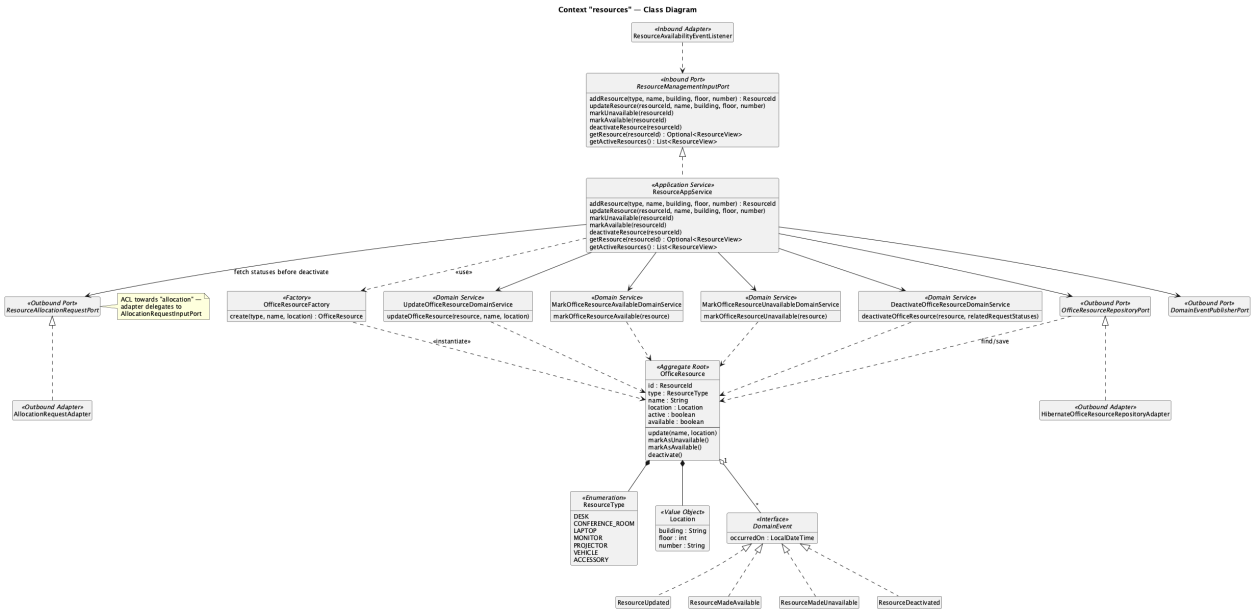


Figure 3.10: Diagram klas — kontekst `resources` (agregat ZasóbBiurowy)

Agregat. `OfficeResource` «korzeń agregatu» (pojęciowo: **ZasóbBiurowy**) reprezentuje pojedynczy zasób biurowy — biurko, salę konferencyjną, laptop, monitor, projektor, pojazd lub akcesorium. Stan obejmuje typ, nazwę, lokalizację oraz dwie niezależne flagi: `active` (czy zasób istnieje w katalogu) i `available` (czy aktualnie można go udostępniać). Operacje biznesowe: `update(name, location)`, `markAsUnavailable()`, `markAsAvailable()`, `deactivate()`.

Obiekty wartości. «obiekt wartości» `ResourceType` (wyliczenie siedmiu typów zasobów: `DESK`, `CONFERENCE_ROOM`, `LAPTOP`, `MONITOR`, `PROJECTOR`, `VEHICLE`, `ACCESSORY`) oraz `Location` (budynek, piętro, numer pomieszczenia).

Fabryka i repozytorium. `OfficeResourceFactory` «fabryka» tworzy nowy zasób w katalogu. `OfficeResourceRepositoryPort` «repozytorium».

Usługi dziedziny. «usługa dziedziny»:

- `UpdateOfficeResourceDomainService`,
- `MarkOfficeResourceAvailableDomainService`,
- `MarkOfficeResourceUnavailableDomainService`,
- `DeactivateOfficeResourceDomainService` (Weryfikacja możliwości modyfikacji zasobu) — przyjmuje listę statusów powiązanych wniosków o udostępnienie (pobraną

przez port antykorupcyjny z kontekstu `allocation`) i blokuje dezaktywację, jeśli zasób ma aktywne, zatwierdzone, złożone lub przeterminowane udostępnienia.

Zdarzenia dziedziny «zdarzenie dziedziny»:

- `ResourceUpdated`
- `ResourceMadeAvailable`
- `ResourceMadeUnavailable`
- `ResourceDeactivated`

Port wejściowy. `ResourceManagementInputPort`, implementowany przez `ResourceAppService`, obsługuje przypadki użycia PU-12 i PU-13 (w tym warianty: dodanie, modyfikacja i dezaktywacja zasobu).

3.5.5. Kanwa agregatu: ZasóbBiurowy

Przejścia stanów. Stan agregatu nie jest pojedynczym polem wyliczeniowym, lecz kombinacją dwóch flag boolowskich: `active` i `available`. W kodzie nie istnieje żadna metoda przywracająca zasób ze stanu wycofanego — `deactivate()` jest przejściem końcowym i nieodwracalnym. Metoda `update(newName, newLocation)` jest dozwolona w stanach *Dostępny* i *Niedostępny* i nie zmienia samego stanu.

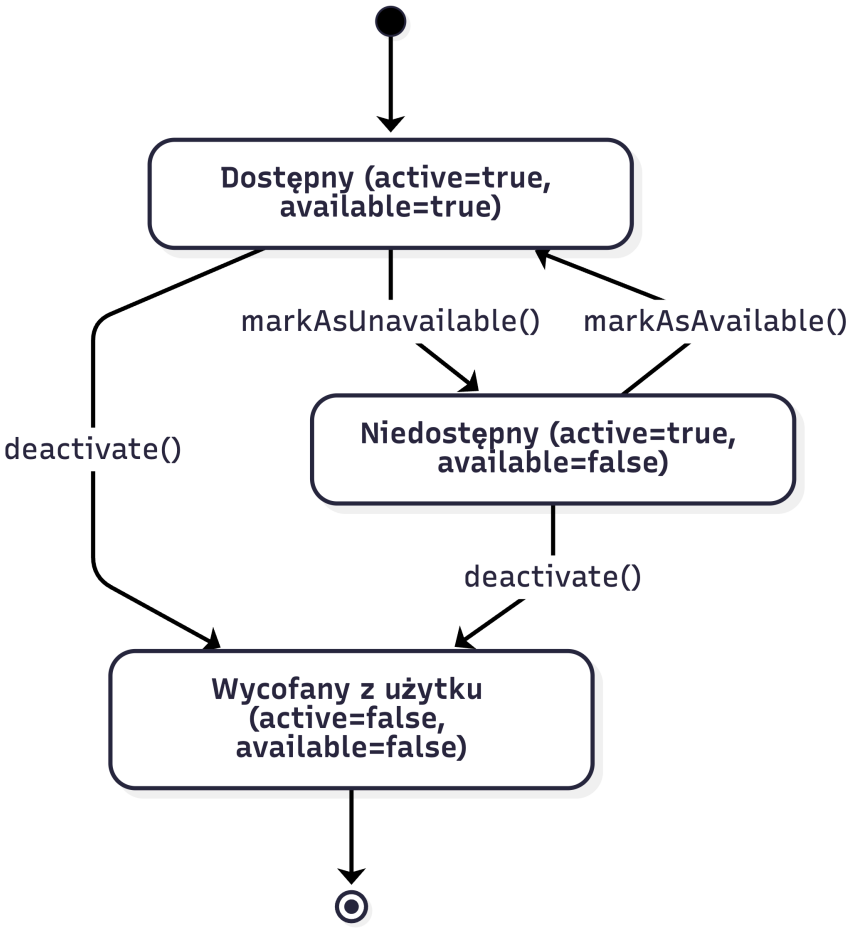


Figure 3.11: Diagram stanów — agregat ZasóbBiurowy

Stan	Znaczenie biznesowe	Metoda wejścia
Dostępny (`active=true`, `available=true`)	zasób istnieje i można go zarezerwować	`create(...)`, `markAsAvailable()`
Niedostępny (`active=true`, `available=false`)	zasób istnieje, ale tymczasowo nie do rezerwacji (np. naprawa, czyszczenie)	`markAsUnavailable()`
Wycofany z użytku (`active=false`, `available=false`)	zasób trwale usunięty z obrotu — stan końcowy	`deactivate()`

Table 3.5: Stany agregatu ZasóbBiurowy

Warto odnotować, że fabryka `create()` — w przeciwieństwie do pozostałych operacji tego agregatu — nie generuje żadnego zdarzenia domenowego; jest to literalny stan kodu, a nie przeoczenie, odnotowane jako odstępstwo od konwencji, w której powstanie agregatu zwykle emituje zdarzenie typu `*Created`.

Niezmienniki wymuszane w agregacie.

1. `update()` (zmiana nazwy/lokalizacji) jest zablokowane, jeśli zasób jest nieaktywny.
2. `markAsUnavailable()` / `markAsAvailable()` wymagają, by zasób był aktywny — nie da się zmieniać dostępności zasobu wycofanego z użytku.
3. `deactivate()` jest niedozwolone, jeśli zasób jest już nieaktywny — gwarancja, że zdarzenie `ResourceDeactivated` nie powstanie dwukrotnie dla tej samej instancji.
4. Obiekt wartości `Location` wymusza niepuste pola budynku i numeru pomieszczenia.
5. Nazwa zasobu nie może być pusta/`blank`.

Polityki korygujące. Zakaz wycofania zasobu z aktywnymi rezerwacjami jest regułą **międzykontekstową**, a nie niezmiennikiem agregatu. `DeactivateOfficeResourceDomainService` przyjmuje listę statusów rezerwacji pochodzącą z kontekstu `allocation` (poprzez port wyjściowy `ResourceAllocationRequestPort`) i blokuje `deactivate()`, jeśli istnieje choć jedna rezerwacja w statusie aktywnym, zatwierdzonym, złożonym lub przeterminowanym. Ponieważ dane o rezerwacjach żyją w innym agregacie/kontekście, reguła ta nie może być częścią lokalnego niezmiennika `OfficeResource` — jest odpytywana synchronicznie w ramach tej samej transakcji modularnego monolitu (wywołanie in-process), co w architekturze rozproszonej wymagałoby już prawdziwej polityki korygującej działającej asynchronicznie, z możliwością tymczasowej niespójności. Nie istnieje natomiast żadna polityka korygująca dla błędnego/przeoczonego `markAsUnavailable()` — jeśli administrator zapomni przywrócić zasób do dostępności po zakończeniu np. konserwacji, nic w systemie nie skoryguje tego automatycznie (brak harmonogramu odpowiadającego `OverdueAllocationScheduler` z modułu `allocation`).

Komenda	**Metoda domenowa**	**Wymagany stan startowy**
`addResource(...)`	`create(...)`	— (nowa instancja)
`updateResource(...)`	`update(name, location)`	aktywny (Dostępny/Niedostępny)
`markUnavailable(resourceId)`	`markAsUnavailable()`	Dostępny
`markAvailable(resourceId)`	`markAsAvailable()`	Niedostępny
`deactivateResource(resourceId)`	`deactivate()`	aktywny, brak otwartych rezerwacji

Table 3.6: Komendy agregatu ZasóbBiurowy

Obsługiwane komendy. Punkt `deactivateResource` wystawiony jest w REST API jako `DELETE /api/resources/{id}` — semantycznie soft-delete, nie usunięcie wiersza.

Metryka	**Średnio (avg)**	**Maksymalnie (max)**
Tempo obsługi komend na instancję	$<$ 1 komenda / dzień dla typowego zasobu	kilka–kilkanaście komend / dzień dla zasobów „gorących” (np. popularna sala konferencyjna w okresie reorganizacji)
Liczba klientów wydających komendy na instancję	1 (zwykle jeden administrator/IT_ADMIN)	2–3 (kilku administratorów różnych pięter/budynków zarządzających równolegle tym samym zasobem)

Table 3.7: Przepustowość agregatu ZasóbBiurowy

Przepustowość. Dla większości zasobów szansa konfliktu współbieżności jest niska; wzrasta dla małej liczby często reorganizowanych zasobów współdzielonych (sale konferencyjne, pojazdy służbowe) zarządzanych przez wielu administratorów — w takim wypadku warto rozważyć optimistic locking (wersjonowanie wiersza) przy zapisie.

Rozmiar. Liczba zdarzeń na instancję jest nieograniczona w teorii (cykle Dostępny $\leftrightarrow$ Niedostępny mogą się powtarzać wielokrotnie przez cały, wieloletni okres eksploatacji zasobu), zakończona pojedynczym, niepowtarzalnym `ResourceDeactivated`. Tempo narastania zdarzeń jest niskie do umiarkowanego — rzędu kilku–kilkudziesięciu zdarzeń rocznie dla typowego zasobu. Czas życia instancji jest długi (lata, tożsamy z fizycznym okresem eksploatacji), do momentu `deactivate()`. Mimo długiego czasu życia rozmiar pojedynczej instancji pozostaje mały — agregat przechowuje tylko aktualny stan, nie listę historycznych zdarzeń (brak event sourcingu), więc narastanie liczby zdarzeń nie wpływa na rozmiar samego agregatu w pamięci/bazie.

3.5.6. Kontekst `administration` — dziedzina pomocnicza

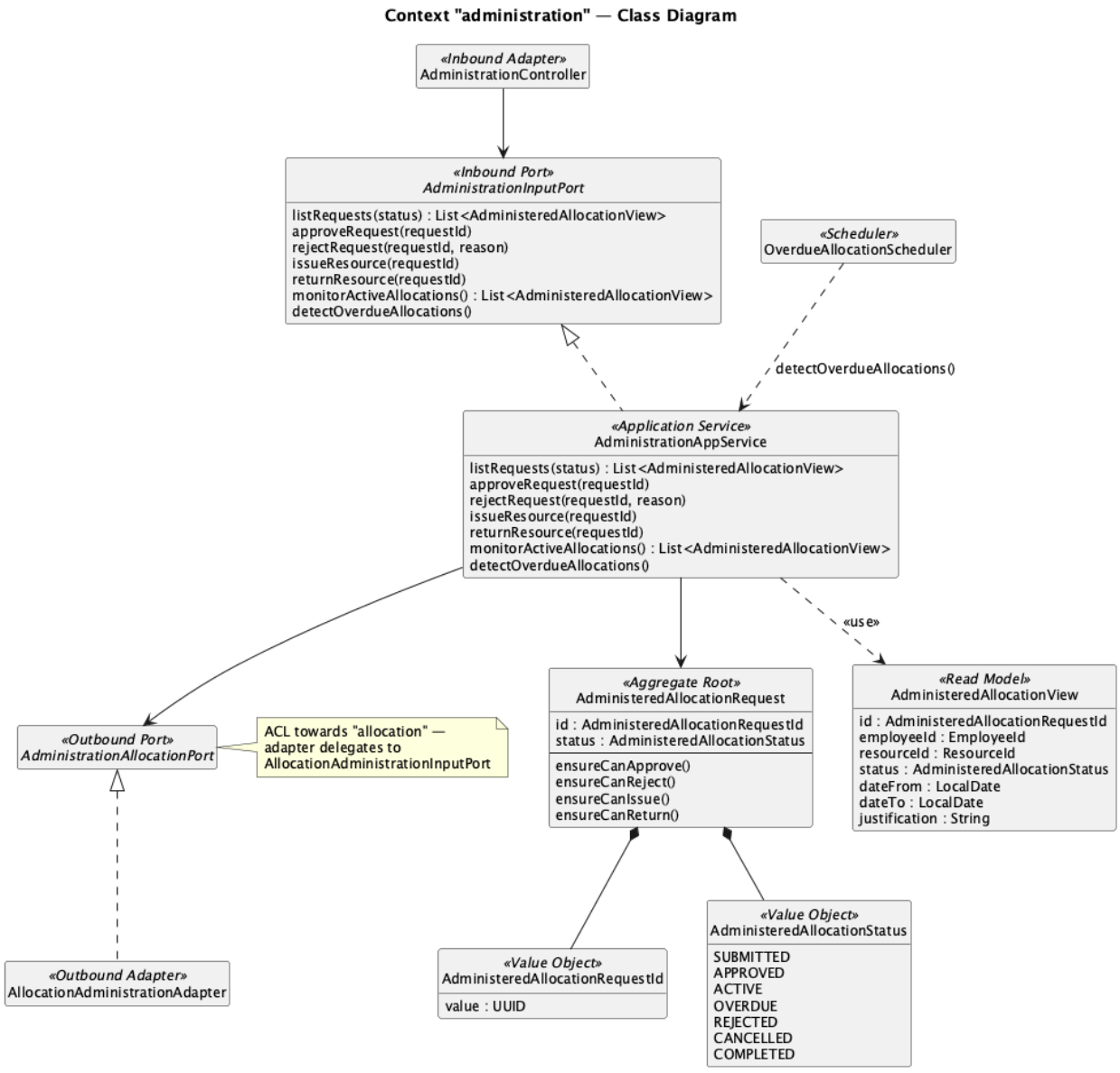


Figure 3.12: Diagram klas — kontekst `administration`

Kontekst `administration` realizuje dziedzinę pomocniczą *Administracja wnioskami*. W odróżnieniu od kontekstów `allocation` i `resources`, nie modeluje on własnego pełnego agregatu biznesowego — stan wniosku i reguły jego zmiany pozostają wyłącznie w kontekście `allocation`, który jest właścicielem tych danych. Kontekst `administration` pełni rolę klienta tego kontekstu, weryfikując status wniosku przed przekazaniem operacji dalej.

Port wyjściowy. `AdministrationAllocationPort`, zaimplementowany przez adapter `AllocationAdministrationAdapter`, przekazuje operacje administracyjne do portu wejściowego

`AllocationAdministrationInputPort` kontekstu `allocation`.

Usługa aplikacyjna i harmonogram. `AdministrationAppService` implementuje port wejściowy `AdministrationInputPort`, obsługujący przypadki użycia PU-05 — PU-09 oraz PU-11. Komponent `OverdueAllocationScheduler` «harmonogram» jest zadaniem cyklicznym Springa (`@Scheduled`), które okresowo wywołuje `detectOverdueAllocations()`, realizując PU-10.

3.5.7. Konteksty generyczne: `identity`, `notifications`, `employees`

Pozostałe trzy konteksty są dziedzinami generycznymi — ich rola w architekturze jest infrastrukturalna względem dziedzin głównej i pomocniczych:

- `identity` — `IdentityAccessAppService` odpowiada za logowanie i wydawanie tokenów JWT; korzysta z danych `Employee` przez własny port antykorupcyjny (uwierzytelnienie na podstawie poświadczeń, autoryzacja na podstawie roli),
- `notifications` — `NotificationAppService` nasłuchuje zdarzeń dziedziny publikowanych przez kontekst `allocation` (`NotificationEventListener`) i wysyła powiadomienia e-mail przez port wyjściowy `NotificationPort` (adapter `SmtpNotificationAdapter`);
- `employees` — agregat `Employee` «korzeń agregatu» (pojęciowo: **Pracownik**) przechowuje dane profilowe pracownika, rolę (`Role`: Pracownik / Kierownik / Informatyk) oraz indywidualne limity udostępnień w podziale na typ zasobu; udostępniany innym kontekstom wyłącznie do odczytu, przez ich własne porty antykorupcyjne (np. `EmployeeProfilePort` w kontekście `allocation`).

3.6. Przypadki użycia — realizacja

Poniższy podrozdział przedstawia szczegółową realizację taktyczną wybranych przypadków użycia z katalogu PU-01 — PU-14. Dla każdego z nich podano pełny opis (zgodny z katalogiem przygotowanym na etapie modelowania strategicznego) oraz diagram sekwencji ilustrujący jego realizację przez elementy modelu taktycznego — agregaty, usługi dziedziny, porty i adaptery. Pominięto przypadki użycia o charakterze czysto odczytowym (PU-01, PU-04, PU-11), które nie wprowadzają nowej logiki biznesowej do zilustrowania na diagramie sekwencji.

3.6.1. PU-02: Składanie wniosku o udostępnienie zasobu biurowego

Identyfikator	PU-02
Nazwa	Składanie wniosku o udostępnienie zasobu biurowego
Domena	Udostępnianie zasobów biurowych
Aktor główny	Pracownik
Cel aktora	Pracownik chce zarezerwować zasób biurowy na wybrany termin, aby móc z niego skorzystać w pracy.
Warunki wstępne	Pracownik jest zalogowany, zasób jest dostępny w wybranym terminie.
Warunki końcowe	Wniosek o udostępnienie zasobu biurowego złożony.
Scenariusz główny	1. Pracownik wybiera zasób biurowy z katalogu. 2. Pracownik wybiera termin udostępnienia. 3. Limit wypożyczeń pracownika jest weryfikowany. 4. Dostępność zasobu biurowego jest sprawdzana. 5. Pracownik składa wniosek o udostępnienie. 6. Pracownik jest powiadamiany o złożeniu wniosku.
Reguły biznesowe	1. O udostępnienie zasobu można wnioskować tylko wtedy, gdy jest aktywny i dostępny w danym terminie. 2. Zasób jest niedostępny, jeśli istnieje zatwierdzony wniosek w nakładającym się terminie. 3. Pracownik nie może złożyć wniosku, jeśli osiągnął limit aktywnych udostępnień zasobu.
Scenariusze alternatywne	**A:** Pracownik przekroczył limit udostępnień — system uniemożliwia złożenie wniosku. **B:** Zasób niedostępny w terminie — system uniemożliwia złożenie wniosku.

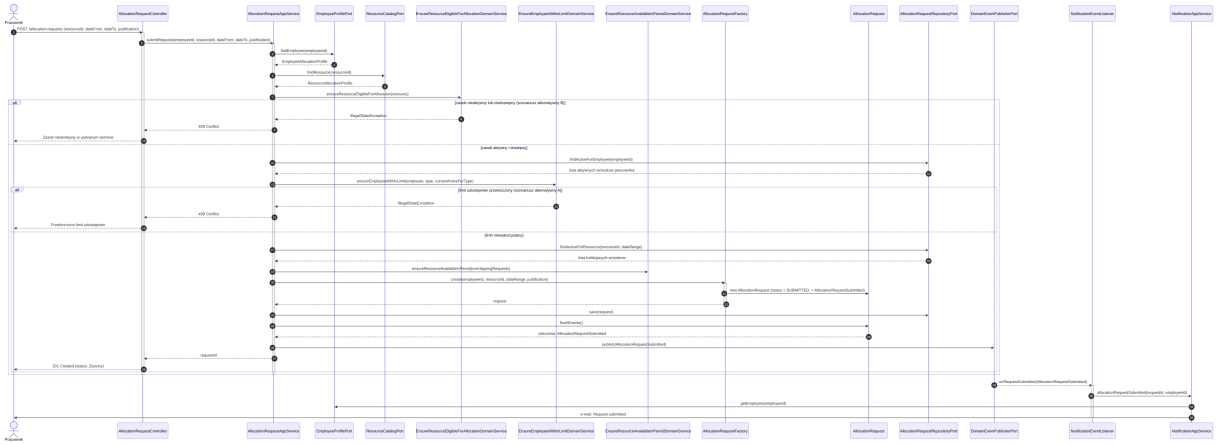


Figure 3.13: Diagram sekwencji — PU-02: Składanie wniosku o udostępnienie zasobu biurowego

3.6.2. PU-03: Anulowanie wniosku o udostępnienie zasobu biurowego

Identyfikator	PU-03
Nazwa	Anulowanie wniosku o udostępnienie zasobu biurowego
Domena	Udostępnianie zasobów biurowych
Aktor główny	Pracownik
Cel aktora	Pracownik chce wycofać swój wniosek o udostępnienie, ponieważ jego plany uległy zmianie.
Warunki wstępne	Pracownik jest zalogowany, jego wniosek oczekuje na decyzję lub został zaakceptowany.
Warunki końcowe	Wniosek anulowany, zasób biurowy ponownie dostępny.
Scenariusz główny	1. Pracownik przegląda swoje wnioski o udostępnienie. 2. Pracownik wybiera własny wniosek, który chce anulować. 3. Pracownik anuluje wybrany wniosek. 4. System nadaje wnioskowi status *Anulowany*. 5. System zwalnia zasób biurowy w terminie objętym wnioskiem.
Reguły biznesowe	1. Pracownik może anulować wyłącznie własny wniosek. 2. Anulować można tylko wniosek nieprzeanalizowany lub zaakceptowany przez Kierownika zasobów.
Scenariusze alternatywne	Pracownik próbuje anulować wniosek, który został już wydany lub zakończony — anulowanie nie jest możliwe.

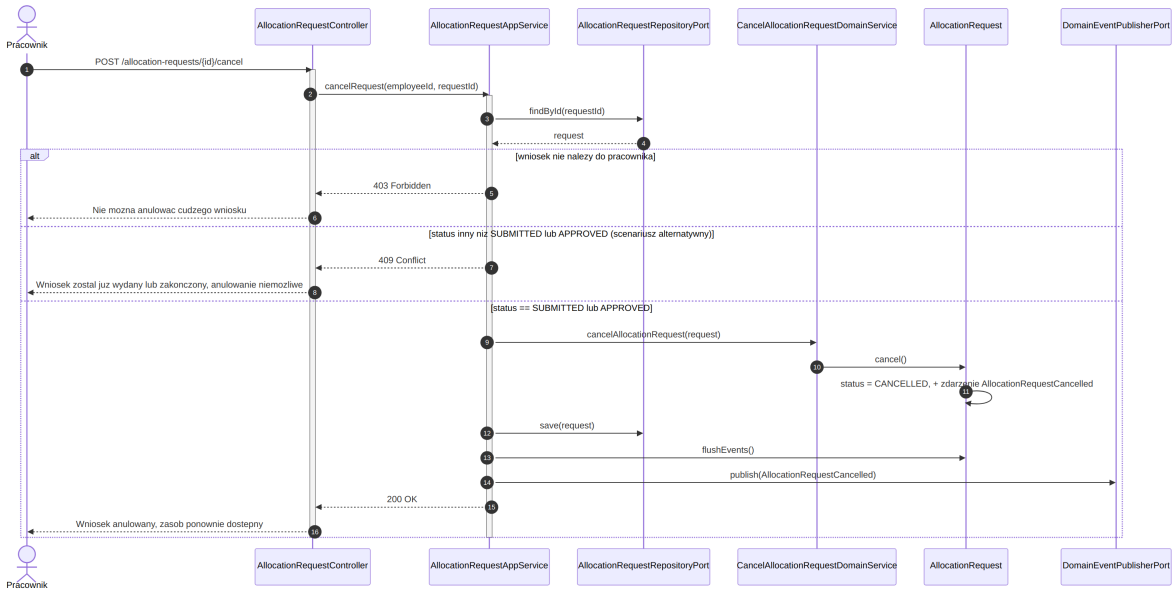


Figure 3.14: Diagram sekwencji — PU-03: Anulowanie wniosku o udostępnienie zasobu biurowego

3.6.3. PU-05: Analiza wniosku o udostępnienie

Identyfikator	PU-05
Nazwa	Analiza wniosku o udostępnienie
Domena	Administracja wnioskami
Aktor główny	Kierownik udostępnień
Cel aktora	Kierownik udostępnień chce ocenić, czy wniosek pracownika może zostać zrealizowany.
Warunki wstępne	Kierownik udostępnień jest zalogowany, wniosek ma status *Złożony*.
Warunki końcowe	Wniosek przeanalizowany — rozszerzenie o PU-06 lub PU-07.
Scenariusz główny	1. Kierownik udostępnień otrzymuje powiadomienie o nowym wniosku. 2. Kierownik udostępnień przegląda listę wniosków o udostępnienie. 3. Kierownik udostępnień analizuje wniosek — sprawdza zasób, termin, pracownika. 4. Kierownik udostępnień zatwierdza lub odrzuca wniosek.
Scenariusze alternatywne	Brak.

Realizacja PU-05 jest przedstawiona na wspólnym diagramie sekwencji wraz z PU-06 i PU-07 (rys. 3.15), ponieważ trzy przypadki użycia tworzą jeden ciągły proces decyzyjny Kierownika udostępnień.

3.6.4. PU-06: Zatwierdzenie wniosku o udostępnienie

Identyfikator	PU-06
Nazwa	Zatwierdzenie wniosku o udostępnienie
Domena	Administracja wnioskami
Aktor główny	Kierownik udostępnień
Warunki wstępne	Wniosek jest złożony.
Warunki końcowe	Wniosek zatwierdzony, dostępność zasobu zaktualizowana.
Scenariusz główny	1. Kierownik udostępnień zatwierdza wniosek. 2. Dostępność zasobu biurowego jest aktualizowana. 3. Pracownik jest powiadamiany o zatwierdzeniu wniosku.
Scenariusze alternatywne	Brak.

3.6.5. PU-07: Odrzucenie wniosku o udostępnienie

Identyfikator	PU-07
Nazwa	Odrzucenie wniosku o udostępnienie
Domena	Administracja wnioskami
Aktor główny	Kierownik udostępnień
Warunki wstępne	Wniosek jest złożony.
Warunki końcowe	Wniosek odrzucony, dostępność zasobu biurowego nie zaktualizowana.
Scenariusz główny	1. Kierownik udostępnień odrzuca wniosek. 2. Pracownik jest powiadamiany o odrzuceniu wniosku.
Scenariusze alternatywne	Brak.

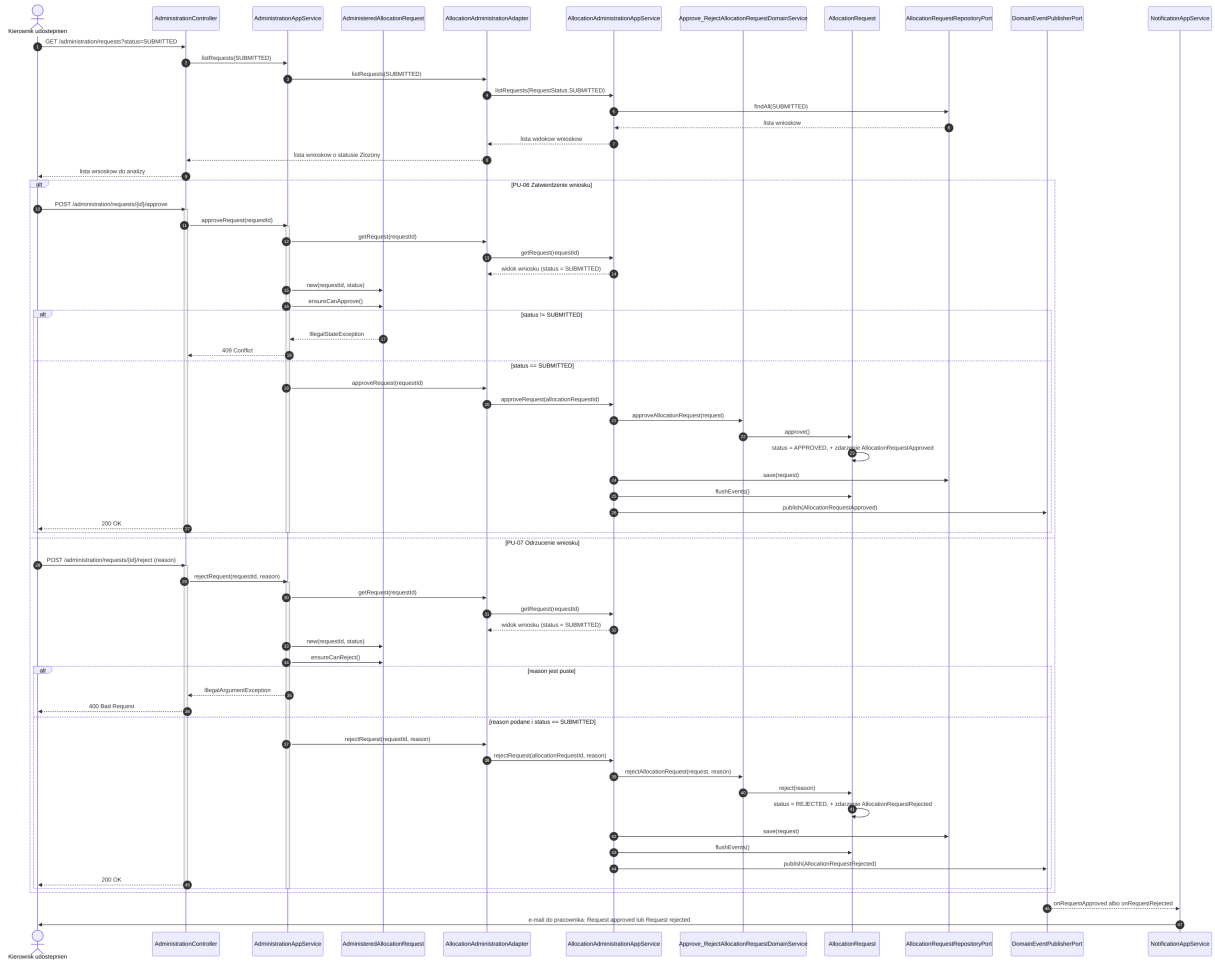


Figure 3.15: Diagram sekwencji — PU-05/PU-06/PU-07: Analiza, zatwierdzenie i odrzucenie wniosku

3.6.6. PU-08: Wydanie zasobu biurowego

Identyfikator	PU-08
Nazwa	Wydanie zasobu biurowego
Domena	Administracja wnioskami
Aktor główny	Kierownik udostępnień
Cel aktora	Kierownik udostępnień chce przekazać zasób pracownikowi, który ma zatwierdzony wniosek.
Warunki wstępne	Wniosek jest zatwierdzony, pracownik zgłosił się po odbiór.
Warunki końcowe	Zasób biurowy wydany, udostępnienie aktywne.
Scenariusz główny	1. Kierownik udostępnień przegląda zatwierdzone wnioski. 2. Kierownik udostępnień wydaje zasób biurowy pracownikowi. 3. Data i godzina wydania zasobu biurowego są rejestrowane. 4. Pracownik jest powiadamiany o wydaniu zasobu biurowego.
Scenariusze alternatywne	Brak.

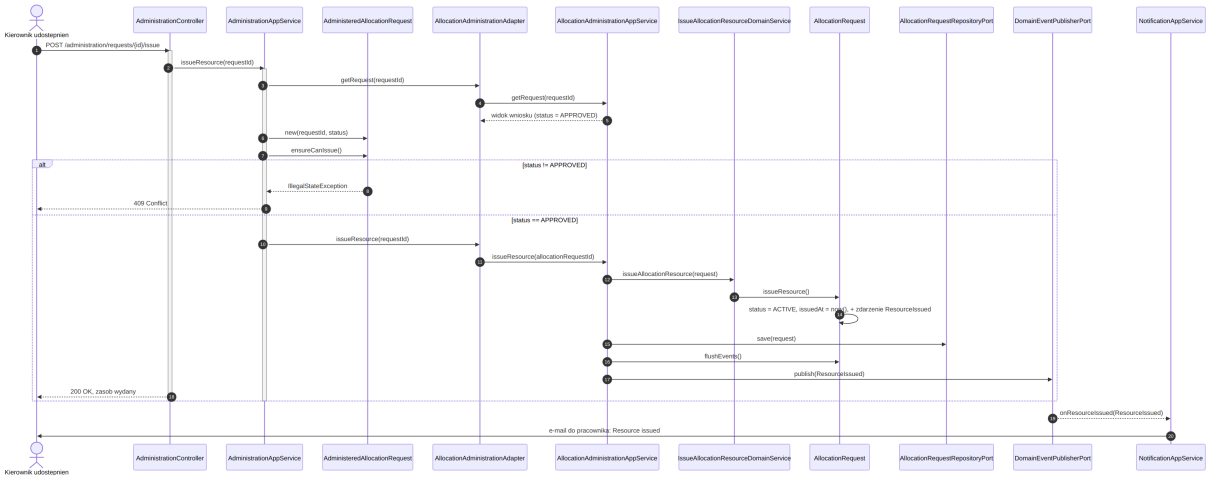


Figure 3.16: Diagram sekwencji — PU-08: Wydanie zasobu biurowego

3.6.7. PU-09: Zwrot zasobu biurowego

Identyfikator	PU-09
Nazwa	Zwrot zasobu biurowego
Domena	Administracja wnioskami
Aktor główny	Kierownik udostępnień
Warunki wstępne	Udostępnienie ma status *Aktywne*.
Warunki końcowe	Zasób biurowy zwrócony, udostępnienie zakończone.
Scenariusz główny	1. Pracownik zwraca zasób biurowy. 2. Kierownik udostępnień odbiera zasób biurowy. 3. Pracownik jest powiadamiany o zakończeniu udostępnienia.
Scenariusze alternatywne	Termin zwrotu przekroczony — udostępnienie znajduje się w statusie *Przeterminowany* (zob. PU-10); zwrot zasobu jest dopuszczalny również z tego statusu.

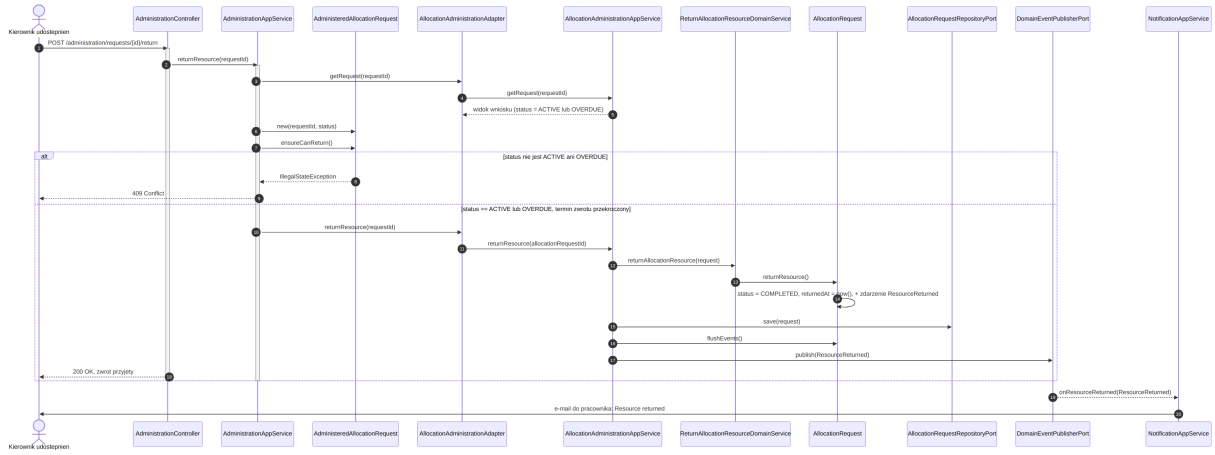


Figure 3.17: Diagram sekwencji — PU-09: Zwrot zasobu biurowego

3.6.8. PU-10: Obsługa przeterminowanego udostępnienia

Identyfikator	PU-10
Nazwa	Obsługa przeterminowanego udostępnienia
Domena	Udostępnianie zasobów biurowych
Aktor główny	System (zdarzenie czasowe)
Warunki wstępne	Udostępnienie ma status *Aktywne*, termin zwrotu minął.
Warunki końcowe	Pracownik i Kierownik udostępnień poinformowani o przekroczeniu terminu.
Scenariusz główny	1. System wykrywa przekroczenie terminu zwrotu zasobu biurowego. 2. System stwierdza przeterminowanie udostępnienia. 3. Pracownik oraz Kierownik udostępnień są powiadamiani o przekroczeniu terminu zwrotu zasobu.
Scenariusze alternatywne	Brak.

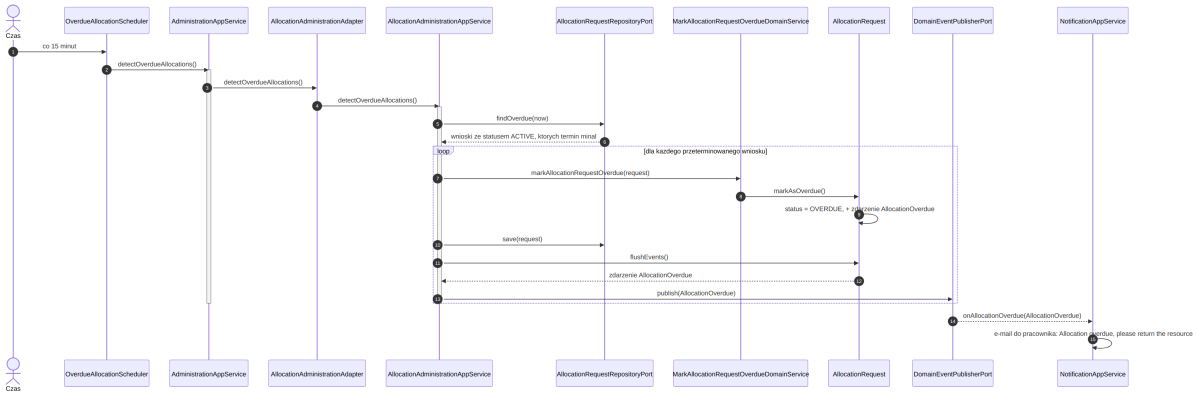


Figure 3.18: Diagram sekwencji — PU-10: Obsługa przeterminowanego udostępnienia

3.6.9. PU-12: Zarządzanie dostępnością zasobów biurowych

Identyfikator	PU-12
Nazwa	Zarządzanie dostępnością zasobów biurowych
Domena	Zarządzanie zasobami biurowymi
Aktor główny	Kierownik zasobów
Cel aktora	Kierownik zasobów chce utrzymać aktualną informację o dostępności zasobów biurowych.
Warunki wstępne	Kierownik zasobów jest zalogowany.
Warunki końcowe	Dostępność zasobów biurowych zaktualizowana.
Scenariusz główny	1. Kierownik zasobów przegląda obłożenie zasobów biurowych. 2. Kierownik zasobów identyfikuje przeciążone strefy lub niedostępne zasoby. 3. Kierownik zasobów oznacza zasób biurowy jako niedostępny.
Scenariusze alternatywne	Zasób ma aktywne udostępnienia.

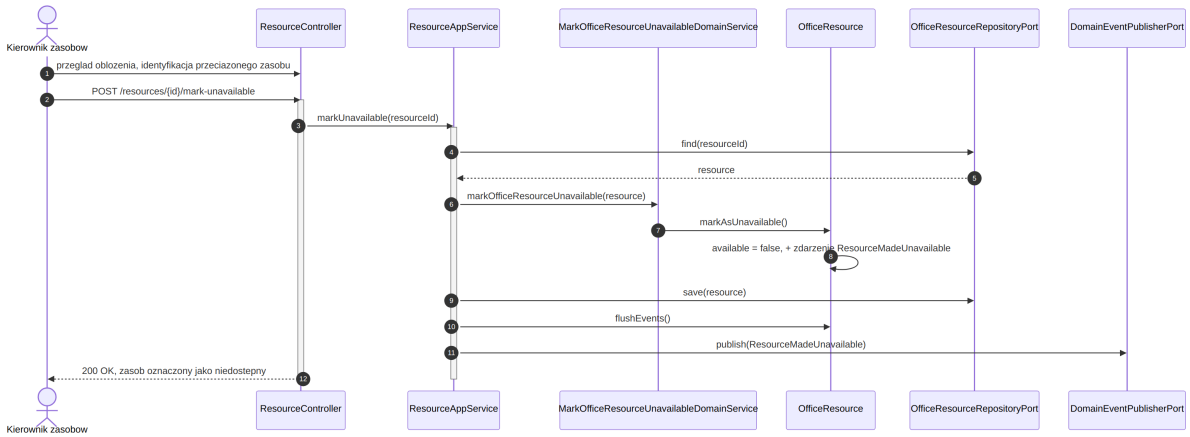


Figure 3.19: Diagram sekwencji — PU-12: Zarządzanie dostępnością zasobów biurowych

3.6.10. PU-13: Zarządzanie katalogiem zasobów biurowych (wariant: dezaktywacja)

Identyfikator	PU-13
Nazwa	Zarządzanie katalogiem zasobów biurowych
Domena	Zarządzanie zasobami biurowymi
Aktor główny	Kierownik zasobów
Cel aktora	Kierownik zasobów chce utrzymać aktualny katalog zasobów biurowych dostępnych dla pracowników.
Warunki wstępne	Kierownik zasobów jest zalogowany.
Warunki końcowe	Katalog zasobów biurowych zaktualizowany.
Scenariusz główny (dezaktywacja)	1. Kierownik zasobów wybiera zasób biurowy do dezaktywacji. 2. Możliwość dezaktywacji jest weryfikowana — jeśli zasób ma aktywne udostępnienia, system uniemożliwia dezaktywację. 3. Zasób biurowy zostaje dezaktywowany. 4. Zasób zostaje usunięty z katalogu zasobów biurowych.
Reguły biznesowe	Dezaktywacja jest niemożliwa, jeśli istnieją aktywne lub zaakceptowane w przyszłości wnioski o udostępnienie zasobu.
Scenariusze alternatywne	Zasób ma aktywne udostępnienia — system blokuje dezaktywację.

Z trzech wariantów PU-13 (dodanie, modyfikacja, dezaktywacja zasobu) na diagramie sekwencji przedstawiono wariant dezaktywacji, ponieważ najlepiej ilustruje współpracę międzykontekstową w przeciwnym kierunku niż poprzednie przypadki użycia: tym razem kontekst

pomocniczy `resources` odpytuje kontekst główny `allocation`, a nie odwrotnie.

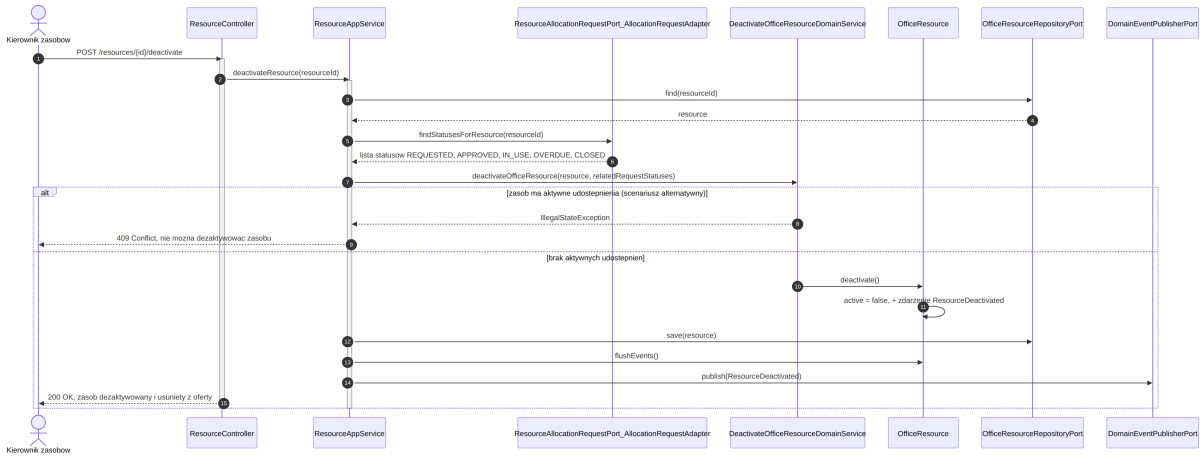


Figure 3.20: Diagram sekwencji — PU-13 (wariant: dezaktywacja zasobu biurowego)

3.6.11. PU-14: Konfiguracja limitu udostępnień

Identyfikator	PU-14
Nazwa	Konfiguracja limitu udostępnień
Domena	Zarządzanie systemem
Aktor główny	Informatyk
Cel aktora	Informatyk chce określić maksymalną liczbę zasobów, o które pracownik może jednocześnie wnioskować, aby kontrolować wykorzystanie zasobów biurowych.
Warunki wstępne	Informatyk jest zalogowany.
Warunki końcowe	Limit udostępnień zaktualizowany.
Scenariusz główny	1. Informatyk przechodzi do ustawień limitów udostępnień. 2. Informatyk wprowadza nowy limit wypożyczeń — indywidualny dla wybranego pracownika lub dla wszystkich.
Scenariusze alternatywne	Informatyk wprowadza nieprawidłową wartość (np. ujemną) — system odrzuca zmianę i informuje o błędzie.

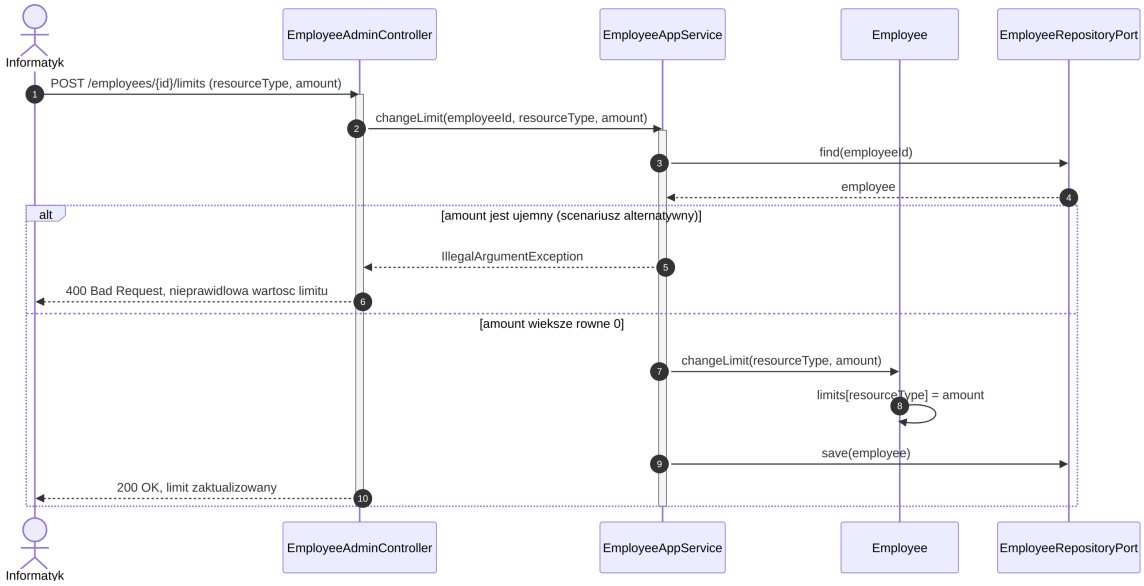


Figure 3.21: Diagram sekwencji — PU-14: Konfiguracja limitu udostępnień

4. Testowanie projektowanego systemu

4.1. Cel etapu

Celem etapu testowania było potwierdzenie, że system *DeskFlow* realizuje wymagania dziedzinowe dotyczące zarządzania zasobami biurowymi oraz że implementacja zachowuje założoną architekturę heksagonalną.

Zakres testowania obejmuje:

- testy jednostkowe modelu domenowego, obiektów wartości i usług dziedzinowych,
- testy jednostkowe usług aplikacyjnych oraz mapperów,
- testy integracyjne adapterów wyjściowych, w szczególności repozytoriów JPA,
- testy integracyjne adapterów wejściowych REST,
- testy systemowe obejmujące pełne ścieżki użycia,
- testy architektury weryfikujące przestrzeganie zasad czystej architektury.

4.2. Zastosowane narzędzia

Backend systemu został zaimplementowany w języku Java 21 z użyciem Spring Boot 4.1.0. Do testowania wykorzystano standardowy zestaw narzędzi ekosystemu Spring oraz dodatkowe biblioteki wspierające testy architektury i integracji:

- JUnit 5 jako podstawowy framework testowy,
- Spring Boot Test oraz moduły `spring-boot-starter-webmvc-test` i `spring-boot-starter-data-jpa-test` do uruchamiania kontekstu aplikacji oraz testów warstwowych,
- MockMvc do testowania endpointów REST bez konieczności uruchamiania zewnętrznego serwera HTTP,
- AssertJ do czytelnych asercji,
- Mockito do podstawiania portów w testach usług aplikacyjnych oraz w testach `@WebMvcTest`, gdzie porty wejściowe są zastępowane przez `@MockitoBean`,
- Spring Security Test do sprawdzania autoryzacji i kontroli dostępu
- Testcontainers z kontenerem PostgreSQL uruchomionym w Dockerze do testów wymagających prawdziwej bazy danych,

- ArchUnit do automatycznej kontroli zależności architektonicznych.

Konfiguracja testowa znajduje się w module backendu. Zależności testowe są zdefiniowane w pliku `deskflow-backend/pom.xml`, a osobna konfiguracja środowiska testowego w pliku `deskflow-backend/src/test/resources/application-test.properties`.

4.3. Organizacja testów w projekcie

Testy backendu znajdują się w katalogu `deskflow-backend/src/test/java/com/deskflow`. Ich struktura odzwierciedla strukturę kodu produkcyjnego i podział na konteksty oraz warstwy:

- `allocation` - testy obsługi wniosków o udostępnienie zasobu,
- `resources` - testy katalogu i cyklu życia zasobów biurowych,
- `administration` - testy czynności administracyjnych, takich jak zatwierdzanie, odrzucanie, wydawanie i oznaczanie przeterminowania,
- `employees` - testy pracowników i limitów alokacji,
- `identity` - testy uwierzytelniania, tokenów JWT i danych użytkownika,
- `notifications` - testy powiadomień i reakcji na zdarzenia domenowe,
- `common` - testy wspólnych obiektów wartości, obsługi błędów, walidacji, REST API i testy systemowe,
- `architecture` - testy architektury systemu.

Taki podział ułatwia powiązanie testów z modelem DDD. Testy dla domeny weryfikują reguły biznesowe bez infrastruktury, testy aplikacyjne sprawdzają orkiestrację przypadków użycia, a testy infrastruktury potwierdzają poprawne podłączenie adapterów do technologii zewnętrznych.

4.4. Testy jednostkowe modelu domenowego

Najważniejsza grupa testów jednostkowych dotyczy modelu domenowego. Celem tych testów było potwierdzenie, że agregaty, obiekty wartości i usługi dziedzinowe samodzielnie egzekwują reguły biznesowe, niezależnie od bazy danych, kontrolerów REST i frameworka Spring.

Przykłady testowanych elementów:

- `AllocationRequestTest` - sprawdza cykl życia wniosku o udostępnienie zasobu,
- `DateRangeTest` - sprawdza poprawność zakresu dat i wykrywanie nakładających się okresów,

- `OfficeResourceTest` - sprawdza zachowanie zasobu biurowego przy aktualizacji, dezaktywacji oraz zmianie dostępności,
- `EmployeeTest` - sprawdza model pracownika i przypisane limity,
- `ResourceIdTest`, `EmployeeIdTest`, `ResourceTypeTest` - sprawdzają poprawne tworzenie i walidację obiektów wartości.

W modelu wniosku `AllocationRequest` testowane są między innymi przejścia stanów:

- `SUBMITTED` po złożeniu wniosku,
- `APPROVED` po zatwierdzeniu,
- `REJECTED` po odrzuceniu,
- `CANCELLED` po anulowaniu,
- `ACTIVE` po wydaniu zasobu,
- `COMPLETED` po zwrocie zasobu,
- `OVERDUE` po oznaczeniu przeterminowania.

Testy sprawdzają również niedozwolone przejścia stanów, na przykład brak możliwości wydania zasobu dla wniosku, który nie został zatwierdzony, albo brak możliwości zwrotu zasobu, który nie jest aktywny. Dzięki temu błędne scenariusze są blokowane w samej domenie, a nie dopiero na poziomie interfejsu użytkownika.

4.5. Testy usług dziedzinowych

Oddzielnie przetestowano usługi dziedzinowe, które reprezentują reguły niedające się naturalnie przypisać do pojedynczego obiektu wartości lub encji. Dotyczy to przede wszystkim alokacji zasobów oraz kontroli limitów.

Przykłady testowanych klas:

- `EmployeeLimitDomainServiceTest` - sprawdza, czy pracownik nie przekracza limitu aktywnych zasobów danego typu,
- `ResourceAvailabilityDomainServicesTest` - sprawdza, czy zasób jest dostępny w wybranym okresie,
- `AllocationLifecycleDomainServicesTest` - sprawdza operacje cyklu życia wniosku: zatwierdzanie, odrzucanie, wydawanie, zwrot i anulowanie,
- `ResourceLifecycleDomainServicesTest` - sprawdza zmianę stanu zasobu,
- `ResourceDeactivationDomainServicesTest` - sprawdza warunki dezaktywacji zasobu.

Testy te są szybkie, ponieważ nie wymagają uruchamiania Springa ani bazy danych. Są one najbliższe idei TDD, ponieważ opisują reguły biznesowe w sposób bezpośredni i stanowią dokumentację zachowania domeny.

4.6. Testy usług aplikacyjnych

Usługi aplikacyjne odpowiadają za realizację przypadków użycia i koordynację pracy domeny, portów oraz publikacji zdarzeń. W testach sprawdzono, czy usługi aplikacyjne wywołują odpowiednie reguły domenowe, korzystają z portów wyjściowych i poprawnie reagują na brak danych.

Przykłady testowanych klas:

- `AllocationRequestAppServiceTest`,
- `AllocationRequestAppServiceMissingDataTest`,
- `AllocationAdministrationAppServiceTest`,
- `ResourceAppServiceTest`,
- `ResourceAppServiceMissingResourceTest`,
- `AdministrationAppServiceTest`,
- `EmployeeAppServiceTest`,
- `IdentityAccessAppServiceTest`,
- `NotificationAppServiceTest`.

W testach tych wykorzystywane są podstawione implementacje portów lub zależności kontrolowane przez test. Dzięki temu można sprawdzić logikę przypadku użycia bez zależności od konkretnej bazy danych, SMTP lub interfejsu REST.

4.7. Testy integracyjne adapterów

Adaptery są częścią infrastruktury, dlatego wymagają osobnego typu testów. Ich zadaniem jest sprawdzenie, czy implementacje portów poprawnie komunikują się z technologią zewnętrzną albo z innym kontekstem systemu.

Przetestowano między innymi:

- `AllocationRequestRepositoryIntegrationTest` - zapis i odczyt wniosków o zasób,
- `AllocationRequestRepositoryDateEdgeCasesTest` - przypadki brzegowe wyszukiwania wniosków po datach,
- `OfficeResourceRepositoryIntegrationTest` - zapis i odczyt zasobów biurowych,
- `EmployeeRepositoryIntegrationTest` - zapis i odczyt pracowników,
- `ResourceCatalogAdapterTest` - adapter katalogu zasobów używany przez kontekst alokacji,

- `EmployeeProfileAdapterTest` - adapter profilu pracownika używany przez kontekst alokacji,
- `AllocationAdministrationAdapterTest` - adapter łączący administrację z alokacją,
- `SmtpNotificationAdapterTest` - adapter powiadomień mailowych,
- `SpringDomainEventPublisherAdapterTest` - publikowanie zdarzeń domenowych przez Spring.

Testy repozytoriów potwierdzają poprawność mapowania między modelem domenowym a encjami relacyjnymi. Jest to istotne, ponieważ w architekturze heksagonalnej domena nie powinna być bezpośrednio związana z JPA. Mappery i adaptery muszą więc poprawnie odtwarzać agregaty z danych zapisanych w tabelach.

4.8. Testy adapterów REST i walidacji

Dla adapterów wejściowych REST przygotowano testy sprawdzające poprawność komunikacji z użytkownikiem systemu przez API HTTP. Testy te obejmują mapowanie danych wejściowych, walidację, obsługę błędów oraz podstawową kontrolę dostępu.

Przykłady testów:

Testy wycinka adaptera REST

- `RestControllerWebMvcTest` – sprawdza działanie kontrolerów REST,
- `AllocationRequestRestMapperTest` – sprawdza mapowanie DTO na dane wymagane przez przypadki użycia,
- `GlobalExceptionHandlerTest` – sprawdza ujednolicony sposób zwracania błędów.

Testy systemowe warstwy HTTP

- `InputValidationSystemTest` – sprawdza odrzucanie niepoprawnych danych wejściowych,
- `AuthorizationSystemTest` – sprawdza zabezpieczenia endpointów i role użytkowników.

Warstwa REST pełni rolę adaptera wejściowego. Testy tej warstwy nie powinny powielać całej logiki domenowej, lecz sprawdzać, czy żądania HTTP są poprawnie tłumaczone na wywołania portów wejściowych aplikacji i czy odpowiedzi są zrozumiałe dla klienta systemu.

4.9. Testy systemowe

Testy systemowe sprawdzają kompletne scenariusze działania systemu od adaptera wejściowego do adapterów wyjściowych. Najważniejszym testem tego typu jest `FullAllocationCycleSystemTest`, który przechodzi przez pełny cykl obsługi wniosku:

- pracownik składa wniosek o udostępnienie zasobu,

- kierownik zatwierdza wniosek,
- kierownik wydaje zasób,
- kierownik przyjmuje zwrot zasobu,
- system aktualizuje status wniosku po każdym kroku.

Ten test odpowiada głównemu przepływowi biznesowemu systemu i łączy kilka warstw: REST API, uwierzytelnienie JWT, usługi aplikacyjne, model domenowy, repozytoria i bazę danych. Dzięki temu można wykryć błędy, które nie byłyby widoczne w samych testach jednostkowych, na przykład niepoprawne mapowanie DTO, błędną konfigurację zabezpieczeń albo nieprawidłowe utrwalanie statusu.

4.10. Testy zdarzeń domenowych

System wykorzystuje podejście EDA w obrębie aplikacji. Agregaty publikują zdarzenia, a ich obsługa i publikacja są weryfikowane na różnych poziomach: od testów jednostkowych modelu, przez usługi, aż po słuchaczy.

Testy obejmują między innymi:

- generowanie zdarzeń przez agregat `AllocationRequest`, np. `AllocationRequestSubmitted`, `AllocationRequestApproved`, `ResourceIssued`, `ResourceReturned`, `AllocationOverdue`,
- generowanie zdarzeń przez agregat `OfficeResource`, np.`ResourceUpdated`, `ResourceMadeUnavailable`, `ResourceMadeAvailable`, `ResourceDeactivated`,
- obsługę zdarzeń przez `ResourceAvailabilityEventListener`,
- obsługę zdarzeń przez `NotificationEventListener`,
- publikację zdarzeń przez `SpringDomainEventPublisherAdapter`.

Dzięki temu przetestowano nie tylko wynik bezpośredniego wywołania metody, ale również skutki uboczne istotne z punktu widzenia integracji wewnątrz systemu.

4.11. Testy architektury

Zgodność z czystą architekturą i architekturą heksagonalną jest weryfikowana automatycznie przez testy ArchUnit. W projekcie znajdują się między innymi testy `CleanArchitectureTest` i `DomainIsolationTest`.

Testy architektury sprawdzają, czy:

- warstwa domeny nie zależy od infrastruktury,
- warstwa domeny nie zależy od adapterów,
- model domenowy nie zależy od usług aplikacyjnych,

- model domenowy nie zależy od portów aplikacyjnych,
- warstwa aplikacji nie zależy od infrastruktury,
- warstwa aplikacji nie zależy od adapterów,
- konteksty domenowe nie tworzą cyklicznych zależności.

Klasa `DomainIsolationTest` uzupełnia powyższe reguły o weryfikację izolacji technologicznej domeny: brak adnotacji JPA, brak importów Spring i brak zależności od `jakarta.persistence` w pakietach `..domain...`.

Takie testy są ważne, ponieważ reguły architektoniczne łatwo naruszyć w trakcie rozwoju systemu. ArchUnit pozwala traktować architekturę jako element testów regresyjnych, a nie tylko jako opis w dokumentacji.

4.12. Uruchamianie testów

Testy backendu można uruchomić z katalogu `deskflow-backend` poleceniem:

```
mvn test
```

W przypadku testów integracyjnych z Testcontainers wymagane jest działające środowisko Docker. Testy uruchamiane są w profilu testowym i korzystają z konfiguracji przeznaczonej do testów, dzięki czemu nie modyfikują danych środowiska deweloperskiego ani produkcyjnego. Brak dostępnego środowiska Docker uniemożliwia uruchomienie testów korzystających z kontenerów PostgreSQL, natomiast testy jednostkowe i testy warstw mogą być wykonywane niezależnie od kontenerów.

4.13. Wnioski z etapu testowania

Etap testowania potwierdził, że system *DeskFlow* został zaprojektowany w sposób umożliwiający testowanie na kilku poziomach. Model domenowy jest odseparowany od infrastruktury, dlatego można go testować szybko i niezależnie. Adaptery są testowane osobno, co pozwala wykrywać błędy integracyjne bez mieszania ich z logiką biznesową. Testy systemowe potwierdzają, że główne scenariusze biznesowe działają po połączeniu wszystkich warstw.

Najważniejszą korzyścią z takiego podejścia jest ograniczenie ryzyka regresji. Zmiany w implementacji można weryfikować automatycznie, a testy architektury dodatkowo chronią projekt przed przypadkowym naruszeniem zasad DDD, SOLID i architektury heksagonalnej.

5. Implementowanie projektowanego systemu

5.1. Cel etapu

Celem etapu implementacji było wykonanie działającego systemu informatycznego *DeskFlow* na podstawie przygotowanego modelu dziedziny, przypadków użycia i założeń architektonicznych. System wspiera zarządzanie zasobami biurowymi, w szczególności składanie wniosków o udostępnienie zasobu, zatwierdzanie lub odrzucanie wniosków, wydawanie zasobów, przyjmowanie zwrotów, obsługę przeterminowań oraz zarządzanie katalogiem zasobów.

Implementacja została wykonana zgodnie z podejściem Domain Driven Design w czystej architekturze hexagonalnej. Najważniejsze reguły biznesowe umieszczono w modelu domenowym, a zależności techniczne odseparowano przez porty i adaptery. Backend, czyli rdzeń systemu, został zaimplementowany w języku Java.

5.2. Ogólna charakterystyka rozwiązania

DeskFlow jest aplikacją webową składającą się z:

- backendu Java/Spring Boot udostępniającego REST API,
- frontendowego klienta webowego Next.js,
- relacyjnej bazy danych PostgreSQL,
- modułu powiadomień wykorzystującego SMTP,
- konfiguracji Docker Compose dla środowiska deweloperskiego i produkcyjnego.

Backend został zorganizowany jako modułowy monolit w stylu architektury heksagonalnej. Z punktu widzenia wymagań projektu jest to rozwiązanie typu SOA na poziomie usług aplikacyjnych i portów, bez rozbijania systemu na osobne procesy mikroserwisowe. Taki wybór ogranicza złożoność wdrożenia, a jednocześnie zachowuje wyraźne granice kontekstów i warstw.

5.3. Technologie użyte w implementacji

W backendzie zastosowano:

- Java 21,

- Spring Boot 4.1.0,
- Spring Web oparty na Spring Framework 7 do implementacji REST API,
- Spring Data JPA do implementacji adapterów repozytoriów,
- Spring Security 7 do uwierzytelniania i autoryzacji,
- JWT do bezstanowej autoryzacji zapytań HTTP,
- Flyway do wersjonowania schematu bazy danych,
- PostgreSQL jako relacyjna baza danych,
- Spring Mail do wysyłania powiadomień,

We frontendzie zastosowano:

- Next.js 16.2.9,
- React 19,
- TypeScript,
- komponenty UI oparte między innymi o Radix UI,
- formularze z React Hook Form i Zod,
- komunikację z backendem przez REST API.

Do uruchamiania systemu przygotowano konfigurację Docker i Docker Compose z rozdzieleniem środowiska deweloperskiego i produkcyjnego. W środowisku deweloperskim uruchamiane są kontenery backendu, frontendu, PostgreSQL oraz MailHog. Wariant produkcyjny jest przeznaczony do uruchamiania na platformie Coolify i obejmuje backend, frontend oraz bazę danych z konfiguracją przekazywaną przez zmienne środowiskowe.

5.4. Struktura backendu

Kod backendu znajduje się w katalogu `deskflow-backend/src/main/java/com/deskflow`. Struktura pakietów została podzielona według kontekstów domenowych i warstw architektonicznych.

Główne obszary implementacji:

- `allocation` - obsługa wniosków o udostępnienie zasobów (kontekst: "Udostępnianie zasobów biurowych"),
- `resources` - katalog i cykl życia zasobów biurowych (kontekst: "Zarządzanie zasobami biurowymi"),
- `administration` - zarządzanie wnioskami przez administratora (kontekst: "Administracja wnioskami"),

- `employees` - zarządzanie systemem: pracownicy, role i limity alokacji (kontekst: "Zarządzanie systemem"),
- `identity` - logowanie, tokeny JWT i dane użytkownika (kontekst: "Uwierzytelnianie i autoryzacja"),
- `notifications` - powiadomienia wynikające ze zdarzeń domenowych (kontekst: "Powiadomienia"),

W każdym głównym kontekście zachowano podział na warstwy:

- `application/domain/model` - agregaty, encje, obiekty wartości, fabryki i zdarzenia domenowe,
- `application/domain/service` - usługi dziedzinowe,
- `application/service` - usługi aplikacyjne orkiestrujące przypadki użycia,
- `application/port/in` - porty wejściowe,
- `application/port/out` - porty wyjściowe,
- `infrastructure/in` - adaptery wejściowe, np. kontrolery REST i listenery zdarzeń,
- `infrastructure/out` - adaptery wyjściowe, np. repozytoria JPA, SMTP i adaptery do komunikacji międzykontekstowej.

5.5. Model domenowy

Najważniejsze elementy domeny zaimplementowano jako klasy modelu domenowego. Agregaty posiadają metody reprezentujące operacje biznesowe i same pilnują poprawności przejść stanów.

Najważniejsze agregaty:

- `AllocationRequest` - wniosek o udostępnienie zasobu,
- `OfficeResource` - zasób biurowy,
- `Employee` - pracownik z rolą i limitami,
- `AdministeredAllocationRequest` - administracyjny widok wniosku wykorzystywany w kontekście administracji.

Najważniejsze obiekty wartości:

- `AllocationRequestId`,
- `ResourceId`,
- `EmployeeId`,
- `DateRange`,

- `ResourceType`,
- `Location`,
- statusy, np. `RequestStatus`, `AllocationUsageStatus`, `AdministeredAllocationStatus`.

Obiekt `AllocationRequest` reprezentuje cykl życia wniosku. Udostępnia operacje domenowe takie jak `approve`, `reject`, `cancel`, `issueResource`, `returnResource` i `markAsOverdue`. Każda z tych metod sprawdza, czy operacja jest dozwolona w aktualnym stanie. Przykładowo zasób może zostać wydany tylko dla wniosku zatwierdzonego, a zwrot może nastąpić tylko dla wniosku aktywnego albo przeterminowanego.

Obiekt `OfficeResource` reprezentuje zasób biurowy i odpowiada za reguły dotyczące jego aktualizacji, dostępności i dezaktywacji. Zasobu nieaktywnego nie można edytować ani oznaczać jako dostępnego. Dezaktywacja powoduje również ustawienie zasobu jako niedostępnego.

5.6. Globalna obsługa wyjątków

W implementacji zastosowano standardowe wyjątki języka Java. Naruszenia reguł stanu są reprezentowane przez `IllegalStateException`, natomiast niepoprawne argumenty i dane wejściowe przez `IllegalArgumentException`. Przykładowo agregat `AllocationRequest` nie pozwala wydać zasobu dla wniosku, który nie jest zatwierdzony, a agregat `OfficeResource` nie pozwala edytować nieaktywnego zasobu. Brak wymaganego obiektu w warstwie aplikacyjnej jest reprezentowany przez `NoSuchElementException`.

Wyjątki te nie są pozostawione domyślnej obsłudze Springa. W pakiecie `common.infrastructure.in.web` znajduje się klasa `GlobalExceptionHandler` oznaczona adnotacją `@RestControllerAdvice`. Odpowiada ona za przekształcenie wyjątków na odpowiedzi HTTP zwracane przez REST API:

- `IllegalStateException` jest mapowany na status `409 Conflict`, ponieważ oznacza konflikt z aktualnym stanem biznesowym zasobu lub wniosku,
- `IllegalArgumentException` jest mapowany na status `400 Bad Request`, ponieważ oznacza niepoprawne dane przekazane do operacji,
- `NoSuchElementException` jest mapowany na status `404 Not Found`, ponieważ oznacza brak szukanego zasobu,
- `MethodArgumentNotValidException` jest mapowany na status `400 Bad Request` dla błędów walidacji danych wejściowych kontrolera.

Dzięki takiemu rozwiązaniu warstwa domenowa może pozostać niezależna od HTTP i Springa, a jednocześnie klient API otrzymuje jednoznaczną odpowiedź zgodną z charakterem błędu. Nieobsłużone wyjątki techniczne nadal byłyby traktowane przez framework jako błędy serwera, natomiast typowe naruszenia reguł biznesowych są jawnie mapowane na statusy `400`, `404` albo `409`.

5.7. Usługi dziedzinowe

W kontekście allocation (kontekst: "Udostępnianie zasobów biurowych") zaimplementowano między innymi:

- `EnsureEmployeeWithinLimitDomainService` - kontrola limitu zasobów przypisanych pracownikowi,
- `EnsureResourceEligibleForAllocationDomainService` - sprawdzenie, czy zasób może być przedmiotem alokacji,
- `EnsureResourceAvailableInPeriodDomainService` - sprawdzenie dostępności zasobu w podanym zakresie dat,
- `ApproveAllocationRequestDomainService`,
- `RejectAllocationRequestDomainService`,
- `CancelAllocationRequestDomainService`,
- `IssueAllocationResourceDomainService`,
- `ReturnAllocationResourceDomainService`,
- `MarkAllocationRequestOverdueDomainService`.

W kontekście resources (kontekst: "Zarządzanie zasobami biurowymi") zaimplementowano między innymi:

- `OfficeResourceFactory`,
- `UpdateOfficeResourceDomainService`,
- `MarkOfficeResourceAvailableDomainService`,
- `MarkOfficeResourceUnavailableDomainService`,
- `DeactivateOfficeResourceDomainService`.

5.8. Usługi aplikacyjne i przypadki użycia

Usługi aplikacyjne orkiestrują przypadki użycia systemu. Nie powinny zawierać szczegółów technologicznych ani rozbudowanej logiki biznesowej. Ich zadaniem jest pobranie wymaganych danych przez porty wyjściowe, wywołanie modelu domenowego lub usług dziedzinowych, zapis zmian i publikacja zdarzeń.

Najważniejsze usługi aplikacyjne:

- `AllocationRequestAppService` - składanie wniosków, anulowanie wniosków, przeglądanie zasobów i historii,

- `AllocationAdministrationAppService` - zatwierdzanie, odrzucanie, wydawanie, zwracanie i oznaczanie wniosków jako przeterminowanych,
- `ResourceAppService` - zarządzanie katalogiem zasobów,
- `AdministrationAppService` - widok i operacje administracyjne (zatwierdzanie/odrzucanie wniosków),
- `EmployeeAppService` - zarządzanie pracownikami i ich limitami,
- `IdentityAccessAppService` - logowanie i dostęp do profilu użytkownika,
- `NotificationAppService` - obsługa powiadomień.

Przykładowo metoda składania wniosku w `AllocationRequestAppService` wykonuje następujące kroki:

- pobiera dane pracownika przez port `EmployeeProfilePort`,
- pobiera dane zasobu przez port `ResourceCatalogPort`,
- deleguje sprawdzenie, czy zasób może być udostępniony, do usługi domenowej `EnsureResourceEligibleForAllocationDomainService`,
- deleguje sprawdzenie limitu aktywnych zasobów pracownika danego typu do usługi domenowej `EnsureEmployeeWithinLimitDomainService`,
- deleguje sprawdzenie dostępności zasobu w wybranym zakresie dat do usługi domenowej `EnsureResourceAvailableInPeriodDomainService`,
- tworzy agregat `AllocationRequest` przez fabrykę,
- zapisuje wniosek przez `AllocationRequestRepositoryPort`,
- publikuje zdarzenia domenowe.

Ten przepływ pokazuje zastosowanie DDD i architektury heksagonalnej: przypadek użycia jest widoczny w warstwie aplikacji, reguły znajdują się w domenie, a infrastruktura jest ukryta za portami.

5.9. Porty i adaptery

Architektura heksagonalna została zaimplementowana przez rozdzielenie portów i adapterów. Porty wejściowe określają, jakie operacje udostępnia aplikacja, np.

- `ResourceManagementInputPort`,
- `EmployeeManagementInputPort`,
- `AllocationRequestInputPort`,

- `AllocationAdministrationInputPort`,
- porty związane z logowaniem i powiadomieniami.

Porty wyjściowe określają, czego aplikacja potrzebuje od świata zewnętrznego lub innych kontekstów, np.:

- `AllocationRequestRepositoryPort`,
- `OfficeResourceRepositoryPort`,
- `EmployeeRepositoryPort`,
- `ResourceCatalogPort`,
- `EmployeeProfilePort`,
- `DomainEventPublisherPort`,
- porty powiadomień i integracji między kontekstami.

Adaptery wejściowe znajdują się głównie w pakietach `infrastructure/in`. Są to kontrolery REST (np. `AllocationRequestController`, `AdministrationController`, `ResourceController`, `EmployeeAdminController`, `AuthController`), listenery zdarzeń (np. `ResourceAvailabilityEventListener`, `NotificationEventListener`) oraz scheduler wywołujący okresowo logikę domenową (`OverdueAllocationScheduler`).

Adaptery wyjściowe znajdują się w pakietach `infrastructure/out`. Są to między innymi adaptery repozytoriów JPA, mappery między encjami JPA i modelem domenowym, adapter SMTP oraz adaptery komunikacji między kontekstami.

Dzięki temu implementacja frameworkowa nie przenika do modelu domenowego. Domenowe klasy nie posiadają adnotacji JPA, kontrolerów ani zależności od Springa.

5.10. Zdarzenia domenowe i EDA

System wykorzystuje zdarzenia domenowe jako sposób informowania innych części aplikacji o istotnych zmianach biznesowych. Agregaty zbierają zdarzenia podczas wykonywania operacji, a usługi aplikacyjne publikują je po zapisie zmian.

Przykłady zdarzeń w kontekście allocation (kontekst: "Udostępnianie zasobów biurowych"):

- `AllocationRequestSubmitted`,
- `AllocationRequestApproved`,
- `AllocationRequestRejected`,
- `AllocationRequestCancelled`,
- `ResourceIssued`,
- `ResourceReturned`,

- `AllocationOverdue`.

Przykłady zdarzeń w kontekście zasobów (kontekst: "Zarządzanie zasobami biurowymi"):

- `ResourceUpdated`,
- `ResourceMadeAvailable`,
- `ResourceMadeUnavailable`,
- `ResourceDeactivated`.

Zdarzenia są publikowane przez port `DomainEventPublisherPort`, którego implementacja infrastrukturalna korzysta z mechanizmów Spring. Słuchacze zdarzeń, np. `ResourceAvailabil ityEventListener` i `NotificationEventListener`, reagują na zmiany i wykonują działania poboczne, takie jak aktualizacja dostępności zasobu albo przygotowanie powiadomienia. Takie rozwiązanie zmniejsza sprzężenie między kontekstami i wspiera podejście Event Driven Architecture.

5.11. Warstwa persystencji

Dane systemu są przechowywane w relacyjnej bazie PostgreSQL. Schemat bazy jest wersjonowany przez Flyway. Pierwsza migracja `V1__init_schema.sql` tworzy tabele:

- `employees`,
- `allocation_limits`,
- `office_resources`,
- `allocation_requests`.

W tabeli `employees` przechowywane są dane pracowników, role i hashe haseł. Tabela `allocation_limits` określa limity zasobów dla pracowników. Tabela `office_resources` przechowuje zasoby biurowe, ich typ, nazwę, lokalizację oraz status aktywności i dostępności. Tabela `allocation_requests` przechowuje wnioski, ich status, zakres dat, uzasadnienie oraz daty wydania i zwrotu.

Persystencja nie jest bezpośrednio częścią modelu domenowego. Dla każdego agregatu istnieją encje JPA oraz mappery, np. `AllocationRequestEntity`, `AllocationRequestMapper`, `OfficeResourceEntity`, `OfficeResourceMapper`, `EmployeeEntity` i `EmployeeMapper`. Adaptery repozytoriów odpowiadają za konwersję między modelem domenowym a modelem relacyjnym.

5.12. Uwierzytelnianie i autoryzacja

Za obszar tożsamości odpowiada kontekst `identity`. Logowanie jest realizowane przez `AuthController`, a dostęp do chronionych endpointów zabezpiecza Spring Security.

System używa tokenów JWT. Klasa `JwtTokenProvider` odpowiada za wystawianie i weryfikację tokenów, natomiast `JwtAuthenticationFilter` odczytuje token z nagłówka

HTTP, tworzy kontekst bezpieczeństwa i przekazuje go do `SecurityContextHolder`. Dane użytkownika są pobierane przez `DeskflowUserDetailsService` oraz adaptery korzystające z danych pracowników.

Autoryzacja opiera się na rolach, np. pracownik może składać i anulować swoje wnioski, a kierownik lub administrator może wykonywać operacje administracyjne, takie jak zatwierdzanie, odrzucanie, wydawanie i przyjmowanie zwrotów.

5.13. Frontend

Frontend znajduje się w katalogu `deskflow-frontend`. Został wykonany jako aplikacja Next.js z React i TypeScript. Jego zadaniem jest udostępnienie użytkownikowi interfejsu do logowania, przeglądania zasobów, składania wniosków, przeglądania historii oraz wykonywania operacji administracyjnych.

Komunikacja z backendem odbywa się przez REST API. Adres backendu jest konfigurowany przez zmienną środowiskową `NEXT_PUBLIC_API_URL`. W kodzie frontendu wydzielono warstwę komunikacji z API w pliku `lib/api.ts` oraz kontekst uwierzytelniania w `lib/auth-context.tsx`.

Frontend nie zawiera logiki domenowej. Reguły biznesowe są egzekwowane przez backend. Interfejs użytkownika odpowiada za prezentację danych, zebranie danych od użytkownika i wysłanie odpowiednich żądań HTTP.

5.14. Integracja komponentów

Komponenty systemu są integrowane przez Docker Compose, przy czym konfiguracja została rozdzielona na wariant deweloperski i produkcyjny. Środowisko deweloperskie opisuje plik `docker-compose.dev.yml`. Jest ono przeznaczone do pracy lokalnej i uruchamia:

- `postgres` - bazę danych PostgreSQL,
- `mailhog` - lokalny serwer testowy do odbierania maili,
- `backend-dev` - backend Spring Boot na porcie 8080,
- `frontend-dev` - frontend Next.js na porcie 3000.

W tym wariancie backend łączy się z PostgreSQL przez JDBC, a z MailHog przez SMTP. Frontend komunikuje się z backendem przez HTTP z użyciem lokalnego adresu API. Zależności startowe w Docker Compose zapewniają, że backend czeka na gotowość bazy danych, a frontend startuje po backendzie. MailHog pełni wyłącznie rolę narzędzia pomocniczego do testowania wiadomości e-mail i nie jest elementem środowiska produkcyjnego.

Wariant produkcyjny opisuje plik `docker-compose.prod.yml`. Jest on używany przy wdrożeniu na platformie Coolify i nie zawiera usług typowo deweloperskich, takich jak MailHog. Zakłada uruchomienie backendu, frontendu i bazy danych jako kontenerów z konfiguracją przekazywaną przez zmienne środowiskowe, takie jak `POSTGRES_PASSWORD`, `JWT_SECRET`, `ALLOWED_ORIGIN`, `NEXT_PUBLIC_API_URL` i opcjonalne dane SMTP. W produkcji adresy usług, sekrety i ustawienia CORS są ustawiane w panelu Coolify, a ruch użytkownika kierowany jest

do kontenera frontendu, który komunikuje się z backendem przez skonfigurowany publiczny adres API.

Różnice między obrazami deweloperskimi i produkcyjnymi wynikają z innego celu ich użycia. Wariant deweloperski ma umożliwiać szybką pracę lokalną, montowanie kodu z repozytorium, automatyczne przeładowanie aplikacji i debugowanie. Wariant produkcyjny ma natomiast uruchamiać gotowe artefakty, ograniczać rozmiar obrazu oraz nie zawierać narzędzi potrzebnych wyłącznie podczas kompilacji.

Table 5.1: Porównanie Dockerfile backendu w środowisku deweloperskim i produkcyjnym

Cecha	**Backend - dev**	**Backend - prod**
Plik	`deskflow-backend/Dockerfile.dev`	Etapy `backend-build` i `backend` w pliku `Dockerfile.prod`
Obraz bazowy	`maven:3.9-eclipse-temurin-21`	Budowanie: `maven:3.9-eclipse-temurin-21`; uruchomienie: `eclipse-temurin:21-jre-alpine`
Sposób uruchomienia	Aplikacja startuje przez `mvn spring-boot:run`.	Aplikacja startuje jako gotowy plik JAR przez `java -jar app.jar`.
Artefakt aplikacji	Kod źródłowy jest kopiowany do kontenera i uruchamiany przez Maven.	Najpierw wykonywane jest `mvn package`, a do obrazu uruchomieniowego kopiowany jest tylko zbudowany plik JAR.
Debugowanie	Wystawiony jest port `5005` dla zdalnego debuggera JVM.	Nie ma portu debugowania; wystawiony jest tylko port aplikacji `8080`.
Użytkownik w kontenerze	Brak osobnego użytkownika aplikacyjnego; kontener działa domyślnie jako `root`.	Tworzony i używany jest nieuprzywilejowany użytkownik `deskflow`.
Cel	Lokalny rozwój, szybkie zmiany kodu i debugowanie.	Stabilne uruchomienie zbudowanego backendu w środowisku produkcyjnym.

W repozytorium znajduje się również plik `deskflow-frontend/Dockerfile.frontend`. Jest to samodzielny produkcyjny Dockerfile dla frontendu, oparty na tym samym podejściu co etap frontendowy w `Dockerfile.prod`: najpierw buduje aplikację Next.js, a następnie uruchamia wynikowy serwer z katalogu `.next/standalone`. W aktualnej konfiguracji wdrożeniowej Coolify używany jest jednak wspólny plik `Dockerfile.prod`, ponieważ pozwala z jednego pliku budować osobne targety dla backendu i frontendu.

5.15. Wdrożenie

Wdrożenie jest bezpośrednim wykorzystaniem wariantu produkcyjnego opisanego w poprzedniej sekcji. Oznacza to, że zamiast uruchamiania deweloperskich serwerów przez Maven i `npm run dev`, platforma Coolify buduje produkcyjne targety z pliku `Dockerfile.prod` i uruchamia je zgodnie z definicją z `docker-compose.prod.yml`.

Table 5.2: Porównanie Dockerfile frontendu w środowisku deweloperskim i produkcyjnym

Cecha	**Frontend - dev**	**Frontend - prod**
Plik	`deskflow-frontend/Dockerfile.dev`	Etapy `frontend-build` i `frontend` w pliku `Dockerfile.prod`
Obraz bazowy	`node:22-alpine`	Budowanie i uruchomienie: `node:22-alpine`, ale w osobnych etapach obrazu.
Sposób uruchomienia	Aplikacja startuje przez `npm run dev`.	Aplikacja startuje przez `node server.js` z katalogu `.next/standalone`.
Budowanie aplikacji	Nie wykonuje się produkcyjnego buildu; działa serwer deweloperski Next.js.	Wykonywane jest `npm run build`, a do obrazu uruchomieniowego kopiowane są tylko pliki potrzebne do działania aplikacji.
Konfiguracja API	Adres API jest przekazywany przez `NEXT_PUBLIC_API_URL` w `docker-compose.dev.yml`.	Adres API jest przekazywany jako argument budowania i zmienna środowiskowa `NEXT_PUBLIC_API_URL`.
Użytkownik w kontenerze	Kontener działa domyślnie jako użytkownik obrazu bazowego, czyli `root`.	Kontener działa jako nieuprzywilejowany użytkownik `node`.
Cel	Lokalny rozwój interfejsu, szybkie odświeżanie i praca z kodem źródłowym.	Uruchomienie zoptymalizowanej aplikacji Next.js w środowisku produkcyjnym.

W środowisku deweloperskim system można uruchomić poleceniem:

```
docker compose -f docker-compose.dev.yml up -build
```

W środowisku produkcyjnym ten sam podział komponentów jest zachowany, ale zmienia się sposób ich przygotowania i konfiguracji. Coolify odpowiada za zbudowanie obrazów, start kontenerów, przekazanie zmiennych środowiskowych, konfigurację domen oraz utrzymywanie działania usług.

Istotne elementy wdrożenia produkcyjnego to healthcheck backendu pod adresem `/actuator/health`, restart kontenerów w trybie `unless-stopped` oraz trwały wolumen bazy danych. Sekrety i wartości zależne od środowiska, takie jak hasło do bazy danych, sekret JWT, adres API i dozwolony adres frontendu, nie są zhardkodowane. Są przekazywane do kontenerów jako zmienne środowiskowe ustawiane w Coolify. Dzięki temu konfiguracja wdrożeniowa pozostaje powtarzalna, ale dane wrażliwe nie są częścią kodu źródłowego.

5.16. Spełnienie zasad DDD, SOLID i czystej architektury

Implementacja spełnia założenia DDD przez:

- jawne wydzielenie kontekstów domenowych,
- użycie agregatów i obiektów wartości,
- umieszczenie reguł biznesowych w modelu domenowym i usługach dziedzinowych,
- zastosowanie zdarzeń domenowych,
- zachowanie nazw zgodnych z językiem dziedziny w kodzie angielskim.

Implementacja spełnia założenia czystej architektury przez:

- oddzielenie domeny od infrastruktury,
- zastosowanie portów wejściowych i wyjściowych,
- implementowanie szczegółów technicznych jako adapterów,
- kontrolowanie kierunku zależności testami ArchUnit.

Zasady SOLID są realizowane przede wszystkim przez:

- pojedynczą odpowiedzialność klas, np. osobne klasy dla kontrolerów, usług aplikacyjnych, usług domenowych, mapperów i adapterów,
- odwrócenie zależności przez porty,
- ograniczenie sprzężenia domeny z technologiami,
- łatwość podstawiania innych implementacji portów, np. innego repozytorium, innego mechanizmu powiadomień albo innego adaptera publikacji zdarzeń.

5.17. Wnioski z etapu implementacji

Etap implementacji doprowadził do utworzenia działającego systemu zgodnego z założeniami projektu. Najważniejsze funkcje biznesowe zostały zaimplementowane w backendzie Java, a interfejs użytkownika i infrastruktura zostały zintegrowane z rdzeniem systemu.

Najważniejsza cecha rozwiązania to oddzielenie logiki domenowej od szczegółów technicznych. Implementacja zachowuje strukturę wynikającą z DDD i architektury hexagonalnej: agregaty, usługi dziedzinowe, przypadki użycia, porty, adaptery i zdarzenia domenowe.

6. Listing kodu źródłowego, testów i skryptów wdrożeniowych

6.1. Kod źródłowy backendu

Source code 6.1: Plik główny: DeskflowApplication.java

```
package com.deskflow;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class DeskflowApplication {
    public static void main(String[] args) {
        SpringApplication.run(DeskflowApplication.class, args);
    }
}
```

6.1.1. Kontekst Allocation - alokacja zasobów

Source code 6.2: allocation/application/domain/model/AllocationRequest.java - AllocationRequest

```
package com.deskflow.allocation.application.domain.model;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.allocation.application.domain.model.events.*;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class AllocationRequest {

    private final AllocationRequestId id;
    private final EmployeeId employeeId;
    private final ResourceId resourceId;
    private final DateRange dateRange;
    private final String justification;
    private final LocalDateTime submittedAt;
    private RequestStatus status;
    private LocalDateTime issuedAt;
    private LocalDateTime returnedAt;
    private String rejectionReason;
    private final List<DomainEvent> events = new ArrayList<>();
```

```
28
29      private AllocationRequest(AllocationRequestId id, EmployeeId employeeId, ResourceId resourceId,
30                                DateRange dateRange, String justification, LocalDateTime submittedAt,
31                                RequestStatus status, LocalDateTime issuedAt,
32                                LocalDateTime returnedAt, String rejectionReason) {
33          this.id = id;
34          this.employeeId = employeeId;
35          this.resourceId = resourceId;
36          this.dateRange = dateRange;
37          this.justification = justification;
38          this.submittedAt = submittedAt;
39          this.status = status;
40          this.issuedAt = issuedAt;
41          this.returnedAt = returnedAt;
42          this.rejectionReason = rejectionReason;
43      }
44
45      public static AllocationRequest create(AllocationRequestId id, EmployeeId employeeId,
46                                             ResourceId resourceId, DateRange dateRange,
47                                             String justification, LocalDateTime submittedAt) {
48          var r = new AllocationRequest(id, employeeId, resourceId, dateRange, justification, submittedAt,
49                  RequestStatus.SUBMITTED, null, null, null);
50          r.events.add(new AllocationRequestSubmitted(id, employeeId, resourceId, dateRange, submittedAt));
51          return r;
52      }
53
54      public static AllocationRequest reconstitute(AllocationRequestId id, EmployeeId employeeId,
55                                                   ResourceId resourceId, DateRange dateRange,
56                                                   String justification, LocalDateTime submittedAt,
57                                                   RequestStatus status, LocalDateTime issuedAt,
58                                                   LocalDateTime returnedAt, String rejectionReason) {
59          return new AllocationRequest(id, employeeId, resourceId, dateRange, justification, submittedAt,
60                  status, issuedAt, returnedAt, rejectionReason);
61      }
62
63      public void approve() {
64          requireStatus(RequestStatus.SUBMITTED, "approve");
65          this.status = RequestStatus.APPROVED;
66          events.add(new AllocationRequestApproved(id, employeeId, LocalDateTime.now()));
67      }
68
69      public void reject(String reason) {
70          requireStatus(RequestStatus.SUBMITTED, "reject");
71          this.status = RequestStatus.REJECTED;
72          this.rejectionReason = reason;
73          events.add(new AllocationRequestRejected(id, employeeId, reason, LocalDateTime.now()));
74      }
75
76      public void cancel() {
77          if (status != RequestStatus.SUBMITTED && status != RequestStatus.APPROVED) {
78              throw new IllegalStateException(
79                      "Can only cancel a SUBMITTED or APPROVED request, current: " + status);
80          }
81          this.status = RequestStatus.CANCELLED;
82          events.add(new AllocationRequestCancelled(id, employeeId, LocalDateTime.now()));
83      }
84
85      public void issueResource() {
86          requireStatus(RequestStatus.APPROVED, "issue resource");
87          this.status = RequestStatus.ACTIVE;
88          this.issuedAt = LocalDateTime.now();
89          events.add(new ResourceIssued(id, employeeId, resourceId, issuedAt));
90      }
91
92      public void returnResource() {
93          if (status != RequestStatus.ACTIVE && status != RequestStatus.OVERDUE) {
94              throw new IllegalStateException(
95                      "Can only return resource for ACTIVE or OVERDUE request, current: " + status);
96          }
97          this.status = RequestStatus.COMPLETED;
```

```
98          this.returnedAt = LocalDateTime.now();
99          events.add(new ResourceReturned(id, employeeId, resourceId, returnedAt));
100     }
101
102     public void markAsOverdue() {
103         requireStatus(RequestStatus.ACTIVE, "mark as overdue");
104         this.status = RequestStatus.OVERDUE;
105         events.add(new AllocationOverdue(id, employeeId, resourceId, LocalDateTime.now()));
106     }
107
108     private void requireStatus(RequestStatus required, String operation) {
109         if (status != required) {
110             throw new IllegalStateException(
111                     "Operation '" + operation + "' requires status " + required + ", current: " + status);
112         }
113     }
114
115     public List<DomainEvent> flushEvents() {
116         var copy = List.copyOf(events);
117         events.clear();
118         return copy;
119     }
120
121     public AllocationRequestId getId() { return id; }
122     public EmployeeId getEmployeeId() { return employeeId; }
123     public ResourceId getResourceId() { return resourceId; }
124     public DateRange getDateRange() { return dateRange; }
125     public String getJustification() { return justification; }
126     public LocalDateTime getSubmittedAt() { return submittedAt; }
127     public RequestStatus getStatus() { return status; }
128     public LocalDateTime getIssuedAt() { return issuedAt; }
129     public LocalDateTime getReturnedAt() { return returnedAt; }
130     public String getRejectionReason() { return rejectionReason; }
131 }
```

Source code 6.3: allocation/application/domain/model/EmployeeAllocationProfile.java - EmployeeAllocationProfile

```
1  package com.deskflow.allocation.application.domain.model;
2
3  import java.util.Map;
4
5  public record EmployeeAllocationProfile(Map<String, Integer> limitsByResourceType) {
6      public int limitForType(String resourceType) {
7          return limitsByResourceType.getOrDefault(resourceType, 1);
8      }
9  }
```

Source code 6.4: allocation/application/domain/model/ResourceAllocationProfile.java - ResourceAllocationProfile

```
1  package com.deskflow.allocation.application.domain.model;
2
3  import com.deskflow.common.application.domain.model.vo.ResourceId;
4
5  public record ResourceAllocationProfile(
6          ResourceId id,
7          String type,
8          String name,
9          String building,
10         int floor,
11         String number,
12         boolean active,
13         boolean available
14 ) {
```

```
    public boolean isActive() {
        return active;
    }

    public boolean isAvailable() {
        return available;
    }
}
```

Source code 6.5: allocation/application/domain/model/events/AllocationOverdue.java - AllocationOverdue

```
package com.deskflow.allocation.application.domain.model.events;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;
import java.util.UUID;

public record AllocationOverdue(
        AllocationRequestId requestId,
        EmployeeId employeeId,
        ResourceId resourceId,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "AllocationOverdue"; }
    @Override
    public UUID aggregateId() { return requestId.getValue(); }
}
```

Source code 6.6: allocation/application/domain/model/events/AllocationRequestApproved.java - AllocationRequestApproved

```
package com.deskflow.allocation.application.domain.model.events;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;

import java.time.LocalDateTime;
import java.util.UUID;

public record AllocationRequestApproved(
        AllocationRequestId requestId,
        EmployeeId employeeId,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "AllocationRequestApproved"; }
    @Override
    public UUID aggregateId() { return requestId.getValue(); }
}
```

Source code 6.7: allocation/application/domain/model/events/AllocationRequestCancelled.java - AllocationRequestCancelled

```java
package com.deskflow.allocation.application.domain.model.events;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;

import java.time.LocalDateTime;
import java.util.UUID;

public record AllocationRequestCancelled(
        AllocationRequestId requestId,
        EmployeeId employeeId,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "AllocationRequestCancelled"; }
    @Override
    public UUID aggregateId() { return requestId.getValue(); }
}
```

Source code 6.8:
allocation/application/domain/model/events/AllocationRequestRejected.java - AllocationRequestRejected

```java
package com.deskflow.allocation.application.domain.model.events;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;

import java.time.LocalDateTime;
import java.util.UUID;

public record AllocationRequestRejected(
        AllocationRequestId requestId,
        EmployeeId employeeId,
        String reason,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "AllocationRequestRejected"; }
    @Override
    public UUID aggregateId() { return requestId.getValue(); }
}
```

Source code 6.9:
allocation/application/domain/model/events/AllocationRequestSubmitted.java - AllocationRequestSubmitted

```java
package com.deskflow.allocation.application.domain.model.events;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;
import java.util.UUID;
```

```
11
12 public record AllocationRequestSubmitted(
13         AllocationRequestId requestId,
14         EmployeeId employeeId,
15         ResourceId resourceId,
16         DateRange dateRange,
17         LocalDateTime occurredAt
18 ) implements DomainEvent {
19     @Override
20     public UUID eventId() { return UUID.randomUUID(); }
21     @Override
22     public String eventType() { return "AllocationRequestSubmitted"; }
23     @Override
24     public UUID aggregateId() { return requestId.getValue(); }
25 }
```

Source code 6.10: allocation/application/domain/model/events/ResourceIssued.java - ResourceIssued

```
1  package com.deskflow.allocation.application.domain.model.events;
2
3  import com.deskflow.common.application.domain.model.vo.EmployeeId;
4  import com.deskflow.common.application.domain.DomainEvent;
5  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
6  import com.deskflow.common.application.domain.model.vo.ResourceId;
7
8  import java.time.LocalDateTime;
9  import java.util.UUID;
10
11 public record ResourceIssued(
12         AllocationRequestId requestId,
13         EmployeeId employeeId,
14         ResourceId resourceId,
15         LocalDateTime occurredAt
16 ) implements DomainEvent {
17     @Override
18     public UUID eventId() { return UUID.randomUUID(); }
19     @Override
20     public String eventType() { return "ResourceIssued"; }
21     @Override
22     public UUID aggregateId() { return requestId.getValue(); }
23 }
```

Source code 6.11: allocation/application/domain/model/events/ResourceReturned.java - ResourceReturned

```
1  package com.deskflow.allocation.application.domain.model.events;
2
3  import com.deskflow.common.application.domain.model.vo.EmployeeId;
4  import com.deskflow.common.application.domain.DomainEvent;
5  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
6  import com.deskflow.common.application.domain.model.vo.ResourceId;
7
8  import java.time.LocalDateTime;
9  import java.util.UUID;
10
11 public record ResourceReturned(
12         AllocationRequestId requestId,
13         EmployeeId employeeId,
14         ResourceId resourceId,
15         LocalDateTime occurredAt
16 ) implements DomainEvent {
17     @Override
18     public UUID eventId() { return UUID.randomUUID(); }
19     @Override
20     public String eventType() { return "ResourceReturned"; }
```

```
21      @Override
22      public UUID aggregateId() { return requestId.getValue(); }
23  }
```

Source code 6.12: allocation/application/domain/model/vo/AllocationRequestId.java - AllocationRequestId

```
1   package com.deskflow.allocation.application.domain.model.vo;
2
3   import java.util.Objects;
4   import java.util.UUID;
5
6   public final class AllocationRequestId {
7       private final UUID value;
8
9       public AllocationRequestId(UUID value) {
10          this.value = Objects.requireNonNull(value);
11      }
12
13      public static AllocationRequestId of(UUID value) {
14          return new AllocationRequestId(value);
15      }
16
17      public static AllocationRequestId of(String value) {
18          return new AllocationRequestId(UUID.fromString(value));
19      }
20
21      public static AllocationRequestId newId() {
22          return new AllocationRequestId(UUID.randomUUID());
23      }
24
25      public UUID getValue() {
26          return value;
27      }
28
29      @Override
30      public boolean equals(Object o) {
31          if (this == o) return true;
32          if (!(o instanceof AllocationRequestId that)) return false;
33          return value.equals(that.value);
34      }
35
36      @Override
37      public int hashCode() {
38          return value.hashCode();
39      }
40
41      @Override
42      public String toString() {
43          return value.toString();
44      }
45  }
```

Source code 6.13: allocation/application/domain/model/vo/DateRange.java - DateRange

```
1   package com.deskflow.allocation.application.domain.model.vo;
2
3   import java.time.LocalDate;
4   import java.util.Objects;
5
6   public final class DateRange {
7       private final LocalDate dateFrom;
8       private final LocalDate dateTo;
9
10      public DateRange(LocalDate dateFrom, LocalDate dateTo) {
```

```
        Objects.requireNonNull(dateFrom, "dateFrom cannot be null");
        Objects.requireNonNull(dateTo, "dateTo cannot be null");
        if (!dateFrom.isBefore(dateTo)) {
            throw new IllegalArgumentException("dateFrom must be before dateTo");
        }
        this.dateFrom = dateFrom;
        this.dateTo = dateTo;
    }

    public LocalDate getDateFrom() { return dateFrom; }
    public LocalDate getDateTo() { return dateTo; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof DateRange that)) return false;
        return dateFrom.equals(that.dateFrom) && dateTo.equals(that.dateTo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(dateFrom, dateTo);
    }

    @Override
    public String toString() {
        return dateFrom + " - " + dateTo;
    }
}
```

Source code 6.14: allocation/application/domain/model/vo/RequestStatus.java - RequestStatus

```
package com.deskflow.allocation.application.domain.model.vo;

public enum RequestStatus {
    SUBMITTED,
    APPROVED,
    ACTIVE,
    COMPLETED,
    REJECTED,
    CANCELLED,
    OVERDUE
}
```

Source code 6.15: allocation/application/domain/service/AllocationRequestFactory.java - AllocationRequestFactory

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;

public class AllocationRequestFactory {

    public AllocationRequest create(EmployeeId employeeId, ResourceId resourceId,
                                    DateRange dateRange, String justification) {
        return AllocationRequest.create(
                AllocationRequestId.newId(),
                employeeId,
```

```
                resourceId,
                dateRange,
                justification,
                LocalDateTime.now()
        );
    }
}
```

Source code 6.16:
allocation/application/domain/service/ApproveAllocationRequestDomainService.java - ApproveAllocationRequestDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

public class ApproveAllocationRequestDomainService {

    public void approveAllocationRequest(AllocationRequest request) {
        request.approve();
    }
}
```

Source code 6.17:
allocation/application/domain/service/CancelAllocationRequestDomainService.java - CancelAllocationRequestDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.common.application.domain.model.vo.EmployeeId;

public class CancelAllocationRequestDomainService {

    public void cancelAllocationRequest(AllocationRequest request, EmployeeId employeeId) {
        if (!request.getEmployeeId().equals(employeeId)) {
            throw new IllegalStateException("Cannot cancel another employee's request");
        }
        request.cancel();
    }
}
```

Source code 6.18:
allocation/application/domain/service/EnsureEmployeeWithinLimitDomainService.java - EnsureEmployeeWithinLimitDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;

public class EnsureEmployeeWithinLimitDomainService {

    public void ensureEmployeeWithinLimit(EmployeeAllocationProfile employee, String resourceType, long
    currentActiveForType) {
        int limit = employee.limitForType(resourceType);
        if (currentActiveForType >= limit) {
            throw new IllegalStateException("Employee has reached the allocation limit for this resource
            type");
        }
    }
}
```

Source code 6.19: allocation/application/domain/service/EnsureResourceAvailableInPeriodDomainService.java - EnsureResourceAvailableInPeriodDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

import java.util.List;

public class EnsureResourceAvailableInPeriodDomainService {

    public void ensureResourceAvailableInPeriod(List<AllocationRequest> overlappingRequests) {
        if (!overlappingRequests.isEmpty()) {
            throw new IllegalStateException("Resource is not available for the requested period");
        }
    }
}
```

Source code 6.20: allocation/application/domain/service/EnsureResourceEligibleForAllocationDomainService.java - EnsureResourceEligibleForAllocationDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;

public class EnsureResourceEligibleForAllocationDomainService {

    public void ensureResourceEligibleForAllocation(ResourceAllocationProfile resource) {
        if (!resource.isActive() || !resource.isAvailable()) {
            throw new IllegalStateException("Resource is not available for allocation");
        }
    }
}
```

Source code 6.21: allocation/application/domain/service/IssueAllocationResourceDomainService.java - IssueAllocationResourceDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

public class IssueAllocationResourceDomainService {

    public void issueAllocationResource(AllocationRequest request) {
        request.issueResource();
    }
}
```

Source code 6.22: allocation/application/domain/service/MarkAllocationRequestOverdueDomainService.java - MarkAllocationRequestOverdueDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

public class MarkAllocationRequestOverdueDomainService {

    public void markAllocationRequestOverdue(AllocationRequest request) {
        request.markAsOverdue();
    }
}
```

Source code 6.23: allocation/application/domain/service/RejectAllocationRequestDomainService.java - RejectAllocationRequestDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

public class RejectAllocationRequestDomainService {

    public void rejectAllocationRequest(AllocationRequest request, String reason) {
        if (reason == null || reason.isBlank()) {
            throw new IllegalArgumentException("Rejection reason cannot be blank");
        }
        request.reject(reason);
    }
}
```

Source code 6.24: allocation/application/domain/service/ReturnAllocationResourceDomainService.java - ReturnAllocationResourceDomainService

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

public class ReturnAllocationResourceDomainService {

    public void returnAllocationResource(AllocationRequest request) {
        request.returnResource();
    }
}
```

Source code 6.25: allocation/application/model/AllocationRequestView.java - AllocationRequestView

```
package com.deskflow.allocation.application.model;

import com.deskflow.allocation.application.domain.model.AllocationRequest;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

public record AllocationRequestView(
        UUID id,
        UUID employeeId,
        UUID resourceId,
        String status,
        LocalDate dateFrom,
        LocalDate dateTo,
        String justification,
        LocalDateTime submittedAt,
        LocalDateTime issuedAt,
        LocalDateTime returnedAt,
        String rejectionReason
) {
    public static AllocationRequestView from(AllocationRequest request) {
        return new AllocationRequestView(
                request.getId().getValue(),
                request.getEmployeeId().getValue(),
                request.getResourceId().getValue(),
                request.getStatus().name(),
```

```
                request.getDateRange().getDateFrom(),
                request.getDateRange().getDateTo(),
                request.getJustification(),
                request.getSubmittedAt(),
                request.getIssuedAt(),
                request.getReturnedAt(),
                request.getRejectionReason()
        );
    }
}
```

Source code 6.26: allocation/application/model/ResourceCatalogItem.java - ResourceCatalogItem

```
package com.deskflow.allocation.application.model;

import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;

import java.util.UUID;

public record ResourceCatalogItem(
        UUID id,
        String type,
        String name,
        String building,
        int floor,
        String number,
        boolean available
) {
    public static ResourceCatalogItem from(ResourceAllocationProfile resource) {
        return new ResourceCatalogItem(
                resource.id().getValue(),
                resource.type(),
                resource.name(),
                resource.building(),
                resource.floor(),
                resource.number(),
                resource.available()
        );
    }
}
```

Source code 6.27: allocation/application/port/in/AllocationAdministrationInputPort.java - AllocationAdministrationInputPort

```
package com.deskflow.allocation.application.port.in;

import com.deskflow.allocation.application.model.AllocationRequestView;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;

import java.util.List;
import java.util.Optional;

public interface AllocationAdministrationInputPort {
    Optional<AllocationRequestView> getRequest(AllocationRequestId requestId);

    List<AllocationRequestView> listRequests(RequestStatus status);

    void approveRequest(AllocationRequestId requestId);

    void rejectRequest(AllocationRequestId requestId, String reason);

    void issueResource(AllocationRequestId requestId);

```

```
21      void returnResource(AllocationRequestId requestId);
22
23      List<AllocationRequestView> monitorActiveAllocations();
24
25      void detectOverdueAllocations();
26  }
```

Source code 6.28: allocation/application/port/in/AllocationRequestInputPort.java - AllocationRequestInputPort

```
1   package com.deskflow.allocation.application.port.in;
2
3   import com.deskflow.allocation.application.model.AllocationRequestView;
4   import com.deskflow.allocation.application.model.ResourceCatalogItem;
5   import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
6   import com.deskflow.common.application.domain.model.vo.EmployeeId;
7   import com.deskflow.common.application.domain.model.vo.ResourceId;
8
9   import java.time.LocalDate;
10  import java.util.List;
11
12  public interface AllocationRequestInputPort {
13
14      AllocationRequestId submitRequest(EmployeeId employeeId, ResourceId resourceId,
15                                        LocalDate dateFrom, LocalDate dateTo, String justification);
16
17      void cancelRequest(AllocationRequestId requestId, EmployeeId employeeId);
18
19      List<ResourceCatalogItem> browseResources();
20
21      List<AllocationRequestView> browseHistory(EmployeeId employeeId);
22
23      List<AllocationRequestView> findActiveForResource(ResourceId resourceId);
24  }
```

Source code 6.29: allocation/application/port/out/AllocationRequestRepositoryPort.java - AllocationRequestRepositoryPort

```
1   package com.deskflow.allocation.application.port.out;
2
3   import com.deskflow.common.application.domain.model.vo.EmployeeId;
4   import com.deskflow.allocation.application.domain.model.AllocationRequest;
5   import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
6   import com.deskflow.allocation.application.domain.model.vo.DateRange;
7   import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
8   import com.deskflow.common.application.domain.model.vo.ResourceId;
9
10  import java.time.LocalDateTime;
11  import java.util.List;
12  import java.util.Optional;
13
14  public interface AllocationRequestRepositoryPort {
15
16      void save(AllocationRequest request);
17
18      Optional<AllocationRequest> find(AllocationRequestId id);
19
20      List<AllocationRequest> findActiveForResource(ResourceId resourceId, DateRange dateRange);
21
22      List<AllocationRequest> findActiveForResource(ResourceId resourceId);
23
24      List<AllocationRequest> findActiveForEmployee(EmployeeId employeeId);
25
26      List<AllocationRequest> findOverdue(LocalDateTime now);
27
```

```
28      List<AllocationRequest> findForEmployee(EmployeeId employeeId);
29
30      List<AllocationRequest> findAll(RequestStatus status);
31
32      List<AllocationRequest> findAll();
33  }
```

Source code 6.30: allocation/application/port/out/EmployeeProfilePort.java - EmployeeProfilePort

```
1   package com.deskflow.allocation.application.port.out;
2
3   import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;
4   import com.deskflow.common.application.domain.model.vo.EmployeeId;
5
6   import java.util.Optional;
7
8   public interface EmployeeProfilePort {
9       Optional<EmployeeAllocationProfile> findEmployee(EmployeeId employeeId);
10  }
```

Source code 6.31: allocation/application/port/out/ResourceCatalogPort.java - ResourceCatalogPort

```
1   package com.deskflow.allocation.application.port.out;
2
3   import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
4   import com.deskflow.common.application.domain.model.vo.ResourceId;
5
6   import java.util.List;
7   import java.util.Optional;
8
9   public interface ResourceCatalogPort {
10      Optional<ResourceAllocationProfile> findResource(ResourceId resourceId);
11
12      List<ResourceAllocationProfile> findActiveResources();
13  }
```

Source code 6.32: allocation/application/service/AllocationAdministrationAppService.java - AllocationAdministrationAppService

```
1   package com.deskflow.allocation.application.service;
2
3   import com.deskflow.allocation.application.model.AllocationRequestView;
4   import com.deskflow.allocation.application.port.in.AllocationAdministrationInputPort;
5   import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
6   import com.deskflow.allocation.application.domain.model.AllocationRequest;
7   import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
8   import com.deskflow.allocation.application.domain.service.ApproveAllocationRequestDomainService;
9   import com.deskflow.allocation.application.domain.service.IssueAllocationResourceDomainService;
10  import com.deskflow.allocation.application.domain.service.MarkAllocationRequestOverdueDomainService;
11  import com.deskflow.allocation.application.domain.service.RejectAllocationRequestDomainService;
12  import com.deskflow.allocation.application.domain.service.ReturnAllocationResourceDomainService;
13  import com.deskflow.common.application.port.out.DomainEventPublisherPort;
14  import com.deskflow.common.application.domain.DomainEvent;
15  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
16  import org.springframework.stereotype.Service;
17  import org.springframework.transaction.annotation.Transactional;
18
19  import java.time.LocalDateTime;
20  import java.util.List;
```

```
21 import java.util.NoSuchElementException;
22 import java.util.Optional;
23
24 @Service
25 @Transactional
26 public class AllocationAdministrationAppService implements AllocationAdministrationInputPort {
27
28     private final AllocationRequestRepositoryPort requestRepository;
29     private final DomainEventPublisherPort eventPublisher;
30     private final ApproveAllocationRequestDomainService approveAllocationRequestDomainService;
31     private final RejectAllocationRequestDomainService rejectAllocationRequestDomainService;
32     private final IssueAllocationResourceDomainService issueAllocationResourceDomainService;
33     private final ReturnAllocationResourceDomainService returnAllocationResourceDomainService;
34     private final MarkAllocationRequestOverdueDomainService markAllocationRequestOverdueDomainService;
35
36     public AllocationAdministrationAppService(AllocationRequestRepositoryPort requestRepository,
37                                               DomainEventPublisherPort eventPublisher,
38                                               ApproveAllocationRequestDomainService
                                                 approveAllocationRequestDomainService,
39                                               RejectAllocationRequestDomainService
                                                 rejectAllocationRequestDomainService,
40                                               IssueAllocationResourceDomainService
                                                 issueAllocationResourceDomainService,
41                                               ReturnAllocationResourceDomainService
                                                 returnAllocationResourceDomainService,
42                                               MarkAllocationRequestOverdueDomainService
                                                 markAllocationRequestOverdueDomainService) {
43         this.requestRepository = requestRepository;
44         this.eventPublisher = eventPublisher;
45         this.approveAllocationRequestDomainService = approveAllocationRequestDomainService;
46         this.rejectAllocationRequestDomainService = rejectAllocationRequestDomainService;
47         this.issueAllocationResourceDomainService = issueAllocationResourceDomainService;
48         this.returnAllocationResourceDomainService = returnAllocationResourceDomainService;
49         this.markAllocationRequestOverdueDomainService = markAllocationRequestOverdueDomainService;
50     }
51
52     @Override
53     @Transactional(readOnly = true)
54     public Optional<AllocationRequestView> getRequest(AllocationRequestId requestId) {
55         return requestRepository.find(requestId).map(AllocationRequestView::from);
56     }
57
58     @Override
59     @Transactional(readOnly = true)
60     public List<AllocationRequestView> listRequests(RequestStatus status) {
61         List<AllocationRequest> requests = status == null ? requestRepository.findAll() :
           requestRepository.findAll(status);
62         return requests.stream().map(AllocationRequestView::from).toList();
63     }
64
65     @Override
66     public void approveRequest(AllocationRequestId requestId) {
67         mutate(requestId, approveAllocationRequestDomainService::approveAllocationRequest);
68     }
69
70     @Override
71     public void rejectRequest(AllocationRequestId requestId, String reason) {
72         mutate(requestId, request -> rejectAllocationRequestDomainService.rejectAllocationRequest(request,
           reason));
73     }
74
75     @Override
76     public void issueResource(AllocationRequestId requestId) {
77         mutate(requestId, issueAllocationResourceDomainService::issueAllocationResource);
78     }
79
80     @Override
81     public void returnResource(AllocationRequestId requestId) {
82         mutate(requestId, returnAllocationResourceDomainService::returnAllocationResource);
83     }
```

```
84
85      @Override
86      @Transactional(readOnly = true)
87      public List<AllocationRequestView> monitorActiveAllocations() {
88          return requestRepository.findAll(RequestStatus.ACTIVE).stream()
89                  .map(AllocationRequestView::from)
90                  .toList();
91      }
92
93      @Override
94      public void detectOverdueAllocations() {
95          List<AllocationRequest> overdue = requestRepository.findOverdue(LocalDateTime.now());
96          for (AllocationRequest request : overdue) {
97              markAllocationRequestOverdueDomainService.markAllocationRequestOverdue(request);
98              requestRepository.save(request);
99              publishAndClear(request);
100         }
101     }
102
103     private void mutate(AllocationRequestId requestId, java.util.function.Consumer<AllocationRequest>
        mutation) {
104         AllocationRequest request = requestRepository.find(requestId)
105                 .orElseThrow(() -> new NoSuchElementException("Allocation request not found: " + requestId));
106         mutation.accept(request);
107         requestRepository.save(request);
108         publishAndClear(request);
109     }
110
111     private void publishAndClear(AllocationRequest request) {
112         for (DomainEvent event : request.flushEvents()) {
113             eventPublisher.publish(event);
114         }
115     }
116 }
```

Source code 6.33: allocation/application/service/AllocationRequestAppService.java - AllocationRequestAppService

```
1  package com.deskflow.allocation.application.service;
2
3  import com.deskflow.allocation.application.port.in.AllocationRequestInputPort;
4  import com.deskflow.allocation.application.model.AllocationRequestView;
5  import com.deskflow.allocation.application.model.ResourceCatalogItem;
6  import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
7  import com.deskflow.allocation.application.port.out.EmployeeProfilePort;
8  import com.deskflow.allocation.application.port.out.ResourceCatalogPort;
9  import com.deskflow.common.application.domain.model.vo.EmployeeId;
10 import com.deskflow.common.application.domain.DomainEvent;
11 import com.deskflow.allocation.application.domain.model.AllocationRequest;
12 import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
13 import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
14 import com.deskflow.allocation.application.domain.service.CancelAllocationRequestDomainService;
15 import com.deskflow.allocation.application.domain.service.EnsureEmployeeWithinLimitDomainService;
16 import com.deskflow.allocation.application.domain.service.EnsureResourceAvailableInPeriodDomainService;
17 import com.deskflow.allocation.application.domain.service.EnsureResourceEligibleForAllocationDomainService;
18 import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
19 import com.deskflow.allocation.application.domain.model.vo.DateRange;
20 import com.deskflow.common.application.port.out.DomainEventPublisherPort;
21 import com.deskflow.common.application.domain.model.vo.ResourceId;
22 import org.springframework.stereotype.Service;
23 import org.springframework.transaction.annotation.Transactional;
24
25 import java.time.LocalDate;
26 import java.util.List;
27 import java.util.NoSuchElementException;
28
29 @Service
30 @Transactional
```

```
31 public class AllocationRequestAppService implements AllocationRequestInputPort {
32
33     private final AllocationRequestRepositoryPort requestRepository;
34     private final ResourceCatalogPort resourceCatalogPort;
35     private final EmployeeProfilePort employeeProfilePort;
36     private final AllocationRequestFactory factory;
37     private final CancelAllocationRequestDomainService cancelAllocationRequestDomainService;
38     private final EnsureResourceEligibleForAllocationDomainService
       ensureResourceEligibleForAllocationDomainService;
39     private final EnsureResourceAvailableInPeriodDomainService ensureResourceAvailableInPeriodDomainService;
40     private final EnsureEmployeeWithinLimitDomainService ensureEmployeeWithinLimitDomainService;
41     private final DomainEventPublisherPort eventPublisher;
42
43     public AllocationRequestAppService(AllocationRequestRepositoryPort requestRepository,
44                                        ResourceCatalogPort resourceCatalogPort,
45                                        EmployeeProfilePort employeeProfilePort,
46                                        AllocationRequestFactory factory,
47                                        CancelAllocationRequestDomainService
                                           cancelAllocationRequestDomainService,
48                                        EnsureResourceEligibleForAllocationDomainService
                                           ensureResourceEligibleForAllocationDomainService,
49                                        EnsureResourceAvailableInPeriodDomainService
                                           ensureResourceAvailableInPeriodDomainService,
50                                        EnsureEmployeeWithinLimitDomainService
                                           ensureEmployeeWithinLimitDomainService,
51                                        DomainEventPublisherPort eventPublisher) {
52         this.requestRepository = requestRepository;
53         this.resourceCatalogPort = resourceCatalogPort;
54         this.employeeProfilePort = employeeProfilePort;
55         this.factory = factory;
56         this.cancelAllocationRequestDomainService = cancelAllocationRequestDomainService;
57         this.ensureResourceEligibleForAllocationDomainService =
           ensureResourceEligibleForAllocationDomainService;
58         this.ensureResourceAvailableInPeriodDomainService = ensureResourceAvailableInPeriodDomainService;
59         this.ensureEmployeeWithinLimitDomainService = ensureEmployeeWithinLimitDomainService;
60         this.eventPublisher = eventPublisher;
61     }
62
63     @Override
64     public AllocationRequestId submitRequest(EmployeeId employeeId, ResourceId resourceId,
65                                              LocalDate dateFrom, LocalDate dateTo, String justification) {
66         var employee = employeeProfilePort.findEmployee(employeeId)
67                 .orElseThrow(() -> new IllegalArgumentException("Employee not found: " + employeeId));
68         ResourceAllocationProfile requestedResource = resourceCatalogPort.findResource(resourceId)
69                 .orElseThrow(() -> new IllegalArgumentException("Resource not found: " + resourceId));
70
71
           ensureResourceEligibleForAllocationDomainService.ensureResourceEligibleForAllocation(requestedResource);
72
73         String requestedType = requestedResource.type();
74         long currentActiveForType = requestRepository.findActiveForEmployee(employeeId).stream()
75                 .map(AllocationRequest::getResourceId)
76                 .map(resourceCatalogPort::findResource)
77                 .flatMap(java.util.Optional::stream)
78                 .filter(resource -> resource.type().equals(requestedType))
79                 .count();
80
81         ensureEmployeeWithinLimitDomainService.ensureEmployeeWithinLimit(employee, requestedType,
           currentActiveForType);
82
83         DateRange dateRange = new DateRange(dateFrom, dateTo);
84
85         List<AllocationRequest> overlappingRequests = requestRepository.findActiveForResource(resourceId,
           dateRange);
86         ensureResourceAvailableInPeriodDomainService.ensureResourceAvailableInPeriod(overlappingRequests);
87
88         AllocationRequest request = factory.create(employeeId, resourceId, dateRange, justification);
89         requestRepository.save(request);
90         publishAndClear(request);
91         return request.getId();
```

```
92      }
93
94      @Override
95      public void cancelRequest(AllocationRequestId requestId, EmployeeId employeeId) {
96          AllocationRequest request = findRequest(requestId);
97          cancelAllocationRequestDomainService.cancelAllocationRequest(request, employeeId);
98          requestRepository.save(request);
99          publishAndClear(request);
100     }
101
102     @Override
103     @Transactional(readOnly = true)
104     public List<ResourceCatalogItem> browseResources() {
105         return resourceCatalogPort.findActiveResources().stream()
106                 .map(ResourceCatalogItem::from)
107                 .toList();
108     }
109
110     @Override
111     @Transactional(readOnly = true)
112     public List<AllocationRequestView> browseHistory(EmployeeId employeeId) {
113         return requestRepository.findForEmployee(employeeId).stream()
114                 .map(AllocationRequestView::from)
115                 .toList();
116     }
117
118     @Override
119     @Transactional(readOnly = true)
120     public List<AllocationRequestView> findActiveForResource(ResourceId resourceId) {
121         return requestRepository.findActiveForResource(resourceId).stream()
122                 .map(AllocationRequestView::from)
123                 .toList();
124     }
125
126     private AllocationRequest findRequest(AllocationRequestId id) {
127         return requestRepository.find(id)
128                 .orElseThrow(() -> new NoSuchElementException("Allocation request not found: " + id));
129     }
130
131     private void publishAndClear(AllocationRequest request) {
132         for (DomainEvent event : request.flushEvents()) {
133             eventPublisher.publish(event);
134         }
135     }
136 }
```

Source code 6.34: allocation/infrastructure/in/web/AllocationRequestController.java - AllocationRequestController

```
1  package com.deskflow.allocation.infrastructure.in.web;
2
3  import com.deskflow.allocation.application.port.out.EmployeeProfilePort;
4  import com.deskflow.allocation.infrastructure.in.web.dto.AllocationRequestDTO;
5  import com.deskflow.allocation.infrastructure.in.web.dto.AllocationRequestResponseDTO;
6  import com.deskflow.allocation.application.port.in.AllocationRequestInputPort;
7  import com.deskflow.common.application.domain.model.vo.EmployeeId;
8  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
9  import com.deskflow.common.application.domain.model.vo.ResourceId;
10 import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
11 import com.deskflow.employees.application.domain.model.Employee;
12 import jakarta.validation.Valid;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.security.core.Authentication;
15 import org.springframework.web.bind.annotation.*;
16
17 import java.net.URI;
18 import java.util.List;
19 import java.util.UUID;
```

```
20
21  @RestController
22  @RequestMapping("/api")
23  public class AllocationRequestController {
24
25      private final AllocationRequestInputPort allocationRequestInputPort;
26      private final CurrentUserResolver userResolver;
27      private final EmployeeManagementInputPort employeeManagementInputPort;
28
29      public AllocationRequestController(AllocationRequestInputPort allocationRequestInputPort,
30                                         CurrentUserResolver userResolver,
31                                         EmployeeManagementInputPort employeeManagementInputPort) {
32          this.allocationRequestInputPort = allocationRequestInputPort;
33          this.userResolver = userResolver;
34          this.employeeManagementInputPort = employeeManagementInputPort;
35      }
36
37      @GetMapping("/resources")
38      public ResponseEntity<?> browseResources() {
39          return ResponseEntity.ok(allocationRequestInputPort.browseResources().stream()
40                  .map(r -> new java.util.LinkedHashMap<>(java.util.Map.of(
41                          "id", r.id(),
42                          "type", r.type(),
43                          "name", r.name(),
44                          "building", r.building(),
45                          "floor", r.floor(),
46                          "number", r.number(),
47                          "available", r.available()
48                  )))
49                  .toList());
50      }
51
52      @PostMapping("/requests")
53      public ResponseEntity<?> submitRequest(@Valid @RequestBody AllocationRequestDTO dto, Authentication auth)
        {
54          EmployeeId employeeId = userResolver.resolveEmployeeId(auth);
55          AllocationRequestId id = allocationRequestInputPort.submitRequest(
56                  employeeId, ResourceId.of(dto.resourceId()),
57                  dto.dateFrom(), dto.dateTo(), dto.justification());
58          return ResponseEntity.created(URI.create("/api/requests/" + id))
59                  .body(java.util.Map.of("requestId", id.getValue()));
60      }
61
62      @PutMapping("/requests/{id}/cancel")
63      public ResponseEntity<Void> cancelRequest(@PathVariable UUID id, Authentication auth) {
64          EmployeeId employeeId = userResolver.resolveEmployeeId(auth);
65          allocationRequestInputPort.cancelRequest(AllocationRequestId.of(id), employeeId);
66          return ResponseEntity.ok().build();
67      }
68
69      @GetMapping("/requests/my")
70      public ResponseEntity<List<AllocationRequestResponseDTO>> browseHistory(Authentication auth) {
71          EmployeeId employeeId = userResolver.resolveEmployeeId(auth);
72          Employee employee = employeeManagementInputPort.getEmployee(employeeId).orElse(null);
73          List<AllocationRequestResponseDTO> history = allocationRequestInputPort.browseHistory(employeeId)
74                  .stream()
75                  .map(r -> AllocationRequestRestMapper.toResponse(r,
76                          employee != null ? employee.getFirstName() : null,
77                          employee != null ? employee.getLastName() : null))
78                  .toList();
79          return ResponseEntity.ok(history);
80      }
81  }
```

Source code 6.35: allocation/infrastructure/in/web/AllocationRequestRestMapper.java - AllocationRequestRestMapper

```
1  package com.deskflow.allocation.infrastructure.in.web;
```

```
2
3  import com.deskflow.allocation.application.model.AllocationRequestView;
4  import com.deskflow.allocation.infrastructure.in.web.dto.AllocationRequestResponseDTO;
5
6  public class AllocationRequestRestMapper {
7
8      public static AllocationRequestResponseDTO toResponse(AllocationRequestView r) {
9          return toResponse(r, null, null);
10     }
11
12     public static AllocationRequestResponseDTO toResponse(AllocationRequestView r, String firstName, String
       lastName) {
13         return new AllocationRequestResponseDTO(
14                 r.id(),
15                 r.employeeId(),
16                 firstName,
17                 lastName,
18                 r.resourceId(),
19                 r.status(),
20                 r.dateFrom(),
21                 r.dateTo(),
22                 r.justification(),
23                 r.submittedAt(),
24                 r.issuedAt(),
25                 r.returnedAt(),
26                 r.rejectionReason()
27         );
28     }
29 }
```

Source code 6.36: allocation/infrastructure/in/web/CurrentUserResolver.java - CurrentUserResolver

```
1  package com.deskflow.allocation.infrastructure.in.web;
2
3  import com.deskflow.common.application.domain.model.vo.EmployeeId;
4  import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
5  import org.springframework.security.core.Authentication;
6  import org.springframework.stereotype.Component;
7
8  @Component
9  public class CurrentUserResolver {
10
11     private final EmployeeManagementInputPort employeeManagementInputPort;
12
13     public CurrentUserResolver(EmployeeManagementInputPort employeeManagementInputPort) {
14         this.employeeManagementInputPort = employeeManagementInputPort;
15     }
16
17     public EmployeeId resolveEmployeeId(Authentication auth) {
18         return employeeManagementInputPort.getEmployeeByEmail(auth.getName())
19                 .map(e -> e.getId())
20                 .orElseThrow(() -> new IllegalStateException("Employee not found for: " + auth.getName()));
21     }
22 }
```

Source code 6.37: allocation/infrastructure/in/web/dto/AllocationRequestDTO.java - AllocationRequestDTO

```
1  package com.deskflow.allocation.infrastructure.in.web.dto;
2
3  import jakarta.validation.constraints.FutureOrPresent;
4  import jakarta.validation.constraints.NotNull;
5
6  import java.time.LocalDate;
```

```
import java.util.UUID;

public record AllocationRequestDTO(
        @NotNull UUID resourceId,
        @NotNull @FutureOrPresent LocalDate dateFrom,
        @NotNull LocalDate dateTo,
        String justification
) {}
```

Source code 6.38: allocation/infrastructure/in/web/dto/AllocationRequestResponseDTO.java - AllocationRequestResponseDTO

```
package com.deskflow.allocation.infrastructure.in.web.dto;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

public record AllocationRequestResponseDTO(
        UUID id,
        UUID employeeId,
        String employeeFirstName,
        String employeeLastName,
        UUID resourceId,
        String status,
        LocalDate dateFrom,
        LocalDate dateTo,
        String justification,
        LocalDateTime submittedAt,
        LocalDateTime issuedAt,
        LocalDateTime returnedAt,
        String rejectionReason
) {}
```

Source code 6.39: allocation/infrastructure/out/employees/EmployeeProfileAdapter.java - EmployeeProfileAdapter

```
package com.deskflow.allocation.infrastructure.out.employees;

import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;
import com.deskflow.allocation.application.port.out.EmployeeProfilePort;
import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import org.springframework.stereotype.Component;

import java.util.Optional;
import java.util.stream.Collectors;

@Component
public class EmployeeProfileAdapter implements EmployeeProfilePort {

    private final EmployeeManagementInputPort employeeManagementInputPort;

    public EmployeeProfileAdapter(EmployeeManagementInputPort employeeManagementInputPort) {
        this.employeeManagementInputPort = employeeManagementInputPort;
    }

    @Override
    public Optional<EmployeeAllocationProfile> findEmployee(EmployeeId employeeId) {
        return employeeManagementInputPort.getEmployee(employeeId)
                .map(employee -> new EmployeeAllocationProfile(
                        employee.getLimits().entrySet().stream()
                                .collect(Collectors.toMap(entry -> entry.getKey().name(),
                                java.util.Map.Entry::getValue))
```

```
                ));
    }
}
```

Source code 6.40: allocation/infrastructure/out/persistence/AllocationRequestEntity.java - AllocationRequestEntity

```
package com.deskflow.allocation.infrastructure.out.persistence;

import jakarta.persistence.*;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "allocation_requests")
class AllocationRequestEntity {

    @Id
    UUID id;

    @Column(name = "employee_id", nullable = false)
    UUID employeeId;

    @Column(name = "resource_id", nullable = false)
    UUID resourceId;

    @Column(nullable = false)
    String status;

    @Column(name = "date_from", nullable = false)
    LocalDate dateFrom;

    @Column(name = "date_to", nullable = false)
    LocalDate dateTo;

    @Column(name = "justification")
    String justification;

    @Column(name = "submitted_at", nullable = false)
    LocalDateTime submittedAt;

    @Column(name = "issued_at")
    LocalDateTime issuedAt;

    @Column(name = "returned_at")
    LocalDateTime returnedAt;

    @Column(name = "rejection_reason")
    String rejectionReason;
}
```

Source code 6.41: allocation/infrastructure/out/persistence/AllocationRequestMapper.java - AllocationRequestMapper

```
package com.deskflow.allocation.infrastructure.out.persistence;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import com.deskflow.common.application.domain.model.vo.ResourceId;
```

```
import org.springframework.stereotype.Component;

@Component
public class AllocationRequestMapper {

    AllocationRequestEntity toEntity(AllocationRequest request) {
        AllocationRequestEntity e = new AllocationRequestEntity();
        updateEntity(request, e);
        return e;
    }

    void updateEntity(AllocationRequest request, AllocationRequestEntity e) {
        e.id = request.getId().getValue();
        e.employeeId = request.getEmployeeId().getValue();
        e.resourceId = request.getResourceId().getValue();
        e.status = request.getStatus().name();
        e.dateFrom = request.getDateRange().getDateFrom();
        e.dateTo = request.getDateRange().getDateTo();
        e.justification = request.getJustification();
        e.submittedAt = request.getSubmittedAt();
        e.issuedAt = request.getIssuedAt();
        e.returnedAt = request.getReturnedAt();
        e.rejectionReason = request.getRejectionReason();
    }

    AllocationRequest toDomain(AllocationRequestEntity e) {
        return AllocationRequest.reconstitute(
                AllocationRequestId.of(e.id),
                EmployeeId.of(e.employeeId),
                ResourceId.of(e.resourceId),
                new DateRange(e.dateFrom, e.dateTo),
                e.justification,
                e.submittedAt,
                RequestStatus.valueOf(e.status),
                e.issuedAt,
                e.returnedAt,
                e.rejectionReason
        );
    }
}
```

Source code 6.42: allocation/infrastructure/out/persistence/HibernateAllocationRequestRepositoryAdapter.java - HibernateAllocationRequestRepositoryAdapter

```
package com.deskflow.allocation.infrastructure.out.persistence;

import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import org.springframework.stereotype.Repository;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Repository
public class HibernateAllocationRequestRepositoryAdapter implements AllocationRequestRepositoryPort {

    private final SpringDataAllocationRequestRepository jpa;
    private final AllocationRequestMapper mapper;

    public HibernateAllocationRequestRepositoryAdapter(SpringDataAllocationRequestRepository jpa,
    AllocationRequestMapper mapper) {
        this.jpa = jpa;
```

```
24          this.mapper = mapper;
25      }
26
27      @Override
28      public void save(AllocationRequest request) {
29          jpa.findById(request.getId().getValue()).ifPresentOrElse(
30                  managed -> mapper.updateEntity(request, managed),
31                  () -> jpa.save(mapper.toEntity(request))
32          );
33      }
34
35      @Override
36      public Optional<AllocationRequest> find(AllocationRequestId id) {
37          return jpa.findById(id.getValue()).map(mapper::toDomain);
38      }
39
40      @Override
41      public List<AllocationRequest> findActiveForResource(ResourceId resourceId, DateRange dateRange) {
42          return jpa.findActiveOverlapping(resourceId.getValue(), dateRange.getDateFrom(),
            dateRange.getDateTo())
43                  .stream().map(mapper::toDomain).toList();
44      }
45
46      @Override
47      public List<AllocationRequest> findActiveForResource(ResourceId resourceId) {
48          return jpa.findActiveByResourceId(resourceId.getValue())
49                  .stream().map(mapper::toDomain).toList();
50      }
51
52      @Override
53      public List<AllocationRequest> findActiveForEmployee(EmployeeId employeeId) {
54          return jpa.findActiveByEmployeeId(employeeId.getValue())
55                  .stream().map(mapper::toDomain).toList();
56      }
57
58      @Override
59      public List<AllocationRequest> findOverdue(LocalDateTime now) {
60          return jpa.findOverdue(now.toLocalDate())
61                  .stream().map(mapper::toDomain).toList();
62      }
63
64      @Override
65      public List<AllocationRequest> findForEmployee(EmployeeId employeeId) {
66          return jpa.findByEmployeeId(employeeId.getValue())
67                  .stream().map(mapper::toDomain).toList();
68      }
69
70      @Override
71      public List<AllocationRequest> findAll(RequestStatus status) {
72          return jpa.findByStatus(status.name())
73                  .stream().map(mapper::toDomain).toList();
74      }
75
76      @Override
77      public List<AllocationRequest> findAll() {
78          return jpa.findAll().stream().map(mapper::toDomain).toList();
79      }
80  }
```

Source code 6.43:
allocation/infrastructure/out/persistence/SpringDataAllocationRequestRepository.java - SpringDataAllocationRequestRepository

```
1   package com.deskflow.allocation.infrastructure.out.persistence;
2
3   import org.springframework.data.jpa.repository.JpaRepository;
4   import org.springframework.data.jpa.repository.Query;
```

```
5  import org.springframework.data.repository.query.Param;
6
7  import java.time.LocalDate;
8  import java.util.List;
9  import java.util.UUID;
10
11 interface SpringDataAllocationRequestRepository extends JpaRepository<AllocationRequestEntity, UUID> {
12
13     List<AllocationRequestEntity> findByEmployeeId(UUID employeeId);
14
15     List<AllocationRequestEntity> findByStatus(String status);
16
17     @Query("""
18             SELECT w FROM AllocationRequestEntity w
19             WHERE w.resourceId = :resourceId
20             AND w.status IN ('SUBMITTED', 'APPROVED', 'ACTIVE', 'OVERDUE')
21             AND w.dateFrom <= :dateTo
22             AND w.dateTo >= :dateFrom
23             """)
24     List<AllocationRequestEntity> findActiveOverlapping(
25             @Param("resourceId") UUID resourceId,
26             @Param("dateFrom") LocalDate dateFrom,
27             @Param("dateTo") LocalDate dateTo
28     );
29
30     @Query("""
31             SELECT w FROM AllocationRequestEntity w
32             WHERE w.employeeId = :employeeId
33             AND w.status IN ('SUBMITTED', 'APPROVED', 'ACTIVE', 'OVERDUE')
34             """)
35     List<AllocationRequestEntity> findActiveByEmployeeId(@Param("employeeId") UUID employeeId);
36
37     @Query("""
38             SELECT w FROM AllocationRequestEntity w
39             WHERE w.resourceId = :resourceId
40             AND w.status IN ('SUBMITTED', 'APPROVED', 'ACTIVE', 'OVERDUE')
41             """)
42     List<AllocationRequestEntity> findActiveByResourceId(@Param("resourceId") UUID resourceId);
43
44     @Query("""
45             SELECT w FROM AllocationRequestEntity w
46             WHERE w.status = 'ACTIVE'
47             AND w.dateTo < :now
48             """)
49     List<AllocationRequestEntity> findOverdue(@Param("now") LocalDate now);
50 }
```

Source code 6.44: allocation/infrastructure/out/resources/ResourceCatalogAdapter.java - ResourceCatalogAdapter

```
1  package com.deskflow.allocation.infrastructure.out.resources;
2
3  import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
4  import com.deskflow.allocation.application.port.out.ResourceCatalogPort;
5  import com.deskflow.resources.application.model.ResourceView;
6  import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
7  import com.deskflow.common.application.domain.model.vo.ResourceId;
8  import org.springframework.stereotype.Component;
9
10 import java.util.List;
11 import java.util.Optional;
12
13 @Component
14 public class ResourceCatalogAdapter implements ResourceCatalogPort {
15
16     private final ResourceManagementInputPort resourceManagementInputPort;
17
18     public ResourceCatalogAdapter(ResourceManagementInputPort resourceManagementInputPort) {
```

```
19          this.resourceManagementInputPort = resourceManagementInputPort;
20      }
21
22      @Override
23      public Optional<ResourceAllocationProfile> findResource(ResourceId resourceId) {
24          return resourceManagementInputPort.getResource(resourceId).map(this::toProfile);
25      }
26
27      @Override
28      public List<ResourceAllocationProfile> findActiveResources() {
29          return resourceManagementInputPort.getActiveResources().stream()
30                  .map(this::toProfile)
31                  .toList();
32      }
33
34      private ResourceAllocationProfile toProfile(ResourceView resource) {
35          return new ResourceAllocationProfile(
36                  ResourceId.of(resource.id()),
37                  resource.type(),
38                  resource.name(),
39                  resource.building(),
40                  resource.floor(),
41                  resource.number(),
42                  resource.active(),
43                  resource.available()
44          );
45      }
46  }
```

6.1.2. Kontekst Resources - zasoby biurowe

Source code 6.45: resources/application/domain/model/OfficeResource.java - OfficeResource

```
1   package com.deskflow.resources.application.domain.model;
2
3   import com.deskflow.common.application.domain.DomainEvent;
4   import com.deskflow.resources.application.domain.model.events.ResourceDeactivated;
5   import com.deskflow.resources.application.domain.model.events.ResourceMadeAvailable;
6   import com.deskflow.resources.application.domain.model.events.ResourceMadeUnavailable;
7   import com.deskflow.resources.application.domain.model.events.ResourceUpdated;
8   import com.deskflow.resources.application.domain.model.vo.Location;
9   import com.deskflow.common.application.domain.model.vo.ResourceType;
10  import com.deskflow.common.application.domain.model.vo.ResourceId;
11
12  import java.time.LocalDateTime;
13  import java.util.ArrayList;
14  import java.util.List;
15
16  public class OfficeResource {
17
18      private final ResourceId id;
19      private final ResourceType type;
20      private String name;
21      private Location location;
22      private boolean active;
23      private boolean available;
24      private final List<DomainEvent> events = new ArrayList<>();
25
26      private OfficeResource(ResourceId id, ResourceType type, String name, Location location,
27                             boolean active, boolean available) {
28          this.id = id;
29          this.type = type;
30          this.name = validateName(name);
31          this.location = location;
32          this.active = active;
```

```
33          this.available = available;
34      }
35
36      public static OfficeResource create(ResourceId id, ResourceType type, String name, Location location) {
37          return new OfficeResource(id, type, name, location, true, true);
38      }
39
40      public static OfficeResource reconstitute(ResourceId id, ResourceType type, String name,
41                                                Location location, boolean active, boolean available) {
42          return new OfficeResource(id, type, name, location, active, available);
43      }
44
45      public void update(String newName, Location newLocation) {
46          if (!active) throw new IllegalStateException("Cannot edit an inactive resource");
47          String prevName = this.name;
48          this.name = validateName(newName);
49          this.location = newLocation;
50          events.add(new ResourceUpdated(id, prevName, newName, LocalDateTime.now()));
51      }
52
53      public void markAsUnavailable() {
54          if (!active) throw new IllegalStateException("Resource is not active");
55          this.available = false;
56          events.add(new ResourceMadeUnavailable(id, LocalDateTime.now()));
57      }
58
59      public void markAsAvailable() {
60          if (!active) throw new IllegalStateException("Resource is not active");
61          this.available = true;
62          events.add(new ResourceMadeAvailable(id, LocalDateTime.now()));
63      }
64
65      public void deactivate() {
66          if (!active) throw new IllegalStateException("Resource is already deactivated");
67          this.active = false;
68          this.available = false;
69          events.add(new ResourceDeactivated(id, LocalDateTime.now()));
70      }
71
72      private String validateName(String name) {
73          if (name == null || name.isBlank()) throw new IllegalArgumentException("Resource name cannot be
            blank");
74          return name;
75      }
76
77      public List<DomainEvent> flushEvents() {
78          var copy = List.copyOf(events);
79          events.clear();
80          return copy;
81      }
82
83      public ResourceId getId() { return id; }
84      public ResourceType getType() { return type; }
85      public String getName() { return name; }
86      public Location getLocation() { return location; }
87      public boolean isActive() { return active; }
88      public boolean isAvailable() { return available; }
89  }
```

Source code 6.46: resources/application/domain/model/events/ResourceDeactivated.java - ResourceDeactivated

```
1   package com.deskflow.resources.application.domain.model.events;
2
3   import com.deskflow.common.application.domain.DomainEvent;
4   import com.deskflow.common.application.domain.model.vo.ResourceId;
5
6   import java.time.LocalDateTime;
```

```
import java.util.UUID;

public record ResourceDeactivated(
        ResourceId resourceId,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "ResourceDeactivated"; }
    @Override
    public UUID aggregateId() { return resourceId.getValue(); }
}
```

Source code 6.47: resources/application/domain/model/events/ResourceMadeAvailable.java - ResourceMadeAvailable

```
package com.deskflow.resources.application.domain.model.events;

import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;
import java.util.UUID;

public record ResourceMadeAvailable(
        ResourceId resourceId,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "ResourceMadeAvailable"; }
    @Override
    public UUID aggregateId() { return resourceId.getValue(); }
}
```

Source code 6.48: resources/application/domain/model/events/ResourceMadeUnavailable.java - ResourceMadeUnavailable

```
package com.deskflow.resources.application.domain.model.events;

import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;
import java.util.UUID;

public record ResourceMadeUnavailable(
        ResourceId resourceId,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "ResourceMadeUnavailable"; }
    @Override
    public UUID aggregateId() { return resourceId.getValue(); }
}
```

Source code 6.49: resources/application/domain/model/events/ResourceUpdated.java - ResourceUpdated

```
package com.deskflow.resources.application.domain.model.events;

import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.time.LocalDateTime;
import java.util.UUID;

public record ResourceUpdated(
        ResourceId resourceId,
        String previousName,
        String newName,
        LocalDateTime occurredAt
) implements DomainEvent {
    @Override
    public UUID eventId() { return UUID.randomUUID(); }
    @Override
    public String eventType() { return "ResourceUpdated"; }
    @Override
    public UUID aggregateId() { return resourceId.getValue(); }
}
```

Source code 6.50: resources/application/domain/model/vo/AllocationUsageStatus.java - AllocationUsageStatus

```
package com.deskflow.resources.application.domain.model.vo;

public enum AllocationUsageStatus {
    REQUESTED,
    APPROVED,
    IN_USE,
    OVERDUE,
    CLOSED
}
```

Source code 6.51: resources/application/domain/model/vo/Location.java - Location

```
package com.deskflow.resources.application.domain.model.vo;

import java.util.Objects;

public final class Location {
    private final String building;
    private final int floor;
    private final String number;

    public Location(String building, int floor, String number) {
        if (building == null || building.isBlank()) throw new IllegalArgumentException("Building cannot be
        blank");
        if (number == null || number.isBlank()) throw new IllegalArgumentException("Number cannot be blank");
        this.building = building;
        this.floor = floor;
        this.number = number;
    }

    public String getBuilding() { return building; }
    public int getFloor() { return floor; }
    public String getNumber() { return number; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Location that)) return false;
        return floor == that.floor && building.equals(that.building) && number.equals(that.number);
```

```
27      }
28
29      @Override
30      public int hashCode() {
31          return Objects.hash(building, floor, number);
32      }
33
34      @Override
35      public String toString() {
36          return building + ", floor " + floor + ", no. " + number;
37      }
38  }
```

Source code 6.52:
resources/application/domain/service/DeactivateOfficeResourceDomainService.java - DeactivateOfficeResourceDomainService

```
1   package com.deskflow.resources.application.domain.service;
2
3   import com.deskflow.resources.application.domain.model.OfficeResource;
4   import com.deskflow.resources.application.domain.model.vo.AllocationUsageStatus;
5
6   import java.util.List;
7
8   public class DeactivateOfficeResourceDomainService {
9
10      public void deactivateOfficeResource(OfficeResource resource, List<AllocationUsageStatus>
        relatedRequestStatuses) {
11          if (!canModifyResource(relatedRequestStatuses)) {
12              throw new IllegalStateException("Cannot deactivate a resource with active allocation requests");
13          }
14          resource.deactivate();
15      }
16
17      private boolean canModifyResource(List<AllocationUsageStatus> relatedRequestStatuses) {
18          return relatedRequestStatuses.stream()
19                  .noneMatch(status -> status == AllocationUsageStatus.IN_USE
20                          || status == AllocationUsageStatus.APPROVED
21                          || status == AllocationUsageStatus.REQUESTED
22                          || status == AllocationUsageStatus.OVERDUE);
23      }
24  }
```

Source code 6.53:
resources/application/domain/service/MarkOfficeResourceAvailableDomainService.java - MarkOfficeResourceAvailableDomainService

```
1   package com.deskflow.resources.application.domain.service;
2
3   import com.deskflow.resources.application.domain.model.OfficeResource;
4
5   public class MarkOfficeResourceAvailableDomainService {
6
7       public void markOfficeResourceAvailable(OfficeResource resource) {
8           resource.markAsAvailable();
9       }
10  }
```

Source code 6.54:
resources/application/domain/service/MarkOfficeResourceUnavailableDomainService.java - MarkOfficeResourceUnavailableDomainService

```
package com.deskflow.resources.application.domain.service;

import com.deskflow.resources.application.domain.model.OfficeResource;

public class MarkOfficeResourceUnavailableDomainService {

    public void markOfficeResourceUnavailable(OfficeResource resource) {
        resource.markAsUnavailable();
    }
}
```

Source code 6.55: resources/application/domain/service/OfficeResourceFactory.java - OfficeResourceFactory

```
package com.deskflow.resources.application.domain.service;

import com.deskflow.resources.application.domain.model.OfficeResource;
import com.deskflow.resources.application.domain.model.vo.Location;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;

import java.util.UUID;

public class OfficeResourceFactory {

    public OfficeResource create(ResourceType type, String name, Location location) {
        return OfficeResource.create(
                ResourceId.of(UUID.randomUUID()),
                type,
                name,
                location
        );
    }
}
```

Source code 6.56: resources/application/domain/service/UpdateOfficeResourceDomainService.java - UpdateOfficeResourceDomainService

```
package com.deskflow.resources.application.domain.service;

import com.deskflow.resources.application.domain.model.OfficeResource;
import com.deskflow.resources.application.domain.model.vo.Location;

public class UpdateOfficeResourceDomainService {

    public void updateOfficeResource(OfficeResource resource, String name, Location location) {
        resource.update(name, location);
    }
}
```

Source code 6.57: resources/application/model/ResourceView.java - ResourceView

```
package com.deskflow.resources.application.model;

import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.resources.application.domain.model.vo.Location;

import java.util.UUID;

```

```
public record ResourceView(
        UUID id,
        String type,
        String name,
        String building,
        int floor,
        String number,
        boolean active,
        boolean available
) {
    public static ResourceView from(ResourceId id, ResourceType type, String name, Location location, boolean
    active, boolean available) {
        return new ResourceView(
                id.getValue(),
                type.name(),
                name,
                location.getBuilding(),
                location.getFloor(),
                location.getNumber(),
                active,
                available
        );
    }
}
```

Source code 6.58: resources/application/port/in/ResourceManagementInputPort.java - ResourceManagementInputPort

```
package com.deskflow.resources.application.port.in;

import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.resources.application.model.ResourceView;

import java.util.List;
import java.util.Optional;

public interface ResourceManagementInputPort {

    ResourceId addResource(ResourceType type, String name, String building, int floor, String number);

    void updateResource(ResourceId resourceId, String name, String building, int floor, String number);

    void markUnavailable(ResourceId resourceId);

    void markAvailable(ResourceId resourceId);

    void deactivateResource(ResourceId resourceId);

    Optional<ResourceView> getResource(ResourceId resourceId);

    List<ResourceView> getActiveResources();
}
```

Source code 6.59: resources/application/port/out/OfficeResourceRepositoryPort.java - OfficeResourceRepositoryPort

```
package com.deskflow.resources.application.port.out;

import com.deskflow.resources.application.domain.model.OfficeResource;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.util.List;
import java.util.Optional;

```

```
public interface OfficeResourceRepositoryPort {

    void save(OfficeResource resource);

    Optional<OfficeResource> find(ResourceId id);

    List<OfficeResource> findActive();
}
```

Source code 6.60: resources/application/port/out/ResourceAllocationRequestPort.java - ResourceAllocationRequestPort

```
package com.deskflow.resources.application.port.out;

import com.deskflow.resources.application.domain.model.vo.AllocationUsageStatus;
import com.deskflow.common.application.domain.model.vo.ResourceId;

import java.util.List;

public interface ResourceAllocationRequestPort {
    List<AllocationUsageStatus> findStatusesForResource(ResourceId resourceId);
}
```

Source code 6.61: resources/application/service/ResourceAppService.java - ResourceAppService

```
package com.deskflow.resources.application.service;

import com.deskflow.common.application.port.out.DomainEventPublisherPort;
import com.deskflow.common.application.domain.DomainEvent;
import com.deskflow.resources.application.model.ResourceView;
import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
import com.deskflow.resources.application.port.out.OfficeResourceRepositoryPort;
import com.deskflow.resources.application.port.out.ResourceAllocationRequestPort;
import com.deskflow.resources.application.domain.model.OfficeResource;
import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
import com.deskflow.resources.application.domain.service.DeactivateOfficeResourceDomainService;
import com.deskflow.resources.application.domain.service.MarkOfficeResourceAvailableDomainService;
import com.deskflow.resources.application.domain.service.MarkOfficeResourceUnavailableDomainService;
import com.deskflow.resources.application.domain.service.UpdateOfficeResourceDomainService;
import com.deskflow.resources.application.domain.model.vo.Location;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.NoSuchElementException;
import java.util.Optional;

@Service
@Transactional
public class ResourceAppService implements ResourceManagementInputPort {

    private final OfficeResourceRepositoryPort resourceRepository;
    private final ResourceAllocationRequestPort allocationRequestPort;
    private final OfficeResourceFactory factory;
    private final UpdateOfficeResourceDomainService updateOfficeResourceDomainService;
    private final MarkOfficeResourceUnavailableDomainService markOfficeResourceUnavailableDomainService;
    private final MarkOfficeResourceAvailableDomainService markOfficeResourceAvailableDomainService;
    private final DeactivateOfficeResourceDomainService deactivateOfficeResourceDomainService;
    private final DomainEventPublisherPort eventPublisher;

    public ResourceAppService(OfficeResourceRepositoryPort resourceRepository,
                              ResourceAllocationRequestPort allocationRequestPort,
```

```
                            OfficeResourceFactory factory,
                            UpdateOfficeResourceDomainService updateOfficeResourceDomainService,
                            MarkOfficeResourceUnavailableDomainService
                            markOfficeResourceUnavailableDomainService,
                            MarkOfficeResourceAvailableDomainService
                            markOfficeResourceAvailableDomainService,
                            DeactivateOfficeResourceDomainService deactivateOfficeResourceDomainService,
                            DomainEventPublisherPort eventPublisher) {
        this.resourceRepository = resourceRepository;
        this.allocationRequestPort = allocationRequestPort;
        this.factory = factory;
        this.updateOfficeResourceDomainService = updateOfficeResourceDomainService;
        this.markOfficeResourceUnavailableDomainService = markOfficeResourceUnavailableDomainService;
        this.markOfficeResourceAvailableDomainService = markOfficeResourceAvailableDomainService;
        this.deactivateOfficeResourceDomainService = deactivateOfficeResourceDomainService;
        this.eventPublisher = eventPublisher;
    }

    @Override
    public ResourceId addResource(ResourceType type, String name, String building, int floor, String number)
    {
        Location location = new Location(building, floor, number);
        OfficeResource resource = factory.create(type, name, location);
        resourceRepository.save(resource);
        return resource.getId();
    }

    @Override
    public void updateResource(ResourceId resourceId, String name, String building, int floor, String number)
    {
        OfficeResource resource = findResource(resourceId);
        Location location = new Location(building, floor, number);
        updateOfficeResourceDomainService.updateOfficeResource(resource, name, location);
        resourceRepository.save(resource);
        publishAndClear(resource);
    }

    @Override
    public void markUnavailable(ResourceId resourceId) {
        OfficeResource resource = findResource(resourceId);
        markOfficeResourceUnavailableDomainService.markOfficeResourceUnavailable(resource);
        resourceRepository.save(resource);
        publishAndClear(resource);
    }

    @Override
    public void markAvailable(ResourceId resourceId) {
        OfficeResource resource = findResource(resourceId);
        markOfficeResourceAvailableDomainService.markOfficeResourceAvailable(resource);
        resourceRepository.save(resource);
        publishAndClear(resource);
    }

    @Override
    public void deactivateResource(ResourceId resourceId) {
        OfficeResource resource = findResource(resourceId);
        deactivateOfficeResourceDomainService.deactivateOfficeResource(resource,
        allocationRequestPort.findStatusesForResource(resourceId));
        resourceRepository.save(resource);
        publishAndClear(resource);
    }

    @Override
    @Transactional(readOnly = true)
    public Optional<ResourceView> getResource(ResourceId resourceId) {
        return resourceRepository.find(resourceId).map(this::toView);
    }

    @Override
    @Transactional(readOnly = true)
```

```
105     public List<ResourceView> getActiveResources() {
106         return resourceRepository.findActive().stream().map(this::toView).toList();
107     }
108 
109     private ResourceView toView(OfficeResource r) {
110         return ResourceView.from(r.getId(), r.getType(), r.getName(), r.getLocation(), r.isActive(),
        r.isAvailable());
111     }
112 
113     private OfficeResource findResource(ResourceId id) {
114         return resourceRepository.find(id)
115                 .orElseThrow(() -> new NoSuchElementException("Resource not found: " + id));
116     }
117 
118     private void publishAndClear(OfficeResource resource) {
119         for (DomainEvent event : resource.flushEvents()) {
120             eventPublisher.publish(event);
121         }
122     }
123 }
```

Source code 6.62: resources/infrastructure/in/events/ResourceAvailabilityEventListener.java - ResourceAvailabilityEventListener

```
1  package com.deskflow.resources.infrastructure.in.events;
2  
3  import com.deskflow.allocation.application.domain.model.events.ResourceIssued;
4  import com.deskflow.allocation.application.domain.model.events.ResourceReturned;
5  import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
6  import org.springframework.stereotype.Component;
7  import org.springframework.transaction.event.TransactionPhase;
8  import org.springframework.transaction.event.TransactionalEventListener;
9  
10 @Component
11 public class ResourceAvailabilityEventListener {
12 
13     private final ResourceManagementInputPort resourceManagementInputPort;
14 
15     public ResourceAvailabilityEventListener(ResourceManagementInputPort resourceManagementInputPort) {
16         this.resourceManagementInputPort = resourceManagementInputPort;
17     }
18 
19     @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
20     public void onResourceIssued(ResourceIssued event) {
21         resourceManagementInputPort.markUnavailable(event.resourceId());
22     }
23 
24     @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
25     public void onResourceReturned(ResourceReturned event) {
26         resourceManagementInputPort.markAvailable(event.resourceId());
27     }
28 }
```

Source code 6.63: resources/infrastructure/in/web/ResourceController.java - ResourceController

```
1  package com.deskflow.resources.infrastructure.in.web;
2  
3  import com.deskflow.resources.infrastructure.in.web.dto.ResourceRequestDTO;
4  import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
5  import com.deskflow.common.application.domain.model.vo.ResourceId;
6  import com.deskflow.common.application.domain.model.vo.ResourceType;
7  import jakarta.validation.Valid;
8  import org.springframework.http.ResponseEntity;
```

```
9  import org.springframework.web.bind.annotation.*;
10
11 import java.net.URI;
12 import java.util.Map;
13 import java.util.UUID;
14
15 @RestController
16 @RequestMapping("/api/resources")
17 public class ResourceController {
18
19     private final ResourceManagementInputPort resourceManagementInputPort;
20
21     public ResourceController(ResourceManagementInputPort resourceManagementInputPort) {
22         this.resourceManagementInputPort = resourceManagementInputPort;
23     }
24
25     @PostMapping
26     public ResponseEntity<?> addResource(@Valid @RequestBody ResourceRequestDTO dto) {
27         ResourceId id = resourceManagementInputPort.addResource(
28                 ResourceType.valueOf(dto.type()), dto.name(), dto.building(), dto.floor(), dto.number());
29         return ResponseEntity.created(URI.create("/api/resources/" + id))
30                 .body(Map.of("resourceId", id.getValue()));
31     }
32
33     @PutMapping("/{id}")
34     public ResponseEntity<Void> updateResource(@PathVariable UUID id, @Valid @RequestBody ResourceRequestDTO
       dto) {
35         resourceManagementInputPort.updateResource(ResourceId.of(id), dto.name(), dto.building(),
           dto.floor(), dto.number());
36         return ResponseEntity.ok().build();
37     }
38
39     @PutMapping("/{id}/unavailable")
40     public ResponseEntity<Void> markUnavailable(@PathVariable UUID id) {
41         resourceManagementInputPort.markUnavailable(ResourceId.of(id));
42         return ResponseEntity.ok().build();
43     }
44
45     @PutMapping("/{id}/available")
46     public ResponseEntity<Void> markAvailable(@PathVariable UUID id) {
47         resourceManagementInputPort.markAvailable(ResourceId.of(id));
48         return ResponseEntity.ok().build();
49     }
50
51     @DeleteMapping("/{id}")
52     public ResponseEntity<Void> deactivate(@PathVariable UUID id) {
53         resourceManagementInputPort.deactivateResource(ResourceId.of(id));
54         return ResponseEntity.ok().build();
55     }
56 }
```

Source code 6.64: resources/infrastructure/in/web/dto/ResourceRequestDTO.java - ResourceRequestDTO

```
1  package com.deskflow.resources.infrastructure.in.web.dto;
2
3  import jakarta.validation.constraints.NotBlank;
4  import jakarta.validation.constraints.NotNull;
5
6  public record ResourceRequestDTO(
7          @NotNull String type,
8          @NotBlank String name,
9          @NotBlank String building,
10         int floor,
11         @NotBlank String number
12 ) {}
```

Source code 6.65: resources/infrastructure/out/allocation/AllocationRequestAdapter.java - AllocationRequestAdapter

```
package com.deskflow.resources.infrastructure.out.allocation;

import com.deskflow.allocation.application.port.in.AllocationRequestInputPort;
import com.deskflow.resources.application.port.out.ResourceAllocationRequestPort;
import com.deskflow.resources.application.domain.model.vo.AllocationUsageStatus;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
public class AllocationRequestAdapter implements ResourceAllocationRequestPort {

    private final AllocationRequestInputPort allocationRequestInputPort;

    public AllocationRequestAdapter(@Lazy AllocationRequestInputPort allocationRequestInputPort) {
        this.allocationRequestInputPort = allocationRequestInputPort;
    }

    @Override
    public List<AllocationUsageStatus> findStatusesForResource(ResourceId resourceId) {
        return allocationRequestInputPort.findActiveForResource(resourceId).stream()
                .map(request -> request.status())
                .map(this::toUsageStatus)
                .toList();
    }

    private AllocationUsageStatus toUsageStatus(String status) {
        return switch (status) {
            case "SUBMITTED" -> AllocationUsageStatus.REQUESTED;
            case "APPROVED" -> AllocationUsageStatus.APPROVED;
            case "ACTIVE" -> AllocationUsageStatus.IN_USE;
            case "OVERDUE" -> AllocationUsageStatus.OVERDUE;
            case "REJECTED", "CANCELLED", "COMPLETED" -> AllocationUsageStatus.CLOSED;
            default -> throw new IllegalArgumentException("Unknown allocation status: " + status);
        };
    }
}
```

Source code 6.66: resources/infrastructure/out/persistence/HibernateOfficeResourceRepositoryAdapter.java - HibernateOfficeResourceRepositoryAdapter

```
package com.deskflow.resources.infrastructure.out.persistence;

import com.deskflow.resources.application.port.out.OfficeResourceRepositoryPort;
import com.deskflow.resources.application.domain.model.OfficeResource;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public class HibernateOfficeResourceRepositoryAdapter implements OfficeResourceRepositoryPort {

    private final SpringDataOfficeResourceRepository jpa;
    private final OfficeResourceMapper mapper;

    public HibernateOfficeResourceRepositoryAdapter(SpringDataOfficeResourceRepository jpa,
    OfficeResourceMapper mapper) {
        this.jpa = jpa;
```

```
        this.mapper = mapper;
    }

    @Override
    public void save(OfficeResource resource) {
        jpa.findById(resource.getId().getValue()).ifPresentOrElse(
                managed -> mapper.updateEntity(resource, managed),
                () -> jpa.save(mapper.toEntity(resource))
        );
    }

    @Override
    public Optional<OfficeResource> find(ResourceId id) {
        return jpa.findById(id.getValue()).map(mapper::toDomain);
    }

    @Override
    public List<OfficeResource> findActive() {
        return jpa.findByActiveTrue().stream().map(mapper::toDomain).toList();
    }
}
```

Source code 6.67: resources/infrastructure/out/persistence/OfficeResourceEntity.java - OfficeResourceEntity

```
package com.deskflow.resources.infrastructure.out.persistence;

import jakarta.persistence.*;

import java.util.UUID;

@Entity
@Table(name = "office_resources")
public class OfficeResourceEntity {

    @Id
    private UUID id;

    @Column(name = "type", nullable = false)
    private String type;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "building", nullable = false)
    private String building;

    @Column(name = "floor", nullable = false)
    private int floor;

    @Column(name = "number", nullable = false)
    private String number;

    @Column(name = "active", nullable = false)
    private boolean active;

    @Column(name = "available", nullable = false)
    private boolean available;

    public UUID getId() { return id; }
    public void setId(UUID id) { this.id = id; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getBuilding() { return building; }
    public void setBuilding(String building) { this.building = building; }
    public int getFloor() { return floor; }
```

```
    public void setFloor(int floor) { this.floor = floor; }
    public String getNumber() { return number; }
    public void setNumber(String number) { this.number = number; }
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }
}
```

Source code 6.68: resources/infrastructure/out/persistence/OfficeResourceMapper.java - OfficeResourceMapper

```
package com.deskflow.resources.infrastructure.out.persistence;

import com.deskflow.resources.application.domain.model.OfficeResource;
import com.deskflow.resources.application.domain.model.vo.Location;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import org.springframework.stereotype.Component;

@Component
class OfficeResourceMapper {

    OfficeResourceEntity toEntity(OfficeResource resource) {
        OfficeResourceEntity e = new OfficeResourceEntity();
        updateEntity(resource, e);
        return e;
    }

    void updateEntity(OfficeResource resource, OfficeResourceEntity e) {
        e.setId(resource.getId().getValue());
        e.setType(resource.getType().name());
        e.setName(resource.getName());
        e.setBuilding(resource.getLocation().getBuilding());
        e.setFloor(resource.getLocation().getFloor());
        e.setNumber(resource.getLocation().getNumber());
        e.setActive(resource.isActive());
        e.setAvailable(resource.isAvailable());
    }

    OfficeResource toDomain(OfficeResourceEntity e) {
        return OfficeResource.reconstitute(
                ResourceId.of(e.getId()),
                ResourceType.valueOf(e.getType()),
                e.getName(),
                new Location(e.getBuilding(), e.getFloor(), e.getNumber()),
                e.isActive(),
                e.isAvailable()
        );
    }
}
```

Source code 6.69: resources/infrastructure/out/persistence/SpringDataOfficeResourceRepository.java - SpringDataOfficeResourceRepository

```
package com.deskflow.resources.infrastructure.out.persistence;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.UUID;

public interface SpringDataOfficeResourceRepository extends JpaRepository<OfficeResourceEntity, UUID> {
    List<OfficeResourceEntity> findByActiveTrue();
}
```

6.1.3. Kontekst Administration - administracja

Source code 6.70:
administration/application/domain/model/AdministeredAllocationRequest.java - AdministeredAllocationRequest

```
package com.deskflow.administration.application.domain.model;

public record AdministeredAllocationRequest(
        AdministeredAllocationRequestId id,
        AdministeredAllocationStatus status
) {
    public void ensureCanApprove() {
        requireStatus(AdministeredAllocationStatus.SUBMITTED, "approve");
    }

    public void ensureCanReject() {
        requireStatus(AdministeredAllocationStatus.SUBMITTED, "reject");
    }

    public void ensureCanIssue() {
        requireStatus(AdministeredAllocationStatus.APPROVED, "issue resource");
    }

    public void ensureCanReturn() {
        if (status != AdministeredAllocationStatus.ACTIVE && status != AdministeredAllocationStatus.OVERDUE)
        {
            throw new IllegalStateException(
                    "Operation 'return resource' requires ACTIVE or OVERDUE status, current: " + status);
        }
    }

    private void requireStatus(AdministeredAllocationStatus required, String operation) {
        if (status != required) {
            throw new IllegalStateException(
                    "Operation '" + operation + "' requires status " + required + ", current: " + status);
        }
    }
}
```

Source code 6.71:
administration/application/domain/model/AdministeredAllocationRequestId.java - AdministeredAllocationRequestId

```
package com.deskflow.administration.application.domain.model;

import java.util.Objects;
import java.util.UUID;

public final class AdministeredAllocationRequestId {
    private final UUID value;

    private AdministeredAllocationRequestId(UUID value) {
        this.value = Objects.requireNonNull(value, "value");
    }

    public static AdministeredAllocationRequestId of(UUID value) {
        return new AdministeredAllocationRequestId(value);
    }

    public UUID getValue() {
        return value;
    }

```

```
21      @Override
22      public boolean equals(Object o) {
23          if (this == o) return true;
24          if (!(o instanceof AdministeredAllocationRequestId that)) return false;
25          return value.equals(that.value);
26      }
27
28      @Override
29      public int hashCode() {
30          return value.hashCode();
31      }
32
33      @Override
34      public String toString() {
35          return value.toString();
36      }
37  }
```

Source code 6.72: administration/application/domain/model/AdministeredAllocationStatus.java - AdministeredAllocationStatus

```
1   package com.deskflow.administration.application.domain.model;
2
3   public enum AdministeredAllocationStatus {
4       SUBMITTED,
5       APPROVED,
6       ACTIVE,
7       OVERDUE,
8       REJECTED,
9       CANCELLED,
10      COMPLETED;
11
12      public static AdministeredAllocationStatus from(String status) {
13          return AdministeredAllocationStatus.valueOf(status);
14      }
15  }
```

Source code 6.73: administration/application/model/AdministeredAllocationView.java - AdministeredAllocationView

```
1   package com.deskflow.administration.application.model;
2
3   import java.time.LocalDate;
4   import java.time.LocalDateTime;
5   import java.util.UUID;
6
7   public record AdministeredAllocationView(
8           UUID id,
9           UUID employeeId,
10          UUID resourceId,
11          String status,
12          LocalDate dateFrom,
13          LocalDate dateTo,
14          String justification,
15          LocalDateTime submittedAt,
16          LocalDateTime issuedAt,
17          LocalDateTime returnedAt,
18          String rejectionReason
19  ) {}
```

Source code 6.74: administration/application/port/in/AdministrationInputPort.java - AdministrationInputPort

```java
package com.deskflow.administration.application.port.in;

import com.deskflow.administration.application.model.AdministeredAllocationView;
import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;

import java.util.List;

public interface AdministrationInputPort {

    List<AdministeredAllocationView> listRequests(AdministeredAllocationStatus status);

    void approveRequest(AdministeredAllocationRequestId requestId);

    void rejectRequest(AdministeredAllocationRequestId requestId, String reason);

    void issueResource(AdministeredAllocationRequestId requestId);

    void returnResource(AdministeredAllocationRequestId requestId);

    List<AdministeredAllocationView> monitorActiveAllocations();

    void detectOverdueAllocations();
}
```

Source code 6.75: administration/application/port/out/AdministrationAllocationPort.java - AdministrationAllocationPort

```java
package com.deskflow.administration.application.port.out;

import com.deskflow.administration.application.model.AdministeredAllocationView;
import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;

import java.util.List;
import java.util.Optional;

public interface AdministrationAllocationPort {
    Optional<AdministeredAllocationView> getRequest(AdministeredAllocationRequestId requestId);

    List<AdministeredAllocationView> listRequests(AdministeredAllocationStatus status);

    void approveRequest(AdministeredAllocationRequestId requestId);

    void rejectRequest(AdministeredAllocationRequestId requestId, String reason);

    void issueResource(AdministeredAllocationRequestId requestId);

    void returnResource(AdministeredAllocationRequestId requestId);

    List<AdministeredAllocationView> monitorActiveAllocations();

    void detectOverdueAllocations();
}
```

Source code 6.76: administration/application/service/AdministrationAppService.java - AdministrationAppService

```java
package com.deskflow.administration.application.service;

import com.deskflow.administration.application.model.AdministeredAllocationView;
import com.deskflow.administration.application.port.in.AdministrationInputPort;
import com.deskflow.administration.application.port.out.AdministrationAllocationPort;
```

```
import com.deskflow.administration.application.domain.model.AdministeredAllocationRequest;
import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.NoSuchElementException;

@Service
@Transactional
public class AdministrationAppService implements AdministrationInputPort {

    private final AdministrationAllocationPort allocationPort;

    public AdministrationAppService(AdministrationAllocationPort allocationPort) {
        this.allocationPort = allocationPort;
    }

    @Override
    @Transactional(readOnly = true)
    public List<AdministeredAllocationView> listRequests(AdministeredAllocationStatus status) {
        return allocationPort.listRequests(status);
    }

    @Override
    public void approveRequest(AdministeredAllocationRequestId requestId) {
        loadRequest(requestId).ensureCanApprove();
        allocationPort.approveRequest(requestId);
    }

    @Override
    public void rejectRequest(AdministeredAllocationRequestId requestId, String reason) {
        loadRequest(requestId).ensureCanReject();
        if (reason == null || reason.isBlank()) {
            throw new IllegalArgumentException("Rejection reason cannot be blank");
        }
        allocationPort.rejectRequest(requestId, reason);
    }

    @Override
    public void issueResource(AdministeredAllocationRequestId requestId) {
        loadRequest(requestId).ensureCanIssue();
        allocationPort.issueResource(requestId);
    }

    @Override
    public void returnResource(AdministeredAllocationRequestId requestId) {
        loadRequest(requestId).ensureCanReturn();
        allocationPort.returnResource(requestId);
    }

    @Override
    @Transactional(readOnly = true)
    public List<AdministeredAllocationView> monitorActiveAllocations() {
        return allocationPort.monitorActiveAllocations();
    }

    @Override
    public void detectOverdueAllocations() {
        allocationPort.detectOverdueAllocations();
    }

    private AdministeredAllocationRequest loadRequest(AdministeredAllocationRequestId requestId) {
        AdministeredAllocationView view = allocationPort.getRequest(requestId)
                .orElseThrow(() -> new NoSuchElementException("Allocation request not found: " + requestId));
        return new AdministeredAllocationRequest(
                requestId,
                AdministeredAllocationStatus.from(view.status())
        );
```

```
76      }
77  }
```

Source code 6.77: administration/application/service/OverdueAllocationScheduler.java - OverdueAllocationScheduler

```
1   package com.deskflow.administration.application.service;
2
3   import org.springframework.scheduling.annotation.Scheduled;
4   import org.springframework.stereotype.Component;
5
6   @Component
7   public class OverdueAllocationScheduler {
8
9       private final AdministrationAppService administrationService;
10
11      public OverdueAllocationScheduler(AdministrationAppService administrationService) {
12          this.administrationService = administrationService;
13      }
14
15      @Scheduled(fixedRateString = "${app.scheduler.przeterminowania-rate-ms:900000}")
16      public void detectOverdueAllocations() {
17          administrationService.detectOverdueAllocations();
18      }
19  }
```

Source code 6.78: administration/infrastructure/in/web/AdministrationController.java - AdministrationController

```
1   package com.deskflow.administration.infrastructure.in.web;
2
3   import com.deskflow.administration.application.model.AdministeredAllocationView;
4   import com.deskflow.administration.infrastructure.in.web.dto.RejectRequestDTO;
5   import com.deskflow.allocation.infrastructure.in.web.dto.AllocationRequestResponseDTO;
6   import com.deskflow.administration.application.port.in.AdministrationInputPort;
7   import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
8   import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;
9   import com.deskflow.common.application.domain.model.vo.EmployeeId;
10  import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
11  import com.deskflow.employees.application.domain.model.Employee;
12  import jakarta.validation.Valid;
13  import org.springframework.http.ResponseEntity;
14  import org.springframework.web.bind.annotation.*;
15
16  import java.util.List;
17  import java.util.Map;
18  import java.util.UUID;
19  import java.util.stream.Collectors;
20
21  @RestController
22  @RequestMapping("/api/administration")
23  public class AdministrationController {
24
25      private final AdministrationInputPort administrationInputPort;
26      private final EmployeeManagementInputPort employeeManagementInputPort;
27
28      public AdministrationController(AdministrationInputPort administrationInputPort,
29                                      EmployeeManagementInputPort employeeManagementInputPort) {
30          this.administrationInputPort = administrationInputPort;
31          this.employeeManagementInputPort = employeeManagementInputPort;
32      }
33
34      @GetMapping("/requests")
35      public ResponseEntity<List<AllocationRequestResponseDTO>> listRequests(
36              @RequestParam(required = false) String status) {
```

```
37          AdministeredAllocationStatus statusEnum = status != null ?
            AdministeredAllocationStatus.valueOf(status) : null;
38          return ResponseEntity.ok(enrich(administrationInputPort.listRequests(statusEnum)));
39      }
40
41      @PutMapping("/requests/{id}/approve")
42      public ResponseEntity<Void> approve(@PathVariable UUID id) {
43          administrationInputPort.approveRequest(AdministeredAllocationRequestId.of(id));
44          return ResponseEntity.ok().build();
45      }
46
47      @PutMapping("/requests/{id}/reject")
48      public ResponseEntity<Void> reject(@PathVariable UUID id, @Valid @RequestBody RejectRequestDTO dto) {
49          administrationInputPort.rejectRequest(AdministeredAllocationRequestId.of(id), dto.reason());
50          return ResponseEntity.ok().build();
51      }
52
53      @PutMapping("/requests/{id}/issue")
54      public ResponseEntity<Void> issue(@PathVariable UUID id) {
55          administrationInputPort.issueResource(AdministeredAllocationRequestId.of(id));
56          return ResponseEntity.ok().build();
57      }
58
59      @PutMapping("/requests/{id}/return")
60      public ResponseEntity<Void> returnResource(@PathVariable UUID id) {
61          administrationInputPort.returnResource(AdministeredAllocationRequestId.of(id));
62          return ResponseEntity.ok().build();
63      }
64
65      @GetMapping("/allocations")
66      public ResponseEntity<List<AllocationRequestResponseDTO>> monitorActive() {
67          return ResponseEntity.ok(enrich(administrationInputPort.monitorActiveAllocations()));
68      }
69
70      private List<AllocationRequestResponseDTO> enrich(List<AdministeredAllocationView> requests) {
71          Map<UUID, Employee> cache = requests.stream()
72                  .map(AdministeredAllocationView::employeeId)
73                  .distinct()
74                  .collect(Collectors.toMap(
75                          id -> id,
76                          id -> employeeManagementInputPort.getEmployee(EmployeeId.of(id)).orElse(null),
77                          (a, b) -> a
78                  ));
79          return requests.stream()
80                  .map(r -> {
81                      Employee emp = cache.get(r.employeeId());
82                      return toResponse(r,
83                              emp != null ? emp.getFirstName() : null,
84                              emp != null ? emp.getLastName() : null);
85                  })
86                  .toList();
87      }
88
89      private AllocationRequestResponseDTO toResponse(AdministeredAllocationView r, String firstName, String
        lastName) {
90          return new AllocationRequestResponseDTO(
91                  r.id(),
92                  r.employeeId(),
93                  firstName,
94                  lastName,
95                  r.resourceId(),
96                  r.status(),
97                  r.dateFrom(),
98                  r.dateTo(),
99                  r.justification(),
100                 r.submittedAt(),
101                 r.issuedAt(),
102                 r.returnedAt(),
103                 r.rejectionReason()
104         );
```

```
    }
}
```

Source code 6.79: administration/infrastructure/in/web/dto/RejectRequestDTO.java - RejectRequestDTO

```
package com.deskflow.administration.infrastructure.in.web.dto;

import jakarta.validation.constraints.NotBlank;

public record RejectRequestDTO(@NotBlank String reason) {}
```

Source code 6.80: administration/infrastructure/out/allocation/AllocationAdministrationAdapter.java - AllocationAdministrationAdapter

```
package com.deskflow.administration.infrastructure.out.allocation;

import com.deskflow.administration.application.model.AdministeredAllocationView;
import com.deskflow.administration.application.port.out.AdministrationAllocationPort;
import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;
import com.deskflow.allocation.application.model.AllocationRequestView;
import com.deskflow.allocation.application.port.in.AllocationAdministrationInputPort;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Optional;

@Component
public class AllocationAdministrationAdapter implements AdministrationAllocationPort {

    private final AllocationAdministrationInputPort allocationAdministrationInputPort;

    public AllocationAdministrationAdapter(AllocationAdministrationInputPort
    allocationAdministrationInputPort) {
        this.allocationAdministrationInputPort = allocationAdministrationInputPort;
    }

    @Override
    public Optional<AdministeredAllocationView> getRequest(AdministeredAllocationRequestId requestId) {
        return
        allocationAdministrationInputPort.getRequest(toAllocationId(requestId)).map(this::toAdminView);
    }

    @Override
    public List<AdministeredAllocationView> listRequests(AdministeredAllocationStatus status) {
        RequestStatus allocationStatus = status != null ? RequestStatus.valueOf(status.name()) : null;
        return allocationAdministrationInputPort.listRequests(allocationStatus).stream()
                .map(this::toAdminView)
                .toList();
    }

    @Override
    public void approveRequest(AdministeredAllocationRequestId requestId) {
        allocationAdministrationInputPort.approveRequest(toAllocationId(requestId));
    }

    @Override
    public void rejectRequest(AdministeredAllocationRequestId requestId, String reason) {
        allocationAdministrationInputPort.rejectRequest(toAllocationId(requestId), reason);
    }
```

```
47
48      @Override
49      public void issueResource(AdministeredAllocationRequestId requestId) {
50          allocationAdministrationInputPort.issueResource(toAllocationId(requestId));
51      }
52
53      @Override
54      public void returnResource(AdministeredAllocationRequestId requestId) {
55          allocationAdministrationInputPort.returnResource(toAllocationId(requestId));
56      }
57
58      @Override
59      public List<AdministeredAllocationView> monitorActiveAllocations() {
60          return allocationAdministrationInputPort.monitorActiveAllocations().stream()
61                  .map(this::toAdminView)
62                  .toList();
63      }
64
65      @Override
66      public void detectOverdueAllocations() {
67          allocationAdministrationInputPort.detectOverdueAllocations();
68      }
69
70      private AllocationRequestId toAllocationId(AdministeredAllocationRequestId requestId) {
71          return AllocationRequestId.of(requestId.getValue());
72      }
73
74      private AdministeredAllocationView toAdminView(AllocationRequestView view) {
75          return new AdministeredAllocationView(
76                  view.id(),
77                  view.employeeId(),
78                  view.resourceId(),
79                  view.status(),
80                  view.dateFrom(),
81                  view.dateTo(),
82                  view.justification(),
83                  view.submittedAt(),
84                  view.issuedAt(),
85                  view.returnedAt(),
86                  view.rejectionReason()
87          );
88      }
89  }
```

6.1.4. Kontekst Employees - pracownicy

Source code 6.81: employees/application/domain/model/Employee.java - Employee

```
1   package com.deskflow.employees.application.domain.model;
2
3   import com.deskflow.common.application.domain.model.vo.EmployeeId;
4   import com.deskflow.employees.application.domain.model.vo.Role;
5   import com.deskflow.common.application.domain.model.vo.ResourceType;
6
7   import java.util.EnumMap;
8   import java.util.Map;
9
10  public class Employee {
11
12      private final EmployeeId id;
13      private final String email;
14      private final String firstName;
15      private final String lastName;
16      private final Role role;
17      private final Map<ResourceType, Integer> limits;
18
```

```
    public Employee(EmployeeId id, String email, String firstName, String lastName,
                    Role role, Map<ResourceType, Integer> limits) {
        this.id = id;
        this.email = email;
        this.firstName = firstName;
        this.lastName = lastName;
        this.role = role;
        this.limits = new EnumMap<>(ResourceType.class);
        this.limits.putAll(limits);
    }

    public int limitForType(ResourceType type) {
        return limits.getOrDefault(type, 1);
    }

    public void changeLimit(ResourceType type, int amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Allocation limit cannot be negative");
        }
        limits.put(type, amount);
    }

    public EmployeeId getId() { return id; }
    public String getEmail() { return email; }
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
    public Role getRole() { return role; }
    public Map<ResourceType, Integer> getLimits() { return Map.copyOf(limits); }
}
```

Source code 6.82: employees/application/domain/model/vo/Role.java - Role

```
package com.deskflow.employees.application.domain.model.vo;

public enum Role {
    EMPLOYEE,
    MANAGER,
    IT_ADMIN
}
```

Source code 6.83: employees/application/model/EmployeeCredentials.java - EmployeeCredentials

```
package com.deskflow.employees.application.model;

public record EmployeeCredentials(String email, String passwordHash, String role) {}
```

Source code 6.84: employees/application/port/in/EmployeeManagementInputPort.java - EmployeeManagementInputPort

```
package com.deskflow.employees.application.port.in;

import com.deskflow.employees.application.model.EmployeeCredentials;
import com.deskflow.employees.application.domain.model.Employee;
import com.deskflow.common.application.domain.model.vo.EmployeeId;

import java.util.Optional;

public interface EmployeeManagementInputPort {
    Optional<Employee> getEmployee(EmployeeId id);
    Optional<Employee> getEmployeeByEmail(String email);
    Optional<EmployeeCredentials> getEmployeeCredentialsByEmail(String email);
    java.util.List<Employee> listEmployees();
    void upsertLimit(EmployeeId employeeId, String resourceType, int amount);
}
```

Source code 6.85: employees/application/port/out/EmployeeRepositoryPort.java - EmployeeRepositoryPort

```
package com.deskflow.employees.application.port.out;

import com.deskflow.employees.application.model.EmployeeCredentials;
import com.deskflow.employees.application.domain.model.Employee;
import com.deskflow.common.application.domain.model.vo.EmployeeId;

import java.util.List;
import java.util.Optional;

public interface EmployeeRepositoryPort {

    Optional<Employee> find(EmployeeId id);

    Optional<Employee> findByEmail(String email);

    Optional<EmployeeCredentials> findCredentialsByEmail(String email);

    void save(Employee employee);

    List<Employee> findAll();
}
```

Source code 6.86: employees/application/service/EmployeeAppService.java - EmployeeAppService

```
package com.deskflow.employees.application.service;

import com.deskflow.employees.application.model.EmployeeCredentials;
import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import com.deskflow.employees.application.port.out.EmployeeRepositoryPort;
import com.deskflow.employees.application.domain.model.Employee;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.NoSuchElementException;
import java.util.Optional;

@Service
@Transactional(readOnly = true)
public class EmployeeAppService implements EmployeeManagementInputPort {

    private final EmployeeRepositoryPort employeeRepository;

    public EmployeeAppService(EmployeeRepositoryPort employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    @Override
    public Optional<Employee> getEmployee(EmployeeId id) {
        return employeeRepository.find(id);
    }

    @Override
    public Optional<Employee> getEmployeeByEmail(String email) {
        return employeeRepository.findByEmail(email);
    }

    @Override
    public Optional<EmployeeCredentials> getEmployeeCredentialsByEmail(String email) {
        return employeeRepository.findCredentialsByEmail(email);
```

```
    }

    @Override
    public List<Employee> listEmployees() {
        return employeeRepository.findAll();
    }

    @Override
    @Transactional
    public void upsertLimit(EmployeeId employeeId, String resourceType, int amount) {
        Employee employee = employeeRepository.find(employeeId)
                .orElseThrow(() -> new NoSuchElementException("Employee not found: " + employeeId));
        employee.changeLimit(ResourceType.valueOf(resourceType), amount);
        employeeRepository.save(employee);
    }
}
```

Source code 6.87: employees/infrastructure/in/web/EmployeeAdminController.java - EmployeeAdminController

```
package com.deskflow.employees.infrastructure.in.web;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.employees.application.domain.model.Employee;
import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.UUID;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/administration/employees")
public class EmployeeAdminController {

    private final EmployeeManagementInputPort employeeManagementInputPort;

    public EmployeeAdminController(EmployeeManagementInputPort employeeManagementInputPort) {
        this.employeeManagementInputPort = employeeManagementInputPort;
    }

    @GetMapping
    public ResponseEntity<List<Map<String, Object>>> listEmployees() {
        List<Map<String, Object>> result = employeeManagementInputPort.listEmployees().stream()
                .map(e -> Map.<String, Object>of(
                        "id", e.getId().getValue(),
                        "email", e.getEmail(),
                        "firstName", e.getFirstName(),
                        "lastName", e.getLastName(),
                        "role", e.getRole().name(),
                        "limits", buildLimits(e)
                ))
                .collect(Collectors.toList());
        return ResponseEntity.ok(result);
    }

```

```
49      @PutMapping("/{id}/limits")
50      public ResponseEntity<Void> upsertLimit(@PathVariable UUID id, @Valid @RequestBody LimitRequest body) {
51          ResourceType.valueOf(body.resourceType());
52          employeeManagementInputPort.upsertLimit(EmployeeId.of(id), body.resourceType(), body.amount());
53          return ResponseEntity.ok().build();
54      }
55
56      private Map<String, Integer> buildLimits(Employee e) {
57          return Arrays.stream(ResourceType.values())
58                  .collect(Collectors.toMap(
59                          Enum::name,
60                          t -> e.limitForType(t)
61                  ));
62      }
63
64      public record LimitRequest(
65              @NotBlank String resourceType,
66              @Min(0) int amount
67      ) {}
68  }
```

Source code 6.88: employees/infrastructure/out/persistence/AllocationLimitEntity.java - AllocationLimitEntity

```
1   package com.deskflow.employees.infrastructure.out.persistence;
2
3   import jakarta.persistence.*;
4
5   import java.util.UUID;
6
7   @Entity
8   @Table(name = "allocation_limits")
9   public class AllocationLimitEntity {
10
11      @Id
12      private UUID id;
13
14      @ManyToOne(fetch = FetchType.LAZY)
15      @JoinColumn(name = "employee_id", nullable = false)
16      private EmployeeEntity employee;
17
18      @Column(name = "resource_type", nullable = false)
19      private String resourceType;
20
21      @Column(name = "limit_amount", nullable = false)
22      private int limitAmount;
23
24      public UUID getId() { return id; }
25      public void setId(UUID id) { this.id = id; }
26      public void setEmployee(EmployeeEntity employee) { this.employee = employee; }
27      public String getResourceType() { return resourceType; }
28      public void setResourceType(String resourceType) { this.resourceType = resourceType; }
29      public int getLimitAmount() { return limitAmount; }
30      public void setLimitAmount(int limitAmount) { this.limitAmount = limitAmount; }
31  }
```

Source code 6.89: employees/infrastructure/out/persistence/EmployeeEntity.java - EmployeeEntity

```
1   package com.deskflow.employees.infrastructure.out.persistence;
2
3   import jakarta.persistence.*;
4
5   import java.util.ArrayList;
6   import java.util.List;
```

```
7  import java.util.UUID;
8
9  @Entity
10 @Table(name = "employees")
11 public class EmployeeEntity {
12
13     @Id
14     private UUID id;
15
16     @Column(nullable = false, unique = true)
17     private String email;
18
19     @Column(name = "first_name", nullable = false)
20     private String firstName;
21
22     @Column(name = "last_name", nullable = false)
23     private String lastName;
24
25     @Column(name = "role", nullable = false)
26     private String role;
27
28     @Column(name = "password_hash", nullable = false)
29     private String passwordHash;
30
31     @OneToMany(mappedBy = "employee", cascade = CascadeType.ALL, orphanRemoval = true, fetch =
       FetchType.EAGER)
32     private List<AllocationLimitEntity> allocationLimits = new ArrayList<>();
33
34     public UUID getId() { return id; }
35     public void setId(UUID id) { this.id = id; }
36     public String getEmail() { return email; }
37     public void setEmail(String email) { this.email = email; }
38     public String getFirstName() { return firstName; }
39     public void setFirstName(String firstName) { this.firstName = firstName; }
40     public String getLastName() { return lastName; }
41     public void setLastName(String lastName) { this.lastName = lastName; }
42     public String getRole() { return role; }
43     public void setRole(String role) { this.role = role; }
44     public String getPasswordHash() { return passwordHash; }
45     public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
46     public List<AllocationLimitEntity> getAllocationLimits() { return allocationLimits; }
47
48     public void addAllocationLimit(AllocationLimitEntity limit) {
49         limit.setEmployee(this);
50         allocationLimits.add(limit);
51     }
52 }
```

Source code 6.90: employees/infrastructure/out/persistence/EmployeeMapper.java - EmployeeMapper

```
1  package com.deskflow.employees.infrastructure.out.persistence;
2
3  import com.deskflow.employees.application.domain.model.Employee;
4  import com.deskflow.common.application.domain.model.vo.EmployeeId;
5  import com.deskflow.employees.application.domain.model.vo.Role;
6  import com.deskflow.common.application.domain.model.vo.ResourceType;
7  import org.springframework.stereotype.Component;
8
9  import java.util.Map;
10 import java.util.stream.Collectors;
11
12 @Component
13 public class EmployeeMapper {
14
15     public Employee toDomain(EmployeeEntity e) {
16         Map<ResourceType, Integer> limits = e.getAllocationLimits().stream()
17                 .collect(Collectors.toMap(
```

```
                        l -> ResourceType.valueOf(l.getResourceType()),
                        AllocationLimitEntity::getLimitAmount
                ));
        return new Employee(
                EmployeeId.of(e.getId()),
                e.getEmail(), e.getFirstName(), e.getLastName(),
                Role.valueOf(e.getRole()),
                limits
        );
    }
}
```

Source code 6.91:
employees/infrastructure/out/persistence/HibernateEmployeeRepositoryAdapter.java - HibernateEmployeeRepositoryAdapter

```
package com.deskflow.employees.infrastructure.out.persistence;

import com.deskflow.employees.application.model.EmployeeCredentials;
import com.deskflow.employees.application.port.out.EmployeeRepositoryPort;
import com.deskflow.employees.application.domain.model.Employee;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;
import java.util.UUID;

@Repository
public class HibernateEmployeeRepositoryAdapter implements EmployeeRepositoryPort {

    private final SpringDataEmployeeRepository jpa;
    private final SpringDataAllocationLimitRepository limitJpa;
    private final EmployeeMapper mapper;

    public HibernateEmployeeRepositoryAdapter(SpringDataEmployeeRepository jpa,
                                              SpringDataAllocationLimitRepository limitJpa,
                                              EmployeeMapper mapper) {
        this.jpa = jpa;
        this.limitJpa = limitJpa;
        this.mapper = mapper;
    }

    @Override
    public Optional<Employee> find(EmployeeId id) {
        return jpa.findById(id.getValue()).map(mapper::toDomain);
    }

    @Override
    public Optional<Employee> findByEmail(String email) {
        return jpa.findByEmail(email).map(mapper::toDomain);
    }

    @Override
    public Optional<EmployeeCredentials> findCredentialsByEmail(String email) {
        return jpa.findByEmail(email)
                .map(employee -> new EmployeeCredentials(employee.getEmail(), employee.getPasswordHash(),
                employee.getRole()));
    }

    @Override
    public void save(Employee employee) {
        EmployeeEntity entity = jpa.findById(employee.getId().getValue())
                .orElseThrow(() -> new IllegalArgumentException("Employee not found: " + employee.getId()));
        employee.getLimits().forEach((type, amount) -> upsertLimit(entity, type.name(), amount));
        jpa.save(entity);
    }
```

```
51
52      @Override
53      public List<Employee> findAll() {
54          return jpa.findAll().stream().map(mapper::toDomain).toList();
55      }
56
57      private void upsertLimit(EmployeeEntity employee, String resourceType, int amount) {
58          AllocationLimitEntity limit = limitJpa
59                  .findByEmployeeIdAndResourceType(employee.getId(), resourceType)
60                  .orElseGet(() -> {
61                      AllocationLimitEntity l = new AllocationLimitEntity();
62                      l.setId(UUID.randomUUID());
63                      l.setEmployee(employee);
64                      l.setResourceType(resourceType);
65                      return l;
66                  });
67          limit.setLimitAmount(amount);
68          limitJpa.save(limit);
69      }
70  }
```

Source code 6.92:
employees/infrastructure/out/persistence/SpringDataAllocationLimitRepository.java - SpringDataAllocationLimitRepository

```
1  package com.deskflow.employees.infrastructure.out.persistence;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4
5  import java.util.Optional;
6  import java.util.UUID;
7
8  public interface SpringDataAllocationLimitRepository extends JpaRepository<AllocationLimitEntity, UUID> {
9      Optional<AllocationLimitEntity> findByEmployeeIdAndResourceType(UUID employeeId, String resourceType);
10 }
```

Source code 6.93:
employees/infrastructure/out/persistence/SpringDataEmployeeRepository.java - SpringDataEmployeeRepository

```
1  package com.deskflow.employees.infrastructure.out.persistence;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4
5  import java.util.Optional;
6  import java.util.UUID;
7
8  public interface SpringDataEmployeeRepository extends JpaRepository<EmployeeEntity, UUID> {
9      Optional<EmployeeEntity> findByEmail(String email);
10 }
```

6.1.5. Kontekst Identity - tożsamość i uwierzytelnianie

Source code 6.94: identity/application/model/AuthenticatedIdentity.java - AuthenticatedIdentity

```
1  package com.deskflow.identity.application.model;
2
3  public record AuthenticatedIdentity(
4          String token,
```

```
        String email,
        String role,
        String firstName,
        String lastName,
        String employeeId
) {}
```

Source code 6.95: identity/application/model/IdentityProfile.java - IdentityProfile

```
package com.deskflow.identity.application.model;

public record IdentityProfile(
        String email,
        String role,
        String firstName,
        String lastName,
        String employeeId
) {}
```

Source code 6.96: identity/application/model/UserCredentials.java - UserCredentials

```
package com.deskflow.identity.application.model;

public record UserCredentials(String email, String passwordHash, String role) {}
```

Source code 6.97: identity/application/port/in/IdentityAccessInputPort.java - IdentityAccessInputPort

```
package com.deskflow.identity.application.port.in;

import com.deskflow.identity.application.model.AuthenticatedIdentity;
import com.deskflow.identity.application.model.IdentityProfile;

import java.util.Optional;

public interface IdentityAccessInputPort {
    AuthenticatedIdentity login(String email, String password);

    Optional<IdentityProfile> currentIdentity(String email);
}
```

Source code 6.98: identity/application/port/out/AccessTokenIssuerPort.java - AccessTokenIssuerPort

```
package com.deskflow.identity.application.port.out;

public interface AccessTokenIssuerPort {
    String issueAccessToken(String email, String role);
}
```

Source code 6.99: identity/application/port/out/CredentialsAuthenticationPort.java - CredentialsAuthenticationPort

```
package com.deskflow.identity.application.port.out;

public interface CredentialsAuthenticationPort {
    void authenticate(String email, String password);
}
```

Source code 6.100: identity/application/port/out/IdentityEmployeeProfilePort.java - IdentityEmployeeProfilePort

```
package com.deskflow.identity.application.port.out;

import com.deskflow.identity.application.model.IdentityProfile;

import java.util.Optional;

public interface IdentityEmployeeProfilePort {
    Optional<IdentityProfile> findByEmail(String email);
}
```

Source code 6.101: identity/application/port/out/UserCredentialsPort.java - UserCredentialsPort

```
package com.deskflow.identity.application.port.out;

import com.deskflow.identity.application.model.UserCredentials;

import java.util.Optional;

public interface UserCredentialsPort {
    Optional<UserCredentials> findByEmail(String email);
}
```

Source code 6.102: identity/application/service/IdentityAccessAppService.java - IdentityAccessAppService

```
package com.deskflow.identity.application.service;

import com.deskflow.identity.application.model.AuthenticatedIdentity;
import com.deskflow.identity.application.model.IdentityProfile;
import com.deskflow.identity.application.port.in.IdentityAccessInputPort;
import com.deskflow.identity.application.port.out.AccessTokenIssuerPort;
import com.deskflow.identity.application.port.out.CredentialsAuthenticationPort;
import com.deskflow.identity.application.port.out.IdentityEmployeeProfilePort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Optional;

@Service
@Transactional(readOnly = true)
public class IdentityAccessAppService implements IdentityAccessInputPort {

    private final CredentialsAuthenticationPort credentialsAuthenticationPort;
    private final AccessTokenIssuerPort accessTokenIssuerPort;
    private final IdentityEmployeeProfilePort employeeProfilePort;

    public IdentityAccessAppService(CredentialsAuthenticationPort credentialsAuthenticationPort,
                                    AccessTokenIssuerPort accessTokenIssuerPort,
                                    IdentityEmployeeProfilePort employeeProfilePort) {
        this.credentialsAuthenticationPort = credentialsAuthenticationPort;
        this.accessTokenIssuerPort = accessTokenIssuerPort;
        this.employeeProfilePort = employeeProfilePort;
    }

    @Override
    public AuthenticatedIdentity login(String email, String password) {
        credentialsAuthenticationPort.authenticate(email, password);
        IdentityProfile profile = employeeProfilePort.findByEmail(email).orElseThrow();
        String token = accessTokenIssuerPort.issueAccessToken(email, profile.role());
        return new AuthenticatedIdentity(
```

```
                token, profile.email(), profile.role(),
                profile.firstName(), profile.lastName(), profile.employeeId()
        );
    }

    @Override
    public Optional<IdentityProfile> currentIdentity(String email) {
        return employeeProfilePort.findByEmail(email);
    }
}
```

Source code 6.103: identity/infrastructure/in/web/AuthController.java - AuthController

```
package com.deskflow.identity.infrastructure.in.web;

import com.deskflow.identity.application.model.AuthenticatedIdentity;
import com.deskflow.identity.application.model.IdentityProfile;
import com.deskflow.identity.application.port.in.IdentityAccessInputPort;
import com.deskflow.identity.infrastructure.in.web.dto.LoginRequest;
import com.deskflow.identity.infrastructure.in.web.dto.LoginResponse;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final IdentityAccessInputPort identityAccessInputPort;

    public AuthController(IdentityAccessInputPort identityAccessInputPort) {
        this.identityAccessInputPort = identityAccessInputPort;
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest req) {
        return ResponseEntity.ok(toResponse(identityAccessInputPort.login(req.email(), req.password())));
    }

    @GetMapping("/me")
    public ResponseEntity<?> me(Authentication auth) {
        if (auth == null) return ResponseEntity.status(401).build();
        return identityAccessInputPort.currentIdentity(auth.getName())
                .<ResponseEntity<?>>map(profile -> ResponseEntity.ok(toResponse(profile)))
                .orElse(ResponseEntity.notFound().build());
    }

    private LoginResponse toResponse(AuthenticatedIdentity identity) {
        return new LoginResponse(
                identity.token(), identity.email(), identity.role(),
                identity.firstName(), identity.lastName(), identity.employeeId()
        );
    }

    private LoginResponse toResponse(IdentityProfile profile) {
        return new LoginResponse(
                null, profile.email(), profile.role(),
                profile.firstName(), profile.lastName(), profile.employeeId()
        );
    }
}
```

Source code 6.104: identity/infrastructure/in/web/dto/LoginRequest.java - LoginRequest

```
package com.deskflow.identity.infrastructure.in.web.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record LoginRequest(@Email @NotBlank String email, @NotBlank String password) {}
```

Source code 6.105: identity/infrastructure/in/web/dto/LoginResponse.java - LoginResponse

```
package com.deskflow.identity.infrastructure.in.web.dto;

public record LoginResponse(String token, String email, String role, String firstName, String lastName,
String employeeId) {}
```

Source code 6.106: identity/infrastructure/out/employees/EmployeeIdentityProfileAdapter.java - EmployeeIdentityProfileAdapter

```
package com.deskflow.identity.infrastructure.out.employees;

import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import com.deskflow.identity.application.model.IdentityProfile;
import com.deskflow.identity.application.port.out.IdentityEmployeeProfilePort;
import org.springframework.stereotype.Component;

import java.util.Optional;

@Component
public class EmployeeIdentityProfileAdapter implements IdentityEmployeeProfilePort {

    private final EmployeeManagementInputPort employeeManagementInputPort;

    public EmployeeIdentityProfileAdapter(EmployeeManagementInputPort employeeManagementInputPort) {
        this.employeeManagementInputPort = employeeManagementInputPort;
    }

    @Override
    public Optional<IdentityProfile> findByEmail(String email) {
        return employeeManagementInputPort.getEmployeeByEmail(email)
                .map(employee -> new IdentityProfile(
                        employee.getEmail(),
                        employee.getRole().name(),
                        employee.getFirstName(),
                        employee.getLastName(),
                        employee.getId().toString()
                ));
    }
}
```

Source code 6.107: identity/infrastructure/out/employees/EmployeeUserCredentialsAdapter.java - EmployeeUserCredentialsAdapter

```
package com.deskflow.identity.infrastructure.out.employees;

import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import com.deskflow.identity.application.model.UserCredentials;
import com.deskflow.identity.application.port.out.UserCredentialsPort;
```

```
import org.springframework.stereotype.Component;

import java.util.Optional;

@Component
public class EmployeeUserCredentialsAdapter implements UserCredentialsPort {

    private final EmployeeManagementInputPort employeeManagementInputPort;

    public EmployeeUserCredentialsAdapter(EmployeeManagementInputPort employeeManagementInputPort) {
        this.employeeManagementInputPort = employeeManagementInputPort;
    }

    @Override
    public Optional<UserCredentials> findByEmail(String email) {
        return employeeManagementInputPort.getEmployeeCredentialsByEmail(email)
                .map(credentials -> new UserCredentials(credentials.email(), credentials.passwordHash(),
                credentials.role()));
    }
}
```

Source code 6.108: identity/infrastructure/out/security/SpringCredentialsAuthenticationAdapter.java - SpringCredentialsAuthenticationAdapter

```
package com.deskflow.identity.infrastructure.out.security;

import com.deskflow.identity.application.port.out.CredentialsAuthenticationPort;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.stereotype.Component;

@Component
public class SpringCredentialsAuthenticationAdapter implements CredentialsAuthenticationPort {

    private final AuthenticationManager authenticationManager;

    public SpringCredentialsAuthenticationAdapter(AuthenticationManager authenticationManager) {
        this.authenticationManager = authenticationManager;
    }

    @Override
    public void authenticate(String email, String password) {
        authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(email, password));
    }
}
```

Source code 6.109: identity/infrastructure/security/DeskflowUserDetailsService.java - DeskflowUserDetailsService

```
package com.deskflow.identity.infrastructure.security;

import com.deskflow.identity.application.port.out.UserCredentialsPort;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class DeskflowUserDetailsService implements UserDetailsService {

```

```
    private final UserCredentialsPort userCredentialsPort;

    public DeskflowUserDetailsService(UserCredentialsPort userCredentialsPort) {
        this.userCredentialsPort = userCredentialsPort;
    }

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        return userCredentialsPort.findByEmail(email)
                .map(credentials -> new User(
                        credentials.email(), credentials.passwordHash(),
                        List.of(new SimpleGrantedAuthority("ROLE_" + credentials.role()))
                ))
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " + email));
    }
}
```

Source code 6.110: identity/infrastructure/security/JwtAuthenticationFilter.java - JwtAuthenticationFilter

```
package com.deskflow.identity.infrastructure.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;

    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider) {
        this.tokenProvider = tokenProvider;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        String token = resolveToken(request);
        if (token != null && tokenProvider.validateToken(token)) {
            String email = tokenProvider.getEmailFromToken(token);
            String role = tokenProvider.getRoleFromToken(token);
            var auth = new UsernamePasswordAuthenticationToken(
                    email, null, List.of(new SimpleGrantedAuthority("ROLE_" + role))
            );
            var context = SecurityContextHolder.createEmptyContext();
            context.setAuthentication(auth);
            SecurityContextHolder.setContext(context);
        }
        chain.doFilter(request, response);
    }

    private String resolveToken(HttpServletRequest request) {
        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            return header.substring(7);
        }
        return null;
    }
```

```
}
```

Source code 6.111: identity/infrastructure/security/JwtTokenProvider.java - JwtTokenProvider

```
package com.deskflow.identity.infrastructure.security;

import com.deskflow.identity.application.port.out.AccessTokenIssuerPort;
import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtTokenProvider implements AccessTokenIssuerPort {

    private final SecretKey secretKey;
    private final long expirationMs;

    public JwtTokenProvider(
            @Value("${app.jwt.secret}") String secret,
            @Value("${app.jwt.expiration-ms:86400000}") long expirationMs) {
        this.secretKey = Keys.hmacShaKeyFor(sha256(secret));
        this.expirationMs = expirationMs;
    }

    private static byte[] sha256(String secret) {
        String value = (secret == null || secret.isBlank())
                ? "deskflow-secret-key-at-least-32-characters-long-for-hs256"
                : secret;
        try {
            return MessageDigest.getInstance("SHA-256").digest(value.getBytes(StandardCharsets.UTF_8));
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 is not available", e);
        }
    }

    @Override
    public String issueAccessToken(String email, String role) {
        return Jwts.builder()
                .subject(email)
                .claim("role", role)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + expirationMs))
                .signWith(secretKey)
                .compact();
    }

    public String getEmailFromToken(String token) {
        return parseClaims(token).getSubject();
    }

    public String getRoleFromToken(String token) {
        return parseClaims(token).get("role", String.class);
    }

    public boolean validateToken(String token) {
        try {
            parseClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
```

```
        }
    }

    private Claims parseClaims(String token) {
        return Jwts.parser()
                .verifyWith(secretKey)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }
}
```

Source code 6.112: identity/infrastructure/security/SecurityConfig.java - SecurityConfig

```
package com.deskflow.identity.infrastructure.security;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;
    private final String allowedOrigin;

    public SecurityConfig(JwtAuthenticationFilter jwtFilter,
                          @Value("${app.cors.allowed-origin:http://localhost:3000}") String allowedOrigin) {
        this.jwtFilter = jwtFilter;
        this.allowedOrigin = allowedOrigin;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/resources").authenticated()
                .requestMatchers("/api/administration/**").hasAnyRole("MANAGER", "IT_ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/resources").hasAnyRole("IT_ADMIN", "MANAGER")
                .requestMatchers(HttpMethod.PUT, "/api/resources/**").hasAnyRole("IT_ADMIN", "MANAGER")
                .requestMatchers(HttpMethod.DELETE, "/api/resources/**").hasAnyRole("IT_ADMIN")
                .anyRequest().authenticated()
            )
```

```
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of(allowedOrigin));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}
```

6.1.6. Kontekst Notifications - powiadomienia

Source code 6.113: notifications/application/port/in/NotificationEventInputPort.java - NotificationEventInputPort

```
package com.deskflow.notifications.application.port.in;

import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.common.application.domain.model.vo.EmployeeId;

public interface NotificationEventInputPort {
    void allocationRequestSubmitted(AllocationRequestId requestId, EmployeeId employeeId);

    void allocationRequestApproved(AllocationRequestId requestId, EmployeeId employeeId);

    void allocationRequestRejected(AllocationRequestId requestId, EmployeeId employeeId, String reason);

    void allocationRequestCancelled(AllocationRequestId requestId, EmployeeId employeeId);

    void resourceIssued(AllocationRequestId requestId, EmployeeId employeeId);

    void resourceReturned(AllocationRequestId requestId, EmployeeId employeeId);

    void allocationOverdue(AllocationRequestId requestId, EmployeeId employeeId);
}
```

Source code 6.114: notifications/application/port/out/NotificationPort.java - NotificationPort

```
package com.deskflow.notifications.application.port.out;

public interface NotificationPort {
    void send(String toEmail, String subject, String body);
}
```

Source code 6.115: notifications/application/service/NotificationAppService.java - NotificationAppService

```
1  package com.deskflow.notifications.application.service;
2
3  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
4  import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
5  import com.deskflow.common.application.domain.model.vo.EmployeeId;
6  import com.deskflow.notifications.application.port.in.NotificationEventInputPort;
7  import com.deskflow.notifications.application.port.out.NotificationPort;
8  import org.slf4j.Logger;
9  import org.slf4j.LoggerFactory;
10 import org.springframework.stereotype.Service;
11 import org.springframework.transaction.annotation.Transactional;
12
13 @Service
14 @Transactional(readOnly = true)
15 public class NotificationAppService implements NotificationEventInputPort {
16
17     private static final Logger log = LoggerFactory.getLogger(NotificationAppService.class);
18
19     private final NotificationPort notifications;
20     private final EmployeeManagementInputPort employeeManagementInputPort;
21
22     public NotificationAppService(NotificationPort notifications,
23                                   EmployeeManagementInputPort employeeManagementInputPort) {
24         this.notifications = notifications;
25         this.employeeManagementInputPort = employeeManagementInputPort;
26     }
27
28     @Override
29     public void allocationRequestSubmitted(AllocationRequestId requestId, EmployeeId employeeId) {
30         emailEmployee(employeeId,
31                 "Request submitted",
32                 "Your allocation request has been submitted (ID: " + requestId + ").");
33     }
34
35     @Override
36     public void allocationRequestApproved(AllocationRequestId requestId, EmployeeId employeeId) {
37         emailEmployee(employeeId,
38                 "Request approved",
39                 "Your request has been approved (ID: " + requestId + "). Please collect the resource.");
40     }
41
42     @Override
43     public void allocationRequestRejected(AllocationRequestId requestId, EmployeeId employeeId, String
       reason) {
44         emailEmployee(employeeId,
45                 "Request rejected",
46                 "Your request (ID: " + requestId + ") was rejected. Reason: " + reason);
47     }
48
49     @Override
50     public void allocationRequestCancelled(AllocationRequestId requestId, EmployeeId employeeId) {
51         emailEmployee(employeeId,
52                 "Request cancelled",
53                 "Your request (ID: " + requestId + ") has been cancelled.");
54     }
55
56     @Override
57     public void resourceIssued(AllocationRequestId requestId, EmployeeId employeeId) {
58         emailEmployee(employeeId,
59                 "Resource issued",
60                 "The resource from request (ID: " + requestId + ") has been issued to you.");
61     }
62
63     @Override
64     public void resourceReturned(AllocationRequestId requestId, EmployeeId employeeId) {
65         emailEmployee(employeeId,
```

```
                "Resource returned",
                "We confirm receipt of the resource from request (ID: " + requestId + ").");
    }

    @Override
    public void allocationOverdue(AllocationRequestId requestId, EmployeeId employeeId) {
        emailEmployee(employeeId,
                "Allocation overdue",
                "Your allocation (ID: " + requestId + ") is past its return date. Please return the resource
                immediately.");
    }

    private void emailEmployee(EmployeeId employeeId, String subject, String body) {
        try {
            employeeManagementInputPort.getEmployee(employeeId)
                    .ifPresent(employee -> notifications.send(employee.getEmail(), subject, body));
        } catch (Exception e) {
            log.warn("Failed to send notification to employee {}: {}", employeeId, e.getMessage(), e);
        }
    }
}
```

Source code 6.116: notifications/infrastructure/in/events/NotificationEventListener.java - NotificationEventListener

```
package com.deskflow.notifications.infrastructure.in.events;

import com.deskflow.allocation.application.domain.model.events.AllocationOverdue;
import com.deskflow.allocation.application.domain.model.events.AllocationRequestApproved;
import com.deskflow.allocation.application.domain.model.events.AllocationRequestCancelled;
import com.deskflow.allocation.application.domain.model.events.AllocationRequestRejected;
import com.deskflow.allocation.application.domain.model.events.AllocationRequestSubmitted;
import com.deskflow.allocation.application.domain.model.events.ResourceIssued;
import com.deskflow.allocation.application.domain.model.events.ResourceReturned;
import com.deskflow.notifications.application.port.in.NotificationEventInputPort;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

@Component
public class NotificationEventListener {

    private final NotificationEventInputPort notificationEventInputPort;

    public NotificationEventListener(NotificationEventInputPort notificationEventInputPort) {
        this.notificationEventInputPort = notificationEventInputPort;
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onRequestSubmitted(AllocationRequestSubmitted event) {
        notificationEventInputPort.allocationRequestSubmitted(event.requestId(), event.employeeId());
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onRequestApproved(AllocationRequestApproved event) {
        notificationEventInputPort.allocationRequestApproved(event.requestId(), event.employeeId());
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onRequestRejected(AllocationRequestRejected event) {
        notificationEventInputPort.allocationRequestRejected(event.requestId(), event.employeeId(),
        event.reason());
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onRequestCancelled(AllocationRequestCancelled event) {
        notificationEventInputPort.allocationRequestCancelled(event.requestId(), event.employeeId());
    }
```

```
43
44      @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
45      public void onResourceIssued(ResourceIssued event) {
46          notificationEventInputPort.resourceIssued(event.requestId(), event.employeeId());
47      }
48
49      @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
50      public void onResourceReturned(ResourceReturned event) {
51          notificationEventInputPort.resourceReturned(event.requestId(), event.employeeId());
52      }
53
54      @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
55      public void onAllocationOverdue(AllocationOverdue event) {
56          notificationEventInputPort.allocationOverdue(event.requestId(), event.employeeId());
57      }
58  }
```

Source code 6.117: notifications/infrastructure/out/smtp/SmtpNotificationAdapter.java - SmtpNotificationAdapter

```
1   package com.deskflow.notifications.infrastructure.out.smtp;
2
3   import com.deskflow.notifications.application.port.out.NotificationPort;
4   import org.slf4j.Logger;
5   import org.slf4j.LoggerFactory;
6   import org.springframework.mail.MailException;
7   import org.springframework.mail.SimpleMailMessage;
8   import org.springframework.mail.javamail.JavaMailSender;
9   import org.springframework.stereotype.Component;
10
11  @Component
12  public class SmtpNotificationAdapter implements NotificationPort {
13
14      private static final Logger log = LoggerFactory.getLogger(SmtpNotificationAdapter.class);
15
16      private final JavaMailSender mailSender;
17
18      public SmtpNotificationAdapter(JavaMailSender mailSender) {
19          this.mailSender = mailSender;
20      }
21
22      @Override
23      public void send(String toEmail, String subject, String body) {
24          try {
25              SimpleMailMessage message = new SimpleMailMessage();
26              message.setFrom("noreply@deskflow.local");
27              message.setTo(toEmail);
28              message.setSubject(subject);
29              message.setText(body);
30              mailSender.send(message);
31          } catch (MailException e) {
32              log.warn("Failed to send email to {}: {}", toEmail, e.getMessage());
33          }
34      }
35  }
```

6.1.7. Kontekst Common - kod współdzielony

Source code 6.118: common/application/domain/model/vo/EmployeeId.java - EmployeeId

```
1   package com.deskflow.common.application.domain.model.vo;
2
```

```
3  import java.util.Objects;
4  import java.util.UUID;
5
6  public final class EmployeeId {
7      private final UUID value;
8
9      public EmployeeId(UUID value) {
10         this.value = Objects.requireNonNull(value);
11     }
12
13     public static EmployeeId of(UUID value) {
14         return new EmployeeId(value);
15     }
16
17     public static EmployeeId of(String value) {
18         return new EmployeeId(UUID.fromString(value));
19     }
20
21     public UUID getValue() {
22         return value;
23     }
24
25     @Override
26     public boolean equals(Object o) {
27         if (this == o) return true;
28         if (!(o instanceof EmployeeId that)) return false;
29         return value.equals(that.value);
30     }
31
32     @Override
33     public int hashCode() {
34         return value.hashCode();
35     }
36
37     @Override
38     public String toString() {
39         return value.toString();
40     }
41 }
```

Source code 6.119: common/application/domain/model/vo/ResourceId.java - ResourceId

```
1  package com.deskflow.common.application.domain.model.vo;
2
3  import java.util.Objects;
4  import java.util.UUID;
5
6  public final class ResourceId {
7      private final UUID value;
8
9      public ResourceId(UUID value) {
10         this.value = Objects.requireNonNull(value);
11     }
12
13     public static ResourceId of(UUID value) {
14         return new ResourceId(value);
15     }
16
17     public static ResourceId of(String value) {
18         return new ResourceId(UUID.fromString(value));
19     }
20
21     public UUID getValue() {
22         return value;
23     }
24
25     @Override
26     public boolean equals(Object o) {
```

```
27          if (this == o) return true;
28          if (!(o instanceof ResourceId that)) return false;
29          return value.equals(that.value);
30      }
31
32      @Override
33      public int hashCode() {
34          return value.hashCode();
35      }
36
37      @Override
38      public String toString() {
39          return value.toString();
40      }
41  }
```

Source code 6.120: common/application/domain/model/vo/ResourceType.java - ResourceType

```
1   package com.deskflow.common.application.domain.model.vo;
2
3   public enum ResourceType {
4       DESK,
5       CONFERENCE_ROOM,
6       LAPTOP,
7       MONITOR,
8       PROJECTOR,
9       VEHICLE
10  }
```

Source code 6.121: common/application/port/out/DomainEventPublisherPort.java - DomainEventPublisherPort

```
1   package com.deskflow.common.application.port.out;
2
3   import com.deskflow.common.application.domain.DomainEvent;
4
5   public interface DomainEventPublisherPort {
6       void publish(DomainEvent event);
7   }
```

Source code 6.122: common/infrastructure/config/DataInitializer.java - DataInitializer

```
1   package com.deskflow.common.infrastructure.config;
2
3   import com.deskflow.employees.infrastructure.out.persistence.AllocationLimitEntity;
4   import com.deskflow.employees.infrastructure.out.persistence.EmployeeEntity;
5   import com.deskflow.employees.infrastructure.out.persistence.SpringDataEmployeeRepository;
6   import com.deskflow.resources.infrastructure.out.persistence.OfficeResourceEntity;
7   import com.deskflow.resources.infrastructure.out.persistence.SpringDataOfficeResourceRepository;
8   import org.springframework.boot.CommandLineRunner;
9   import org.springframework.context.annotation.Profile;
10  import org.springframework.security.crypto.password.PasswordEncoder;
11  import org.springframework.stereotype.Component;
12
13  import java.util.UUID;
14
15  @Component
16  @Profile("!test")
17  public class DataInitializer implements CommandLineRunner {
18
19      private final SpringDataEmployeeRepository employeeRepo;
20      private final SpringDataOfficeResourceRepository resourceRepo;
```

```
21      private final PasswordEncoder passwordEncoder;
22
23      public DataInitializer(SpringDataEmployeeRepository employeeRepo,
24                             SpringDataOfficeResourceRepository resourceRepo,
25                             PasswordEncoder passwordEncoder) {
26          this.employeeRepo = employeeRepo;
27          this.resourceRepo = resourceRepo;
28          this.passwordEncoder = passwordEncoder;
29      }
30
31      @Override
32      public void run(String... args) {
33          if (employeeRepo.count() > 0) return;
34
35          // Employees (test data - names and emails stay in Polish per project requirements)
36          EmployeeEntity employeeOne = new EmployeeEntity();
37          employeeOne.setId(UUID.randomUUID());
38          employeeOne.setEmail("anna.kowalska@firma.pl");
39          employeeOne.setFirstName("Anna");
40          employeeOne.setLastName("Kowalska");
41          employeeOne.setRole("EMPLOYEE");
42          employeeOne.setPasswordHash(passwordEncoder.encode("password123"));
43          addLimit(employeeOne, "DESK", 2);
44          addLimit(employeeOne, "LAPTOP", 1);
45
46          EmployeeEntity employeeTwo = new EmployeeEntity();
47          employeeTwo.setId(UUID.randomUUID());
48          employeeTwo.setEmail("jan.nowak@firma.pl");
49          employeeTwo.setFirstName("Jan");
50          employeeTwo.setLastName("Nowak");
51          employeeTwo.setRole("EMPLOYEE");
52          employeeTwo.setPasswordHash(passwordEncoder.encode("password123"));
53          addLimit(employeeTwo, "DESK", 1);
54          addLimit(employeeTwo, "CONFERENCE_ROOM", 3);
55
56          EmployeeEntity manager = new EmployeeEntity();
57          manager.setId(UUID.randomUUID());
58          manager.setEmail("marek.kierownik@firma.pl");
59          manager.setFirstName("Marek");
60          manager.setLastName("Kierownik");
61          manager.setRole("MANAGER");
62          manager.setPasswordHash(passwordEncoder.encode("password123"));
63
64          EmployeeEntity admin = new EmployeeEntity();
65          admin.setId(UUID.randomUUID());
66          admin.setEmail("admin@firma.pl");
67          admin.setFirstName("Admin");
68          admin.setLastName("IT");
69          admin.setRole("IT_ADMIN");
70          admin.setPasswordHash(passwordEncoder.encode("admin123"));
71          addLimit(admin, "DESK", 5);
72          addLimit(admin, "LAPTOP", 3);
73          addLimit(admin, "CONFERENCE_ROOM", 5);
74
75          employeeRepo.save(employeeOne);
76          employeeRepo.save(employeeTwo);
77          employeeRepo.save(manager);
78          employeeRepo.save(admin);
79
80          // Office resources (test data - names stay in Polish per project requirements)
81          createResource("DESK", "Biurko A-101", "A", 1, "101");
82          createResource("DESK", "Biurko A-102", "A", 1, "102");
83          createResource("DESK", "Biurko B-201", "B", 2, "201");
84          createResource("CONFERENCE_ROOM", "Sala Alpha", "A", 0, "S1");
85          createResource("CONFERENCE_ROOM", "Sala Beta", "B", 1, "S2");
86          createResource("LAPTOP", "Laptop Dell XPS", "A", 0, "IT-01");
87          createResource("LAPTOP", "Laptop Lenovo ThinkPad", "A", 0, "IT-02");
88          createResource("MONITOR", "Monitor Samsung 27\"", "A", 1, "M-01");
89          createResource("PROJECTOR", "Projektor Epson", "A", 0, "P-01");
90          createResource("VEHICLE", "Samochód VW Golf", "Parking", 0, "GAR-01");
```

```
91      }
92
93      private void addLimit(EmployeeEntity employee, String resourceType, int amount) {
94          AllocationLimitEntity l = new AllocationLimitEntity();
95          l.setId(UUID.randomUUID());
96          l.setResourceType(resourceType);
97          l.setLimitAmount(amount);
98          employee.addAllocationLimit(l);
99      }
100
101     private void createResource(String type, String name, String building, int floor, String number) {
102         OfficeResourceEntity resource = new OfficeResourceEntity();
103         resource.setId(UUID.randomUUID());
104         resource.setType(type);
105         resource.setName(name);
106         resource.setBuilding(building);
107         resource.setFloor(floor);
108         resource.setNumber(number);
109         resource.setActive(true);
110         resource.setAvailable(true);
111         resourceRepo.save(resource);
112     }
113 }
```

Source code 6.123: common/infrastructure/config/DomainBeanConfig.java - DomainBeanConfig

```
1   package com.deskflow.common.infrastructure.config;
2
3   import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
4   import com.deskflow.allocation.application.domain.service.ApproveAllocationRequestDomainService;
5   import com.deskflow.allocation.application.domain.service.CancelAllocationRequestDomainService;
6   import com.deskflow.allocation.application.domain.service.EnsureEmployeeWithinLimitDomainService;
7   import com.deskflow.allocation.application.domain.service.EnsureResourceAvailableInPeriodDomainService;
8   import com.deskflow.allocation.application.domain.service.EnsureResourceEligibleForAllocationDomainService;
9   import com.deskflow.allocation.application.domain.service.IssueAllocationResourceDomainService;
10  import com.deskflow.allocation.application.domain.service.MarkAllocationRequestOverdueDomainService;
11  import com.deskflow.allocation.application.domain.service.RejectAllocationRequestDomainService;
12  import com.deskflow.allocation.application.domain.service.ReturnAllocationResourceDomainService;
13  import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
14  import com.deskflow.resources.application.domain.service.DeactivateOfficeResourceDomainService;
15  import com.deskflow.resources.application.domain.service.MarkOfficeResourceAvailableDomainService;
16  import com.deskflow.resources.application.domain.service.MarkOfficeResourceUnavailableDomainService;
17  import com.deskflow.resources.application.domain.service.UpdateOfficeResourceDomainService;
18  import org.springframework.context.annotation.Bean;
19  import org.springframework.context.annotation.Configuration;
20
21  @Configuration
22  public class DomainBeanConfig {
23
24      @Bean
25      public AllocationRequestFactory allocationRequestFactory() {
26          return new AllocationRequestFactory();
27      }
28
29      @Bean
30      public OfficeResourceFactory officeResourceFactory() {
31          return new OfficeResourceFactory();
32      }
33
34      @Bean
35      public EnsureResourceEligibleForAllocationDomainService
        ensureResourceEligibleForAllocationDomainService() {
36          return new EnsureResourceEligibleForAllocationDomainService();
37      }
38
39      @Bean
40      public EnsureResourceAvailableInPeriodDomainService ensureResourceAvailableInPeriodDomainService() {
```

```
41          return new EnsureResourceAvailableInPeriodDomainService();
42      }
43
44      @Bean
45      public EnsureEmployeeWithinLimitDomainService ensureEmployeeWithinLimitDomainService() {
46          return new EnsureEmployeeWithinLimitDomainService();
47      }
48
49      @Bean
50      public CancelAllocationRequestDomainService cancelAllocationRequestDomainService() {
51          return new CancelAllocationRequestDomainService();
52      }
53
54      @Bean
55      public ApproveAllocationRequestDomainService approveAllocationRequestDomainService() {
56          return new ApproveAllocationRequestDomainService();
57      }
58
59      @Bean
60      public RejectAllocationRequestDomainService rejectAllocationRequestDomainService() {
61          return new RejectAllocationRequestDomainService();
62      }
63
64      @Bean
65      public IssueAllocationResourceDomainService issueAllocationResourceDomainService() {
66          return new IssueAllocationResourceDomainService();
67      }
68
69      @Bean
70      public ReturnAllocationResourceDomainService returnAllocationResourceDomainService() {
71          return new ReturnAllocationResourceDomainService();
72      }
73
74      @Bean
75      public MarkAllocationRequestOverdueDomainService markAllocationRequestOverdueDomainService() {
76          return new MarkAllocationRequestOverdueDomainService();
77      }
78
79      @Bean
80      public UpdateOfficeResourceDomainService updateOfficeResourceDomainService() {
81          return new UpdateOfficeResourceDomainService();
82      }
83
84      @Bean
85      public MarkOfficeResourceUnavailableDomainService markOfficeResourceUnavailableDomainService() {
86          return new MarkOfficeResourceUnavailableDomainService();
87      }
88
89      @Bean
90      public MarkOfficeResourceAvailableDomainService markOfficeResourceAvailableDomainService() {
91          return new MarkOfficeResourceAvailableDomainService();
92      }
93
94      @Bean
95      public DeactivateOfficeResourceDomainService deactivateOfficeResourceDomainService() {
96          return new DeactivateOfficeResourceDomainService();
97      }
98  }
```

Source code 6.124: common/infrastructure/in/web/GlobalExceptionHandler.java - GlobalExceptionHandler

```
1   package com.deskflow.common.infrastructure.in.web;
2
3   import org.springframework.http.HttpStatus;
4   import org.springframework.http.ResponseEntity;
5   import org.springframework.web.bind.MethodArgumentNotValidException;
6   import org.springframework.web.bind.annotation.ExceptionHandler;
```

```
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.Map;
import java.util.NoSuchElementException;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalStateException.class)
    public ResponseEntity<?> handleBusinessRule(IllegalStateException e) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<?> handleBadArgument(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<?> handleNotFound(NoSuchElementException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<?> handleValidation(MethodArgumentNotValidException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("error", "Validation error", "details", e.getBindingResult().toString()));
    }
}
```

Source code 6.125: common/infrastructure/out/events/SpringDomainEventPublisherAdapter.java - SpringDomainEventPublisherAdapter

```
package com.deskflow.common.infrastructure.out.events;

import com.deskflow.common.application.port.out.DomainEventPublisherPort;
import com.deskflow.common.application.domain.DomainEvent;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Component;

@Component
public class SpringDomainEventPublisherAdapter implements DomainEventPublisherPort {

    private final ApplicationEventPublisher publisher;

    public SpringDomainEventPublisherAdapter(ApplicationEventPublisher publisher) {
        this.publisher = publisher;
    }

    @Override
    public void publish(DomainEvent event) {
        publisher.publishEvent(event);
    }
}
```

Source code 6.126: common/application/domain/DomainEvent.java - DomainEvent

```
package com.deskflow.common.application.domain;

import java.time.LocalDateTime;
import java.util.UUID;

public interface DomainEvent {
    UUID eventId();
```

```
8       String eventType();
9       UUID aggregateId();
10      LocalDateTime occurredAt();
11  }
```

6.2. Kod źródłowy testów

Source code 6.127: architecture/CleanArchitectureTest.java - CleanArchitectureTest

```
package com.deskflow.architecture;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.lang.ArchRule;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.slices;

class CleanArchitectureTest {

    private static JavaClasses classes;

    @BeforeAll
    static void importClasses() {
        classes = new ClassFileImporter().importPackages("com.deskflow");
    }

    @Test
    void domain_must_not_depend_on_infrastructure() {
        ArchRule rule = noClasses().that().resideInAPackage("..domain..")
                .should().dependOnClassesThat().resideInAPackage("..infrastructure..");
        rule.check(classes);
    }

    @Test
    void domain_must_not_depend_on_adapters() {
        ArchRule rule = noClasses().that().resideInAPackage("..domain..")
                .should().dependOnClassesThat().resideInAPackage("..adapters..");
        rule.check(classes);
    }

    @Test
    void domain_must_not_depend_on_application_service() {
        ArchRule rule = noClasses().that().resideInAPackage("..application.domain..")
                .should().dependOnClassesThat().resideInAPackage("..application.service..");
        rule.check(classes);
    }

    @Test
    void domain_must_not_depend_on_application_port() {
        ArchRule rule = noClasses().that().resideInAPackage("..application.domain..")
                .should().dependOnClassesThat().resideInAPackage("..application.port..");
        rule.check(classes);
    }

    @Test
    void application_must_not_depend_on_infrastructure() {
        ArchRule rule = noClasses().that().resideInAPackage("..application..")
                .should().dependOnClassesThat().resideInAPackage("..infrastructure..");
        rule.check(classes);
    }

    @Test
    void application_must_not_depend_on_adapters() {
        ArchRule rule = noClasses().that().resideInAPackage("..application..")
                .should().dependOnClassesThat().resideInAPackage("..adapters..");
        rule.check(classes);
    }

    @Test
    void domain_contexts_have_no_cyclic_dependencies() {
```

```
65          slices().matching("com.deskflow.(*)..domain..").should().beFreeOfCycles()
66                  .check(classes);
67      }
68  }
```

Source code 6.128: architecture/DomainIsolationTest.java - DomainIsolationTest

```
1   package com.deskflow.architecture;
2
3   import com.tngtech.archunit.core.domain.JavaClasses;
4   import com.tngtech.archunit.core.importer.ClassFileImporter;
5   import com.tngtech.archunit.lang.ArchRule;
6   import org.junit.jupiter.api.BeforeAll;
7   import org.junit.jupiter.api.Test;
8
9   import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
10
11  class DomainIsolationTest {
12
13      private static JavaClasses classes;
14
15      @BeforeAll
16      static void importClasses() {
17          classes = new ClassFileImporter().importPackages("com.deskflow");
18      }
19
20      @Test
21      void domain_must_not_have_jpa_annotations() {
22          ArchRule rule = noClasses().that().resideInAPackage("..domain..")
23                  .should().beAnnotatedWith("jakarta.persistence.Entity")
24                  .orShould().beAnnotatedWith("jakarta.persistence.Table")
25                  .orShould().beAnnotatedWith("jakarta.persistence.Column");
26          rule.check(classes);
27      }
28
29      @Test
30      void domain_must_not_import_spring() {
31          ArchRule rule = noClasses().that().resideInAPackage("..domain..")
32                  .should().dependOnClassesThat().resideInAPackage("org.springframework..")
33                  .because("Domain layer must be framework-independent");
34          rule.check(classes);
35      }
36
37      @Test
38      void domain_must_not_import_jakarta_persistence() {
39          ArchRule rule = noClasses().that().resideInAPackage("..domain..")
40                  .should().dependOnClassesThat().resideInAPackage("jakarta.persistence..");
41          rule.check(classes);
42      }
43  }
```

6.2.1. Testy - Allocation

Source code 6.129: allocation/application/domain/model/AllocationRequestTest.java - AllocationRequestTest

```
1   package com.deskflow.allocation.application.domain.model;
2
3   import com.deskflow.common.application.domain.model.vo.EmployeeId;
4   import com.deskflow.allocation.application.domain.model.events.*;
5   import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
6   import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
7   import com.deskflow.allocation.application.domain.model.vo.DateRange;
```

```
8  import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
9  import com.deskflow.common.application.domain.model.vo.ResourceId;
10 import org.junit.jupiter.api.BeforeEach;
11 import org.junit.jupiter.api.Test;
12
13 import java.time.LocalDate;
14 import java.time.LocalDateTime;
15 import java.util.UUID;
16
17 import static org.assertj.core.api.Assertions.*;
18
19 class AllocationRequestTest {
20
21     private static final EmployeeId EMPLOYEE_ID = EmployeeId.of(UUID.randomUUID());
22     private static final ResourceId RESOURCE_ID = ResourceId.of(UUID.randomUUID());
23     private static final DateRange DATE_RANGE = new DateRange(
24             LocalDate.of(2025, 6, 10),
25             LocalDate.of(2025, 6, 15)
26     );
27
28     private AllocationRequest request;
29
30     @BeforeEach
31     void setUp() {
32         AllocationRequestFactory factory = new AllocationRequestFactory();
33         request = factory.create(EMPLOYEE_ID, RESOURCE_ID, DATE_RANGE, "Potrzebuję do projektu");
34     }
35
36     @Test
37     void new_request_has_submitted_status() {
38         assertThat(request.getStatus()).isEqualTo(RequestStatus.SUBMITTED);
39     }
40
41     @Test
42     void new_request_publishes_submitted_event() {
43         assertThat(request.flushEvents())
44                 .hasSize(1)
45                 .first()
46                 .isInstanceOf(AllocationRequestSubmitted.class);
47     }
48
49     @Test
50     void approve_changes_status_to_approved() {
51         request.flushEvents();
52         request.approve();
53         assertThat(request.getStatus()).isEqualTo(RequestStatus.APPROVED);
54     }
55
56     @Test
57     void approve_publishes_approved_event() {
58         request.flushEvents();
59         request.approve();
60         assertThat(request.flushEvents())
61                 .hasSize(1)
62                 .first()
63                 .isInstanceOf(AllocationRequestApproved.class);
64     }
65
66     @Test
67     void reject_changes_status_and_sets_reason() {
68         request.reject("Brak dostępności");
69         assertThat(request.getStatus()).isEqualTo(RequestStatus.REJECTED);
70         assertThat(request.getRejectionReason()).isEqualTo("Brak dostępności");
71     }
72
73     @Test
74     void reject_publishes_rejected_event() {
75         request.flushEvents();
76         request.reject("powód");
77         assertThat(request.flushEvents())
```

```
78                  .hasSize(1)
79                  .first()
80                  .isInstanceOf(AllocationRequestRejected.class);
81      }
82
83      @Test
84      void cancel_from_submitted_changes_status() {
85          request.cancel();
86          assertThat(request.getStatus()).isEqualTo(RequestStatus.CANCELLED);
87      }
88
89      @Test
90      void cancel_from_approved_changes_status() {
91          request.approve();
92          request.cancel();
93          assertThat(request.getStatus()).isEqualTo(RequestStatus.CANCELLED);
94      }
95
96      @Test
97      void cancel_from_active_throws() {
98          request.approve();
99          request.issueResource();
100         assertThatThrownBy(request::cancel)
101                 .isInstanceOf(IllegalStateException.class);
102     }
103
104     @Test
105     void issue_resource_changes_status_to_active() {
106         request.approve();
107         request.issueResource();
108         assertThat(request.getStatus()).isEqualTo(RequestStatus.ACTIVE);
109         assertThat(request.getIssuedAt()).isNotNull();
110     }
111
112     @Test
113     void issue_resource_from_submitted_throws() {
114         assertThatThrownBy(request::issueResource)
115                 .isInstanceOf(IllegalStateException.class);
116     }
117
118     @Test
119     void return_resource_from_active_changes_status_to_completed() {
120         request.approve();
121         request.issueResource();
122         request.returnResource();
123         assertThat(request.getStatus()).isEqualTo(RequestStatus.COMPLETED);
124         assertThat(request.getReturnedAt()).isNotNull();
125     }
126
127     @Test
128     void return_resource_from_overdue_changes_status_to_completed() {
129         request.approve();
130         request.issueResource();
131         request.markAsOverdue();
132         request.returnResource();
133         assertThat(request.getStatus()).isEqualTo(RequestStatus.COMPLETED);
134     }
135
136     @Test
137     void mark_as_overdue_from_active_changes_status() {
138         request.approve();
139         request.issueResource();
140         request.markAsOverdue();
141         assertThat(request.getStatus()).isEqualTo(RequestStatus.OVERDUE);
142     }
143
144     @Test
145     void mark_as_overdue_from_submitted_throws() {
146         assertThatThrownBy(request::markAsOverdue)
147                 .isInstanceOf(IllegalStateException.class);
```

```
148     }
149
150     @Test
151     void approve_after_rejection_throws() {
152         request.reject("za późno");
153         assertThatThrownBy(request::approve)
154                 .isInstanceOf(IllegalStateException.class);
155     }
156
157     @Test
158     void approve_after_cancellation_throws() {
159         request.cancel();
160         assertThatThrownBy(request::approve)
161                 .isInstanceOf(IllegalStateException.class);
162     }
163
164     @Test
165     void request_stores_input_data() {
166         assertThat(request.getEmployeeId()).isEqualTo(EMPLOYEE_ID);
167         assertThat(request.getResourceId()).isEqualTo(RESOURCE_ID);
168         assertThat(request.getDateRange()).isEqualTo(DATE_RANGE);
169         assertThat(request.getJustification()).isEqualTo("Potrzebuję do projektu");
170         assertThat(request.getSubmittedAt()).isNotNull();
171     }
172
173     @Test
174     void reconstituted_request_has_no_events() {
175         AllocationRequest reconstituted = AllocationRequest.reconstitute(
176                 AllocationRequestId.newId(), EMPLOYEE_ID, RESOURCE_ID, DATE_RANGE,
177                 "uzasadnienie", LocalDateTime.now(),
178                 RequestStatus.APPROVED, null, null, null
179         );
180         assertThat(reconstituted.flushEvents()).isEmpty();
181         assertThat(reconstituted.getStatus()).isEqualTo(RequestStatus.APPROVED);
182     }
183 }
```

Source code 6.130: allocation/application/domain/model/DateRangeTest.java - DateRangeTest

```
1  package com.deskflow.allocation.application.domain.model;
2
3  import com.deskflow.allocation.application.domain.model.vo.DateRange;
4  import org.junit.jupiter.api.Test;
5
6  import java.time.LocalDate;
7
8  import static org.assertj.core.api.Assertions.*;
9
10 class DateRangeTest {
11
12     private static final LocalDate MON = LocalDate.of(2025, 6, 2);
13     private static final LocalDate WED = LocalDate.of(2025, 6, 4);
14     private static final LocalDate FRI = LocalDate.of(2025, 6, 6);
15     private static final LocalDate MON2 = LocalDate.of(2025, 6, 9);
16
17     @Test
18     void creates_date_range_when_from_before_to() {
19         DateRange r = new DateRange(MON, FRI);
20         assertThat(r.getDateFrom()).isEqualTo(MON);
21         assertThat(r.getDateTo()).isEqualTo(FRI);
22     }
23
24     @Test
25     void throws_when_date_from_equals_date_to() {
26         assertThatThrownBy(() -> new DateRange(MON, MON))
27                 .isInstanceOf(IllegalArgumentException.class)
28                 .hasMessageContaining("dateFrom must be before dateTo");
```

```
    }

    @Test
    void throws_when_date_from_after_date_to() {
        assertThatThrownBy(() -> new DateRange(FRI, MON))
                .isInstanceOf(IllegalArgumentException.class);
    }

    @Test
    void throws_when_date_from_null() {
        assertThatThrownBy(() -> new DateRange(null, FRI))
                .isInstanceOf(NullPointerException.class);
    }

    @Test
    void value_object_equality() {
        DateRange r1 = new DateRange(MON, FRI);
        DateRange r2 = new DateRange(MON, FRI);
        assertThat(r1).isEqualTo(r2);
        assertThat(r1.hashCode()).isEqualTo(r2.hashCode());
    }

    @Test
    void not_equal_when_different_dates() {
        DateRange r1 = new DateRange(MON, FRI);
        DateRange r2 = new DateRange(WED, MON2);
        assertThat(r1).isNotEqualTo(r2);
    }
}
```

Source code 6.131:
allocation/application/domain/service/AllocationLifecycleDomainServicesTest.java - AllocationLifecycleDomainServicesTest

```
package com.deskflow.allocation.application.domain.service;

import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class AllocationLifecycleDomainServicesTest {

    private final ApproveAllocationRequestDomainService approveService = new
    ApproveAllocationRequestDomainService();
    private final RejectAllocationRequestDomainService rejectService = new
    RejectAllocationRequestDomainService();
    private final IssueAllocationResourceDomainService issueService = new
    IssueAllocationResourceDomainService();
    private final ReturnAllocationResourceDomainService returnService = new
    ReturnAllocationResourceDomainService();
    private final MarkAllocationRequestOverdueDomainService overdueService = new
    MarkAllocationRequestOverdueDomainService();
    private final CancelAllocationRequestDomainService cancelService = new
    CancelAllocationRequestDomainService();

    private final EmployeeId employeeId = EmployeeId.of(UUID.randomUUID());
```

```
28
29      @Test
30      void approve_service_approves_submitted_request() {
31          AllocationRequest request = request(RequestStatus.SUBMITTED);
32
33          approveService.approveAllocationRequest(request);
34
35          assertThat(request.getStatus()).isEqualTo(RequestStatus.APPROVED);
36      }
37
38      @Test
39      void reject_service_rejects_request_and_stores_reason() {
40          AllocationRequest request = request(RequestStatus.SUBMITTED);
41
42          rejectService.rejectAllocationRequest(request, "No resource available");
43
44          assertThat(request.getStatus()).isEqualTo(RequestStatus.REJECTED);
45          assertThat(request.getRejectionReason()).isEqualTo("No resource available");
46      }
47
48      @Test
49      void reject_service_rejects_blank_reason() {
50          AllocationRequest request = request(RequestStatus.SUBMITTED);
51
52          assertThatThrownBy(() -> rejectService.rejectAllocationRequest(request, " "))
53                  .isInstanceOf(IllegalArgumentException.class)
54                  .hasMessageContaining("cannot be blank");
55      }
56
57      @Test
58      void issue_service_issues_approved_request() {
59          AllocationRequest request = request(RequestStatus.APPROVED);
60
61          issueService.issueAllocationResource(request);
62
63          assertThat(request.getStatus()).isEqualTo(RequestStatus.ACTIVE);
64          assertThat(request.getIssuedAt()).isNotNull();
65      }
66
67      @Test
68      void return_service_completes_active_request() {
69          AllocationRequest request = request(RequestStatus.ACTIVE);
70
71          returnService.returnAllocationResource(request);
72
73          assertThat(request.getStatus()).isEqualTo(RequestStatus.COMPLETED);
74          assertThat(request.getReturnedAt()).isNotNull();
75      }
76
77      @Test
78      void overdue_service_marks_active_request_as_overdue() {
79          AllocationRequest request = request(RequestStatus.ACTIVE);
80
81          overdueService.markAllocationRequestOverdue(request);
82
83          assertThat(request.getStatus()).isEqualTo(RequestStatus.OVERDUE);
84      }
85
86      @Test
87      void cancel_service_cancels_own_request() {
88          AllocationRequest request = request(RequestStatus.SUBMITTED);
89
90          cancelService.cancelAllocationRequest(request, employeeId);
91
92          assertThat(request.getStatus()).isEqualTo(RequestStatus.CANCELLED);
93      }
94
95      @Test
96      void cancel_service_rejects_cancelling_another_employees_request() {
97          AllocationRequest request = request(RequestStatus.SUBMITTED);
```

```
98
99          assertThatThrownBy(() -> cancelService.cancelAllocationRequest(request,
            EmployeeId.of(UUID.randomUUID())))
100                 .isInstanceOf(IllegalStateException.class)
101                 .hasMessageContaining("another employee");
102     }
103
104     private AllocationRequest request(RequestStatus status) {
105         return AllocationRequest.reconstitute(
106                 AllocationRequestId.newId(), employeeId, ResourceId.of(UUID.randomUUID()),
107                 new DateRange(LocalDate.of(2025, 7, 1), LocalDate.of(2025, 7, 5)),
108                 "test", LocalDateTime.now(), status, null, null, null
109         );
110     }
111 }
```

Source code 6.132:
allocation/application/domain/service/EmployeeLimitDomainServiceTest.java - EmployeeLimitDomainServiceTest

```
1   package com.deskflow.allocation.application.domain.service;
2
3   import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;
4   import com.deskflow.common.application.domain.model.vo.ResourceType;
5   import org.junit.jupiter.api.Test;
6
7   import java.util.Map;
8
9   import static org.assertj.core.api.Assertions.assertThatCode;
10  import static org.assertj.core.api.Assertions.assertThatThrownBy;
11
12  class EmployeeLimitDomainServiceTest {
13
14      private final EnsureEmployeeWithinLimitDomainService service = new
        EnsureEmployeeWithinLimitDomainService();
15
16      @Test
17      void allows_allocation_when_current_usage_is_below_limit() {
18          EmployeeAllocationProfile employee = new EmployeeAllocationProfile(Map.of(ResourceType.DESK.name(),
            2));
19
20          assertThatCode(() -> service.ensureEmployeeWithinLimit(employee, ResourceType.DESK.name(), 1))
21                  .doesNotThrowAnyException();
22      }
23
24      @Test
25      void rejects_allocation_when_current_usage_equals_limit() {
26          EmployeeAllocationProfile employee = new EmployeeAllocationProfile(Map.of(ResourceType.DESK.name(),
            2));
27
28          assertThatThrownBy(() -> service.ensureEmployeeWithinLimit(employee, ResourceType.DESK.name(), 2))
29                  .isInstanceOf(IllegalStateException.class)
30                  .hasMessageContaining("allocation limit");
31      }
32
33      @Test
34      void rejects_allocation_when_current_usage_exceeds_limit() {
35          EmployeeAllocationProfile employee = new EmployeeAllocationProfile(Map.of(ResourceType.DESK.name(),
            2));
36
37          assertThatThrownBy(() -> service.ensureEmployeeWithinLimit(employee, ResourceType.DESK.name(), 3))
38                  .isInstanceOf(IllegalStateException.class)
39                  .hasMessageContaining("allocation limit");
40      }
41  }
```

Source code 6.133:
allocation/application/domain/service/ResourceAvailabilityDomainServicesTest.java - ResourceAvailabilityDomainServicesTest

```
1  package com.deskflow.allocation.application.domain.service;
2
3  import com.deskflow.allocation.application.domain.model.AllocationRequest;
4  import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
5  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
6  import com.deskflow.allocation.application.domain.model.vo.DateRange;
7  import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
8  import com.deskflow.common.application.domain.model.vo.EmployeeId;
9  import com.deskflow.common.application.domain.model.vo.ResourceId;
10 import com.deskflow.common.application.domain.model.vo.ResourceType;
11 import org.junit.jupiter.api.Test;
12
13 import java.time.LocalDate;
14 import java.time.LocalDateTime;
15 import java.util.Collections;
16 import java.util.List;
17 import java.util.UUID;
18
19 import static org.assertj.core.api.Assertions.assertThatCode;
20 import static org.assertj.core.api.Assertions.assertThatThrownBy;
21
22 class ResourceAvailabilityDomainServicesTest {
23
24     private final EnsureResourceAvailableInPeriodDomainService ensureResourceAvailableInPeriodDomainService =
       new EnsureResourceAvailableInPeriodDomainService();
25     private final EnsureResourceEligibleForAllocationDomainService
       ensureResourceEligibleForAllocationDomainService = new
       EnsureResourceEligibleForAllocationDomainService();
26
27     @Test
28     void requires_available_when_no_overlapping_requests() {
29         assertThatCode(() ->
           ensureResourceAvailableInPeriodDomainService.ensureResourceAvailableInPeriod(Collections.emptyList()))
30                 .doesNotThrowAnyException();
31     }
32
33     @Test
34     void throws_when_overlapping_requests_exist() {
35         AllocationRequest existing = AllocationRequest.reconstitute(
36                 AllocationRequestId.newId(), EmployeeId.of(UUID.randomUUID()),
                   ResourceId.of(UUID.randomUUID()),
37                 new DateRange(LocalDate.of(2025, 7, 1), LocalDate.of(2025, 7, 10)),
38                 "test", LocalDateTime.now(), RequestStatus.ACTIVE, null, null, null
39         );
40
41         assertThatThrownBy(() ->
           ensureResourceAvailableInPeriodDomainService.ensureResourceAvailableInPeriod(List.of(existing)))
42                 .isInstanceOf(IllegalStateException.class)
43                 .hasMessageContaining("not available for the requested period");
44     }
45
46     @Test
47     void requires_eligible_for_allocation_does_not_throw_when_active_and_available() {
48         ResourceAllocationProfile resource = new ResourceAllocationProfile(
49                 ResourceId.of(UUID.randomUUID()), ResourceType.DESK.name(), "D1", "A", 1, "1", true, true);
50         assertThatCode(() ->
           ensureResourceEligibleForAllocationDomainService.ensureResourceEligibleForAllocation(resource))
51                 .doesNotThrowAnyException();
52     }
53
54     @Test
55     void requires_eligible_throws_when_unavailable() {
56         ResourceAllocationProfile resource = new ResourceAllocationProfile(
57                 ResourceId.of(UUID.randomUUID()), ResourceType.DESK.name(), "D1", "A", 1, "1", true, false);
```

```
58          assertThatThrownBy(() ->
            ensureResourceEligibleForAllocationDomainService.ensureResourceEligibleForAllocation(resource))
59                  .isInstanceOf(IllegalStateException.class)
60                  .hasMessageContaining("Resource is not available for allocation");
61      }
62  }
```

Source code 6.134: allocation/application/model/AllocationRequestViewTest.java - AllocationRequestViewTest

```
1   package com.deskflow.allocation.application.model;
2
3   import com.deskflow.allocation.application.domain.model.AllocationRequest;
4   import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
5   import com.deskflow.allocation.application.domain.model.vo.DateRange;
6   import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
7   import com.deskflow.common.application.domain.model.vo.EmployeeId;
8   import com.deskflow.common.application.domain.model.vo.ResourceId;
9   import org.junit.jupiter.api.Test;
10
11  import java.time.LocalDate;
12  import java.time.LocalDateTime;
13  import java.util.UUID;
14
15  import static org.assertj.core.api.Assertions.*;
16
17  class AllocationRequestViewTest {
18
19      @Test
20      void from_creates_view_with_all_fields() {
21          var id = AllocationRequestId.newId();
22          var employeeId = EmployeeId.of(UUID.randomUUID());
23          var resourceId = ResourceId.of(UUID.randomUUID());
24          var dateRange = new DateRange(LocalDate.of(2025, 6, 10), LocalDate.of(2025, 6, 15));
25          var submittedAt = LocalDateTime.of(2025, 6, 1, 10, 0);
26          var request = AllocationRequest.reconstitute(id, employeeId, resourceId, dateRange,
27                  "justification", submittedAt, RequestStatus.SUBMITTED, null, null, null);
28
29          var view = AllocationRequestView.from(request);
30
31          assertThat(view.id()).isEqualTo(id.getValue());
32          assertThat(view.employeeId()).isEqualTo(employeeId.getValue());
33          assertThat(view.resourceId()).isEqualTo(resourceId.getValue());
34          assertThat(view.status()).isEqualTo("SUBMITTED");
35          assertThat(view.dateFrom()).isEqualTo(LocalDate.of(2025, 6, 10));
36          assertThat(view.dateTo()).isEqualTo(LocalDate.of(2025, 6, 15));
37          assertThat(view.justification()).isEqualTo("justification");
38          assertThat(view.submittedAt()).isEqualTo(submittedAt);
39          assertThat(view.issuedAt()).isNull();
40          assertThat(view.returnedAt()).isNull();
41          assertThat(view.rejectionReason()).isNull();
42      }
43
44      @Test
45      void from_creates_view_for_completed_request() {
46          var id = AllocationRequestId.newId();
47          var employeeId = EmployeeId.of(UUID.randomUUID());
48          var resourceId = ResourceId.of(UUID.randomUUID());
49          var dateRange = new DateRange(LocalDate.of(2025, 6, 10), LocalDate.of(2025, 6, 15));
50          var submittedAt = LocalDateTime.of(2025, 6, 1, 10, 0);
51          var issuedAt = LocalDateTime.of(2025, 6, 2, 9, 0);
52          var returnedAt = LocalDateTime.of(2025, 6, 14, 16, 0);
53          var request = AllocationRequest.reconstitute(id, employeeId, resourceId, dateRange,
54                  "test", submittedAt, RequestStatus.COMPLETED, issuedAt, returnedAt, null);
55
56          var view = AllocationRequestView.from(request);
57
58          assertThat(view.status()).isEqualTo("COMPLETED");
```

```
59          assertThat(view.issuedAt()).isEqualTo(issuedAt);
60          assertThat(view.returnedAt()).isEqualTo(returnedAt);
61      }
62
63      @Test
64      void from_creates_view_for_rejected_request() {
65          var id = AllocationRequestId.newId();
66          var employeeId = EmployeeId.of(UUID.randomUUID());
67          var resourceId = ResourceId.of(UUID.randomUUID());
68          var dateRange = new DateRange(LocalDate.of(2025, 6, 10), LocalDate.of(2025, 6, 15));
69          var request = AllocationRequest.reconstitute(id, employeeId, resourceId, dateRange,
70                  "test", LocalDateTime.now(), RequestStatus.REJECTED, null, null, "Brak miejsca");
71
72          var view = AllocationRequestView.from(request);
73
74          assertThat(view.status()).isEqualTo("REJECTED");
75          assertThat(view.rejectionReason()).isEqualTo("Brak miejsca");
76      }
77  }
```

Source code 6.135:
allocation/application/service/AllocationAdministrationAppServiceTest.java - AllocationAdministrationAppServiceTest

```
1   package com.deskflow.allocation.application.service;
2
3   import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
4   import com.deskflow.allocation.application.domain.model.AllocationRequest;
5   import com.deskflow.allocation.application.domain.model.events.AllocationRequestApproved;
6   import com.deskflow.allocation.application.domain.model.events.AllocationRequestRejected;
7   import com.deskflow.allocation.application.domain.model.events.ResourceIssued;
8   import com.deskflow.allocation.application.domain.model.events.ResourceReturned;
9   import com.deskflow.allocation.application.domain.model.vo.DateRange;
10  import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
11  import com.deskflow.allocation.application.domain.service.ApproveAllocationRequestDomainService;
12  import com.deskflow.allocation.application.domain.service.IssueAllocationResourceDomainService;
13  import com.deskflow.allocation.application.domain.service.MarkAllocationRequestOverdueDomainService;
14  import com.deskflow.allocation.application.domain.service.RejectAllocationRequestDomainService;
15  import com.deskflow.allocation.application.domain.service.ReturnAllocationResourceDomainService;
16  import com.deskflow.common.application.port.out.DomainEventPublisherPort;
17  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
18  import com.deskflow.common.application.domain.model.vo.EmployeeId;
19  import com.deskflow.common.application.domain.model.vo.ResourceId;
20  import org.junit.jupiter.api.BeforeEach;
21  import org.junit.jupiter.api.Test;
22  import org.junit.jupiter.api.extension.ExtendWith;
23  import org.mockito.Mock;
24  import org.mockito.junit.jupiter.MockitoExtension;
25
26  import java.time.LocalDate;
27  import java.time.LocalDateTime;
28  import java.util.List;
29  import java.util.NoSuchElementException;
30  import java.util.Optional;
31  import java.util.UUID;
32
33  import static org.assertj.core.api.Assertions.assertThat;
34  import static org.assertj.core.api.Assertions.assertThatThrownBy;
35  import static org.mockito.Mockito.any;
36  import static org.mockito.Mockito.verify;
37  import static org.mockito.Mockito.when;
38
39  @ExtendWith(MockitoExtension.class)
40  class AllocationAdministrationAppServiceTest {
41
42      @Mock private AllocationRequestRepositoryPort requestRepository;
43      @Mock private DomainEventPublisherPort eventPublisher;
```

```
44
45      private AllocationAdministrationAppService appService;
46      private EmployeeId employeeId;
47      private ResourceId resourceId;
48
49      @BeforeEach
50      void setUp() {
51          appService = new AllocationAdministrationAppService(
52                  requestRepository,
53                  eventPublisher,
54                  new ApproveAllocationRequestDomainService(),
55                  new RejectAllocationRequestDomainService(),
56                  new IssueAllocationResourceDomainService(),
57                  new ReturnAllocationResourceDomainService(),
58                  new MarkAllocationRequestOverdueDomainService());
59          employeeId = EmployeeId.of(UUID.randomUUID());
60          resourceId = ResourceId.of(UUID.randomUUID());
61      }
62
63      private AllocationRequest createRequest(RequestStatus status) {
64          return AllocationRequest.reconstitute(
65                  AllocationRequestId.newId(), employeeId, resourceId,
66                  new DateRange(LocalDate.of(2025, 7, 1), LocalDate.of(2025, 7, 5)),
67                  "test", LocalDateTime.now(),
68                  status, null, null, null
69          );
70      }
71
72      @Test
73      void approve_request_changes_status_and_publishes_event() {
74          AllocationRequest request = createRequest(RequestStatus.SUBMITTED);
75          when(requestRepository.find(request.getId())).thenReturn(Optional.of(request));
76
77          appService.approveRequest(request.getId());
78
79          assertThat(request.getStatus()).isEqualTo(RequestStatus.APPROVED);
80          verify(requestRepository).save(request);
81          verify(eventPublisher).publish(any(AllocationRequestApproved.class));
82      }
83
84      @Test
85      void reject_request_changes_status_and_publishes_event() {
86          AllocationRequest request = createRequest(RequestStatus.SUBMITTED);
87          when(requestRepository.find(request.getId())).thenReturn(Optional.of(request));
88
89          appService.rejectRequest(request.getId(), "brak");
90
91          assertThat(request.getStatus()).isEqualTo(RequestStatus.REJECTED);
92          assertThat(request.getRejectionReason()).isEqualTo("brak");
93          verify(eventPublisher).publish(any(AllocationRequestRejected.class));
94      }
95
96      @Test
97      void issue_request_changes_status_and_publishes_event() {
98          AllocationRequest request = createRequest(RequestStatus.APPROVED);
99          when(requestRepository.find(request.getId())).thenReturn(Optional.of(request));
100
101         appService.issueResource(request.getId());
102
103         assertThat(request.getStatus()).isEqualTo(RequestStatus.ACTIVE);
104         verify(eventPublisher).publish(any(ResourceIssued.class));
105     }
106
107     @Test
108     void return_resource_changes_status_and_publishes_event() {
109         AllocationRequest request = createRequest(RequestStatus.ACTIVE);
110         when(requestRepository.find(request.getId())).thenReturn(Optional.of(request));
111
112         appService.returnResource(request.getId());
113
```

```
        assertThat(request.getStatus()).isEqualTo(RequestStatus.COMPLETED);
        verify(eventPublisher).publish(any(ResourceReturned.class));
    }

    @Test
    void detect_overdue_marks_active_past_due_requests() {
        AllocationRequest request = createRequest(RequestStatus.ACTIVE);
        when(requestRepository.findOverdue(any(LocalDateTime.class))).thenReturn(List.of(request));

        appService.detectOverdueAllocations();

        assertThat(request.getStatus()).isEqualTo(RequestStatus.OVERDUE);
        verify(requestRepository).save(request);
    }

    @Test
    void approve_throws_when_request_missing() {
        AllocationRequestId requestId = AllocationRequestId.newId();
        when(requestRepository.find(requestId)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> appService.approveRequest(requestId))
                .isInstanceOf(NoSuchElementException.class)
                .hasMessageContaining("Allocation request not found");
    }
}
```

Source code 6.136:
allocation/application/service/AllocationRequestAppServiceMissingDataTest.java - AllocationRequestAppServiceMissingDataTest

```
package com.deskflow.allocation.application.service;

import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
import com.deskflow.allocation.application.port.out.EmployeeProfilePort;
import com.deskflow.allocation.application.port.out.ResourceCatalogPort;
import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;
import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
import com.deskflow.allocation.application.domain.service.CancelAllocationRequestDomainService;
import com.deskflow.allocation.application.domain.service.EnsureEmployeeWithinLimitDomainService;
import com.deskflow.allocation.application.domain.service.EnsureResourceAvailableInPeriodDomainService;
import com.deskflow.allocation.application.domain.service.EnsureResourceEligibleForAllocationDomainService;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.common.application.port.out.DomainEventPublisherPort;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDate;
import java.util.Map;
import java.util.NoSuchElementException;
import java.util.Optional;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class AllocationRequestAppServiceMissingDataTest {

    @Mock private AllocationRequestRepositoryPort requestRepository;
    @Mock private ResourceCatalogPort resourceCatalogPort;
```

```
38      @Mock private EmployeeProfilePort employeeProfilePort;
39      @Mock private DomainEventPublisherPort eventPublisher;
40
41      private AllocationRequestAppService appService;
42      private EmployeeId employeeId;
43      private ResourceId resourceId;
44      private ResourceAllocationProfile resource;
45      private EmployeeAllocationProfile employee;
46
47      @BeforeEach
48      void setUp() {
49          employeeId = EmployeeId.of(UUID.randomUUID());
50          resourceId = ResourceId.of(UUID.randomUUID());
51
52          resource = new ResourceAllocationProfile(resourceId, ResourceType.DESK.name(), "Test desk", "A", 1,
            "1", true, true);
53          employee = new EmployeeAllocationProfile(Map.of(ResourceType.DESK.name(), 3));
54
55          appService = new AllocationRequestAppService(
56                  requestRepository, resourceCatalogPort, employeeProfilePort,
57                  new AllocationRequestFactory(), new CancelAllocationRequestDomainService(),
58                  new EnsureResourceEligibleForAllocationDomainService(), new
                    EnsureResourceAvailableInPeriodDomainService(),
59                  new EnsureEmployeeWithinLimitDomainService(), eventPublisher
60          );
61      }
62
63      @Test
64      void submit_throws_when_employee_missing() {
65          when(employeeProfilePort.findEmployee(employeeId)).thenReturn(Optional.empty());
66
67          assertThatThrownBy(() -> appService.submitRequest(employeeId, resourceId, LocalDate.of(2025, 7, 1),
            LocalDate.of(2025, 7, 5), "test"))
68                  .isInstanceOf(IllegalArgumentException.class)
69                  .hasMessageContaining("Employee not found");
70      }
71
72      @Test
73      void submit_throws_when_resource_missing() {
74          when(employeeProfilePort.findEmployee(employeeId)).thenReturn(Optional.of(employee));
75          when(resourceCatalogPort.findResource(resourceId)).thenReturn(Optional.empty());
76
77          assertThatThrownBy(() -> appService.submitRequest(employeeId, resourceId, LocalDate.of(2025, 7, 1),
            LocalDate.of(2025, 7, 5), "test"))
78                  .isInstanceOf(IllegalArgumentException.class)
79                  .hasMessageContaining("Resource not found");
80      }
81
82      @Test
83      void cancel_throws_when_request_missing() {
84          AllocationRequestId requestId = AllocationRequestId.newId();
85          when(requestRepository.find(requestId)).thenReturn(Optional.empty());
86
87          assertThatThrownBy(() -> appService.cancelRequest(requestId, employeeId))
88                  .isInstanceOf(NoSuchElementException.class)
89                  .hasMessageContaining("Allocation request not found");
90      }
91  }
```

Source code 6.137: allocation/application/service/AllocationRequestAppServiceTest.java - AllocationRequestAppServiceTest

```
1   package com.deskflow.allocation.application.service;
2
3   import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
4   import com.deskflow.allocation.application.port.out.EmployeeProfilePort;
5   import com.deskflow.allocation.application.port.out.ResourceCatalogPort;
```

```
import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;
import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
import com.deskflow.allocation.application.domain.service.CancelAllocationRequestDomainService;
import com.deskflow.allocation.application.domain.service.EnsureEmployeeWithinLimitDomainService;
import com.deskflow.allocation.application.domain.service.EnsureResourceAvailableInPeriodDomainService;
import com.deskflow.allocation.application.domain.service.EnsureResourceEligibleForAllocationDomainService;
import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.common.application.port.out.DomainEventPublisherPort;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.UUID;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class AllocationRequestAppServiceTest {

    @Mock private AllocationRequestRepositoryPort requestRepository;
    @Mock private ResourceCatalogPort resourceCatalogPort;
    @Mock private EmployeeProfilePort employeeProfilePort;
    @Mock private DomainEventPublisherPort eventPublisher;

    private AllocationRequestAppService appService;
    private EmployeeId employeeId;
    private ResourceId resourceId;
    private ResourceAllocationProfile resource;
    private EmployeeAllocationProfile employee;

    @BeforeEach
    void setUp() {
        employeeId = EmployeeId.of(UUID.randomUUID());
        resourceId = ResourceId.of(UUID.randomUUID());

        resource = new ResourceAllocationProfile(resourceId, ResourceType.DESK.name(), "Test desk", "A", 1,
        "1", true, true);
        employee = new EmployeeAllocationProfile(Map.of(ResourceType.DESK.name(), 3));

        appService = new AllocationRequestAppService(
                requestRepository, resourceCatalogPort, employeeProfilePort,
                new AllocationRequestFactory(), new CancelAllocationRequestDomainService(),
                new EnsureResourceEligibleForAllocationDomainService(), new
                EnsureResourceAvailableInPeriodDomainService(),
                new EnsureEmployeeWithinLimitDomainService(), eventPublisher
        );
    }

    @Test
    void submit_saves_request_when_within_limit_and_resource_available() {
        when(employeeProfilePort.findEmployee(employeeId)).thenReturn(Optional.of(employee));
        when(resourceCatalogPort.findResource(resourceId)).thenReturn(Optional.of(resource));
        when(requestRepository.findActiveForEmployee(employeeId)).thenReturn(Collections.emptyList());
```

```
73          when(requestRepository.findActiveForResource(eq(resourceId),
            any(DateRange.class))).thenReturn(Collections.emptyList());
74
75          AllocationRequestId id = appService.submitRequest(employeeId, resourceId, LocalDate.of(2025, 7, 1),
            LocalDate.of(2025, 7, 5), "test");
76
77          assertThat(id).isNotNull();
78          verify(requestRepository).save(any(AllocationRequest.class));
79      }
80
81      @Test
82      void submit_throws_when_limit_exceeded() {
83          AllocationRequest a1 = AllocationRequest.reconstitute(
84                  AllocationRequestId.newId(), employeeId, resourceId,
85                  new DateRange(LocalDate.of(2025, 6, 1), LocalDate.of(2025, 6, 10)),
86                  "x", LocalDateTime.now(), RequestStatus.ACTIVE, LocalDateTime.now(), null, null
87          );
88          AllocationRequest a2 = AllocationRequest.reconstitute(
89                  AllocationRequestId.newId(), employeeId, resourceId,
90                  new DateRange(LocalDate.of(2025, 6, 11), LocalDate.of(2025, 6, 20)),
91                  "x", LocalDateTime.now(), RequestStatus.ACTIVE, LocalDateTime.now(), null, null
92          );
93          AllocationRequest a3 = AllocationRequest.reconstitute(
94                  AllocationRequestId.newId(), employeeId, resourceId,
95                  new DateRange(LocalDate.of(2025, 6, 21), LocalDate.of(2025, 6, 30)),
96                  "x", LocalDateTime.now(), RequestStatus.ACTIVE, LocalDateTime.now(), null, null
97          );
98          when(employeeProfilePort.findEmployee(employeeId)).thenReturn(Optional.of(employee));
99          when(resourceCatalogPort.findResource(resourceId)).thenReturn(Optional.of(resource));
100         when(requestRepository.findActiveForEmployee(employeeId))
101                 .thenReturn(List.of(a1, a2, a3));
102
103         assertThatThrownBy(() -> appService.submitRequest(employeeId, resourceId, LocalDate.of(2025, 7, 1),
            LocalDate.of(2025, 7, 5), "test"))
104                 .isInstanceOf(IllegalStateException.class)
105                 .hasMessageContaining("limit");
106     }
107
108     @Test
109     void submit_throws_when_resource_unavailable() {
110         when(employeeProfilePort.findEmployee(employeeId)).thenReturn(Optional.of(employee));
111         when(resourceCatalogPort.findResource(resourceId)).thenReturn(Optional.of(resource));
112         when(requestRepository.findActiveForEmployee(employeeId)).thenReturn(Collections.emptyList());
113
114         AllocationRequest blocking = AllocationRequest.reconstitute(
115                 AllocationRequestId.newId(), EmployeeId.of(UUID.randomUUID()), resourceId,
116                 new DateRange(LocalDate.of(2025, 7, 1), LocalDate.of(2025, 7, 10)),
117                 "x", LocalDateTime.now(), RequestStatus.ACTIVE, LocalDateTime.now(), null, null
118         );
119         when(requestRepository.findActiveForResource(eq(resourceId), any(DateRange.class)))
120                 .thenReturn(List.of(blocking));
121
122         assertThatThrownBy(() -> appService.submitRequest(employeeId, resourceId, LocalDate.of(2025, 7, 1),
            LocalDate.of(2025, 7, 5), "test"))
123                 .isInstanceOf(IllegalStateException.class)
124                 .hasMessageContaining("available");
125     }
126
127     @Test
128     void cancel_changes_status_and_saves() {
129         AllocationRequestId requestId = AllocationRequestId.newId();
130         AllocationRequest request = AllocationRequest.reconstitute(
131                 requestId, employeeId, resourceId,
132                 new DateRange(LocalDate.of(2025, 7, 1), LocalDate.of(2025, 7, 5)),
133                 "test", LocalDateTime.now(),
134                 RequestStatus.SUBMITTED, null, null, null
135         );
136         when(requestRepository.find(requestId)).thenReturn(Optional.of(request));
137
138         appService.cancelRequest(requestId, employeeId);
```

```
139
140         assertThat(request.getStatus()).isEqualTo(RequestStatus.CANCELLED);
141         verify(requestRepository).save(request);
142     }
143 }
```

Source code 6.138:
allocation/infrastructure/in/web/AllocationRequestRestMapperTest.java - AllocationRequestRestMapperTest

```
1   package com.deskflow.allocation.infrastructure.in.web;
2
3   import com.deskflow.allocation.application.model.AllocationRequestView;
4   import com.deskflow.allocation.infrastructure.in.web.dto.AllocationRequestResponseDTO;
5   import org.junit.jupiter.api.Test;
6
7   import java.time.LocalDate;
8   import java.time.LocalDateTime;
9   import java.util.UUID;
10
11  import static org.assertj.core.api.Assertions.*;
12
13  class AllocationRequestRestMapperTest {
14
15      private final UUID id = UUID.randomUUID();
16      private final UUID employeeId = UUID.randomUUID();
17      private final UUID resourceId = UUID.randomUUID();
18      private final LocalDate dateFrom = LocalDate.of(2025, 6, 10);
19      private final LocalDate dateTo = LocalDate.of(2025, 6, 15);
20      private final LocalDateTime submittedAt = LocalDateTime.of(2025, 6, 1, 10, 0);
21
22      @Test
23      void toResponse_maps_all_fields() {
24          var view = new AllocationRequestView(id, employeeId, resourceId, "SUBMITTED",
25                  dateFrom, dateTo, "justification", submittedAt, null, null, null);
26
27          var response = AllocationRequestRestMapper.toResponse(view);
28
29          assertThat(response.id()).isEqualTo(id);
30          assertThat(response.employeeId()).isEqualTo(employeeId);
31          assertThat(response.resourceId()).isEqualTo(resourceId);
32          assertThat(response.status()).isEqualTo("SUBMITTED");
33          assertThat(response.dateFrom()).isEqualTo(dateFrom);
34          assertThat(response.dateTo()).isEqualTo(dateTo);
35          assertThat(response.justification()).isEqualTo("justification");
36          assertThat(response.submittedAt()).isEqualTo(submittedAt);
37          assertThat(response.issuedAt()).isNull();
38          assertThat(response.returnedAt()).isNull();
39          assertThat(response.rejectionReason()).isNull();
40          assertThat(response.employeeFirstName()).isNull();
41          assertThat(response.employeeLastName()).isNull();
42      }
43
44      @Test
45      void toResponse_with_employee_names() {
46          var view = new AllocationRequestView(id, employeeId, resourceId, "APPROVED",
47                  dateFrom, dateTo, "test", submittedAt, LocalDateTime.now(), null, null);
48
49          var response = AllocationRequestRestMapper.toResponse(view, "Anna", "Kowalska");
50
51          assertThat(response.employeeFirstName()).isEqualTo("Anna");
52          assertThat(response.employeeLastName()).isEqualTo("Kowalska");
53      }
54
55      @Test
56      void toResponse_maps_completed_request_with_all_fields() {
57          var issuedAt = LocalDateTime.of(2025, 6, 11, 9, 0);
```

```
        var returnedAt = LocalDateTime.of(2025, 6, 14, 16, 0);
        var view = new AllocationRequestView(id, employeeId, resourceId, "COMPLETED",
                dateFrom, dateTo, "projekt", submittedAt, issuedAt, returnedAt, null);

        var response = AllocationRequestRestMapper.toResponse(view, "Jan", "Test");

        assertThat(response.status()).isEqualTo("COMPLETED");
        assertThat(response.issuedAt()).isEqualTo(issuedAt);
        assertThat(response.returnedAt()).isEqualTo(returnedAt);
        assertThat(response.employeeFirstName()).isEqualTo("Jan");
        assertThat(response.employeeLastName()).isEqualTo("Test");
    }

    @Test
    void toResponse_maps_rejected_request() {
        var view = new AllocationRequestView(id, employeeId, resourceId, "REJECTED",
                dateFrom, dateTo, "test", submittedAt, null, null, "Brak miejsca");

        var response = AllocationRequestRestMapper.toResponse(view);

        assertThat(response.status()).isEqualTo("REJECTED");
        assertThat(response.rejectionReason()).isEqualTo("Brak miejsca");
    }
}
```

Source code 6.139:
allocation/infrastructure/out/employees/EmployeeProfileAdapterTest.java - EmployeeProfileAdapterTest

```
package com.deskflow.allocation.infrastructure.out.employees;

import com.deskflow.allocation.application.domain.model.EmployeeAllocationProfile;
import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import com.deskflow.employees.application.domain.model.Employee;
import com.deskflow.employees.application.domain.model.vo.Role;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Map;
import java.util.Optional;
import java.util.UUID;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class EmployeeProfileAdapterTest {

    @Mock private EmployeeManagementInputPort employeeManagementInputPort;

    private EmployeeProfileAdapter adapter;
    private EmployeeId employeeId;

    @BeforeEach
    void setUp() {
        adapter = new EmployeeProfileAdapter(employeeManagementInputPort);
        employeeId = EmployeeId.of(UUID.randomUUID());
    }

    @Test
    void findEmployee_returns_profile_with_limits() {
        var employee = new Employee(employeeId, "test@company.test", "Jan", "Kowalski", Role.EMPLOYEE,
```

```
                Map.of(ResourceType.DESK, 3, ResourceType.LAPTOP, 1));
        when(employeeManagementInputPort.getEmployee(employeeId)).thenReturn(Optional.of(employee));

        var result = adapter.findEmployee(employeeId);

        assertThat(result).isPresent();
        assertThat(result.get().limitsByResourceType()).containsEntry("DESK", 3);
        assertThat(result.get().limitsByResourceType()).containsEntry("LAPTOP", 1);
    }

    @Test
    void findEmployee_returns_empty_when_not_found() {
        when(employeeManagementInputPort.getEmployee(employeeId)).thenReturn(Optional.empty());

        var result = adapter.findEmployee(employeeId);

        assertThat(result).isEmpty();
    }

    @Test
    void findEmployee_empty_limits_when_employee_has_none() {
        var employee = new Employee(employeeId, "test@company.test", "Jan", "Kowalski", Role.EMPLOYEE,
        Map.of());
        when(employeeManagementInputPort.getEmployee(employeeId)).thenReturn(Optional.of(employee));

        var result = adapter.findEmployee(employeeId);

        assertThat(result).isPresent();
        assertThat(result.get().limitsByResourceType()).isEmpty();
    }
}
```

Source code 6.140: allocation/infrastructure/out/persistence/AllocationRequestRepositoryDateEdgeCasesTest.java - AllocationRequestRepositoryDateEdgeCasesTest

```
package com.deskflow.allocation.infrastructure.out.persistence;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.allocation.application.domain.model.AllocationRequest;
import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
import com.deskflow.allocation.application.domain.model.vo.DateRange;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.jdbc.test.autoconfigure.AutoConfigureTestDatabase;
import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.ActiveProfiles;

import java.time.LocalDate;
import java.util.List;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;

@DataJpaTest
@ActiveProfiles("test")
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Import({

        com.deskflow.allocation.infrastructure.out.persistence.HibernateAllocationRequestRepositoryAdapter.class,
        com.deskflow.allocation.infrastructure.out.persistence.AllocationRequestMapper.class
})
class AllocationRequestRepositoryDateEdgeCasesIntegrationTest {

    @Autowired
```

```
32      private AllocationRequestRepositoryPort requestRepository;
33
34      private final AllocationRequestFactory factory = new AllocationRequestFactory();
35      private final EmployeeId employeeId = EmployeeId.of(UUID.randomUUID());
36      private final ResourceId resourceId = ResourceId.of(UUID.randomUUID());
37
38      @Test
39      void overlapping_when_search_starts_on_request_end_is_included() {
40          DateRange requestRange = new DateRange(LocalDate.of(2025, 8, 1), LocalDate.of(2025, 8, 10));
41          AllocationRequest request = factory.create(employeeId, resourceId, requestRange, "test");
42          requestRepository.save(request);
43
44          DateRange searchRange = new DateRange(LocalDate.of(2025, 8, 10), LocalDate.of(2025, 8, 12));
45          List<AllocationRequest> results = requestRepository.findActiveForResource(resourceId, searchRange);
46
47          assertThat(results).anyMatch(r -> r.getId().equals(request.getId()));
48      }
49
50      @Test
51      void no_overlap_when_request_ends_before_search_starts_is_not_included() {
52          DateRange requestRange = new DateRange(LocalDate.of(2025, 8, 1), LocalDate.of(2025, 8, 10));
53          AllocationRequest request = factory.create(employeeId, resourceId, requestRange, "test");
54          requestRepository.save(request);
55
56          DateRange searchRange = new DateRange(LocalDate.of(2025, 8, 11), LocalDate.of(2025, 8, 12));
57          List<AllocationRequest> results = requestRepository.findActiveForResource(resourceId, searchRange);
58
59          assertThat(results).noneMatch(r -> r.getId().equals(request.getId()));
60      }
61
62      @Test
63      void overlapping_when_request_inside_search_is_included() {
64          DateRange requestRange = new DateRange(LocalDate.of(2025, 8, 5), LocalDate.of(2025, 8, 6));
65          AllocationRequest request = factory.create(employeeId, resourceId, requestRange, "test");
66          requestRepository.save(request);
67
68          DateRange searchRange = new DateRange(LocalDate.of(2025, 8, 1), LocalDate.of(2025, 8, 10));
69          List<AllocationRequest> results = requestRepository.findActiveForResource(resourceId, searchRange);
70
71          assertThat(results).anyMatch(r -> r.getId().equals(request.getId()));
72      }
73  }
```

Source code 6.141:
allocation/infrastructure/out/persistence/AllocationRequestRepositoryIntegrationTest.java - AllocationRequestRepositoryIntegrationTest

```
1   package com.deskflow.allocation.infrastructure.out.persistence;
2
3   import com.deskflow.common.application.domain.model.vo.EmployeeId;
4   import com.deskflow.allocation.application.domain.model.AllocationRequest;
5   import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
6   import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
7   import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
8   import com.deskflow.allocation.application.domain.model.vo.DateRange;
9   import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
10  import com.deskflow.common.application.domain.model.vo.ResourceId;
11  import org.junit.jupiter.api.Test;
12  import org.springframework.beans.factory.annotation.Autowired;
13  import org.springframework.boot.jdbc.test.autoconfigure.AutoConfigureTestDatabase;
14  import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;
15  import org.springframework.context.annotation.Import;
16  import org.springframework.test.context.ActiveProfiles;
17
18  import java.time.LocalDate;
19  import java.time.LocalDateTime;
20  import java.util.List;
```

```
21 import java.util.Optional;
22 import java.util.UUID;
23
24 import static org.assertj.core.api.Assertions.assertThat;
25
26 @DataJpaTest
27 @ActiveProfiles("test")
28 @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
29 @Import({
30
        com.deskflow.allocation.infrastructure.out.persistence.HibernateAllocationRequestRepositoryAdapter.class,
31      com.deskflow.allocation.infrastructure.out.persistence.AllocationRequestMapper.class
32 })
33 class AllocationRequestRepositoryIntegrationTest {
34
35     @Autowired
36     private AllocationRequestRepositoryPort requestRepository;
37
38     private final AllocationRequestFactory factory = new AllocationRequestFactory();
39     private final EmployeeId employeeId = EmployeeId.of(UUID.randomUUID());
40     private final ResourceId resourceId = ResourceId.of(UUID.randomUUID());
41
42     @Test
43     void save_and_find_request() {
44         DateRange dateRange = new DateRange(LocalDate.of(2025, 7, 1), LocalDate.of(2025, 7, 5));
45         AllocationRequest request = factory.create(employeeId, resourceId, dateRange, "test");
46
47         requestRepository.save(request);
48
49         Optional<AllocationRequest> found = requestRepository.find(request.getId());
50         assertThat(found).isPresent();
51         assertThat(found.get().getStatus()).isEqualTo(RequestStatus.SUBMITTED);
52         assertThat(found.get().getEmployeeId()).isEqualTo(employeeId);
53     }
54
55     @Test
56     void find_overdue_returns_active_past_due_requests() {
57         DateRange pastRange = new DateRange(LocalDate.of(2024, 1, 1), LocalDate.of(2024, 1, 10));
58         AllocationRequest request = AllocationRequest.reconstitute(
59                 AllocationRequestId.newId(),
60                 employeeId, resourceId, pastRange, "test",
61                 LocalDateTime.of(2024, 1, 1, 9, 0),
62                 RequestStatus.ACTIVE, LocalDateTime.of(2024, 1, 1, 10, 0), null, null
63         );
64         requestRepository.save(request);
65
66         List<AllocationRequest> overdue = requestRepository.findOverdue(LocalDateTime.now());
67
68         assertThat(overdue).anyMatch(r -> r.getId().equals(request.getId()));
69     }
70
71     @Test
72     void find_active_for_resource_with_overlapping_date_range() {
73         DateRange dateRange = new DateRange(LocalDate.of(2025, 8, 1), LocalDate.of(2025, 8, 10));
74         AllocationRequest request = factory.create(employeeId, resourceId, dateRange, "test");
75         requestRepository.save(request);
76
77         DateRange searchRange = new DateRange(LocalDate.of(2025, 8, 5), LocalDate.of(2025, 8, 15));
78         List<AllocationRequest> active = requestRepository.findActiveForResource(resourceId, searchRange);
79
80         assertThat(active).anyMatch(r -> r.getId().equals(request.getId()));
81     }
82 }
```

Source code 6.142:
allocation/infrastructure/out/resources/ResourceCatalogAdapterTest.java - ResourceCatalogAdapterTest

```
package com.deskflow.allocation.infrastructure.out.resources;

import com.deskflow.allocation.application.domain.model.ResourceAllocationProfile;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.resources.application.model.ResourceView;
import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;
import java.util.UUID;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ResourceCatalogAdapterTest {

    @Mock private ResourceManagementInputPort resourceManagementInputPort;

    private ResourceCatalogAdapter adapter;
    private ResourceId resourceId;

    @BeforeEach
    void setUp() {
        adapter = new ResourceCatalogAdapter(resourceManagementInputPort);
        resourceId = ResourceId.of(UUID.randomUUID());
    }

    @Test
    void findResource_returns_profile_when_found() {
        var view = new ResourceView(resourceId.getValue(), "DESK", "Biurko A", "A", 1, "101", true, true);
        when(resourceManagementInputPort.getResource(resourceId)).thenReturn(Optional.of(view));

        var result = adapter.findResource(resourceId);

        assertThat(result).isPresent();
        assertThat(result.get().id()).isEqualTo(resourceId);
        assertThat(result.get().type()).isEqualTo("DESK");
        assertThat(result.get().name()).isEqualTo("Biurko A");
        assertThat(result.get().building()).isEqualTo("A");
        assertThat(result.get().floor()).isEqualTo(1);
        assertThat(result.get().number()).isEqualTo("101");
        assertThat(result.get().active()).isTrue();
        assertThat(result.get().available()).isTrue();
    }

    @Test
    void findResource_returns_empty_when_not_found() {
        when(resourceManagementInputPort.getResource(resourceId)).thenReturn(Optional.empty());

        var result = adapter.findResource(resourceId);

        assertThat(result).isEmpty();
    }

    @Test
    void findActiveResources_returns_all_active() {
        var view1 = new ResourceView(UUID.randomUUID(), "DESK", "Biurko A", "A", 1, "101", true, true);
        var view2 = new ResourceView(UUID.randomUUID(), "LAPTOP", "Laptop X", "B", 2, "201", true, false);
        when(resourceManagementInputPort.getActiveResources()).thenReturn(List.of(view1, view2));

        var profiles = adapter.findActiveResources();

        assertThat(profiles).hasSize(2);
```

```
70          assertThat(profiles).extracting(ResourceAllocationProfile::active).containsOnly(true);
71      }
72
73      @Test
74      void findActiveResources_returns_empty_when_none() {
75          when(resourceManagementInputPort.getActiveResources()).thenReturn(List.of());
76
77          var profiles = adapter.findActiveResources();
78
79          assertThat(profiles).isEmpty();
80      }
81  }
```

6.2.2. Testy - Resources

Source code 6.143: resources/application/domain/model/LocationTest.java - LocationTest

```
1   package com.deskflow.resources.application.domain.model;
2
3   import com.deskflow.resources.application.domain.model.vo.Location;
4   import org.junit.jupiter.api.Test;
5
6   import static org.assertj.core.api.Assertions.*;
7
8   class LocationTest {
9
10      @Test
11      void creates_location() {
12          Location l = new Location("Building A", 2, "101");
13          assertThat(l.getBuilding()).isEqualTo("Building A");
14          assertThat(l.getFloor()).isEqualTo(2);
15          assertThat(l.getNumber()).isEqualTo("101");
16      }
17
18      @Test
19      void throws_when_building_blank() {
20          assertThatThrownBy(() -> new Location("", 1, "1"))
21                  .isInstanceOf(IllegalArgumentException.class);
22      }
23
24      @Test
25      void throws_when_number_blank() {
26          assertThatThrownBy(() -> new Location("Building A", 1, ""))
27                  .isInstanceOf(IllegalArgumentException.class);
28      }
29
30      @Test
31      void value_object_equality() {
32          Location l1 = new Location("A", 1, "101");
33          Location l2 = new Location("A", 1, "101");
34          assertThat(l1).isEqualTo(l2);
35          assertThat(l1.hashCode()).isEqualTo(l2.hashCode());
36      }
37
38      @Test
39      void not_equal_when_different_fields() {
40          assertThat(new Location("A", 1, "101")).isNotEqualTo(new Location("B", 1, "101"));
41          assertThat(new Location("A", 1, "101")).isNotEqualTo(new Location("A", 2, "101"));
42          assertThat(new Location("A", 1, "101")).isNotEqualTo(new Location("A", 1, "102"));
43      }
44  }
```

Source code 6.144: resources/application/domain/model/OfficeResourceTest.java - OfficeResourceTest

```
package com.deskflow.resources.application.domain.model;

import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
import com.deskflow.resources.application.domain.model.vo.Location;
import com.deskflow.common.application.domain.model.vo.ResourceId;
import com.deskflow.common.application.domain.model.vo.ResourceType;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.UUID;

import static org.assertj.core.api.Assertions.*;

class OfficeResourceTest {

    private OfficeResourceFactory factory;
    private static final Location LOC = new Location("A", 1, "101");

    @BeforeEach
    void setUp() {
        factory = new OfficeResourceFactory();
    }

    @Test
    void new_resource_is_active_and_available() {
        OfficeResource r = factory.create(ResourceType.DESK, "Desk 1", LOC);
        assertThat(r.isActive()).isTrue();
        assertThat(r.isAvailable()).isTrue();
    }

    @Test
    void throws_when_name_blank() {
        assertThatThrownBy(() -> factory.create(ResourceType.DESK, "", LOC))
                .isInstanceOf(IllegalArgumentException.class);
    }

    @Test
    void mark_as_unavailable() {
        OfficeResource r = factory.create(ResourceType.DESK, "Desk 1", LOC);
        r.markAsUnavailable();
        assertThat(r.isAvailable()).isFalse();
        assertThat(r.isActive()).isTrue();
    }

    @Test
    void mark_as_available_after_unavailable() {
        OfficeResource r = factory.create(ResourceType.DESK, "Desk 1", LOC);
        r.markAsUnavailable();
        r.markAsAvailable();
        assertThat(r.isAvailable()).isTrue();
    }

    @Test
    void deactivate_resource() {
        OfficeResource r = factory.create(ResourceType.DESK, "Desk 1", LOC);
        r.deactivate();
        assertThat(r.isActive()).isFalse();
        assertThat(r.isAvailable()).isFalse();
    }

    @Test
    void deactivate_twice_throws() {
        OfficeResource r = factory.create(ResourceType.DESK, "Desk 1", LOC);
        r.deactivate();
        assertThatThrownBy(r::deactivate).isInstanceOf(IllegalStateException.class);
    }

    @Test
    void update_changes_name_and_location() {
```

```
        OfficeResource r = factory.create(ResourceType.DESK, "Old name", LOC);
        Location newLoc = new Location("B", 3, "305");
        r.update("New name", newLoc);
        assertThat(r.getName()).isEqualTo("New name");
        assertThat(r.getLocation()).isEqualTo(newLoc);
    }

    @Test
    void cannot_mark_unavailable_on_deactivated_resource() {
        OfficeResource r = factory.create(ResourceType.DESK, "Desk", LOC);
        r.deactivate();
        assertThatThrownBy(r::markAsUnavailable).isInstanceOf(IllegalStateException.class);
    }

    @Test
    void resource_has_correct_type_and_location() {
        OfficeResource r = factory.create(ResourceType.CONFERENCE_ROOM, "Room A", LOC);
        assertThat(r.getType()).isEqualTo(ResourceType.CONFERENCE_ROOM);
        assertThat(r.getLocation()).isEqualTo(LOC);
    }

    @Test
    void reconstituted_resource_preserves_state() {
        OfficeResource r = OfficeResource.reconstitute(
                ResourceId.of(UUID.randomUUID()), ResourceType.LAPTOP, "Laptop X",
                LOC, false, false
        );
        assertThat(r.isActive()).isFalse();
        assertThat(r.isAvailable()).isFalse();
    }
}
```

Source code 6.145:
resources/application/domain/model/ResourceDeactivationDomainServicesTest.java - ResourceDeactivationDomainServicesTest

```
package com.deskflow.resources.application.domain.model;

import com.deskflow.common.application.domain.model.vo.ResourceType;
import com.deskflow.resources.application.domain.model.vo.Location;
import com.deskflow.resources.application.domain.model.vo.AllocationUsageStatus;
import com.deskflow.resources.application.domain.service.DeactivateOfficeResourceDomainService;
import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.EnumSource;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class ResourceDeactivationDomainServicesTest {

    private final DeactivateOfficeResourceDomainService deactivateOfficeResourceDomainService =
            new DeactivateOfficeResourceDomainService();

    @ParameterizedTest
    @EnumSource(value = AllocationUsageStatus.class, names = {"IN_USE", "APPROVED", "REQUESTED", "OVERDUE"})
    void deactivate_throws_when_resource_has_blocking_allocation_status(AllocationUsageStatus status) {
        OfficeResource resource = new OfficeResourceFactory().create(ResourceType.DESK, "Desk", new
        Location("A", 1, "101"));

        assertThatThrownBy(() -> deactivateOfficeResourceDomainService.deactivateOfficeResource(resource,
        List.of(status)))
                .isInstanceOf(IllegalStateException.class)
                .hasMessageContaining("active allocation requests");
```

```
30      }
31
32      @Test
33      void deactivate_resource_when_no_related_allocations() {
34          OfficeResource resource = new OfficeResourceFactory().create(ResourceType.DESK, "Desk", new
            Location("A", 1, "101"));
35
36          deactivateOfficeResourceDomainService.deactivateOfficeResource(resource, List.of());
37
38          assertThat(resource.isActive()).isFalse();
39          assertThat(resource.isAvailable()).isFalse();
40      }
41  }
```

Source code 6.146:
resources/application/domain/model/ResourceLifecycleDomainServicesTest.java - ResourceLifecycleDomainServicesTest

```
1   package com.deskflow.resources.application.domain.model;
2
3   import com.deskflow.common.application.domain.model.vo.ResourceType;
4   import com.deskflow.resources.application.domain.model.events.ResourceMadeAvailable;
5   import com.deskflow.resources.application.domain.model.events.ResourceMadeUnavailable;
6   import com.deskflow.resources.application.domain.model.events.ResourceUpdated;
7   import com.deskflow.resources.application.domain.model.vo.Location;
8   import com.deskflow.resources.application.domain.service.MarkOfficeResourceAvailableDomainService;
9   import com.deskflow.resources.application.domain.service.MarkOfficeResourceUnavailableDomainService;
10  import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
11  import com.deskflow.resources.application.domain.service.UpdateOfficeResourceDomainService;
12  import org.junit.jupiter.api.Test;
13
14  import static org.assertj.core.api.Assertions.assertThat;
15
16  class ResourceLifecycleDomainServicesTest {
17
18      private final UpdateOfficeResourceDomainService updateService = new UpdateOfficeResourceDomainService();
19      private final MarkOfficeResourceUnavailableDomainService unavailableService = new
        MarkOfficeResourceUnavailableDomainService();
20      private final MarkOfficeResourceAvailableDomainService availableService = new
        MarkOfficeResourceAvailableDomainService();
21
22      @Test
23      void update_service_updates_name_and_location_and_records_event() {
24          OfficeResource resource = resource();
25          Location newLocation = new Location("B", 2, "202");
26
27          updateService.updateOfficeResource(resource, "New desk", newLocation);
28
29          assertThat(resource.getName()).isEqualTo("New desk");
30          assertThat(resource.getLocation()).isEqualTo(newLocation);
31          assertThat(resource.flushEvents()).hasAtLeastOneElementOfType(ResourceUpdated.class);
32      }
33
34      @Test
35      void mark_unavailable_service_marks_resource_unavailable_and_records_event() {
36          OfficeResource resource = resource();
37
38          unavailableService.markOfficeResourceUnavailable(resource);
39
40          assertThat(resource.isAvailable()).isFalse();
41          assertThat(resource.flushEvents()).hasAtLeastOneElementOfType(ResourceMadeUnavailable.class);
42      }
43
44      @Test
45      void mark_available_service_marks_resource_available_and_records_event() {
46          OfficeResource resource = resource();
47          resource.markAsUnavailable();
```

```
48          resource.flushEvents();
49
50          availableService.markOfficeResourceAvailable(resource);
51
52          assertThat(resource.isAvailable()).isTrue();
53          assertThat(resource.flushEvents()).hasAtLeastOneElementOfType(ResourceMadeAvailable.class);
54      }
55
56      private OfficeResource resource() {
57          return new OfficeResourceFactory().create(ResourceType.DESK, "Desk", new Location("A", 1, "101"));
58      }
59  }
```

Source code 6.147:
resources/application/service/ResourceAppServiceMissingResourceTest.java - ResourceAppServiceMissingResourceTest

```
1   package com.deskflow.resources.application.service;
2
3   import com.deskflow.common.application.port.out.DomainEventPublisherPort;
4   import com.deskflow.resources.application.port.out.OfficeResourceRepositoryPort;
5   import com.deskflow.resources.application.port.out.ResourceAllocationRequestPort;
6   import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
7   import com.deskflow.resources.application.domain.service.DeactivateOfficeResourceDomainService;
8   import com.deskflow.resources.application.domain.service.MarkOfficeResourceAvailableDomainService;
9   import com.deskflow.resources.application.domain.service.MarkOfficeResourceUnavailableDomainService;
10  import com.deskflow.resources.application.domain.service.UpdateOfficeResourceDomainService;
11  import com.deskflow.common.application.domain.model.vo.ResourceId;
12  import org.junit.jupiter.api.BeforeEach;
13  import org.junit.jupiter.api.Test;
14  import org.junit.jupiter.api.extension.ExtendWith;
15  import org.mockito.Mock;
16  import org.mockito.junit.jupiter.MockitoExtension;
17
18  import java.util.NoSuchElementException;
19  import java.util.Optional;
20
21  import static org.assertj.core.api.Assertions.assertThatThrownBy;
22  import static org.mockito.Mockito.when;
23
24  @ExtendWith(MockitoExtension.class)
25  class ResourceAppServiceMissingResourceTest {
26
27      @Mock private OfficeResourceRepositoryPort resourceRepository;
28      @Mock private ResourceAllocationRequestPort allocationRequestPort;
29      @Mock private DomainEventPublisherPort eventPublisher;
30
31      private ResourceAppService appService;
32
33      @BeforeEach
34      void setUp() {
35          appService = new ResourceAppService(resourceRepository, allocationRequestPort, new
            OfficeResourceFactory(),
36                  new UpdateOfficeResourceDomainService(), new MarkOfficeResourceUnavailableDomainService(),
37                  new MarkOfficeResourceAvailableDomainService(),
38                  new DeactivateOfficeResourceDomainService(),
39                  eventPublisher);
40      }
41
42      @Test
43      void update_throws_when_resource_missing() {
44          ResourceId resourceId = ResourceId.of(java.util.UUID.randomUUID());
45          when(resourceRepository.find(resourceId)).thenReturn(Optional.empty());
46
47          assertThatThrownBy(() -> appService.updateResource(resourceId, "Desk", "A", 1, "101"))
48                  .isInstanceOf(NoSuchElementException.class)
49                  .hasMessageContaining("Resource not found");
```

```
50      }
51  }
```

Source code 6.148: resources/application/service/ResourceAppServiceTest.java - ResourceAppServiceTest

```
1   package com.deskflow.resources.application.service;
2
3   import com.deskflow.common.application.port.out.DomainEventPublisherPort;
4   import com.deskflow.common.application.domain.model.vo.ResourceId;
5   import com.deskflow.common.application.domain.model.vo.ResourceType;
6   import com.deskflow.resources.application.port.out.OfficeResourceRepositoryPort;
7   import com.deskflow.resources.application.port.out.ResourceAllocationRequestPort;
8   import com.deskflow.resources.application.domain.model.OfficeResource;
9   import com.deskflow.resources.application.domain.model.events.ResourceDeactivated;
10  import com.deskflow.resources.application.domain.model.events.ResourceMadeUnavailable;
11  import com.deskflow.resources.application.domain.model.events.ResourceUpdated;
12  import com.deskflow.resources.application.domain.model.vo.Location;
13  import com.deskflow.resources.application.domain.service.DeactivateOfficeResourceDomainService;
14  import com.deskflow.resources.application.domain.service.MarkOfficeResourceAvailableDomainService;
15  import com.deskflow.resources.application.domain.service.MarkOfficeResourceUnavailableDomainService;
16  import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
17  import com.deskflow.resources.application.domain.service.UpdateOfficeResourceDomainService;
18  import org.junit.jupiter.api.BeforeEach;
19  import org.junit.jupiter.api.Test;
20  import org.junit.jupiter.api.extension.ExtendWith;
21  import org.mockito.ArgumentCaptor;
22  import org.mockito.Mock;
23  import org.mockito.junit.jupiter.MockitoExtension;
24
25  import java.util.List;
26  import java.util.NoSuchElementException;
27  import java.util.Optional;
28  import java.util.UUID;
29
30  import static org.assertj.core.api.Assertions.assertThat;
31  import static org.assertj.core.api.Assertions.assertThatThrownBy;
32  import static org.mockito.Mockito.any;
33  import static org.mockito.Mockito.verify;
34  import static org.mockito.Mockito.when;
35
36  @ExtendWith(MockitoExtension.class)
37  class ResourceAppServiceTest {
38
39      @Mock private OfficeResourceRepositoryPort resourceRepository;
40      @Mock private ResourceAllocationRequestPort allocationRequestPort;
41      @Mock private DomainEventPublisherPort eventPublisher;
42
43      private ResourceAppService appService;
44
45      @BeforeEach
46      void setUp() {
47          appService = new ResourceAppService(resourceRepository, allocationRequestPort, new
            OfficeResourceFactory(),
48                  new UpdateOfficeResourceDomainService(), new MarkOfficeResourceUnavailableDomainService(),
49                  new MarkOfficeResourceAvailableDomainService(),
50                  new DeactivateOfficeResourceDomainService(),
51                  eventPublisher);
52      }
53
54      @Test
55      void add_resource_creates_active_available_resource_and_saves() {
56          ArgumentCaptor<OfficeResource> resourceCaptor = ArgumentCaptor.forClass(OfficeResource.class);
57
58          ResourceId id = appService.addResource(ResourceType.DESK, "Desk 1", "A", 1, "101");
59
60          verify(resourceRepository).save(resourceCaptor.capture());
61          OfficeResource saved = resourceCaptor.getValue();
```

```
62          assertThat(id).isEqualTo(saved.getId());
63          assertThat(saved.getType()).isEqualTo(ResourceType.DESK);
64          assertThat(saved.getName()).isEqualTo("Desk 1");
65          assertThat(saved.isActive()).isTrue();
66          assertThat(saved.isAvailable()).isTrue();
67      }
68
69      @Test
70      void update_resource_saves_and_publishes_update_event() {
71          OfficeResource resource = resource();
72          when(resourceRepository.find(resource.getId())).thenReturn(Optional.of(resource));
73
74          appService.updateResource(resource.getId(), "New name", "B", 2, "202");
75
76          verify(resourceRepository).save(resource);
77          verify(eventPublisher).publish(any(ResourceUpdated.class));
78          assertThat(resource.getName()).isEqualTo("New name");
79          assertThat(resource.getLocation()).isEqualTo(new Location("B", 2, "202"));
80      }
81
82      @Test
83      void mark_unavailable_saves_and_publishes_event() {
84          OfficeResource resource = resource();
85          when(resourceRepository.find(resource.getId())).thenReturn(Optional.of(resource));
86
87          appService.markUnavailable(resource.getId());
88
89          verify(resourceRepository).save(resource);
90          verify(eventPublisher).publish(any(ResourceMadeUnavailable.class));
91          assertThat(resource.isAvailable()).isFalse();
92      }
93
94      @Test
95      void deactivate_resource_checks_related_allocations_saves_and_publishes_event() {
96          OfficeResource resource = resource();
97          when(resourceRepository.find(resource.getId())).thenReturn(Optional.of(resource));
98          when(allocationRequestPort.findStatusesForResource(resource.getId())).thenReturn(List.of());
99
100         appService.deactivateResource(resource.getId());
101
102         verify(allocationRequestPort).findStatusesForResource(resource.getId());
103         verify(resourceRepository).save(resource);
104         verify(eventPublisher).publish(any(ResourceDeactivated.class));
105         assertThat(resource.isActive()).isFalse();
106     }
107
108     @Test
109     void mark_unavailable_throws_when_resource_missing() {
110         ResourceId resourceId = ResourceId.of(UUID.randomUUID());
111         when(resourceRepository.find(resourceId)).thenReturn(Optional.empty());
112
113         assertThatThrownBy(() -> appService.markUnavailable(resourceId))
114                 .isInstanceOf(NoSuchElementException.class)
115                 .hasMessageContaining("Resource not found");
116     }
117
118     private OfficeResource resource() {
119         return new OfficeResourceFactory().create(ResourceType.DESK, "Desk", new Location("A", 1, "101"));
120     }
121 }
```

Source code 6.149:
resources/infrastructure/in/events/ResourceAvailabilityEventListenerTest.java - ResourceAvailabilityEventListenerTest

```
1  package com.deskflow.resources.infrastructure.in.events;
2
```

```
3  import com.deskflow.allocation.application.domain.model.events.ResourceIssued;
4  import com.deskflow.allocation.application.domain.model.events.ResourceReturned;
5  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
6  import com.deskflow.common.application.domain.model.vo.EmployeeId;
7  import com.deskflow.common.application.domain.model.vo.ResourceId;
8  import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
9  import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.Test;
11 import org.junit.jupiter.api.extension.ExtendWith;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.time.LocalDateTime;
16 import java.util.UUID;
17
18 import static org.mockito.Mockito.*;
19
20 @ExtendWith(MockitoExtension.class)
21 class ResourceAvailabilityEventListenerTest {
22
23     @Mock private ResourceManagementInputPort resourceManagementInputPort;
24
25     private ResourceAvailabilityEventListener listener;
26     private ResourceId resourceId;
27
28     @BeforeEach
29     void setUp() {
30         listener = new ResourceAvailabilityEventListener(resourceManagementInputPort);
31         resourceId = ResourceId.of(UUID.randomUUID());
32     }
33
34     @Test
35     void onResourceIssued_marks_resource_unavailable() {
36         var event = new ResourceIssued(AllocationRequestId.newId(), EmployeeId.of(UUID.randomUUID()),
           resourceId, LocalDateTime.now());
37
38         listener.onResourceIssued(event);
39
40         verify(resourceManagementInputPort).markUnavailable(resourceId);
41     }
42
43     @Test
44     void onResourceReturned_marks_resource_available() {
45         var event = new ResourceReturned(AllocationRequestId.newId(), EmployeeId.of(UUID.randomUUID()),
           resourceId, LocalDateTime.now());
46
47         listener.onResourceReturned(event);
48
49         verify(resourceManagementInputPort).markAvailable(resourceId);
50     }
51 }
```

Source code 6.150:
resources/infrastructure/out/allocation/AllocationRequestAdapterTest.java - AllocationRequestAdapterTest

```
1  package com.deskflow.resources.infrastructure.out.allocation;
2
3  import com.deskflow.allocation.application.model.AllocationRequestView;
4  import com.deskflow.allocation.application.port.in.AllocationRequestInputPort;
5  import com.deskflow.common.application.domain.model.vo.ResourceId;
6  import com.deskflow.resources.application.domain.model.vo.AllocationUsageStatus;
7  import org.junit.jupiter.api.BeforeEach;
8  import org.junit.jupiter.api.Test;
9  import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.Mock;
11 import org.mockito.junit.jupiter.MockitoExtension;
```

```
12
13  import java.time.LocalDate;
14  import java.time.LocalDateTime;
15  import java.util.List;
16  import java.util.UUID;
17
18  import static org.assertj.core.api.Assertions.*;
19  import static org.mockito.Mockito.*;
20
21  @ExtendWith(MockitoExtension.class)
22  class AllocationRequestAdapterTest {
23
24      @Mock private AllocationRequestInputPort allocationRequestInputPort;
25
26      private AllocationRequestAdapter adapter;
27      private ResourceId resourceId;
28
29      @BeforeEach
30      void setUp() {
31          adapter = new AllocationRequestAdapter(allocationRequestInputPort);
32          resourceId = ResourceId.of(UUID.randomUUID());
33      }
34
35      private AllocationRequestView viewWithStatus(String status) {
36          return new AllocationRequestView(UUID.randomUUID(), UUID.randomUUID(), resourceId.getValue(),
37                  status, LocalDate.now(), LocalDate.now().plusDays(1), "test",
38                  LocalDateTime.now(), null, null, null);
39      }
40
41      @Test
42      void submitted_maps_to_requested() {
43          when(allocationRequestInputPort.findActiveForResource(resourceId))
44                  .thenReturn(List.of(viewWithStatus("SUBMITTED")));
45
46          var statuses = adapter.findStatusesForResource(resourceId);
47
48          assertThat(statuses).containsExactly(AllocationUsageStatus.REQUESTED);
49      }
50
51      @Test
52      void approved_maps_to_approved() {
53          when(allocationRequestInputPort.findActiveForResource(resourceId))
54                  .thenReturn(List.of(viewWithStatus("APPROVED")));
55
56          var statuses = adapter.findStatusesForResource(resourceId);
57
58          assertThat(statuses).containsExactly(AllocationUsageStatus.APPROVED);
59      }
60
61      @Test
62      void active_maps_to_in_use() {
63          when(allocationRequestInputPort.findActiveForResource(resourceId))
64                  .thenReturn(List.of(viewWithStatus("ACTIVE")));
65
66          var statuses = adapter.findStatusesForResource(resourceId);
67
68          assertThat(statuses).containsExactly(AllocationUsageStatus.IN_USE);
69      }
70
71      @Test
72      void overdue_maps_to_overdue() {
73          when(allocationRequestInputPort.findActiveForResource(resourceId))
74                  .thenReturn(List.of(viewWithStatus("OVERDUE")));
75
76          var statuses = adapter.findStatusesForResource(resourceId);
77
78          assertThat(statuses).containsExactly(AllocationUsageStatus.OVERDUE);
79      }
80
81      @Test
```

```
82      void rejected_cancelled_completed_map_to_closed() {
83          when(allocationRequestInputPort.findActiveForResource(resourceId))
84                  .thenReturn(List.of(
85                          viewWithStatus("REJECTED"),
86                          viewWithStatus("CANCELLED"),
87                          viewWithStatus("COMPLETED")
88                  ));
89
90          var statuses = adapter.findStatusesForResource(resourceId);
91
92          assertThat(statuses).allMatch(s -> s == AllocationUsageStatus.CLOSED);
93      }
94
95      @Test
96      void unknown_status_throws() {
97          when(allocationRequestInputPort.findActiveForResource(resourceId))
98                  .thenReturn(List.of(viewWithStatus("UNKNOWN")));
99
100         assertThatThrownBy(() -> adapter.findStatusesForResource(resourceId))
101                 .isInstanceOf(IllegalArgumentException.class)
102                 .hasMessageContaining("Unknown allocation status");
103     }
104
105     @Test
106     void returns_multiple_statuses() {
107         when(allocationRequestInputPort.findActiveForResource(resourceId))
108                 .thenReturn(List.of(
109                         viewWithStatus("APPROVED"),
110                         viewWithStatus("ACTIVE")
111                 ));
112
113         var statuses = adapter.findStatusesForResource(resourceId);
114
115         assertThat(statuses).containsExactly(AllocationUsageStatus.APPROVED, AllocationUsageStatus.IN_USE);
116     }
117 }
```

Source code 6.151:
resources/infrastructure/out/persistence/OfficeResourceRepositoryIntegrationTest.java - OfficeResourceRepositoryIntegrationTest

```
1   package com.deskflow.resources.infrastructure.out.persistence;
2
3   import com.deskflow.resources.application.domain.model.OfficeResource;
4   import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
5   import com.deskflow.resources.application.port.out.OfficeResourceRepositoryPort;
6   import com.deskflow.resources.application.domain.model.vo.Location;
7   import com.deskflow.common.application.domain.model.vo.ResourceId;
8   import com.deskflow.common.application.domain.model.vo.ResourceType;
9   import org.junit.jupiter.api.Test;
10  import org.springframework.beans.factory.annotation.Autowired;
11  import org.springframework.boot.jdbc.test.autoconfigure.AutoConfigureTestDatabase;
12  import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;
13  import org.springframework.context.annotation.Import;
14  import org.springframework.test.context.ActiveProfiles;
15
16  import java.util.List;
17  import java.util.UUID;
18
19  import static org.assertj.core.api.Assertions.assertThat;
20
21  @DataJpaTest
22  @ActiveProfiles("test")
23  @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
24  @Import({
25          com.deskflow.resources.infrastructure.out.persistence.HibernateOfficeResourceRepositoryAdapter.class,
26          com.deskflow.resources.infrastructure.out.persistence.OfficeResourceMapper.class
```

```
})
class OfficeResourceRepositoryIntegrationTest {

    @Autowired
    private OfficeResourceRepositoryPort resourceRepository;

    private final OfficeResourceFactory factory = new OfficeResourceFactory();

    @Test
    void save_and_find_resource() {
        OfficeResource resource = factory.create(ResourceType.DESK, "Desk A1", new Location("A", 1, "101"));

        resourceRepository.save(resource);

        ResourceId id = resource.getId();
        OfficeResource found = resourceRepository.find(id).orElseThrow();
        assertThat(found.getId()).isEqualTo(id);
        assertThat(found.getType()).isEqualTo(ResourceType.DESK);
        assertThat(found.getLocation()).isEqualTo(resource.getLocation());
    }

    @Test
    void find_active_returns_only_active_resources() {
        OfficeResource active = factory.create(ResourceType.DESK, "Desk A2", new Location("A", 1, "102"));
        OfficeResource inactive = factory.create(ResourceType.DESK, "Desk A3", new Location("A", 1, "103"));
        inactive.deactivate();

        resourceRepository.save(active);
        resourceRepository.save(inactive);

        List<OfficeResource> activeResources = resourceRepository.findActive();

        assertThat(activeResources).anyMatch(r -> r.getId().equals(active.getId()));
        assertThat(activeResources).noneMatch(r -> r.getId().equals(inactive.getId()));
    }

    @Test
    void find_active_returns_empty_when_no_active_resources() {
        OfficeResource inactive = factory.create(ResourceType.DESK, "Desk A4", new Location("A", 1, "104"));
        inactive.deactivate();
        resourceRepository.save(inactive);

        List<OfficeResource> activeResources = resourceRepository.findActive();

        assertThat(activeResources).isEmpty();
    }

    @Test
    void find_returns_empty_when_resource_missing() {
        var missingId = ResourceId.of(UUID.randomUUID());

        assertThat(resourceRepository.find(missingId)).isEmpty();
    }
}
```

6.2.3. Testy - Administration

Source code 6.152:
administration/application/domain/model/AdministeredAllocationRequestIdTest.java - AdministeredAllocationRequestIdTest

```
package com.deskflow.administration.application.domain.model;

import org.junit.jupiter.api.Test;

```

```
import java.util.UUID;

import static org.assertj.core.api.Assertions.*;

class AdministeredAllocationRequestIdTest {

    @Test
    void creates_from_uuid() {
        UUID uuid = UUID.randomUUID();
        var id = AdministeredAllocationRequestId.of(uuid);
        assertThat(id.getValue()).isEqualTo(uuid);
    }

    @Test
    void rejects_null() {
        assertThatThrownBy(() -> AdministeredAllocationRequestId.of(null))
                .isInstanceOf(NullPointerException.class);
    }

    @Test
    void equality() {
        UUID uuid = UUID.randomUUID();

        assertThat(AdministeredAllocationRequestId.of(uuid)).isEqualTo(AdministeredAllocationRequestId.of(uuid));
    }

    @Test
    void inequality() {
        assertThat(AdministeredAllocationRequestId.of(UUID.randomUUID()))
                .isNotEqualTo(AdministeredAllocationRequestId.of(UUID.randomUUID()));
    }

    @Test
    void toString_returns_uuid_string() {
        UUID uuid = UUID.randomUUID();
        assertThat(AdministeredAllocationRequestId.of(uuid)).hasToString(uuid.toString());
    }
}
```

Source code 6.153:
administration/application/domain/model/AdministeredAllocationRequestTest.java - AdministeredAllocationRequestTest

```
package com.deskflow.administration.application.domain.model;

import org.junit.jupiter.api.Test;

import java.util.UUID;

import static org.assertj.core.api.Assertions.*;

class AdministeredAllocationRequestTest {

    private AdministeredAllocationRequestId id = AdministeredAllocationRequestId.of(UUID.randomUUID());

    @Test
    void approve_from_submitted_passes() {
        var req = new AdministeredAllocationRequest(id, AdministeredAllocationStatus.SUBMITTED);

        req.ensureCanApprove();
    }

    @Test
    void approve_from_non_submitted_throws() {
        assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
        AdministeredAllocationStatus.APPROVED).ensureCanApprove())
                .isInstanceOf(IllegalStateException.class)
```

```
24                 .hasMessageContaining("approve");
25         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.ACTIVE).ensureCanApprove())
26                 .isInstanceOf(IllegalStateException.class);
27         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.REJECTED).ensureCanApprove())
28                 .isInstanceOf(IllegalStateException.class);
29         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.CANCELLED).ensureCanApprove())
30                 .isInstanceOf(IllegalStateException.class);
31         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.COMPLETED).ensureCanApprove())
32                 .isInstanceOf(IllegalStateException.class);
33         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.OVERDUE).ensureCanApprove())
34                 .isInstanceOf(IllegalStateException.class);
35     }
36
37     @Test
38     void reject_from_submitted_passes() {
39         var req = new AdministeredAllocationRequest(id, AdministeredAllocationStatus.SUBMITTED);
40
41         req.ensureCanReject();
42     }
43
44     @Test
45     void reject_from_non_submitted_throws() {
46         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.APPROVED).ensureCanReject())
47                 .isInstanceOf(IllegalStateException.class)
48                 .hasMessageContaining("reject");
49     }
50
51     @Test
52     void issue_from_approved_passes() {
53         var req = new AdministeredAllocationRequest(id, AdministeredAllocationStatus.APPROVED);
54
55         req.ensureCanIssue();
56     }
57
58     @Test
59     void issue_from_non_approved_throws() {
60         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.SUBMITTED).ensureCanIssue())
61                 .isInstanceOf(IllegalStateException.class)
62                 .hasMessageContaining("issue resource");
63     }
64
65     @Test
66     void return_from_active_passes() {
67         var req = new AdministeredAllocationRequest(id, AdministeredAllocationStatus.ACTIVE);
68
69         req.ensureCanReturn();
70     }
71
72     @Test
73     void return_from_overdue_passes() {
74         var req = new AdministeredAllocationRequest(id, AdministeredAllocationStatus.OVERDUE);
75
76         req.ensureCanReturn();
77     }
78
79     @Test
80     void return_from_non_active_or_overdue_throws() {
81         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.SUBMITTED).ensureCanReturn())
82                 .isInstanceOf(IllegalStateException.class)
83                 .hasMessageContaining("return resource");
84         assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
           AdministeredAllocationStatus.APPROVED).ensureCanReturn())
```

```
85                  .isInstanceOf(IllegalStateException.class);
86          assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
            AdministeredAllocationStatus.COMPLETED).ensureCanReturn())
87                  .isInstanceOf(IllegalStateException.class);
88          assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
            AdministeredAllocationStatus.REJECTED).ensureCanReturn())
89                  .isInstanceOf(IllegalStateException.class);
90          assertThatThrownBy(() -> new AdministeredAllocationRequest(id,
            AdministeredAllocationStatus.CANCELLED).ensureCanReturn())
91                  .isInstanceOf(IllegalStateException.class);
92      }
93  }
```

Source code 6.154:
administration/application/domain/model/AdministeredAllocationStatusTest.java - AdministeredAllocationStatusTest

```
1   package com.deskflow.administration.application.domain.model;
2
3   import org.junit.jupiter.api.Test;
4
5   import static org.assertj.core.api.Assertions.*;
6
7   class AdministeredAllocationStatusTest {
8
9       @Test
10      void from_returns_correct_enum() {
11
            assertThat(AdministeredAllocationStatus.from("SUBMITTED")).isEqualTo(AdministeredAllocationStatus.SUBMITTED);
12
            assertThat(AdministeredAllocationStatus.from("APPROVED")).isEqualTo(AdministeredAllocationStatus.APPROVED);
13
            assertThat(AdministeredAllocationStatus.from("ACTIVE")).isEqualTo(AdministeredAllocationStatus.ACTIVE);
14
            assertThat(AdministeredAllocationStatus.from("OVERDUE")).isEqualTo(AdministeredAllocationStatus.OVERDUE);
15
            assertThat(AdministeredAllocationStatus.from("REJECTED")).isEqualTo(AdministeredAllocationStatus.REJECTED);
16
            assertThat(AdministeredAllocationStatus.from("CANCELLED")).isEqualTo(AdministeredAllocationStatus.CANCELLED);
17
            assertThat(AdministeredAllocationStatus.from("COMPLETED")).isEqualTo(AdministeredAllocationStatus.COMPLETED);
18      }
19
20      @Test
21      void from_invalid_throws() {
22          assertThatThrownBy(() -> AdministeredAllocationStatus.from("INVALID"))
23                  .isInstanceOf(IllegalArgumentException.class);
24      }
25  }
```

Source code 6.155:
administration/application/service/AdministrationAppServiceTest.java - AdministrationAppServiceTest

```
1   package com.deskflow.administration.application.service;
2
3   import com.deskflow.administration.application.port.out.AdministrationAllocationPort;
4   import com.deskflow.administration.application.model.AdministeredAllocationView;
5   import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
6   import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;
7   import org.junit.jupiter.api.BeforeEach;
8   import org.junit.jupiter.api.Test;
9   import org.junit.jupiter.api.extension.ExtendWith;
10  import org.mockito.Mock;
```

```
11  import org.mockito.junit.jupiter.MockitoExtension;
12
13  import java.time.LocalDate;
14  import java.time.LocalDateTime;
15  import java.util.Optional;
16  import java.util.UUID;
17
18  import static org.assertj.core.api.Assertions.assertThatThrownBy;
19  import static org.mockito.Mockito.verify;
20  import static org.mockito.Mockito.when;
21
22  @ExtendWith(MockitoExtension.class)
23  class AdministrationAppServiceTest {
24
25      @Mock private AdministrationAllocationPort allocationPort;
26
27      private AdministrationAppService appService;
28
29      @BeforeEach
30      void setUp() {
31          appService = new AdministrationAppService(allocationPort);
32      }
33
34      private AdministeredAllocationView requestView(AdministeredAllocationRequestId requestId,
        AdministeredAllocationStatus status) {
35          return new AdministeredAllocationView(
36                  requestId.getValue(),
37                  UUID.randomUUID(),
38                  UUID.randomUUID(),
39                  status.name(),
40                  LocalDate.of(2025, 7, 1),
41                  LocalDate.of(2025, 7, 5),
42                  "test",
43                  LocalDateTime.now(),
44                  null,
45                  null,
46                  null
47          );
48      }
49
50      @Test
51      void approve_request_delegates_to_allocation_port() {
52          AdministeredAllocationRequestId requestId = AdministeredAllocationRequestId.of(UUID.randomUUID());
53          when(allocationPort.getRequest(requestId)).thenReturn(Optional.of(requestView(requestId,
            AdministeredAllocationStatus.SUBMITTED)));
54
55          appService.approveRequest(requestId);
56
57          verify(allocationPort).approveRequest(requestId);
58      }
59
60      @Test
61      void list_requests_delegates_to_allocation_port() {
62          appService.listRequests(AdministeredAllocationStatus.SUBMITTED);
63
64          verify(allocationPort).listRequests(AdministeredAllocationStatus.SUBMITTED);
65      }
66
67      @Test
68      void detect_overdue_delegates_to_allocation_port() {
69          appService.detectOverdueAllocations();
70
71          verify(allocationPort).detectOverdueAllocations();
72      }
73
74      @Test
75      void approve_rejects_non_submitted_request_before_calling_allocation() {
76          AdministeredAllocationRequestId requestId = AdministeredAllocationRequestId.of(UUID.randomUUID());
77          when(allocationPort.getRequest(requestId)).thenReturn(Optional.of(requestView(requestId,
            AdministeredAllocationStatus.ACTIVE)));
```

```
78
79          assertThatThrownBy(() -> appService.approveRequest(requestId))
80                  .isInstanceOf(IllegalStateException.class)
81                  .hasMessageContaining("requires status SUBMITTED");
82      }
83  }
```

Source code 6.156:
administration/application/service/OverdueAllocationSchedulerTest.java - OverdueAllocationSchedulerTest

```
1   package com.deskflow.administration.application.service;
2
3   import com.deskflow.administration.application.port.out.AdministrationAllocationPort;
4   import org.junit.jupiter.api.Test;
5   import org.junit.jupiter.api.extension.ExtendWith;
6   import org.mockito.Mock;
7   import org.mockito.junit.jupiter.MockitoExtension;
8
9   import static org.mockito.Mockito.*;
10
11  @ExtendWith(MockitoExtension.class)
12  class OverdueAllocationSchedulerTest {
13
14      @Mock private AdministrationAllocationPort allocationPort;
15
16      @Test
17      void detectOverdueAllocations_delegates_to_administration_service() {
18          var adminService = new AdministrationAppService(allocationPort);
19          var scheduler = new OverdueAllocationScheduler(adminService);
20
21          scheduler.detectOverdueAllocations();
22
23          verify(allocationPort).detectOverdueAllocations();
24      }
25  }
```

Source code 6.157:
administration/infrastructure/out/allocation/AllocationAdministrationAdapterTest.java - AllocationAdministrationAdapterTest

```
1   package com.deskflow.administration.infrastructure.out.allocation;
2
3   import com.deskflow.administration.application.model.AdministeredAllocationView;
4   import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
5   import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;
6   import com.deskflow.allocation.application.model.AllocationRequestView;
7   import com.deskflow.allocation.application.port.in.AllocationAdministrationInputPort;
8   import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
9   import org.junit.jupiter.api.BeforeEach;
10  import org.junit.jupiter.api.Test;
11  import org.junit.jupiter.api.extension.ExtendWith;
12  import org.mockito.Mock;
13  import org.mockito.junit.jupiter.MockitoExtension;
14
15  import java.time.LocalDate;
16  import java.time.LocalDateTime;
17  import java.util.List;
18  import java.util.Optional;
19  import java.util.UUID;
20
21  import static org.assertj.core.api.Assertions.*;
22  import static org.mockito.Mockito.*;
23
```

```
@ExtendWith(MockitoExtension.class)
class AllocationAdministrationAdapterTest {

    @Mock private AllocationAdministrationInputPort allocationAdministrationInputPort;

    private AllocationAdministrationAdapter adapter;

    @BeforeEach
    void setUp() {
        adapter = new AllocationAdministrationAdapter(allocationAdministrationInputPort);
    }

    @Test
    void getRequest_returns_admin_view_when_found() {
        var adminId = AdministeredAllocationRequestId.of(UUID.randomUUID());
        var allocView = new AllocationRequestView(adminId.getValue(), UUID.randomUUID(), UUID.randomUUID(),
                "SUBMITTED", LocalDate.now(), LocalDate.now().plusDays(1), "test",
                LocalDateTime.now(), null, null, null);
        when(allocationAdministrationInputPort.getRequest(AllocationRequestId.of(adminId.getValue())))
                .thenReturn(Optional.of(allocView));

        var result = adapter.getRequest(adminId);

        assertThat(result).isPresent();
        assertThat(result.get().id()).isEqualTo(adminId.getValue());
        assertThat(result.get().status()).isEqualTo("SUBMITTED");
    }

    @Test
    void getRequest_returns_empty_when_not_found() {
        var adminId = AdministeredAllocationRequestId.of(UUID.randomUUID());
        when(allocationAdministrationInputPort.getRequest(any())).thenReturn(Optional.empty());

        var result = adapter.getRequest(adminId);

        assertThat(result).isEmpty();
    }

    @Test
    void listRequests_converts_status_and_maps() {
        when(allocationAdministrationInputPort.listRequests(any()))
                .thenReturn(List.of(
                        new AllocationRequestView(UUID.randomUUID(), UUID.randomUUID(), UUID.randomUUID(),
                                "SUBMITTED", LocalDate.now(), LocalDate.now().plusDays(1), "test",
                                LocalDateTime.now(), null, null, null)
                ));

        var result = adapter.listRequests(AdministeredAllocationStatus.SUBMITTED);

        assertThat(result).hasSize(1);

        verify(allocationAdministrationInputPort).listRequests(com.deskflow.allocation.application.domain.model.vo.RequestStatu
    }

    @Test
    void listRequests_passes_null_status() {
        when(allocationAdministrationInputPort.listRequests(null)).thenReturn(List.of());

        var result = adapter.listRequests(null);

        assertThat(result).isEmpty();
    }

    @Test
    void approveRequest_delegates() {
        var adminId = AdministeredAllocationRequestId.of(UUID.randomUUID());

        adapter.approveRequest(adminId);

        verify(allocationAdministrationInputPort).approveRequest(AllocationRequestId.of(adminId.getValue()));
```

```
93      }
94
95      @Test
96      void rejectRequest_delegates() {
97          var adminId = AdministeredAllocationRequestId.of(UUID.randomUUID());
98
99          adapter.rejectRequest(adminId, "No space");
100
101         verify(allocationAdministrationInputPort).rejectRequest(AllocationRequestId.of(adminId.getValue()),
            "No space");
102     }
103
104     @Test
105     void issueResource_delegates() {
106         var adminId = AdministeredAllocationRequestId.of(UUID.randomUUID());
107
108         adapter.issueResource(adminId);
109
110         verify(allocationAdministrationInputPort).issueResource(AllocationRequestId.of(adminId.getValue()));
111     }
112
113     @Test
114     void returnResource_delegates() {
115         var adminId = AdministeredAllocationRequestId.of(UUID.randomUUID());
116
117         adapter.returnResource(adminId);
118
119         verify(allocationAdministrationInputPort).returnResource(AllocationRequestId.of(adminId.getValue()));
120     }
121
122     @Test
123     void monitorActiveAllocations_returns_mapped_views() {
124         when(allocationAdministrationInputPort.monitorActiveAllocations())
125                 .thenReturn(List.of(
126                         new AllocationRequestView(UUID.randomUUID(), UUID.randomUUID(), UUID.randomUUID(),
127                                 "ACTIVE", LocalDate.now(), LocalDate.now().plusDays(1), "test",
128                                 LocalDateTime.now(), LocalDateTime.now(), null, null)
129                 ));
130
131         var result = adapter.monitorActiveAllocations();
132
133         assertThat(result).hasSize(1);
134         assertThat(result.getFirst().status()).isEqualTo("ACTIVE");
135     }
136
137     @Test
138     void detectOverdueAllocations_delegates() {
139         adapter.detectOverdueAllocations();
140
141         verify(allocationAdministrationInputPort).detectOverdueAllocations();
142     }
143 }
```

6.2.4. Testy - Employees

Source code 6.158: employees/application/domain/model/EmployeeTest.java - EmployeeTest

```
1  package com.deskflow.employees.application.domain.model;
2
3  import com.deskflow.common.application.domain.model.vo.EmployeeId;
4  import com.deskflow.common.application.domain.model.vo.ResourceType;
5  import com.deskflow.employees.application.domain.model.vo.Role;
6  import org.junit.jupiter.api.Test;
7
8  import java.util.Map;
```

```
9  import java.util.UUID;
10
11 import static org.assertj.core.api.Assertions.assertThat;
12 import static org.assertj.core.api.Assertions.assertThatThrownBy;
13
14 class EmployeeTest {
15
16     @Test
17     void limit_for_missing_resource_type_defaults_to_one() {
18         Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
19                 Role.EMPLOYEE, Map.of(ResourceType.DESK, 3));
20
21         assertThat(employee.limitForType(ResourceType.LAPTOP)).isEqualTo(1);
22     }
23
24     @Test
25     void change_limit_updates_limit_for_resource_type() {
26         Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
27                 Role.EMPLOYEE, Map.of());
28
29         employee.changeLimit(ResourceType.DESK, 0);
30
31         assertThat(employee.limitForType(ResourceType.DESK)).isZero();
32     }
33
34     @Test
35     void change_limit_rejects_negative_amount() {
36         Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
37                 Role.EMPLOYEE, Map.of());
38
39         assertThatThrownBy(() -> employee.changeLimit(ResourceType.DESK, -1))
40                 .isInstanceOf(IllegalArgumentException.class)
41                 .hasMessageContaining("cannot be negative");
42     }
43
44     @Test
45     void getters_return_expected_values() {
46         var id = EmployeeId.of(UUID.randomUUID());
47         var employee = new Employee(id, "test@test.pl", "Jan", "Kowalski", Role.MANAGER,
48                 Map.of(ResourceType.DESK, 2));
49
50         assertThat(employee.getId()).isEqualTo(id);
51         assertThat(employee.getEmail()).isEqualTo("test@test.pl");
52         assertThat(employee.getFirstName()).isEqualTo("Jan");
53         assertThat(employee.getLastName()).isEqualTo("Kowalski");
54         assertThat(employee.getRole()).isEqualTo(Role.MANAGER);
55     }
56
57     @Test
58     void change_limit_overwrites_existing_limit() {
59         Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
60                 Role.EMPLOYEE, Map.of(ResourceType.DESK, 3));
61
62         employee.changeLimit(ResourceType.DESK, 5);
63
64         assertThat(employee.limitForType(ResourceType.DESK)).isEqualTo(5);
65     }
66
67     @Test
68     void constructor_makes_defensive_copy_of_limits() {
69         var mutableLimits = new java.util.HashMap<ResourceType, Integer>();
70         mutableLimits.put(ResourceType.DESK, 3);
71         Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
72                 Role.EMPLOYEE, mutableLimits);
73
74         mutableLimits.put(ResourceType.LAPTOP, 2);
75
76         assertThat(employee.limitForType(ResourceType.LAPTOP)).isEqualTo(1);
77     }
78
```

```
79      @Test
80      void getLimits_returns_unmodifiable_map() {
81          Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
82                  Role.EMPLOYEE, Map.of(ResourceType.DESK, 3));
83
84          assertThatThrownBy(() -> employee.getLimits().put(ResourceType.LAPTOP, 2))
85                  .isInstanceOf(UnsupportedOperationException.class);
86      }
87
88      @Test
89      void limit_for_zero_is_valid() {
90          Employee employee = new Employee(EmployeeId.of(UUID.randomUUID()), "a@test.pl", "Anna", "Test",
91                  Role.EMPLOYEE, Map.of(ResourceType.DESK, 0));
92
93          assertThat(employee.limitForType(ResourceType.DESK)).isZero();
94      }
95  }
```

Source code 6.159: employees/application/service/EmployeeAppServiceTest.java - EmployeeAppServiceTest

```
1   package com.deskflow.employees.application.service;
2
3   import com.deskflow.common.application.domain.model.vo.EmployeeId;
4   import com.deskflow.common.application.domain.model.vo.ResourceType;
5   import com.deskflow.employees.application.port.out.EmployeeRepositoryPort;
6   import com.deskflow.employees.application.domain.model.Employee;
7   import com.deskflow.employees.application.domain.model.vo.Role;
8   import org.junit.jupiter.api.BeforeEach;
9   import org.junit.jupiter.api.Test;
10  import org.junit.jupiter.api.extension.ExtendWith;
11  import org.mockito.Mock;
12  import org.mockito.junit.jupiter.MockitoExtension;
13
14  import java.util.Map;
15  import java.util.NoSuchElementException;
16  import java.util.Optional;
17  import java.util.UUID;
18
19  import static org.assertj.core.api.Assertions.assertThat;
20  import static org.assertj.core.api.Assertions.assertThatThrownBy;
21  import static org.mockito.Mockito.verify;
22  import static org.mockito.Mockito.when;
23
24  @ExtendWith(MockitoExtension.class)
25  class EmployeeAppServiceTest {
26
27      @Mock private EmployeeRepositoryPort employeeRepository;
28
29      private EmployeeAppService appService;
30
31      @BeforeEach
32      void setUp() {
33          appService = new EmployeeAppService(employeeRepository);
34      }
35
36      @Test
37      void upsert_limit_changes_employee_limit_and_saves() {
38          EmployeeId employeeId = EmployeeId.of(UUID.randomUUID());
39          Employee employee = new Employee(employeeId, "a@test.pl", "Anna", "Test", Role.EMPLOYEE, Map.of());
40          when(employeeRepository.find(employeeId)).thenReturn(Optional.of(employee));
41
42          appService.upsertLimit(employeeId, "DESK", 4);
43
44          assertThat(employee.limitForType(ResourceType.DESK)).isEqualTo(4);
45          verify(employeeRepository).save(employee);
46      }
47
```

```
48      @Test
49      void upsert_limit_throws_when_employee_missing() {
50          EmployeeId employeeId = EmployeeId.of(UUID.randomUUID());
51          when(employeeRepository.find(employeeId)).thenReturn(Optional.empty());
52
53          assertThatThrownBy(() -> appService.upsertLimit(employeeId, "DESK", 4))
54                  .isInstanceOf(NoSuchElementException.class)
55                  .hasMessageContaining("Employee not found");
56      }
57
58      @Test
59      void upsert_limit_rejects_invalid_resource_type() {
60          EmployeeId employeeId = EmployeeId.of(UUID.randomUUID());
61          Employee employee = new Employee(employeeId, "a@test.pl", "Anna", "Test", Role.EMPLOYEE, Map.of());
62          when(employeeRepository.find(employeeId)).thenReturn(Optional.of(employee));
63
64          assertThatThrownBy(() -> appService.upsertLimit(employeeId, "UNKNOWN", 4))
65                  .isInstanceOf(IllegalArgumentException.class);
66      }
67  }
```

Source code 6.160:
employees/infrastructure/out/persistence/EmployeeRepositoryIntegrationTest.java - EmployeeRepositoryIntegrationTest

```
1   package com.deskflow.employees.infrastructure.out.persistence;
2
3   import org.junit.jupiter.api.Test;
4   import org.springframework.beans.factory.annotation.Autowired;
5   import org.springframework.boot.jdbc.test.autoconfigure.AutoConfigureTestDatabase;
6   import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;
7   import org.springframework.test.context.ActiveProfiles;
8
9   import java.util.Map;
10  import java.util.UUID;
11
12  import static org.assertj.core.api.Assertions.*;
13
14  @DataJpaTest
15  @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
16  @ActiveProfiles("test")
17  class EmployeeRepositoryIntegrationTest {
18
19      @Autowired private SpringDataEmployeeRepository jpa;
20
21      @Test
22      void save_and_find_employee() {
23          EmployeeEntity entity = new EmployeeEntity();
24          entity.setId(UUID.randomUUID());
25          entity.setEmail("test@company.test");
26          entity.setFirstName("Jan");
27          entity.setLastName("Kowalski");
28          entity.setRole("EMPLOYEE");
29          entity.setPasswordHash("hash");
30          jpa.save(entity);
31
32          var found = jpa.findById(entity.getId());
33
34          assertThat(found).isPresent();
35          assertThat(found.get().getEmail()).isEqualTo("test@company.test");
36          assertThat(found.get().getFirstName()).isEqualTo("Jan");
37          assertThat(found.get().getLastName()).isEqualTo("Kowalski");
38          assertThat(found.get().getRole()).isEqualTo("EMPLOYEE");
39      }
40
41      @Test
42      void find_by_email() {
```

```
43          EmployeeEntity entity = new EmployeeEntity();
44          entity.setId(UUID.randomUUID());
45          entity.setEmail("unique@company.test");
46          entity.setFirstName("A");
47          entity.setLastName("B");
48          entity.setRole("MANAGER");
49          entity.setPasswordHash("hash");
50          jpa.save(entity);
51
52          var found = jpa.findByEmail("unique@company.test");
53
54          assertThat(found).isPresent();
55          assertThat(found.get().getId()).isEqualTo(entity.getId());
56      }
57
58      @Test
59      void find_by_email_returns_empty_when_missing() {
60          var found = jpa.findByEmail("nonexistent@company.test");
61
62          assertThat(found).isEmpty();
63      }
64
65      @Test
66      void find_returns_empty_when_missing() {
67          var found = jpa.findById(UUID.randomUUID());
68
69          assertThat(found).isEmpty();
70      }
71
72      @Test
73      void saves_and_loads_allocation_limits() {
74          EmployeeEntity entity = new EmployeeEntity();
75          entity.setId(UUID.randomUUID());
76          entity.setEmail("limits@company.test");
77          entity.setFirstName("A");
78          entity.setLastName("B");
79          entity.setRole("EMPLOYEE");
80          entity.setPasswordHash("hash");
81
82          AllocationLimitEntity limit = new AllocationLimitEntity();
83          limit.setId(UUID.randomUUID());
84          limit.setResourceType("DESK");
85          limit.setLimitAmount(3);
86          entity.addAllocationLimit(limit);
87
88          jpa.save(entity);
89
90          var found = jpa.findById(entity.getId()).orElseThrow();
91          assertThat(found.getAllocationLimits()).hasSize(1);
92          assertThat(found.getAllocationLimits().getFirst().getResourceType()).isEqualTo("DESK");
93          assertThat(found.getAllocationLimits().getFirst().getLimitAmount()).isEqualTo(3);
94      }
95
96      @Test
97      void find_all_returns_all_employees() {
98          EmployeeEntity e1 = new EmployeeEntity();
99          e1.setId(UUID.randomUUID());
100         e1.setEmail("e1@company.test");
101         e1.setFirstName("E1");
102         e1.setLastName("N1");
103         e1.setRole("EMPLOYEE");
104         e1.setPasswordHash("hash");
105
106         EmployeeEntity e2 = new EmployeeEntity();
107         e2.setId(UUID.randomUUID());
108         e2.setEmail("e2@company.test");
109         e2.setFirstName("E2");
110         e2.setLastName("N2");
111         e2.setRole("MANAGER");
112         e2.setPasswordHash("hash");
```

```
113
114         jpa.save(e1);
115         jpa.save(e2);
116
117         var all = jpa.findAll();
118
119         assertThat(all).hasSize(2);
120     }
121 }
```

6.2.5. Testy - Identity

Source code 6.161: identity/application/service/IdentityAccessAppServiceTest.java - IdentityAccessAppServiceTest

```
1  package com.deskflow.identity.application.service;
2
3  import com.deskflow.identity.application.model.IdentityProfile;
4  import com.deskflow.identity.application.port.out.AccessTokenIssuerPort;
5  import com.deskflow.identity.application.port.out.CredentialsAuthenticationPort;
6  import com.deskflow.identity.application.port.out.IdentityEmployeeProfilePort;
7  import org.junit.jupiter.api.BeforeEach;
8  import org.junit.jupiter.api.Test;
9  import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.Mock;
11 import org.mockito.junit.jupiter.MockitoExtension;
12
13 import java.util.NoSuchElementException;
14 import java.util.Optional;
15
16 import static org.assertj.core.api.Assertions.assertThat;
17 import static org.assertj.core.api.Assertions.assertThatThrownBy;
18 import static org.mockito.Mockito.verify;
19 import static org.mockito.Mockito.when;
20
21 @ExtendWith(MockitoExtension.class)
22 class IdentityAccessAppServiceTest {
23
24     @Mock private CredentialsAuthenticationPort credentialsAuthenticationPort;
25     @Mock private AccessTokenIssuerPort accessTokenIssuerPort;
26     @Mock private IdentityEmployeeProfilePort employeeProfilePort;
27
28     private IdentityAccessAppService appService;
29
30     @BeforeEach
31     void setUp() {
32         appService = new IdentityAccessAppService(credentialsAuthenticationPort, accessTokenIssuerPort,
           employeeProfilePort);
33     }
34
35     @Test
36     void login_authenticates_user_and_returns_profile_with_token() {
37         IdentityProfile profile = new IdentityProfile("user@test.pl", "MANAGER", "Jan", "Testowy", "emp-1");
38         when(employeeProfilePort.findByEmail("user@test.pl")).thenReturn(Optional.of(profile));
39         when(accessTokenIssuerPort.issueAccessToken("user@test.pl", "MANAGER")).thenReturn("token-123");
40
41         var identity = appService.login("user@test.pl", "password");
42
43         verify(credentialsAuthenticationPort).authenticate("user@test.pl", "password");
44         assertThat(identity.token()).isEqualTo("token-123");
45         assertThat(identity.email()).isEqualTo("user@test.pl");
46         assertThat(identity.role()).isEqualTo("MANAGER");
47         assertThat(identity.firstName()).isEqualTo("Jan");
48         assertThat(identity.lastName()).isEqualTo("Testowy");
49         assertThat(identity.employeeId()).isEqualTo("emp-1");
50     }
```

```
51
52      @Test
53      void login_throws_when_authenticated_user_has_no_employee_profile() {
54          when(employeeProfilePort.findByEmail("missing@test.pl")).thenReturn(Optional.empty());
55
56          assertThatThrownBy(() -> appService.login("missing@test.pl", "password"))
57                  .isInstanceOf(NoSuchElementException.class);
58
59          verify(credentialsAuthenticationPort).authenticate("missing@test.pl", "password");
60      }
61
62      @Test
63      void current_identity_returns_profile_by_email() {
64          IdentityProfile profile = new IdentityProfile("user@test.pl", "EMPLOYEE", "Anna", "Nowak", "emp-2");
65          when(employeeProfilePort.findByEmail("user@test.pl")).thenReturn(Optional.of(profile));
66
67          assertThat(appService.currentIdentity("user@test.pl")).contains(profile);
68      }
69  }
```

Source code 6.162:
identity/infrastructure/out/employees/EmployeeIdentityProfileAdapterTest.java - EmployeeIdentityProfileAdapterTest

```
1   package com.deskflow.identity.infrastructure.out.employees;
2
3   import com.deskflow.common.application.domain.model.vo.EmployeeId;
4   import com.deskflow.common.application.domain.model.vo.ResourceType;
5   import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
6   import com.deskflow.employees.application.domain.model.Employee;
7   import com.deskflow.employees.application.domain.model.vo.Role;
8   import org.junit.jupiter.api.BeforeEach;
9   import org.junit.jupiter.api.Test;
10  import org.junit.jupiter.api.extension.ExtendWith;
11  import org.mockito.Mock;
12  import org.mockito.junit.jupiter.MockitoExtension;
13
14  import java.util.Map;
15  import java.util.Optional;
16  import java.util.UUID;
17
18  import static org.assertj.core.api.Assertions.*;
19  import static org.mockito.Mockito.*;
20
21  @ExtendWith(MockitoExtension.class)
22  class EmployeeIdentityProfileAdapterTest {
23
24      @Mock private EmployeeManagementInputPort employeeManagementInputPort;
25
26      private EmployeeIdentityProfileAdapter adapter;
27
28      @BeforeEach
29      void setUp() {
30          adapter = new EmployeeIdentityProfileAdapter(employeeManagementInputPort);
31      }
32
33      @Test
34      void findByEmail_returns_mapped_profile_when_found() {
35          var employeeId = EmployeeId.of(UUID.randomUUID());
36          var employee = new Employee(employeeId, "test@company.test", "Jan", "Kowalski", Role.MANAGER,
            Map.of());
37          when(employeeManagementInputPort.getEmployeeByEmail("test@company.test"))
38                  .thenReturn(Optional.of(employee));
39
40          var result = adapter.findByEmail("test@company.test");
41
42          assertThat(result).isPresent();
```

```
        assertThat(result.get().email()).isEqualTo("test@company.test");
        assertThat(result.get().role()).isEqualTo("MANAGER");
        assertThat(result.get().firstName()).isEqualTo("Jan");
        assertThat(result.get().lastName()).isEqualTo("Kowalski");
        assertThat(result.get().employeeId()).isEqualTo(employeeId.toString());
    }

    @Test
    void findByEmail_returns_empty_when_not_found() {
        when(employeeManagementInputPort.getEmployeeByEmail("missing@company.test"))
                .thenReturn(Optional.empty());

        var result = adapter.findByEmail("missing@company.test");

        assertThat(result).isEmpty();
    }
}
```

Source code 6.163:
identity/infrastructure/out/employees/EmployeeUserCredentialsAdapterTest.java - EmployeeUserCredentialsAdapterTest

```
package com.deskflow.identity.infrastructure.out.employees;

import com.deskflow.employees.application.model.EmployeeCredentials;
import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class EmployeeUserCredentialsAdapterTest {

    @Mock private EmployeeManagementInputPort employeeManagementInputPort;

    private EmployeeUserCredentialsAdapter adapter;

    @BeforeEach
    void setUp() {
        adapter = new EmployeeUserCredentialsAdapter(employeeManagementInputPort);
    }

    @Test
    void findByEmail_returns_mapped_credentials_when_found() {
        var employeeCredentials = new EmployeeCredentials("test@company.test", "hashed_pass", "EMPLOYEE");
        when(employeeManagementInputPort.getEmployeeCredentialsByEmail("test@company.test"))
                .thenReturn(Optional.of(employeeCredentials));

        var result = adapter.findByEmail("test@company.test");

        assertThat(result).isPresent();
        assertThat(result.get().email()).isEqualTo("test@company.test");
        assertThat(result.get().passwordHash()).isEqualTo("hashed_pass");
        assertThat(result.get().role()).isEqualTo("EMPLOYEE");
    }

    @Test
    void findByEmail_returns_empty_when_not_found() {
        when(employeeManagementInputPort.getEmployeeCredentialsByEmail("missing@company.test"))
                .thenReturn(Optional.empty());
```

```
46
47          var result = adapter.findByEmail("missing@company.test");
48
49          assertThat(result).isEmpty();
50      }
51  }
```

Source code 6.164: identity/infrastructure/security/DeskflowUserDetailsServiceTest.java - DeskflowUserDetailsServiceTest

```
1   package com.deskflow.identity.infrastructure.security;
2
3   import com.deskflow.identity.application.model.UserCredentials;
4   import com.deskflow.identity.application.port.out.UserCredentialsPort;
5   import org.junit.jupiter.api.BeforeEach;
6   import org.junit.jupiter.api.Test;
7   import org.junit.jupiter.api.extension.ExtendWith;
8   import org.mockito.Mock;
9   import org.mockito.junit.jupiter.MockitoExtension;
10  import org.springframework.security.core.userdetails.UsernameNotFoundException;
11
12  import java.util.Optional;
13
14  import static org.assertj.core.api.Assertions.*;
15  import static org.mockito.Mockito.*;
16
17  @ExtendWith(MockitoExtension.class)
18  class DeskflowUserDetailsServiceTest {
19
20      @Mock private UserCredentialsPort userCredentialsPort;
21
22      private DeskflowUserDetailsService service;
23
24      @BeforeEach
25      void setUp() {
26          service = new DeskflowUserDetailsService(userCredentialsPort);
27      }
28
29      @Test
30      void loadUserByUsername_returns_user_details() {
31          when(userCredentialsPort.findByEmail("test@company.test"))
32                  .thenReturn(Optional.of(new UserCredentials("test@company.test", "hashed_pass",
                    "EMPLOYEE")));
33
34          var userDetails = service.loadUserByUsername("test@company.test");
35
36          assertThat(userDetails.getUsername()).isEqualTo("test@company.test");
37          assertThat(userDetails.getPassword()).isEqualTo("hashed_pass");
38          assertThat(userDetails.getAuthorities()).hasSize(1);
39          assertThat(userDetails.getAuthorities().iterator().next().getAuthority()).isEqualTo("ROLE_EMPLOYEE");
40      }
41
42      @Test
43      void loadUserByUsername_throws_when_not_found() {
44          when(userCredentialsPort.findByEmail("missing@company.test")).thenReturn(Optional.empty());
45
46          assertThatThrownBy(() -> service.loadUserByUsername("missing@company.test"))
47                  .isInstanceOf(UsernameNotFoundException.class)
48                  .hasMessageContaining("missing@company.test");
49      }
50  }
```

Source code 6.165: identity/infrastructure/security/JwtTokenProviderTest.java - JwtTokenProviderTest

```
1   package com.deskflow.identity.infrastructure.security;
```

```
2
3  import org.junit.jupiter.api.Test;
4
5  import static org.assertj.core.api.Assertions.assertThat;
6
7  class JwtTokenProviderTest {
8
9      private static final String SECRET = "test-secret-key-at-least-32-characters-long-for-hs256-tests";
10
11     @Test
12     void generated_token_contains_email_and_role() {
13         JwtTokenProvider provider = new JwtTokenProvider(SECRET, 60_000);
14
15         String token = provider.issueAccessToken("user@test.pl", "IT_ADMIN");
16
17         assertThat(provider.validateToken(token)).isTrue();
18         assertThat(provider.getEmailFromToken(token)).isEqualTo("user@test.pl");
19         assertThat(provider.getRoleFromToken(token)).isEqualTo("IT_ADMIN");
20     }
21
22     @Test
23     void invalid_token_is_rejected() {
24         JwtTokenProvider provider = new JwtTokenProvider(SECRET, 60_000);
25
26         assertThat(provider.validateToken("not-a-jwt")).isFalse();
27     }
28
29     @Test
30     void expired_token_is_rejected() throws InterruptedException {
31         JwtTokenProvider provider = new JwtTokenProvider(SECRET, 1);
32         String token = provider.issueAccessToken("user@test.pl", "EMPLOYEE");
33
34         Thread.sleep(5);
35
36         assertThat(provider.validateToken(token)).isFalse();
37     }
38 }
```

6.2.6. Testy - Notifications

Source code 6.166: notifications/application/service/NotificationAppServiceTest.java - NotificationAppServiceTest

```
1  package com.deskflow.notifications.application.service;
2
3  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
4  import com.deskflow.common.application.domain.model.vo.EmployeeId;
5  import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
6  import com.deskflow.employees.application.domain.model.Employee;
7  import com.deskflow.employees.application.domain.model.vo.Role;
8  import com.deskflow.notifications.application.port.out.NotificationPort;
9  import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.Test;
11 import org.junit.jupiter.api.extension.ExtendWith;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.util.Optional;
16 import java.util.UUID;
17
18 import static org.mockito.Mockito.*;
19
20 @ExtendWith(MockitoExtension.class)
21 class NotificationAppServiceTest {
22
23     @Mock private NotificationPort notifications;
```

```
24      @Mock private EmployeeManagementInputPort employeeManagement;
25
26      private NotificationAppService appService;
27      private AllocationRequestId requestId;
28      private EmployeeId employeeId;
29      private Employee employee;
30
31      @BeforeEach
32      void setUp() {
33          requestId = AllocationRequestId.newId();
34          employeeId = EmployeeId.of(UUID.randomUUID());
35          employee = new Employee(employeeId, "test@company.test", "Jan", "Kowalski", Role.EMPLOYEE,
            java.util.Map.of());
36          appService = new NotificationAppService(notifications, employeeManagement);
37      }
38
39      @Test
40      void submitted_sends_email() {
41          when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
42
43          appService.allocationRequestSubmitted(requestId, employeeId);
44
45          verify(notifications).send("test@company.test", "Request submitted",
46                  "Your allocation request has been submitted (ID: " + requestId + ").");
47      }
48
49      @Test
50      void approved_sends_email() {
51          when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
52
53          appService.allocationRequestApproved(requestId, employeeId);
54
55          verify(notifications).send("test@company.test", "Request approved",
56                  "Your request has been approved (ID: " + requestId + "). Please collect the resource.");
57      }
58
59      @Test
60      void rejected_sends_email_with_reason() {
61          when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
62
63          appService.allocationRequestRejected(requestId, employeeId, "Brak miejsca");
64
65          verify(notifications).send("test@company.test", "Request rejected",
66                  "Your request (ID: " + requestId + ") was rejected. Reason: Brak miejsca");
67      }
68
69      @Test
70      void cancelled_sends_email() {
71          when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
72
73          appService.allocationRequestCancelled(requestId, employeeId);
74
75          verify(notifications).send("test@company.test", "Request cancelled",
76                  "Your request (ID: " + requestId + ") has been cancelled.");
77      }
78
79      @Test
80      void resourceIssued_sends_email() {
81          when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
82
83          appService.resourceIssued(requestId, employeeId);
84
85          verify(notifications).send("test@company.test", "Resource issued",
86                  "The resource from request (ID: " + requestId + ") has been issued to you.");
87      }
88
89      @Test
90      void resourceReturned_sends_email() {
91          when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
92
```

```
93          appService.resourceReturned(requestId, employeeId);
94
95          verify(notifications).send("test@company.test", "Resource returned",
96                  "We confirm receipt of the resource from request (ID: " + requestId + ").");
97      }
98
99      @Test
100     void overdue_sends_email() {
101         when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
102
103         appService.allocationOverdue(requestId, employeeId);
104
105         verify(notifications).send("test@company.test", "Allocation overdue",
106                 "Your allocation (ID: " + requestId + ") is past its return date. Please return the resource
                    immediately.");
107     }
108
109     @Test
110     void employee_not_found_skips_email() {
111         when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.empty());
112
113         appService.allocationRequestSubmitted(requestId, employeeId);
114
115         verify(notifications, never()).send(any(), any(), any());
116     }
117
118     @Test
119     void employee_lookup_failure_does_not_propagate() {
120         when(employeeManagement.getEmployee(employeeId)).thenThrow(new RuntimeException("DB down"));
121
122         appService.allocationRequestSubmitted(requestId, employeeId);
123
124         verify(notifications, never()).send(any(), any(), any());
125     }
126
127     @Test
128     void notification_failure_does_not_propagate() {
129         when(employeeManagement.getEmployee(employeeId)).thenReturn(Optional.of(employee));
130         doThrow(new RuntimeException("SMTP down")).when(notifications).send(any(), any(), any());
131
132         appService.allocationRequestSubmitted(requestId, employeeId);
133     }
134 }
```

Source code 6.167:
notifications/infrastructure/in/events/NotificationEventListenerTest.java - NotificationEventListenerTest

```
1  package com.deskflow.notifications.infrastructure.in.events;
2
3  import com.deskflow.allocation.application.domain.model.events.*;
4  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
5  import com.deskflow.allocation.application.domain.model.vo.DateRange;
6  import com.deskflow.common.application.domain.model.vo.EmployeeId;
7  import com.deskflow.common.application.domain.model.vo.ResourceId;
8  import com.deskflow.notifications.application.port.in.NotificationEventInputPort;
9  import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.Test;
11 import org.junit.jupiter.api.extension.ExtendWith;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.time.LocalDate;
16 import java.time.LocalDateTime;
17 import java.util.UUID;
18
19 import static org.mockito.Mockito.*;
```

```
20
21  @ExtendWith(MockitoExtension.class)
22  class NotificationEventListenerTest {
23
24      @Mock private NotificationEventInputPort notificationInputPort;
25
26      private NotificationEventListener listener;
27      private AllocationRequestId requestId;
28      private EmployeeId employeeId;
29
30      private ResourceId resourceId;
31      private DateRange dateRange;
32      private LocalDateTime now;
33
34      @BeforeEach
35      void setUp() {
36          listener = new NotificationEventListener(notificationInputPort);
37          requestId = AllocationRequestId.newId();
38          employeeId = EmployeeId.of(UUID.randomUUID());
39          resourceId = ResourceId.of(UUID.randomUUID());
40          dateRange = new DateRange(LocalDate.now(), LocalDate.now().plusDays(1));
41          now = LocalDateTime.now();
42      }
43
44      @Test
45      void onRequestSubmitted_delegates() {
46          var event = new AllocationRequestSubmitted(requestId, employeeId, resourceId, dateRange, now);
47
48          listener.onRequestSubmitted(event);
49
50          verify(notificationInputPort).allocationRequestSubmitted(requestId, employeeId);
51      }
52
53      @Test
54      void onRequestApproved_delegates() {
55          var event = new AllocationRequestApproved(requestId, employeeId, now);
56
57          listener.onRequestApproved(event);
58
59          verify(notificationInputPort).allocationRequestApproved(requestId, employeeId);
60      }
61
62      @Test
63      void onRequestRejected_delegates() {
64          var event = new AllocationRequestRejected(requestId, employeeId, "No space", now);
65
66          listener.onRequestRejected(event);
67
68          verify(notificationInputPort).allocationRequestRejected(requestId, employeeId, "No space");
69      }
70
71      @Test
72      void onRequestCancelled_delegates() {
73          var event = new AllocationRequestCancelled(requestId, employeeId, now);
74
75          listener.onRequestCancelled(event);
76
77          verify(notificationInputPort).allocationRequestCancelled(requestId, employeeId);
78      }
79
80      @Test
81      void onResourceIssued_delegates() {
82          var event = new ResourceIssued(requestId, employeeId, resourceId, now);
83
84          listener.onResourceIssued(event);
85
86          verify(notificationInputPort).resourceIssued(requestId, employeeId);
87      }
88
89      @Test
```

```
90      void onResourceReturned_delegates() {
91          var event = new ResourceReturned(requestId, employeeId, resourceId, now);
92
93          listener.onResourceReturned(event);
94
95          verify(notificationInputPort).resourceReturned(requestId, employeeId);
96      }
97
98      @Test
99      void onAllocationOverdue_delegates() {
100         var event = new AllocationOverdue(requestId, employeeId, resourceId, now);
101
102         listener.onAllocationOverdue(event);
103
104         verify(notificationInputPort).allocationOverdue(requestId, employeeId);
105     }
106 }
```

Source code 6.168:
notifications/infrastructure/out/smtp/SmtpNotificationAdapterTest.java - SmtpNotificationAdapterTest

```
1   package com.deskflow.notifications.infrastructure.out.smtp;
2
3   import org.junit.jupiter.api.Test;
4   import org.junit.jupiter.api.extension.ExtendWith;
5   import org.mockito.ArgumentCaptor;
6   import org.mockito.Captor;
7   import org.mockito.Mock;
8   import org.mockito.junit.jupiter.MockitoExtension;
9   import org.springframework.mail.MailException;
10  import org.springframework.mail.SimpleMailMessage;
11  import org.springframework.mail.javamail.JavaMailSender;
12
13  import static org.assertj.core.api.Assertions.assertThat;
14  import static org.mockito.Mockito.*;
15
16  @ExtendWith(MockitoExtension.class)
17  class SmtpNotificationAdapterTest {
18
19      @Mock private JavaMailSender mailSender;
20      @Captor private ArgumentCaptor<SimpleMailMessage> messageCaptor;
21
22      @Test
23      void sends_email_with_correct_fields() {
24          SmtpNotificationAdapter adapter = new SmtpNotificationAdapter(mailSender);
25
26          adapter.send("user@test.com", "Subject", "Body text");
27
28          verify(mailSender).send(messageCaptor.capture());
29          SimpleMailMessage msg = messageCaptor.getValue();
30          assertThat(msg.getTo()).containsExactly("user@test.com");
31          assertThat(msg.getSubject()).isEqualTo("Subject");
32          assertThat(msg.getText()).isEqualTo("Body text");
33          assertThat(msg.getFrom()).isEqualTo("noreply@deskflow.local");
34      }
35
36      @Test
37      void mail_exception_is_caught_and_logged() {
38          SmtpNotificationAdapter adapter = new SmtpNotificationAdapter(mailSender);
39          doThrow(new MailException("SMTP error") {}).when(mailSender).send(any(SimpleMailMessage.class));
40
41          adapter.send("user@test.com", "Subject", "Body");
42      }
43
44      @Test
45      void sends_to_different_recipients() {
```

```
46          SmtpNotificationAdapter adapter = new SmtpNotificationAdapter(mailSender);
47
48          adapter.send("alice@test.com", "A", "msg1");
49          adapter.send("bob@test.com", "B", "msg2");
50
51          verify(mailSender, times(2)).send(any(SimpleMailMessage.class));
52      }
53  }
```

6.2.7. Testy - Common

Source code 6.169: common/application/domain/model/vo/EmployeeIdTest.java - EmployeeIdTest

```
1   package com.deskflow.common.application.domain.model.vo;
2
3   import org.junit.jupiter.api.Test;
4
5   import java.util.UUID;
6
7   import static org.assertj.core.api.Assertions.*;
8
9   class EmployeeIdTest {
10
11      @Test
12      void creates_from_uuid() {
13          UUID uuid = UUID.randomUUID();
14          EmployeeId id = EmployeeId.of(uuid);
15          assertThat(id.getValue()).isEqualTo(uuid);
16      }
17
18      @Test
19      void creates_from_string() {
20          UUID uuid = UUID.randomUUID();
21          EmployeeId id = EmployeeId.of(uuid.toString());
22          assertThat(id.getValue()).isEqualTo(uuid);
23      }
24
25      @Test
26      void rejects_null() {
27          assertThatThrownBy(() -> new EmployeeId(null))
28                  .isInstanceOf(NullPointerException.class);
29      }
30
31      @Test
32      void rejects_null_via_factory() {
33          assertThatThrownBy(() -> EmployeeId.of((UUID) null))
34                  .isInstanceOf(NullPointerException.class);
35      }
36
37      @Test
38      void equality() {
39          UUID uuid = UUID.randomUUID();
40          assertThat(EmployeeId.of(uuid)).isEqualTo(EmployeeId.of(uuid));
41      }
42
43      @Test
44      void inequality() {
45          assertThat(EmployeeId.of(UUID.randomUUID())).isNotEqualTo(EmployeeId.of(UUID.randomUUID()));
46      }
47
48      @Test
49      void hashCode_is_consistent() {
50          UUID uuid = UUID.randomUUID();
51          assertThat(EmployeeId.of(uuid)).hasSameHashCodeAs(EmployeeId.of(uuid));
52      }
```

```
53
54      @Test
55      void toString_returns_uuid_string() {
56          UUID uuid = UUID.randomUUID();
57          assertThat(EmployeeId.of(uuid)).hasToString(uuid.toString());
58      }
59  }
```

Source code 6.170: common/application/domain/model/vo/ResourceIdTest.java - ResourceIdTest

```
1   package com.deskflow.common.application.domain.model.vo;
2
3   import org.junit.jupiter.api.Test;
4
5   import java.util.UUID;
6
7   import static org.assertj.core.api.Assertions.*;
8
9   class ResourceIdTest {
10
11      @Test
12      void creates_from_uuid() {
13          UUID uuid = UUID.randomUUID();
14          ResourceId id = ResourceId.of(uuid);
15          assertThat(id.getValue()).isEqualTo(uuid);
16      }
17
18      @Test
19      void creates_from_string() {
20          UUID uuid = UUID.randomUUID();
21          ResourceId id = ResourceId.of(uuid.toString());
22          assertThat(id.getValue()).isEqualTo(uuid);
23      }
24
25      @Test
26      void rejects_null() {
27          assertThatThrownBy(() -> new ResourceId(null))
28                  .isInstanceOf(NullPointerException.class);
29      }
30
31      @Test
32      void equality() {
33          UUID uuid = UUID.randomUUID();
34          assertThat(ResourceId.of(uuid)).isEqualTo(ResourceId.of(uuid));
35      }
36
37      @Test
38      void inequality() {
39          assertThat(ResourceId.of(UUID.randomUUID())).isNotEqualTo(ResourceId.of(UUID.randomUUID()));
40      }
41
42      @Test
43      void toString_returns_uuid_string() {
44          UUID uuid = UUID.randomUUID();
45          assertThat(ResourceId.of(uuid)).hasToString(uuid.toString());
46      }
47  }
```

Source code 6.171: common/application/domain/model/vo/ResourceTypeTest.java - ResourceTypeTest

```
1   package com.deskflow.common.application.domain.model.vo;
2
3   import org.junit.jupiter.api.Test;
```

```
4
5  import static org.assertj.core.api.Assertions.*;
6
7  class ResourceTypeTest {
8
9      @Test
10     void has_expected_values() {
11         assertThat(ResourceType.values()).containsExactly(
12                 ResourceType.DESK,
13                 ResourceType.CONFERENCE_ROOM,
14                 ResourceType.LAPTOP,
15                 ResourceType.MONITOR,
16                 ResourceType.PROJECTOR,
17                 ResourceType.VEHICLE
18         );
19     }
20
21     @Test
22     void valueOf_returns_correct_enum() {
23         assertThat(ResourceType.valueOf("DESK")).isEqualTo(ResourceType.DESK);
24         assertThat(ResourceType.valueOf("LAPTOP")).isEqualTo(ResourceType.LAPTOP);
25     }
26
27     @Test
28     void valueOf_invalid_throws() {
29         assertThatThrownBy(() -> ResourceType.valueOf("INVALID"))
30                 .isInstanceOf(IllegalArgumentException.class);
31     }
32 }
```

Source code 6.172: common/infrastructure/in/web/AuthorizationSystemTest.java - AuthorizationSystemTest

```
1  package com.deskflow.common.infrastructure.in.web;
2
3  import com.deskflow.identity.infrastructure.security.JwtTokenProvider;
4  import org.junit.jupiter.api.BeforeEach;
5  import org.junit.jupiter.api.Test;
6  import org.springframework.beans.factory.annotation.Autowired;
7  import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
8  import org.springframework.boot.test.context.SpringBootTest;
9  import org.springframework.test.context.ActiveProfiles;
10 import org.springframework.test.web.servlet.MockMvc;
11
12 import java.util.UUID;
13
14 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
15 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
16 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
17 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
18
19 @SpringBootTest
20 @AutoConfigureMockMvc
21 @ActiveProfiles("test")
22 class AuthorizationSystemTest {
23
24     @Autowired
25     private MockMvc mockMvc;
26
27     @Autowired
28     private JwtTokenProvider tokenProvider;
29
30     private String employeeToken;
31     private String managerToken;
32     private String adminToken;
33
34     @BeforeEach
35     void setUp() {
```

```
        employeeToken = tokenProvider.issueAccessToken("test.employee@company.test", "EMPLOYEE");
        managerToken = tokenProvider.issueAccessToken("test.manager@company.test", "MANAGER");
        adminToken = tokenProvider.issueAccessToken("test.admin@company.test", "IT_ADMIN");
    }

    @Test
    void admin_endpoint_requires_authentication() throws Exception {
        mockMvc.perform(get("/api/administration/requests"))
                .andExpect(status().isUnauthorized());
    }

    @Test
    void invalid_token_is_unauthorized() throws Exception {
        mockMvc.perform(get("/api/administration/requests")
                        .header("Authorization", "Bearer invalid-token"))
                .andExpect(status().isUnauthorized());
    }

    @Test
    void employee_role_cannot_access_admin_endpoints() throws Exception {
        mockMvc.perform(get("/api/administration/requests")
                        .header("Authorization", "Bearer " + employeeToken))
                .andExpect(status().isForbidden());
    }

    @Test
    void manager_role_can_access_admin_endpoints() throws Exception {
        mockMvc.perform(get("/api/administration/requests")
                        .header("Authorization", "Bearer " + managerToken))
                .andExpect(status().isOk());
    }

    @Test
    void resources_browse_requires_authentication() throws Exception {
        mockMvc.perform(get("/api/resources"))
                .andExpect(status().isUnauthorized());
    }

    @Test
    void auth_endpoints_are_public_but_still_validate_input() throws Exception {
        mockMvc.perform(post("/api/auth/login")
                        .contentType("application/json")
                        .content("{}"))
                .andExpect(status().isBadRequest());
    }

    @Test
    void manager_role_cannot_delete_resources() throws Exception {
        mockMvc.perform(delete("/api/resources/" + UUID.randomUUID())
                        .header("Authorization", "Bearer " + managerToken))
                .andExpect(status().isForbidden());
    }

    @Test
    void admin_role_is_authorized_to_delete_resources() throws Exception {
        mockMvc.perform(delete("/api/resources/" + UUID.randomUUID())
                        .header("Authorization", "Bearer " + adminToken))
                .andExpect(status().isNotFound());
    }
}
```

Source code 6.173: common/infrastructure/in/web/FullAllocationCycleSystemTest.java - FullAllocationCycleSystemTest

```
package com.deskflow.common.infrastructure.in.web;

import com.deskflow.common.application.domain.model.vo.EmployeeId;
import com.deskflow.allocation.application.domain.model.AllocationRequest;
```

```
5  import com.deskflow.allocation.application.domain.service.AllocationRequestFactory;
6  import com.deskflow.allocation.application.port.out.AllocationRequestRepositoryPort;
7  import com.deskflow.allocation.application.domain.model.vo.AllocationRequestId;
8  import com.deskflow.allocation.application.domain.model.vo.DateRange;
9  import com.deskflow.allocation.application.domain.model.vo.RequestStatus;
10 import com.deskflow.resources.application.domain.model.OfficeResource;
11 import com.deskflow.resources.application.domain.service.OfficeResourceFactory;
12 import com.deskflow.resources.application.port.out.OfficeResourceRepositoryPort;
13 import com.deskflow.resources.application.domain.model.vo.Location;
14 import com.deskflow.common.application.domain.model.vo.ResourceType;
15 import com.deskflow.employees.infrastructure.out.persistence.AllocationLimitEntity;
16 import com.deskflow.employees.infrastructure.out.persistence.EmployeeEntity;
17 import com.deskflow.employees.infrastructure.out.persistence.SpringDataEmployeeRepository;
18 import com.deskflow.identity.infrastructure.security.JwtTokenProvider;
19 import com.fasterxml.jackson.databind.ObjectMapper;
20 import org.junit.jupiter.api.BeforeEach;
21 import org.junit.jupiter.api.Test;
22 import org.springframework.beans.factory.annotation.Autowired;
23 import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
24 import org.springframework.boot.test.context.SpringBootTest;
25 import org.springframework.http.MediaType;
26 import org.springframework.security.crypto.password.PasswordEncoder;
27 import org.springframework.test.context.ActiveProfiles;
28 import org.springframework.test.web.servlet.MockMvc;
29 import org.springframework.transaction.annotation.Transactional;
30
31 import java.time.LocalDate;
32 import java.util.Map;
33 import java.util.UUID;
34
35 import static org.assertj.core.api.Assertions.assertThat;
36 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
37 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
38
39 @SpringBootTest
40 @AutoConfigureMockMvc
41 @ActiveProfiles("test")
42 @Transactional
43 class FullAllocationCycleSystemTest {
44
45     @Autowired MockMvc mockMvc;
46     @Autowired ObjectMapper objectMapper;
47     @Autowired JwtTokenProvider tokenProvider;
48     @Autowired AllocationRequestRepositoryPort requestRepository;
49     @Autowired OfficeResourceRepositoryPort resourceRepository;
50     @Autowired SpringDataEmployeeRepository employeeRepo;
51     @Autowired PasswordEncoder passwordEncoder;
52
53     private String employeeToken;
54     private String managerToken;
55     private EmployeeId employeeId;
56     private UUID resourceUuid;
57
58     @BeforeEach
59     void setUp() {
60         EmployeeEntity employee = new EmployeeEntity();
61         employee.setId(UUID.randomUUID());
62         employee.setEmail("test.employee@company.test");
63         employee.setFirstName("Test");
64         employee.setLastName("Employee");
65         employee.setRole("EMPLOYEE");
66         employee.setPasswordHash(passwordEncoder.encode("pass"));
67         AllocationLimitEntity limit = new AllocationLimitEntity();
68         limit.setId(UUID.randomUUID());
69         limit.setResourceType("DESK");
70         limit.setLimitAmount(3);
71         employee.addAllocationLimit(limit);
72         employeeRepo.save(employee);
73         employeeId = EmployeeId.of(employee.getId());
74
```

```
        EmployeeEntity manager = new EmployeeEntity();
        manager.setId(UUID.randomUUID());
        manager.setEmail("test.kierownik@firma.pl");
        manager.setFirstName("Test");
        manager.setLastName("Kierownik");
        manager.setRole("MANAGER");
        manager.setPasswordHash(passwordEncoder.encode("pass"));
        employeeRepo.save(manager);

        employeeToken = tokenProvider.issueAccessToken("test.employee@company.test", "EMPLOYEE");
        managerToken = tokenProvider.issueAccessToken("test.kierownik@firma.pl", "MANAGER");

        OfficeResource resource = new OfficeResourceFactory().create(
                ResourceType.DESK, "Biurko testowe", new Location("A", 1, "T01")
        );
        resourceRepository.save(resource);
        resourceUuid = resource.getId().getValue();
    }

    @Test
    void full_cycle_submit_approve_issue_return() throws Exception {
        // UC-02: Submit request
        String requestBody = objectMapper.writeValueAsString(Map.of(
                "resourceId", resourceUuid,
                "dateFrom", LocalDate.now().plusDays(1).toString(),
                "dateTo", LocalDate.now().plusDays(5).toString(),
                "justification", "Test systemowy"
        ));

        String responseBody = mockMvc.perform(post("/api/requests")
                        .header("Authorization", "Bearer " + employeeToken)
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(requestBody))
                .andExpect(status().isCreated())
                .andReturn().getResponse().getContentAsString();

        UUID requestId = UUID.fromString(
                objectMapper.readTree(responseBody).get("requestId").asText()
        );

        assertThat(requestRepository.find(AllocationRequestId.of(requestId)).orElseThrow().getStatus())
                .isEqualTo(RequestStatus.SUBMITTED);

        // UC-05: Approve request
        mockMvc.perform(put("/api/administration/requests/" + requestId + "/approve")
                        .header("Authorization", "Bearer " + managerToken))
                .andExpect(status().isOk());
        assertThat(requestRepository.find(AllocationRequestId.of(requestId)).orElseThrow().getStatus())
                .isEqualTo(RequestStatus.APPROVED);

        // UC-07: Issue resource
        mockMvc.perform(put("/api/administration/requests/" + requestId + "/issue")
                        .header("Authorization", "Bearer " + managerToken))
                .andExpect(status().isOk());
        assertThat(requestRepository.find(AllocationRequestId.of(requestId)).orElseThrow().getStatus())
                .isEqualTo(RequestStatus.ACTIVE);

        // UC-08: Return resource
        mockMvc.perform(put("/api/administration/requests/" + requestId + "/return")
                        .header("Authorization", "Bearer " + managerToken))
                .andExpect(status().isOk());
        assertThat(requestRepository.find(AllocationRequestId.of(requestId)).orElseThrow().getStatus())
                .isEqualTo(RequestStatus.COMPLETED);
    }

    @Test
    void employee_can_cancel_submitted_request() throws Exception {
        DateRange dateRange = new DateRange(LocalDate.now().plusDays(10), LocalDate.now().plusDays(15));
        AllocationRequest request = new AllocationRequestFactory().create(
```

```
                employeeId, com.deskflow.common.application.domain.model.vo.ResourceId.of(resourceUuid),
                dateRange, "test"
        );
        requestRepository.save(request);

        mockMvc.perform(put("/api/requests/" + request.getId().getValue() + "/cancel")
                        .header("Authorization", "Bearer " + employeeToken))
                .andExpect(status().isOk());

        assertThat(requestRepository.find(request.getId()).orElseThrow().getStatus())
                .isEqualTo(RequestStatus.CANCELLED);
    }
}
```

Source code 6.174: common/infrastructure/in/web/GlobalExceptionHandlerTest.java - GlobalExceptionHandlerTest

```
package com.deskflow.common.infrastructure.in.web;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.core.MethodParameter;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.BeanPropertyBindingResult;
import org.springframework.web.bind.MethodArgumentNotValidException;

import java.util.Map;
import java.util.NoSuchElementException;

import static org.assertj.core.api.Assertions.*;

class GlobalExceptionHandlerTest {

    private GlobalExceptionHandler handler;

    @BeforeEach
    void setUp() {
        handler = new GlobalExceptionHandler();
    }

    @Test
    void illegalState_returns_409() {
        var ex = new IllegalStateException("Resource is not available");

        ResponseEntity<?> response = handler.handleBusinessRule(ex);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CONFLICT);
        assertThat(response.getBody()).isEqualTo(Map.of("error", "Resource is not available"));
    }

    @Test
    void illegalArgument_returns_400() {
        var ex = new IllegalArgumentException("Invalid resource type");

        ResponseEntity<?> response = handler.handleBadArgument(ex);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.BAD_REQUEST);
        assertThat(response.getBody()).isEqualTo(Map.of("error", "Invalid resource type"));
    }

    @Test
    void noSuchElement_returns_404() {
        var ex = new NoSuchElementException("Employee not found");

        ResponseEntity<?> response = handler.handleNotFound(ex);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.NOT_FOUND);
```

```
52          assertThat(response.getBody()).isEqualTo(Map.of("error", "Employee not found"));
53      }
54
55      @Test
56      void validationError_returns_400_with_details() throws Exception {
57          var method =
            GlobalExceptionHandlerTest.class.getDeclaredMethod("validationError_returns_400_with_details");
58          var param = new MethodParameter(method, -1);
59          var binding = new BeanPropertyBindingResult(new Object(), "object");
60          var ex = new MethodArgumentNotValidException(param, binding);
61
62          ResponseEntity<?> response = handler.handleValidation(ex);
63
64          assertThat(response.getStatusCode()).isEqualTo(HttpStatus.BAD_REQUEST);
65          assertThat(response.getBody()).isInstanceOf(Map.class);
66          @SuppressWarnings("unchecked")
67          Map<String, Object> body = (Map<String, Object>) response.getBody();
68          assertThat(body).containsKey("error");
69          assertThat(body).containsKey("details");
70      }
71  }
```

Source code 6.175: common/infrastructure/in/web/InputValidationSystemTest.java - InputValidationSystemTest

```
1   package com.deskflow.common.infrastructure.in.web;
2
3   import com.deskflow.employees.infrastructure.out.persistence.AllocationLimitEntity;
4   import com.deskflow.employees.infrastructure.out.persistence.EmployeeEntity;
5   import com.deskflow.employees.infrastructure.out.persistence.SpringDataEmployeeRepository;
6   import com.deskflow.identity.infrastructure.security.JwtTokenProvider;
7   import org.junit.jupiter.api.BeforeEach;
8   import org.junit.jupiter.api.Test;
9   import org.springframework.beans.factory.annotation.Autowired;
10  import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
11  import org.springframework.boot.test.context.SpringBootTest;
12  import org.springframework.security.crypto.password.PasswordEncoder;
13  import org.springframework.test.context.ActiveProfiles;
14  import org.springframework.test.web.servlet.MockMvc;
15  import org.springframework.transaction.annotation.Transactional;
16
17  import java.util.UUID;
18
19  import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
20  import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
21  import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
22
23  @SpringBootTest
24  @AutoConfigureMockMvc
25  @ActiveProfiles("test")
26  @Transactional
27  class InputValidationSystemTest {
28
29      @Autowired
30      private MockMvc mockMvc;
31
32      @Autowired
33      private JwtTokenProvider tokenProvider;
34
35      @Autowired
36      private SpringDataEmployeeRepository employeeRepo;
37
38      @Autowired
39      private PasswordEncoder passwordEncoder;
40
41      private String employeeToken;
42      private String managerToken;
43
```

```
44      @BeforeEach
45      void setUp() {
46          EmployeeEntity employee = new EmployeeEntity();
47          employee.setId(UUID.randomUUID());
48          employee.setEmail("test.employee@company.test");
49          employee.setFirstName("Test");
50          employee.setLastName("Employee");
51          employee.setRole("EMPLOYEE");
52          employee.setPasswordHash(passwordEncoder.encode("pass"));
53          AllocationLimitEntity limit = new AllocationLimitEntity();
54          limit.setId(UUID.randomUUID());
55          limit.setResourceType("DESK");
56          limit.setLimitAmount(3);
57          employee.addAllocationLimit(limit);
58          employeeRepo.save(employee);
59
60          employeeToken = tokenProvider.issueAccessToken("test.employee@company.test", "EMPLOYEE");
61          managerToken = tokenProvider.issueAccessToken("test.kierownik@firma.pl", "MANAGER");
62      }
63
64      @Test
65      void submit_request_requires_resource_id() throws Exception {
66          String body = "{" +
67                  "\"dateFrom\":\"2099-01-01\"," +
68                  "\"dateTo\":\"2099-01-02\"," +
69                  "\"justification\":\"test\"" +
70                  "}";
71
72          mockMvc.perform(post("/api/requests")
73                          .header("Authorization", "Bearer " + employeeToken)
74                          .contentType("application/json")
75                          .content(body))
76                  .andExpect(status().isBadRequest());
77      }
78
79      @Test
80      void submit_request_requires_date_from() throws Exception {
81          String body = "{" +
82                  "\"resourceId\":\"" + UUID.randomUUID() + "\"," +
83                  "\"dateTo\":\"2099-01-02\"," +
84                  "\"justification\":\"test\"" +
85                  "}";
86
87          mockMvc.perform(post("/api/requests")
88                          .header("Authorization", "Bearer " + employeeToken)
89                          .contentType("application/json")
90                          .content(body))
91                  .andExpect(status().isBadRequest());
92      }
93
94      @Test
95      void submit_request_rejects_past_date_from() throws Exception {
96          String body = "{" +
97                  "\"resourceId\":\"" + UUID.randomUUID() + "\"," +
98                  "\"dateFrom\":\"2000-01-01\"," +
99                  "\"dateTo\":\"2099-01-02\"," +
100                 "\"justification\":\"test\"" +
101                 "}";
102
103         mockMvc.perform(post("/api/requests")
104                         .header("Authorization", "Bearer " + employeeToken)
105                         .contentType("application/json")
106                         .content(body))
107                 .andExpect(status().isBadRequest());
108     }
109
110     @Test
111     void submit_request_requires_date_to() throws Exception {
112         String body = "{" +
113                 "\"resourceId\":\"" + UUID.randomUUID() + "\"," +
```

```
                "\"dateFrom\":\"2099-01-01\"," +
                "\"justification\":\"test\"" +
                "}";

        mockMvc.perform(post("/api/requests")
                        .header("Authorization", "Bearer " + employeeToken)
                        .contentType("application/json")
                        .content(body))
                .andExpect(status().isBadRequest());
    }

    @Test
    void add_resource_requires_type() throws Exception {
        String body = "{" +
                "\"name\":\"Desk 1\"," +
                "\"building\":\"A\"," +
                "\"floor\":1," +
                "\"number\":\"101\"" +
                "}";

        mockMvc.perform(post("/api/resources")
                        .header("Authorization", "Bearer " + managerToken)
                        .contentType("application/json")
                        .content(body))
                .andExpect(status().isBadRequest());
    }

    @Test
    void add_resource_requires_name() throws Exception {
        String body = "{" +
                "\"type\":\"DESK\"," +
                "\"name\":\"\"," +
                "\"building\":\"A\"," +
                "\"floor\":1," +
                "\"number\":\"101\"" +
                "}";

        mockMvc.perform(post("/api/resources")
                        .header("Authorization", "Bearer " + managerToken)
                        .contentType("application/json")
                        .content(body))
                .andExpect(status().isBadRequest());
    }

    @Test
    void reject_request_requires_reason() throws Exception {
        String body = "{" +
                "\"reason\":\"\"" +
                "}";

        mockMvc.perform(put("/api/administration/requests/" + UUID.randomUUID() + "/reject")
                        .header("Authorization", "Bearer " + managerToken)
                        .contentType("application/json")
                        .content(body))
                .andExpect(status().isBadRequest());
    }
}
```

Source code 6.176: common/infrastructure/in/web/RestControllerWebMvcTest.java - RestControllerWebMvcTest

```
package com.deskflow.common.infrastructure.in.web;

import com.deskflow.administration.application.domain.model.AdministeredAllocationRequestId;
import com.deskflow.administration.application.domain.model.AdministeredAllocationStatus;
import com.deskflow.administration.application.model.AdministeredAllocationView;
import com.deskflow.administration.application.port.in.AdministrationInputPort;
import com.deskflow.administration.infrastructure.in.web.AdministrationController;
```

```
8  import com.deskflow.allocation.application.model.ResourceCatalogItem;
9  import com.deskflow.allocation.application.port.in.AllocationRequestInputPort;
10 import com.deskflow.allocation.infrastructure.in.web.AllocationRequestController;
11 import com.deskflow.allocation.infrastructure.in.web.CurrentUserResolver;
12 import com.deskflow.common.application.domain.model.vo.EmployeeId;
13 import com.deskflow.common.application.domain.model.vo.ResourceId;
14 import com.deskflow.common.application.domain.model.vo.ResourceType;
15 import com.deskflow.employees.application.domain.model.Employee;
16 import com.deskflow.employees.application.domain.model.vo.Role;
17 import com.deskflow.employees.application.port.in.EmployeeManagementInputPort;
18 import com.deskflow.employees.infrastructure.in.web.EmployeeAdminController;
19 import com.deskflow.identity.application.model.AuthenticatedIdentity;
20 import com.deskflow.identity.application.model.IdentityProfile;
21 import com.deskflow.identity.application.port.in.IdentityAccessInputPort;
22 import com.deskflow.identity.infrastructure.in.web.AuthController;
23 import com.deskflow.identity.infrastructure.security.JwtAuthenticationFilter;
24 import com.deskflow.identity.infrastructure.security.JwtTokenProvider;
25 import com.deskflow.identity.infrastructure.security.SecurityConfig;
26 import com.deskflow.resources.application.port.in.ResourceManagementInputPort;
27 import com.deskflow.resources.infrastructure.in.web.ResourceController;
28 import org.junit.jupiter.api.BeforeEach;
29 import org.junit.jupiter.api.Test;
30 import org.springframework.beans.factory.annotation.Autowired;
31 import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
32 import org.springframework.context.annotation.Import;
33 import org.springframework.http.MediaType;
34 import org.springframework.test.context.bean.override.mockito.MockitoBean;
35 import org.springframework.test.web.servlet.MockMvc;
36 import org.springframework.test.web.servlet.request.RequestPostProcessor;
37
38 import java.time.LocalDate;
39 import java.time.LocalDateTime;
40 import java.util.List;
41 import java.util.Map;
42 import java.util.NoSuchElementException;
43 import java.util.Optional;
44 import java.util.UUID;
45
46 import static org.mockito.ArgumentMatchers.any;
47 import static org.mockito.ArgumentMatchers.eq;
48 import static org.mockito.Mockito.doThrow;
49 import static org.mockito.Mockito.verify;
50 import static org.mockito.Mockito.when;
51 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
52 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
53 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
54 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
55 import static
   org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
56 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
57 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
58
59 @WebMvcTest(controllers = {
60         AllocationRequestController.class,
61         AdministrationController.class,
62         EmployeeAdminController.class,
63         AuthController.class,
64         ResourceController.class
65 })
66 @Import({SecurityConfig.class, JwtAuthenticationFilter.class, GlobalExceptionHandler.class})
67 class RestControllerWebMvcTest {
68
69     @Autowired private MockMvc mockMvc;
70
71     @MockitoBean private JwtTokenProvider tokenProvider;
72     @MockitoBean private AllocationRequestInputPort allocationRequestInputPort;
73     @MockitoBean private AdministrationInputPort administrationInputPort;
74     @MockitoBean private EmployeeManagementInputPort employeeManagementInputPort;
75     @MockitoBean private IdentityAccessInputPort identityAccessInputPort;
76     @MockitoBean private ResourceManagementInputPort resourceManagementInputPort;
```

```
77      @MockitoBean private CurrentUserResolver currentUserResolver;
78
79      private final UUID employeeUuid = UUID.fromString("00000000-0000-0000-0000-000000000001");
80      private final UUID resourceUuid = UUID.fromString("00000000-0000-0000-0000-000000000002");
81      private final UUID requestUuid = UUID.fromString("00000000-0000-0000-0000-000000000003");
82
83      @BeforeEach
84      void setUpTokens() {
85          token("employee-token", "employee@test.pl", "EMPLOYEE");
86          token("manager-token", "manager@test.pl", "MANAGER");
87          token("admin-token", "admin@test.pl", "IT_ADMIN");
88          when(tokenProvider.validateToken("invalid-token")).thenReturn(false);
89      }
90
91      @Test
92      void auth_login_returns_identity_with_token() throws Exception {
93          when(identityAccessInputPort.login("user@test.pl", "secret"))
94                  .thenReturn(new AuthenticatedIdentity("jwt", "user@test.pl", "MANAGER", "Jan", "Kowalski",
                    employeeUuid.toString()));
95
96          mockMvc.perform(post("/api/auth/login")
97                          .contentType(MediaType.APPLICATION_JSON)
98                          .content("{\"email\":\"user@test.pl\",\"password\":\"secret\"}"))
99                  .andExpect(status().isOk())
100                 .andExpect(jsonPath("$.token").value("jwt"))
101                 .andExpect(jsonPath("$.email").value("user@test.pl"))
102                 .andExpect(jsonPath("$.role").value("MANAGER"));
103     }
104
105     @Test
106     void auth_me_returns_current_identity_for_valid_token() throws Exception {
107         when(identityAccessInputPort.currentIdentity("employee@test.pl"))
108                 .thenReturn(Optional.of(new IdentityProfile("employee@test.pl", "EMPLOYEE", "Anna", "Nowak",
                    employeeUuid.toString())));
109
110         mockMvc.perform(get("/api/auth/me").with(authenticated("employee-token")))
111                 .andExpect(status().isOk())
112                 .andExpect(jsonPath("$.email").value("employee@test.pl"))
113                 .andExpect(jsonPath("$.token").doesNotExist());
114     }
115
116     @Test
117     void auth_me_returns_not_found_when_profile_is_missing() throws Exception {
118         when(identityAccessInputPort.currentIdentity("employee@test.pl")).thenReturn(Optional.empty());
119
120         mockMvc.perform(get("/api/auth/me").with(authenticated("employee-token")))
121                 .andExpect(status().isNotFound());
122     }
123
124     @Test
125     void resource_browsing_requires_authentication() throws Exception {
126         mockMvc.perform(get("/api/resources"))
127                 .andExpect(status().isForbidden());
128     }
129
130     @Test
131     void resource_browsing_returns_catalog_for_authenticated_user() throws Exception {
132         when(allocationRequestInputPort.browseResources()).thenReturn(List.of(
133                 new ResourceCatalogItem(resourceUuid, "DESK", "Desk 1", "A", 1, "101", true)
134         ));
135
136         mockMvc.perform(get("/api/resources").with(authenticated("employee-token")))
137                 .andExpect(status().isOk())
138                 .andExpect(jsonPath("$[0].id").value(resourceUuid.toString()))
139                 .andExpect(jsonPath("$[0].available").value(true));
140     }
141
142     @Test
143     void employee_cannot_add_resource() throws Exception {
144         mockMvc.perform(post("/api/resources")
```

```
                        .with(authenticated("employee-token"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(resourceBody("DESK", "Desk 1", "A", "101")))
                .andExpect(status().isForbidden());
    }

    @Test
    void manager_can_add_resource() throws Exception {
        when(resourceManagementInputPort.addResource(ResourceType.DESK, "Desk 1", "A", 1, "101"))
                .thenReturn(ResourceId.of(resourceUuid));

        mockMvc.perform(post("/api/resources")
                        .with(authenticated("manager-token"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(resourceBody("DESK", "Desk 1", "A", "101")))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.resourceId").value(resourceUuid.toString()));

        verify(resourceManagementInputPort).addResource(ResourceType.DESK, "Desk 1", "A", 1, "101");
    }

    @Test
    void add_resource_rejects_blank_building_and_number() throws Exception {
        mockMvc.perform(post("/api/resources")
                        .with(authenticated("manager-token"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(resourceBody("DESK", "Desk 1", "", "")))
                .andExpect(status().isBadRequest());
    }

    @Test
    void add_resource_rejects_invalid_type() throws Exception {
        mockMvc.perform(post("/api/resources")
                        .with(authenticated("manager-token"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(resourceBody("INVALID", "Desk 1", "A", "101")))
                .andExpect(status().isBadRequest());
    }

    @Test
    void add_resource_rejects_malformed_json() throws Exception {
        mockMvc.perform(post("/api/resources")
                        .with(authenticated("manager-token"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{"))
                .andExpect(status().isBadRequest());
    }

    @Test
    void add_resource_rejects_missing_content_type() throws Exception {
        mockMvc.perform(post("/api/resources")
                        .with(authenticated("manager-token"))
                        .content(resourceBody("DESK", "Desk 1", "A", "101")))
                .andExpect(status().isUnsupportedMediaType());
    }

    @Test
    void manager_can_update_resource_availability() throws Exception {
        mockMvc.perform(put("/api/resources/" + resourceUuid + "/unavailable")
                        .with(authenticated("manager-token")))
                .andExpect(status().isOk());

        mockMvc.perform(put("/api/resources/" + resourceUuid + "/available")
                        .with(authenticated("manager-token")))
                .andExpect(status().isOk());

        verify(resourceManagementInputPort).markUnavailable(ResourceId.of(resourceUuid));
        verify(resourceManagementInputPort).markAvailable(ResourceId.of(resourceUuid));
    }

```

```
215     @Test
216     void resource_path_rejects_invalid_uuid() throws Exception {
217         mockMvc.perform(put("/api/resources/not-a-uuid/unavailable")
218                         .with(authenticated("manager-token")))
219                 .andExpect(status().isBadRequest());
220     }
221
222     @Test
223     void only_it_admin_can_delete_resource() throws Exception {
224         mockMvc.perform(delete("/api/resources/" + resourceUuid)
225                         .with(authenticated("manager-token")))
226                 .andExpect(status().isForbidden());
227
228         mockMvc.perform(delete("/api/resources/" + resourceUuid)
229                         .with(authenticated("admin-token")))
230                 .andExpect(status().isOk());
231
232         verify(resourceManagementInputPort).deactivateResource(ResourceId.of(resourceUuid));
233     }
234
235     @Test
236     void employee_admin_lists_employees_with_limits() throws Exception {
237         Employee employee = new Employee(EmployeeId.of(employeeUuid), "employee@test.pl", "Anna", "Nowak",
        Role.EMPLOYEE,
238                 Map.of(ResourceType.DESK, 2));
239         when(employeeManagementInputPort.listEmployees()).thenReturn(List.of(employee));
240
241         mockMvc.perform(get("/api/administration/employees").with(authenticated("manager-token")))
242                 .andExpect(status().isOk())
243                 .andExpect(jsonPath("$[0].id").value(employeeUuid.toString()))
244                 .andExpect(jsonPath("$[0].limits.DESK").value(2));
245     }
246
247     @Test
248     void employee_admin_rejects_invalid_limit_update() throws Exception {
249         mockMvc.perform(put("/api/administration/employees/" + employeeUuid + "/limits")
250                         .with(authenticated("manager-token"))
251                         .contentType(MediaType.APPLICATION_JSON)
252                         .content("{\"resourceType\":\"DESK\",\"amount\":-1}"))
253                 .andExpect(status().isBadRequest());
254
255         mockMvc.perform(put("/api/administration/employees/" + employeeUuid + "/limits")
256                         .with(authenticated("manager-token"))
257                         .contentType(MediaType.APPLICATION_JSON)
258                         .content("{\"resourceType\":\"INVALID\",\"amount\":1}"))
259                 .andExpect(status().isBadRequest());
260     }
261
262     @Test
263     void employee_admin_updates_limit_and_later_lists_updated_employee() throws Exception {
264         mockMvc.perform(put("/api/administration/employees/" + employeeUuid + "/limits")
265                         .with(authenticated("manager-token"))
266                         .contentType(MediaType.APPLICATION_JSON)
267                         .content("{\"resourceType\":\"DESK\",\"amount\":5}"))
268                 .andExpect(status().isOk());
269
270         verify(employeeManagementInputPort).upsertLimit(EmployeeId.of(employeeUuid), "DESK", 5);
271
272         Employee updatedEmployee = new Employee(EmployeeId.of(employeeUuid), "employee@test.pl", "Anna",
        "Nowak", Role.EMPLOYEE,
273                 Map.of(ResourceType.DESK, 5));
274         when(employeeManagementInputPort.listEmployees()).thenReturn(List.of(updatedEmployee));
275
276         mockMvc.perform(get("/api/administration/employees").with(authenticated("manager-token")))
277                 .andExpect(status().isOk())
278                 .andExpect(jsonPath("$[0].id").value(employeeUuid.toString()))
279                 .andExpect(jsonPath("$[0].limits.DESK").value(5));
280     }
281
282     @Test
```

```
283     void administration_lists_requests_with_status_filter_and_employee_names() throws Exception {
284         when(administrationInputPort.listRequests(AdministeredAllocationStatus.SUBMITTED))
285                 .thenReturn(List.of(adminView("SUBMITTED")));
286         when(employeeManagementInputPort.getEmployee(EmployeeId.of(employeeUuid)))
287                 .thenReturn(Optional.of(new Employee(EmployeeId.of(employeeUuid), "employee@test.pl", "Anna",
                    "Nowak", Role.EMPLOYEE, Map.of())));
288
289         mockMvc.perform(get("/api/administration/requests?status=SUBMITTED")
290                 .with(authenticated("manager-token")))
291                 .andExpect(status().isOk())
292                 .andExpect(jsonPath("$[0].id").value(requestUuid.toString()))
293                 .andExpect(jsonPath("$[0].employeeFirstName").value("Anna"));
294
295         verify(administrationInputPort).listRequests(AdministeredAllocationStatus.SUBMITTED);
296     }
297
298     @Test
299     void administration_rejects_invalid_status_filter() throws Exception {
300         mockMvc.perform(get("/api/administration/requests?status=BAD_STATUS")
301                         .with(authenticated("manager-token")))
302                 .andExpect(status().isBadRequest());
303     }
304
305     @Test
306     void administration_returns_conflict_for_invalid_lifecycle_operation() throws Exception {
307         doThrow(new IllegalStateException("Request requires status SUBMITTED"))
308
                .when(administrationInputPort).approveRequest(AdministeredAllocationRequestId.of(requestUuid));
309
310         mockMvc.perform(put("/api/administration/requests/" + requestUuid + "/approve")
311                         .with(authenticated("manager-token")))
312                 .andExpect(status().isConflict())
313                 .andExpect(jsonPath("$.error").value("Request requires status SUBMITTED"));
314     }
315
316     @Test
317     void submit_request_rejects_date_to_before_date_from() throws Exception {
318         when(currentUserResolver.resolveEmployeeId(any())).thenReturn(EmployeeId.of(employeeUuid));
319         doThrow(new IllegalArgumentException("dateTo must not be before dateFrom"))
320                 .when(allocationRequestInputPort).submitRequest(
321                         eq(EmployeeId.of(employeeUuid)), eq(ResourceId.of(resourceUuid)),
322                         eq(LocalDate.of(2099, 1, 10)), eq(LocalDate.of(2099, 1, 1)), eq("test"));
323
324         mockMvc.perform(post("/api/requests")
325                         .with(authenticated("employee-token"))
326                         .contentType(MediaType.APPLICATION_JSON)
327                         .content("{\"resourceId\":\"" + resourceUuid +
                            "\",\"dateFrom\":\"2099-01-10\",\"dateTo\":\"2099-01-01\",\"justification\":\"test\"}"))
328                 .andExpect(status().isBadRequest())
329                 .andExpect(jsonPath("$.error").value("dateTo must not be before dateFrom"));
330     }
331
332     @Test
333     void missing_resource_from_controller_returns_not_found() throws Exception {
334         doThrow(new NoSuchElementException("Resource not found"))
335                 .when(resourceManagementInputPort).updateResource(ResourceId.of(resourceUuid), "Desk 1", "A",
                    1, "101");
336
337         mockMvc.perform(put("/api/resources/" + resourceUuid)
338                         .with(authenticated("manager-token"))
339                         .contentType(MediaType.APPLICATION_JSON)
340                         .content(resourceBody("DESK", "Desk 1", "A", "101")))
341                 .andExpect(status().isNotFound())
342                 .andExpect(jsonPath("$.error").value("Resource not found"));
343     }
344
345     private void token(String token, String email, String role) {
346         when(tokenProvider.validateToken(token)).thenReturn(true);
347         when(tokenProvider.getEmailFromToken(token)).thenReturn(email);
348         when(tokenProvider.getRoleFromToken(token)).thenReturn(role);
```

```
    }

    private RequestPostProcessor authenticated(String token) {
        return switch (token) {
            case "employee-token" -> user("employee@test.pl").roles("EMPLOYEE");
            case "manager-token" -> user("manager@test.pl").roles("MANAGER");
            case "admin-token" -> user("admin@test.pl").roles("IT_ADMIN");
            default -> user("unknown@test.pl");
        };
    }

    private String resourceBody(String type, String name, String building, String number) {
        return "{\"type\":\"" + type + "\",\"name\":\"" + name + "\",\"building\":\"" + building +
                "\",\"floor\":1,\"number\":\"" + number + "\"}";
    }

    private AdministeredAllocationView adminView(String status) {
        return new AdministeredAllocationView(
                requestUuid,
                employeeUuid,
                resourceUuid,
                status,
                LocalDate.of(2099, 1, 1),
                LocalDate.of(2099, 1, 2),
                "test",
                LocalDateTime.of(2098, 12, 1, 10, 0),
                null,
                null,
                null
        );
    }
}
```

Source code 6.177:
common/infrastructure/out/events/SpringDomainEventPublisherAdapterTest.java - SpringDomainEventPublisherAdapterTest

```
package com.deskflow.common.infrastructure.out.events;

import com.deskflow.common.application.domain.DomainEvent;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.context.ApplicationEventPublisher;

import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class SpringDomainEventPublisherAdapterTest {

    @Mock private ApplicationEventPublisher publisher;

    @Test
    void publish_delegates_to_spring_publisher() {
        var adapter = new SpringDomainEventPublisherAdapter(publisher);
        DomainEvent event = mock(DomainEvent.class);

        adapter.publish(event);

        verify(publisher).publishEvent(event);
    }
}
```

6.3. Pliki konfiguracyjne i migracje

Source code 6.178: resources/application-test.properties - application-test.properties

```
spring.datasource.url=jdbc:tc:postgresql:17-alpine:///deskflow
spring.datasource.driver-class-name=org.testcontainers.jdbc.ContainerDatabaseDriver
spring.flyway.enabled=true
spring.jpa.hibernate.ddl-auto=validate
spring.mail.host=localhost
spring.mail.port=3025
app.jwt.secret=test-secret-key-at-least-32-characters-long-for-hs256-tests
```

Source code 6.179: resources/application.properties - application.properties

```
spring.application.name=deskflow-backend

# DataSource
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/deskflow}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:deskflow}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:deskflow}
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.format_sql=true

# Flyway
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration

# Mail
spring.mail.host=${SPRING_MAIL_HOST:localhost}
spring.mail.port=${SPRING_MAIL_PORT:1025}
spring.mail.username=${SPRING_MAIL_USERNAME:}
spring.mail.password=${SPRING_MAIL_PASSWORD:}
spring.mail.properties.mail.smtp.auth=false
spring.mail.properties.mail.smtp.starttls.enable=false

# JWT
app.jwt.secret=${JWT_SECRET:deskflow-secret-key-at-least-32-characters-long-for-hs256}
app.jwt.expiration-ms=86400000

# CORS
app.cors.allowed-origin=${ALLOWED_ORIGIN:http://localhost:3000}

# Scheduler
spring.task.scheduling.enabled=true

# Actuator
management.endpoints.web.exposure.include=health,info
management.health.mail.enabled=false
```

Source code 6.180: resources/db/migration/V1___init_schema.sql - V1___init_schema.sql

```
CREATE TABLE employees (
    id UUID PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL,
```

```
    password_hash VARCHAR(255) NOT NULL
);

CREATE TABLE allocation_limits (
    id UUID PRIMARY KEY,
    employee_id UUID NOT NULL REFERENCES employees(id),
    resource_type VARCHAR(50) NOT NULL,
    limit_amount INT NOT NULL DEFAULT 1,
    UNIQUE (employee_id, resource_type)
);

CREATE TABLE office_resources (
    id UUID PRIMARY KEY,
    type VARCHAR(50) NOT NULL,
    name VARCHAR(255) NOT NULL,
    building VARCHAR(100) NOT NULL,
    floor INT NOT NULL,
    number VARCHAR(50) NOT NULL,
    active BOOLEAN NOT NULL DEFAULT TRUE,
    available BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE allocation_requests (
    id UUID PRIMARY KEY,
    employee_id UUID NOT NULL REFERENCES employees(id),
    resource_id UUID NOT NULL REFERENCES office_resources(id),
    status VARCHAR(50) NOT NULL,
    date_from DATE NOT NULL,
    date_to DATE NOT NULL,
    justification TEXT,
    submitted_at TIMESTAMP NOT NULL,
    issued_at TIMESTAMP,
    returned_at TIMESTAMP,
    rejection_reason TEXT
);

CREATE INDEX idx_allocation_requests_status ON allocation_requests(status);
CREATE INDEX idx_allocation_requests_employee_id ON allocation_requests(employee_id);
CREATE INDEX idx_allocation_requests_resource_id ON allocation_requests(resource_id);
CREATE INDEX idx_allocation_requests_date_to ON allocation_requests(date_to);
```

6.4. Pliki konfiguracji budowania (pom.xml)

Source code 6.181: deskflow-backend/pom.xml - deskflow-backend

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.0</version>
        <relativePath/>
    </parent>

    <groupId>com.deskflow</groupId>
    <artifactId>deskflow-backend</artifactId>
    <version>1.0.0</version>
    <name>deskflow-backend</name>
    <description>DeskFlow - zarządzanie zasobami biurowymi (DDD + hexagonal architecture)</description>

    <properties>
        <java.version>21</java.version>
        <archunit.version>1.3.0</archunit.version>
        <testcontainers.version>1.20.3</testcontainers.version>
        <jjwt.version>0.12.6</jjwt.version>
    </properties>

    <dependencies>
        <!- Spring Boot ->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-mail</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <!- PostgreSQL ->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!- JWT ->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
```

```
            <version>${jjwt.version}</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>${jjwt.version}</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>${jjwt.version}</version>
            <scope>runtime</scope>
        </dependency>

        <!- Flyway ->
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-core</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-database-postgresql</artifactId>
        </dependency>

        <!- Testing ->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>

        <!- ArchUnit ->
        <dependency>
            <groupId>com.tngtech.archunit</groupId>
            <artifactId>archunit-junit5</artifactId>
            <version>${archunit.version}</version>
            <scope>test</scope>
        </dependency>

        <!- TestContainers ->
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${testcontainers.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>postgresql</artifactId>
            <version>${testcontainers.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

```

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration />
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <configuration>
                    <argLine>-Dnet.bytebuddy.experimental=true</argLine>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

6.5. Skrypty wdrożeniowe i konfiguracja Docker

Source code 6.182: .dockerignore

```
.git
.gitignore
.DS_Store
**/.next
**/node_modules
**/target
**/*.log
**/.env*
**/.idea
**/.vscode
```

Source code 6.183: Dockerfile.prod

```
FROM maven:3.9-eclipse-temurin-21 AS backend-build
WORKDIR /app
COPY deskflow-backend/pom.xml .
RUN mvn dependency:go-offline -q
COPY deskflow-backend/src ./src
RUN mvn package -Dmaven.test.skip=true

FROM eclipse-temurin:21-jre-alpine AS backend
RUN apk add -no-cache wget \
    && addgroup -S deskflow \
    && adduser -S deskflow -G deskflow
WORKDIR /app
COPY -from=backend-build /app/target/deskflow-backend-*.jar app.jar
EXPOSE 8080
USER deskflow
ENTRYPOINT ["java", "-jar", "app.jar"]

FROM node:22-alpine AS frontend-build
WORKDIR /app
COPY deskflow-frontend/package.json deskflow-frontend/package-lock.json ./
RUN npm ci
COPY deskflow-frontend .
ARG NEXT_PUBLIC_API_URL=http://localhost:8080
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build

FROM node:22-alpine AS frontend
```

```
WORKDIR /app
ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1
COPY -from=frontend-build /app/.next/standalone ./
COPY -from=frontend-build /app/.next/static ./.next/static
COPY -from=frontend-build /app/public ./public
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

Source code 6.184: deskflow-backend/Dockerfile.dev

```
FROM maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
EXPOSE 8080 5005
CMD ["mvn", "spring-boot:run", \
  "-Dspring-boot.run.jvmArguments=-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005"]
```

Source code 6.185: docker-compose.dev.yml

```
# DeskFlow - lokalne środowisko deweloperskie
# Uruchom: docker compose -f docker-compose.dev.yml up -build

services:
  postgres:
    image: postgres:17-alpine
    container_name: deskflow-postgres
    environment:
      POSTGRES_DB: deskflow
      POSTGRES_USER: deskflow
      POSTGRES_PASSWORD: deskflow
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U deskflow -d deskflow"]
      interval: 5s
      timeout: 5s
      retries: 10

  mailhog:
    image: mailhog/mailhog:latest
    container_name: deskflow-mailhog
    ports:
      - "1025:1025"
      - "8025:8025"

  backend-dev:
    build:
      context: ./deskflow-backend
      dockerfile: Dockerfile.dev
    container_name: deskflow-backend-dev
    ports:
      - "8080:8080"
      - "5005:5005"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/deskflow
      SPRING_DATASOURCE_USERNAME: deskflow
      SPRING_DATASOURCE_PASSWORD: deskflow
      SPRING_MAIL_HOST: mailhog
      SPRING_MAIL_PORT: 1025
```

```
      JWT_SECRET: deskflow-dev-secret-key-change-in-production
      ALLOWED_ORIGIN: http://localhost:3000
      SPRING_JPA_SHOW_SQL: "true"
    volumes:
      - ./deskflow-backend:/app
      - /app/target
    depends_on:
      postgres:
        condition: service_healthy

  frontend-dev:
    build:
      context: ./deskflow-frontend
      dockerfile: Dockerfile.dev
    container_name: deskflow-frontend-dev
    ports:
      - "3000:3000"
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8080
    volumes:
      - ./deskflow-frontend:/app
      - /app/node_modules
      - /app/.next
    depends_on:
      - backend-dev

volumes:
  postgres_data:
```

Source code 6.186: docker-compose.prod.yml

```
# DeskFlow - deployment (Coolify / production)
# Użyj w Coolify jako serwis typu "Docker Compose".
# Zmienne środowiskowe ustaw w panelu Coolify (Stacks → Environment).

services:
  postgres:
    image: postgres:17-alpine
    container_name: deskflow-postgres
    environment:
      POSTGRES_DB: deskflow
      POSTGRES_USER: deskflow
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data_v2:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U deskflow -d deskflow"]
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped

  backend:
    build:
      context: .
      dockerfile: Dockerfile.prod
      target: backend
    container_name: deskflow-backend
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/deskflow
      SPRING_DATASOURCE_USERNAME: deskflow
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
      SPRING_MAIL_HOST: ${SMTP_HOST:-}
      SPRING_MAIL_PORT: ${SMTP_PORT:-}
      SPRING_MAIL_USERNAME: ${SMTP_USERNAME:-}
      SPRING_MAIL_PASSWORD: ${SMTP_PASSWORD:-}
      JWT_SECRET: ${JWT_SECRET}
      ALLOWED_ORIGIN: ${ALLOWED_ORIGIN}
```

```
    depends_on:
      postgres:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-q", "-spider", "http://localhost:8080/actuator/health"]
      interval: 15s
      timeout: 10s
      retries: 5
      start_period: 60s
    restart: unless-stopped

  frontend:
    build:
      context: .
      dockerfile: Dockerfile.prod
      target: frontend
      args:
        NEXT_PUBLIC_API_URL: ${NEXT_PUBLIC_API_URL}
    container_name: deskflow-frontend
    environment:
      NEXT_PUBLIC_API_URL: ${NEXT_PUBLIC_API_URL}
    depends_on:
      backend:
        condition: service_healthy
    restart: unless-stopped

volumes:
  postgres_data_v2:
```

Source code 6.187: .env.example

```
# ------------------------------------------
# DeskFlow - zmienne środowiskowe
# ------------------------------------------
# Dla dev: cp .env.example .env.dev
# Dla prod: ustaw w panelu Coolify

# ---- Postgres ----------------------------------------------------------------
POSTGRES_PASSWORD=deskflow

# ---- Backend JWT -------------------------------------------------------------
# Minimum 32 znaki dla HS256
JWT_SECRET=zmien-mnie-na-cos-bardziej-skomplikowanego

# ---- Backend CORS ------------------------------------------------------------
ALLOWED_ORIGIN=http://localhost:3000

# ---- SMTP (opcjonalny, MailHog domyślnie) ----------
SMTP_HOST=mailhog
SMTP_PORT=1025
SMTP_USERNAME=
SMTP_PASSWORD=

# ---- Frontend ----------------------------------------------------------------
# URL backendu osiągalny z przeglądarki
NEXT_PUBLIC_API_URL=http://localhost:8080
```

6.6. Kod źródłowy frontendu

Source code 6.188: deskflow-frontend/Dockerfile.dev

```
FROM node:22-alpine
WORKDIR /app
```

```
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
EXPOSE 3000
CMD ["npm", "run", "dev"]
```

Source code 6.189: deskflow-frontend/Dockerfile.frontend

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ARG NEXT_PUBLIC_API_URL=http://localhost:8080
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1
COPY -from=build /app/.next/standalone ./
COPY -from=build /app/.next/static ./.next/static
COPY -from=build /app/public ./public
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

Source code 6.190: deskflow-frontend/docker-compose.yml

```
services:
  db:
    image: postgres:17-alpine
    container_name: deskflow-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: deskflow
    ports:
      - '5432:5432'
    volumes:
      - deskflow-db-data:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U postgres -d deskflow']
      interval: 5s
      timeout: 5s
      retries: 10

  app:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    container_name: deskflow-app
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    env_file:
      - .env.local
    environment:
      NODE_ENV: development
      DATABASE_URL: postgres://postgres:postgres@db:5432/deskflow
```

```
      NEXT_TELEMETRY_DISABLED: '1'
    ports:
      - '3000:3000'

volumes:
  deskflow-db-data:
```

Source code 6.191: deskflow-frontend/.gitignore

```
# v0 runtime files
__v0_runtime_loader.js
__v0_devtools.tsx
__v0_jsx-dev-runtime.ts

# Common ignores
node_modules/
.next/
.env*.local
.DS_Store

tsconfig.tsbuildinfo
# clerk configuration (can include secrets)
/.clerk/
```

Source code 6.192: deskflow-frontend/.dockerignore

```
node_modules
.next
.git
.gitignore
pnpm-debug.log*
*.log
.vscode
```

Source code 6.193: deskflow-frontend/package.json

```
{
  "name": "deskflow-frontend",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint ."
  },
  "dependencies": {
    "@hookform/resolvers": "^3.9.1",
    "@radix-ui/react-accordion": "1.2.12",
    "@radix-ui/react-alert-dialog": "1.1.15",
    "@radix-ui/react-avatar": "1.1.11",
    "@radix-ui/react-checkbox": "1.3.3",
    "@radix-ui/react-dialog": "1.1.15",
    "@radix-ui/react-dropdown-menu": "2.1.16",
    "@radix-ui/react-label": "2.1.8",
    "@radix-ui/react-popover": "1.1.15",
    "@radix-ui/react-progress": "1.1.8",
    "@radix-ui/react-radio-group": "1.3.8",
    "@radix-ui/react-scroll-area": "1.2.10",
    "@radix-ui/react-select": "2.2.6",
    "@radix-ui/react-separator": "1.1.8",
    "@radix-ui/react-slot": "1.2.4",
    "@radix-ui/react-tabs": "1.1.13",
```

```
    "@radix-ui/react-toast": "1.2.15",
    "@radix-ui/react-tooltip": "1.2.8",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "1.1.1",
    "date-fns": "4.1.0",
    "lucide-react": "^0.564.0",
    "next": "16.2.9",
    "next-themes": "^0.4.6",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-hook-form": "^7.54.1",
    "recharts": "2.15.0",
    "sonner": "^1.7.1",
    "tailwind-merge": "^3.3.1",
    "zod": "^3.24.1"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4.2.0",
    "@types/node": "^22",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9.39.2",
    "eslint-config-next": "16.2.9",
    "postcss": "^8.5",
    "tailwindcss": "^4.2.0",
    "tw-animate-css": "1.3.3",
    "typescript": "5.7.3"
  }
}
```

Source code 6.194: deskflow-frontend/tsconfig.json

```
{
  "compilerOptions": {
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],
    "allowJs": true,
    "target": "ES6",
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": [
        "./*"
      ]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
```

```
    ".next/dev/types/**/*.ts"
  ],
  "exclude": [
    "node_modules"
  ]
}
```

Source code 6.195: deskflow-frontend/next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'standalone',
  typescript: {
    ignoreBuildErrors: true,
  },
  images: {
    unoptimized: true,
  },
}

export default nextConfig
```

Source code 6.196: deskflow-frontend/eslint.config.mjs

```
import nextVitals from 'eslint-config-next/core-web-vitals'

const eslintConfig = [
  ...nextVitals,
  {
    rules: {
      'react-hooks/immutability': 'off',
      'react-hooks/purity': 'off',
      'react-hooks/set-state-in-effect': 'off',
      'react/no-unescaped-entities': 'off',
    },
  },
]

export default eslintConfig
```

Source code 6.197: deskflow-frontend/postcss.config.mjs

```
/** @type {import('postcss-load-config').Config} */
const config = {
  plugins: {
    '@tailwindcss/postcss': {},
  },
}

export default config
```

Source code 6.198: deskflow-frontend/components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
```

```
    "css": "app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "iconLibrary": "lucide"
}
```

Source code 6.199: deskflow-frontend/next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

Source code 6.200: deskflow-frontend/app/layout.tsx

```
import type { Metadata } from 'next'
import { Toaster } from '@/components/ui/toaster'
import { AuthProvider } from '@/lib/auth-context'
import './globals.css'

export const metadata: Metadata = {
  title: 'DeskFlow - Zarządzanie Zasobami Biurowymi',
  description: 'System udostępniania zasobów biurowych - biurka, sale, sprzęt',
  icons: {
    icon: '/favicon.ico',
  },
}

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode
}>) {
  return (
    <html lang="pl">
      <body className="font-sans antialiased">
        <AuthProvider>
          {children}
          <Toaster />
        </AuthProvider>
      </body>
    </html>
  )
}
```

Source code 6.201: deskflow-frontend/app/globals.css

```
@import 'tailwindcss';
@import 'tw-animate-css';

@custom-variant dark (&:is(.dark *));

:root {
```

```
  -background: oklch(0.985 0 0);
  -foreground: oklch(0.145 0 0);
  -card: oklch(1 0 0);
  -card-foreground: oklch(0.145 0 0);
  -popover: oklch(1 0 0);
  -popover-foreground: oklch(0.145 0 0);
  -primary: oklch(0.55 0.2 250);
  -primary-foreground: oklch(1 0 0);
  -secondary: oklch(0.96 0.01 250);
  -secondary-foreground: oklch(0.25 0.05 250);
  -muted: oklch(0.96 0 0);
  -muted-foreground: oklch(0.5 0 0);
  -accent: oklch(0.7 0.15 165);
  -accent-foreground: oklch(0.2 0.05 165);
  -destructive: oklch(0.55 0.22 27);
  -destructive-foreground: oklch(1 0 0);
  -border: oklch(0.9 0 0);
  -input: oklch(0.9 0 0);
  -ring: oklch(0.55 0.2 250);
  -chart-1: oklch(0.55 0.2 250);
  -chart-2: oklch(0.7 0.15 165);
  -chart-3: oklch(0.65 0.18 45);
  -chart-4: oklch(0.6 0.15 300);
  -chart-5: oklch(0.55 0.2 15);
  -radius: 0.625rem;
  -sidebar: oklch(0.15 0.03 250);
  -sidebar-foreground: oklch(0.95 0 0);
  -sidebar-primary: oklch(0.55 0.2 250);
  -sidebar-primary-foreground: oklch(1 0 0);
  -sidebar-accent: oklch(0.25 0.04 250);
  -sidebar-accent-foreground: oklch(0.95 0 0);
  -sidebar-border: oklch(0.25 0.03 250);
  -sidebar-ring: oklch(0.55 0.2 250);
}

.dark {
  -background: oklch(0.12 0.02 250);
  -foreground: oklch(0.95 0 0);
  -card: oklch(0.15 0.02 250);
  -card-foreground: oklch(0.95 0 0);
  -popover: oklch(0.15 0.02 250);
  -popover-foreground: oklch(0.95 0 0);
  -primary: oklch(0.6 0.2 250);
  -primary-foreground: oklch(1 0 0);
  -secondary: oklch(0.25 0.03 250);
  -secondary-foreground: oklch(0.95 0 0);
  -muted: oklch(0.2 0.02 250);
  -muted-foreground: oklch(0.65 0 0);
  -accent: oklch(0.65 0.15 165);
  -accent-foreground: oklch(0.1 0.02 165);
  -destructive: oklch(0.55 0.22 27);
  -destructive-foreground: oklch(1 0 0);
  -border: oklch(0.25 0.02 250);
  -input: oklch(0.25 0.02 250);
  -ring: oklch(0.6 0.2 250);
  -chart-1: oklch(0.6 0.2 250);
  -chart-2: oklch(0.65 0.15 165);
  -chart-3: oklch(0.7 0.18 45);
  -chart-4: oklch(0.65 0.15 300);
  -chart-5: oklch(0.6 0.2 15);
  -sidebar: oklch(0.12 0.02 250);
  -sidebar-foreground: oklch(0.95 0 0);
  -sidebar-primary: oklch(0.6 0.2 250);
  -sidebar-primary-foreground: oklch(1 0 0);
  -sidebar-accent: oklch(0.2 0.03 250);
  -sidebar-accent-foreground: oklch(0.95 0 0);
  -sidebar-border: oklch(0.2 0.02 250);
  -sidebar-ring: oklch(0.6 0.2 250);
}

```

```
@theme inline {
  -font-sans: 'Geist', 'Geist Fallback';
  -font-mono: 'Geist Mono', 'Geist Mono Fallback';
  -color-background: var(-background);
  -color-foreground: var(-foreground);
  -color-card: var(-card);
  -color-card-foreground: var(-card-foreground);
  -color-popover: var(-popover);
  -color-popover-foreground: var(-popover-foreground);
  -color-primary: var(-primary);
  -color-primary-foreground: var(-primary-foreground);
  -color-secondary: var(-secondary);
  -color-secondary-foreground: var(-secondary-foreground);
  -color-muted: var(-muted);
  -color-muted-foreground: var(-muted-foreground);
  -color-accent: var(-accent);
  -color-accent-foreground: var(-accent-foreground);
  -color-destructive: var(-destructive);
  -color-destructive-foreground: var(-destructive-foreground);
  -color-border: var(-border);
  -color-input: var(-input);
  -color-ring: var(-ring);
  -color-chart-1: var(-chart-1);
  -color-chart-2: var(-chart-2);
  -color-chart-3: var(-chart-3);
  -color-chart-4: var(-chart-4);
  -color-chart-5: var(-chart-5);
  -radius-sm: calc(var(-radius) - 4px);
  -radius-md: calc(var(-radius) - 2px);
  -radius-lg: var(-radius);
  -radius-xl: calc(var(-radius) + 4px);
  -color-sidebar: var(-sidebar);
  -color-sidebar-foreground: var(-sidebar-foreground);
  -color-sidebar-primary: var(-sidebar-primary);
  -color-sidebar-primary-foreground: var(-sidebar-primary-foreground);
  -color-sidebar-accent: var(-sidebar-accent);
  -color-sidebar-accent-foreground: var(-sidebar-accent-foreground);
  -color-sidebar-border: var(-sidebar-border);
  -color-sidebar-ring: var(-sidebar-ring);
}

/* Dynamic Company Branding Variables */
:root {
  /* Floor plan element colors - these will be updated by JavaScript */
  -desk-available: #10b981;
  -desk-reserved: #3b82f6;
  -desk-occupied: #ef4444;
  -desk-maintenance: #f97316;

  -room-available: #10b981;
  -room-reserved: #3b82f6;
  -room-occupied: #ef4444;
  -room-maintenance: #f97316;

  /* Company colors - these will be updated by the BrandingContext */
  -company-primary: #3b82f6;
  -company-secondary: #10b981;
  -company-text: #111827;
  -company-active-button-text: #ffffff;
  -company-primary-rgb: 59, 130, 246;
  -company-secondary-rgb: 16, 185, 129;
  -company-text-rgb: 17, 24, 39;
  -company-active-button-text-rgb: 255, 255, 255;
}

/* Utility classes for dynamic theming */
.company-primary {
  background-color: var(-company-primary);
}

```

```
.company-secondary {
  background-color: var(-company-secondary);
}

.text-company-primary {
  color: var(-company-primary);
}

.text-company-secondary {
  color: var(-company-secondary);
}

.text-company-text {
  color: var(-company-text);
}

.text-company-active-button {
  color: var(-company-active-button-text);
}

.border-company-primary {
  border-color: var(-company-primary);
}

.border-company-secondary {
  border-color: var(-company-secondary);
}

/* Floor plan specific styles */
.floor-plan-desk-available {
  background-color: var(-desk-available);
}

.floor-plan-desk-reserved {
  background-color: var(-desk-reserved);
}

.floor-plan-desk-occupied {
  background-color: var(-desk-occupied);
}

.floor-plan-room-available {
  background-color: var(-room-available);
}

.floor-plan-room-reserved {
  background-color: var(-room-reserved);
}

.floor-plan-room-occupied {
  background-color: var(-room-occupied);
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground;
  }
}
```

Source code 6.202: deskflow-frontend/app/(auth)/login/page.tsx

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
```

```
import { Building2 } from 'lucide-react';
import { useAuth } from '@/lib/auth-context';
import { authApi } from '@/lib/api';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@/components/ui/card';
import { Label } from '@/components/ui/label';
import { useToast } from '@/hooks/use-toast';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const router = useRouter();
  const { toast } = useToast();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    try {
      const res = await authApi.login(email, password);
      login(res.token, {
        email: res.email,
        role: res.role,
        firstName: res.firstName,
        lastName: res.lastName,
        employeeId: res.employeeId,
      });
      router.push('/');
    } catch (err: unknown) {
      toast({
        title: 'Błąd logowania',
        description: err instanceof Error ? err.message : 'Nieprawidłowe dane logowania',
        variant: 'destructive',
      });
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50 px-4">
      <Card className="w-full max-w-md">
        <CardHeader className="text-center space-y-2">
          <div className="flex justify-center">
            <Building2 className="h-10 w-10 text-blue-600" />
          </div>
          <CardTitle className="text-2xl font-bold">DeskFlow</CardTitle>
          <CardDescription>Zarządzanie zasobami biurowymi</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleSubmit} className="space-y-4">
            <div className="space-y-2">
              <Label htmlFor="email">Email</Label>
              <Input
                id="email"
                type="email"
                value={email}
                onChange={e => setEmail(e.target.value)}
                placeholder="jan.kowalski@firma.pl"
                required
                autoFocus
              />
            </div>
            <div className="space-y-2">
              <Label htmlFor="password">Hasło</Label>
              <Input
                id="password"
                type="password"
```

```
                value={password}
                onChange={e => setPassword(e.target.value)}
                required
              />
            </div>
            <Button type="submit" className="w-full" disabled={loading}>
              {loading ? 'Logowanie...' : 'Zaloguj się'}
            </Button>
          </form>
          <div className="mt-6 p-4 bg-slate-50 border rounded-lg text-sm text-slate-600">
            <p className="font-semibold mb-2">Dane testowe:</p>
            <div className="space-y-1">
              <p>Pracownik: anna.kowalska@firma.pl / password123</p>
              <p>Kierownik: marek.kierownik@firma.pl / password123</p>
              <p>Admin IT: admin@firma.pl / admin123</p>
            </div>
          </div>
        </CardContent>
      </Card>
    </div>
  );
}
```

Source code 6.203: deskflow-frontend/app/(auth)/register/page.tsx

```
"use client"

import { useState } from "react"
import Link from "next/link"
import { Building2, Eye, EyeOff } from "lucide-react"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card"
import { Checkbox } from "@/components/ui/checkbox"
import { Field, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select"

export default function RegisterPage() {
  const [showPassword, setShowPassword] = useState(false)

  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="w-full max-w-md">
        <div className="flex flex-col items-center mb-8">
          <div className="flex h-14 w-14 items-center justify-center rounded-xl bg-primary mb-4">
            <Building2 className="h-8 w-8 text-primary-foreground" />
          </div>
          <h1 className="text-2xl font-bold">DeskFlow</h1>
          <p className="text-muted-foreground">System rezerwacji biura</p>
        </div>

        <Card>
          <CardHeader className="text-center">
            <CardTitle>Utworz konto</CardTitle>
            <CardDescription>
              Wypelnij formularz, aby zarejestrowac sie w systemie
            </CardDescription>
          </CardHeader>
          <CardContent>
            <form className="space-y-4">
              <div className="grid grid-cols-2 gap-4">
                <FieldGroup>
                  <Field>
                    <FieldLabel>Imie</FieldLabel>
                    <Input placeholder="Jan" />
                  </Field>
                </FieldGroup>
                <FieldGroup>
```

```
                    <Field>
                      <FieldLabel>Nazwisko</FieldLabel>
                      <Input placeholder="Kowalski" />
                    </Field>
                  </FieldGroup>
                </div>

                <FieldGroup>
                  <Field>
                    <FieldLabel>Email sluzbowy</FieldLabel>
                    <Input
                      type="email"
                      placeholder="jan.kowalski@firma.pl"
                    />
                  </Field>
                </FieldGroup>

                <FieldGroup>
                  <Field>
                    <FieldLabel>Dzial</FieldLabel>
                    <Select>
                      <SelectTrigger>
                        <SelectValue placeholder="Wybierz dzial" />
                      </SelectTrigger>
                      <SelectContent>
                        <SelectItem value="it">IT</SelectItem>
                        <SelectItem value="hr">HR</SelectItem>
                        <SelectItem value="sales">Sprzedaz</SelectItem>
                        <SelectItem value="marketing">Marketing</SelectItem>
                        <SelectItem value="finance">Finanse</SelectItem>
                        <SelectItem value="operations">Operacje</SelectItem>
                      </SelectContent>
                    </Select>
                  </Field>
                </FieldGroup>

                <FieldGroup>
                  <Field>
                    <FieldLabel>Haslo</FieldLabel>
                    <div className="relative">
                      <Input
                        type={showPassword ? "text" : "password"}
                        placeholder="Minimum 8 znakow"
                        className="pr-10"
                      />
                      <button
                        type="button"
                        onClick={() => setShowPassword(!showPassword)}
                        className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground
                        hover:text-foreground"
                      >
                        {showPassword ? (
                          <EyeOff className="h-4 w-4" />
                        ) : (
                          <Eye className="h-4 w-4" />
                        )}
                      </button>
                    </div>
                  </Field>
                </FieldGroup>

                <FieldGroup>
                  <Field>
                    <FieldLabel>Powtorz haslo</FieldLabel>
                    <Input type="password" placeholder="Powtorz haslo" />
                  </Field>
                </FieldGroup>

                <div className="flex items-start gap-2">
                  <Checkbox id="terms" className="mt-1" />
```

```
                <label
                  htmlFor="terms"
                  className="text-sm text-muted-foreground cursor-pointer"
                >
                  Akceptuje{" "}
                  <Link href="#" className="text-primary hover:underline">
                    Regulamin
                  </Link>{" "}
                  oraz{" "}
                  <Link href="#" className="text-primary hover:underline">
                    Polityke prywatnosci
                  </Link>
                </label>
              </div>

              <Link href="/">
                <Button type="button" className="w-full">
                  Zarejestruj sie
                </Button>
              </Link>
            </form>

            <div className="mt-6 text-center text-sm">
              <span className="text-muted-foreground">Masz juz konto? </span>
              <Link href="/login" className="text-primary hover:underline font-medium">
                Zaloguj sie
              </Link>
            </div>
          </CardContent>
        </Card>
      </div>
    </div>
  )
}
```

Source code 6.204: deskflow-frontend/app/(dashboard)/layout.tsx

```
export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return <>{children}</>;
}
```

Source code 6.205: deskflow-frontend/app/(dashboard)/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import Link from 'next/link';
import { Monitor, Users, Package, Calendar, ArrowRight, Clock, Building2, LogOut, Settings } from
'lucide-react';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@/components/ui/card';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import { useAuth } from '@/lib/auth-context';
import { requestsApi, resourcesApi, AllocationRequest, Resource } from '@/lib/api';
import { useRouter } from 'next/navigation';

const STATUS_LABELS: Record<string, { label: string; color: string }> = {
  SUBMITTED: { label: 'Złożony', color: 'bg-yellow-100 text-yellow-800' },
  APPROVED: { label: 'Zatwierdzony', color: 'bg-blue-100 text-blue-800' },
  ACTIVE: { label: 'Aktywny', color: 'bg-green-100 text-green-800' },
  COMPLETED: { label: 'Zakończony', color: 'bg-gray-100 text-gray-800' },
  REJECTED: { label: 'Odrzucony', color: 'bg-red-100 text-red-800' },
  CANCELLED: { label: 'Anulowany', color: 'bg-gray-100 text-gray-500' },
  OVERDUE: { label: 'Przeterminowane', color: 'bg-orange-100 text-orange-800' },
};
```

```
22
23  export default function DashboardPage() {
24    const { user, loading: authLoading, logout, isManager, isItAdmin } = useAuth();
25    const router = useRouter();
26    const [myRequests, setMyRequests] = useState<AllocationRequest[]>([]);
27    const [resources, setResources] = useState<Resource[]>([]);
28    const [loading, setLoading] = useState(true);
29
30    useEffect(() => {
31      if (authLoading) return;
32
33      if (!user) {
34        router.push('/login');
35        return;
36      }
37      Promise.all([
38        requestsApi.my().then(setMyRequests).catch(() => {}),
39        resourcesApi.list().then(setResources).catch(() => {}),
40      ]).finally(() => setLoading(false));
41    }, [user, authLoading, router]);
42
43    if (authLoading || !user) return null;
44
45    const activeRequests = myRequests.filter(r => ['SUBMITTED', 'APPROVED', 'ACTIVE'].includes(r.status));
46    const availableResources = resources.filter(r => r.available);
47
48    return (
49      <div className="min-h-screen bg-gray-50">
50        {/* Navbar */}
51        <nav className="bg-white border-b px-6 py-3 flex items-center justify-between">
52          <div className="flex items-center gap-2">
53            <Building2 className="h-6 w-6 text-blue-600" />
54            <span className="font-bold text-lg">DeskFlow</span>
55          </div>
56          <div className="flex items-center gap-4">
57            <span className="text-sm text-gray-600">
58              {user.firstName} {user.lastName} ({user.role})
59            </span>
60            <div className="flex gap-2">
61              <Link href="/wnioski">
62                <Button variant="ghost" size="sm">Moje wnioski</Button>
63              </Link>
64              <Link href="/zasoby">
65                <Button variant="ghost" size="sm">Zasoby</Button>
66              </Link>
67              {isManager() && (
68                <Link href="/administracja">
69                  <Button variant="ghost" size="sm">Wnioski</Button>
70                </Link>
71              )}
72              {isItAdmin() && (
73                <Link href="/admin/it">
74                  <Button variant="ghost" size="sm">IT Admin</Button>
75                </Link>
76              )}
77              <Button variant="ghost" size="sm" onClick={logout}>
78                <LogOut className="h-4 w-4" />
79              </Button>
80            </div>
81          </div>
82        </nav>
83
84        <div className="p-8 max-w-7xl mx-auto">
85          <div className="mb-8">
86            <h1 className="text-2xl font-bold">Witaj, {user.firstName}!</h1>
87            <p className="text-gray-600 mt-1">System zarządzania zasobami biurowymi</p>
88          </div>
89
90          {/* Stats */}
91          <div className="grid gap-4 md:grid-cols-3 mb-8">
```

```
92            <Card>
93              <CardContent className="p-6">
94                <div className="flex items-center justify-between">
95                  <div>
96                    <p className="text-sm text-gray-600">Moje aktywne wnioski</p>
97                    <p className="text-3xl font-bold mt-1">{activeRequests.length}</p>
98                  </div>
99                  <Calendar className="h-8 w-8 text-blue-500" />
100               </div>
101             </CardContent>
102           </Card>
103           <Card>
104             <CardContent className="p-6">
105               <div className="flex items-center justify-between">
106                 <div>
107                   <p className="text-sm text-gray-600">Dostępne zasoby</p>
108                   <p className="text-3xl font-bold mt-1">{availableResources.length}</p>
109                 </div>
110                 <Monitor className="h-8 w-8 text-green-500" />
111               </div>
112             </CardContent>
113           </Card>
114           <Card>
115             <CardContent className="p-6">
116               <div className="flex items-center justify-between">
117                 <div>
118                   <p className="text-sm text-gray-600">Wszystkie zasoby</p>
119                   <p className="text-3xl font-bold mt-1">{resources.length}</p>
120                 </div>
121                 <Package className="h-8 w-8 text-purple-500" />
122               </div>
123             </CardContent>
124           </Card>
125         </div>
126
127         {/* Quick Actions */}
128         <Card className="mb-6">
129           <CardHeader>
130             <CardTitle>Szybkie akcje</CardTitle>
131           </CardHeader>
132           <CardContent>
133             <div className="flex flex-wrap gap-3">
134               <Link href="/zasoby">
135                 <Button className="gap-2">
136                   <Monitor className="h-4 w-4" />
137                   Przeglądaj zasoby i złóż wniosek
138                 </Button>
139               </Link>
140               <Link href="/wnioski">
141                 <Button variant="outline" className="gap-2">
142                   <Clock className="h-4 w-4" />
143                   Historia wniosków
144                 </Button>
145               </Link>
146               {isManager() && (
147                 <Link href="/administracja">
148                   <Button variant="outline" className="gap-2">
149                     <Users className="h-4 w-4" />
150                     Panel administracji
151                   </Button>
152                 </Link>
153               )}
154               {isItAdmin() && (
155                 <Link href="/admin/it">
156                   <Button variant="outline" className="gap-2">
157                     <Settings className="h-4 w-4" />
158                     IT Admin
159                   </Button>
160                 </Link>
161               )}
```

```
            </div>
          </CardContent>
        </Card>

        {/* Active requests */}
        <Card>
          <CardHeader className="flex flex-row items-center justify-between">
            <div>
              <CardTitle>Moje aktywne wnioski</CardTitle>
              <CardDescription>Wnioski oczekujące i aktywne udostępnienia</CardDescription>
            </div>
            <Link href="/wnioski">
              <Button variant="outline" size="sm" className="gap-2">
                Wszystkie <ArrowRight className="h-4 w-4" />
              </Button>
            </Link>
          </CardHeader>
          <CardContent>
            {loading ? (
              <p className="text-gray-500 text-sm">Ładowanie...</p>
            ) : activeRequests.length === 0 ? (
              <p className="text-gray-500 text-sm">Brak aktywnych wniosków</p>
            ) : (
              <div className="space-y-3">
                {activeRequests.slice(0, 5).map(r => (
                  <div key={r.id} className="flex items-center justify-between p-3 bg-gray-50 rounded-lg">
                    <div>
                      <div className="flex items-center gap-2 mb-1">
                        <span className={`text-xs px-2 py-0.5 rounded-full font-medium
                        ${STATUS_LABELS[r.status]?.color}`}>
                          {STATUS_LABELS[r.status]?.label}
                        </span>
                      </div>
                      <p className="text-sm font-medium">{r.dateFrom} → {r.dateTo}</p>
                      {r.justification && (
                        <p className="text-xs text-gray-500">{r.justification}</p>
                      )}
                    </div>
                  </div>
                ))}
              </div>
            )}
          </CardContent>
        </Card>
      </div>
    </div>
  );
}
```

Source code 6.206: deskflow-frontend/app/(dashboard)/biurka/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { ArrowLeft, Plus, MapPin, CheckCircle, XCircle } from 'lucide-react';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from '@/components/ui/dialog';
import { useAuth } from '@/lib/auth-context';
import { resourcesApi, requestsApi, Resource } from '@/lib/api';
import { useToast } from '@/hooks/use-toast';

const TYPE_LABELS: Record<string, string> = {
  DESK: 'Biurko', CONFERENCE_ROOM: 'Sala', LAPTOP: 'Laptop',
```

```
  MONITOR: 'Monitor', PROJECTOR: 'Projektor', VEHICLE: 'Pojazd', ACCESSORY: 'Akcesoria',
};

const today = new Date().toISOString().split('T')[0];

function nextDay(date: string) {
  const match = /^(\d{4})-(\d{2})-(\d{2})$/.exec(date);
  if (!match) return today;

  const [, year, month, day] = match.map(Number);
  const value = new Date(Date.UTC(year, month - 1, day));
  if (
    Number.isNaN(value.getTime()) ||
    value.getUTCFullYear() !== year ||
    value.getUTCMonth() !== month - 1 ||
    value.getUTCDate() !== day
  ) return today;

  value.setUTCDate(value.getUTCDate() + 1);
  return value.toISOString().split('T')[0];
}

export default function ResourcesPage() {
  const { user, loading: authLoading } = useAuth();
  const router = useRouter();
  const { toast } = useToast();
  const [resources, setResources] = useState<Resource[]>([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [selected, setSelected] = useState<Resource | null>(null);
  const [form, setForm] = useState({ dateFrom: '', dateTo: '', justification: '' });

  const load = () => resourcesApi.list().then(setResources).catch(() => {}).finally(() => setLoading(false));

  useEffect(() => {
    if (authLoading) return;

    if (!user) { router.push('/login'); return; }
    load();
  }, [user, authLoading, router]);

  const handleSubmit = async () => {
    if (!selected) return;

    if (form.dateFrom < today) {
      toast({ title: 'Błąd', description: 'Data od nie może być wcześniejsza niż dzisiaj.', variant:
      'destructive' });
      return;
    }

    if (form.dateTo <= form.dateFrom) {
      toast({ title: 'Błąd', description: 'Data do musi być późniejsza niż Data od.', variant: 'destructive'
      });
      return;
    }

    try {
      await requestsApi.submit({ resourceId: selected.id, ...form });
      toast({ title: 'Wniosek złożony pomyślnie' });
      setOpen(false);
      setForm({ dateFrom: '', dateTo: '', justification: '' });
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    }
  };

  return (
    <div className="min-h-screen bg-gray-50 p-8">
      <div className="max-w-6xl mx-auto">
        <div className="flex items-center gap-4 mb-6">
```

```
          <Link href="/"><Button variant="ghost" size="sm"><ArrowLeft className="h-4 w-4 mr-1" />Powrót</Button></Link>
          <div>
            <h1 className="text-2xl font-bold">Oferta zasobów</h1>
          </div>
        </div>

        {loading ? (
          <p className="text-gray-500">Ładowanie zasobów...</p>
        ) : (
          <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
            {resources.map(r => (
              <Card key={r.id} className={`${!r.available ? 'opacity-60' : 'hover:shadow-md transition-shadow'}`}>
                <CardHeader className="pb-2">
                  <div className="flex items-start justify-between">
                    <CardTitle className="text-base">{r.name}</CardTitle>
                    {r.available ? (
                      <span className="flex items-center gap-1 text-xs text-green-600">
                        <CheckCircle className="h-3 w-3" />Dostępny
                      </span>
                    ) : (
                      <span className="flex items-center gap-1 text-xs text-red-500">
                        <XCircle className="h-3 w-3" />Niedostępny
                      </span>
                    )}
                  </div>
                </CardHeader>
                <CardContent>
                  <div className="space-y-1 mb-3">
                    <span className="text-xs bg-blue-100 text-blue-700 px-2 py-0.5 rounded-full">
                      {TYPE_LABELS[r.type] || r.type}
                    </span>
                    <p className="text-sm text-gray-600 flex items-center gap-1 mt-2">
                      <MapPin className="h-3 w-3" />
                      {r.building}, piętro {r.floor}, nr {r.number}
                    </p>
                  </div>
                  <Dialog open={open && selected?.id === r.id} onOpenChange={v => { setOpen(v); if (!v) setSelected(null); }}>
                    <DialogTrigger asChild>
                      <Button
                        size="sm" className="w-full gap-2"
                        disabled={!r.available}
                        onClick={() => { setSelected(r); setOpen(true); }}
                      >
                        <Plus className="h-4 w-4" />Złóż wniosek
                      </Button>
                    </DialogTrigger>
                    <DialogContent>
                      <DialogHeader>
                        <DialogTitle>Złóż wniosek: {r.name}</DialogTitle>
                      </DialogHeader>
                      <div className="space-y-4 pt-2">
                        <div className="grid grid-cols-2 gap-3">
                          <div>
                            <Label>Data od</Label>
                            <Input
                              type="date"
                              min={today}
                              value={form.dateFrom}
                              onChange={e => setForm(f => ({ ...f, dateFrom: e.target.value }))}
                            />
                          </div>
                          <div>
                            <Label>Data do</Label>
                            <Input
                              type="date"
                              min={form.dateFrom ? nextDay(form.dateFrom) : today}
                              value={form.dateTo}
```

```
                            onChange={e => setForm(f => ({ ...f, dateTo: e.target.value }))}
                          />
                        </div>
                      </div>
                      <div>
                        <Label>Uzasadnienie</Label>
                        <Input
                          value={form.justification}
                          onChange={e => setForm(f => ({ ...f, justification: e.target.value }))}
                          placeholder="Cel wypożyczenia..."
                        />
                      </div>
                      <Button className="w-full" onClick={handleSubmit} disabled={!form.dateFrom ||
                      !form.dateTo}>
                        Złóż wniosek
                      </Button>
                    </div>
                  </DialogContent>
                </Dialog>
              </CardContent>
            </Card>
          ))}
        </div>
      )}
    </div>
  </div>
  );
}
```

Source code 6.207: deskflow-frontend/app/(dashboard)/rezerwacje/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { ArrowLeft, XCircle, Box, Calendar, MapPin } from 'lucide-react';
import { Card, CardContent } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { useAuth } from '@/lib/auth-context';
import { requestsApi, resourcesApi, AllocationRequest, Resource } from '@/lib/api';
import { useToast } from '@/hooks/use-toast';

const STATUS_LABELS: Record<string, { label: string; color: string }> = {
  SUBMITTED: { label: 'Złożony', color: 'bg-yellow-100 text-yellow-800' },
  APPROVED: { label: 'Zatwierdzony', color: 'bg-blue-100 text-blue-800' },
  ACTIVE: { label: 'Aktywny', color: 'bg-green-100 text-green-800' },
  COMPLETED: { label: 'Zakończony', color: 'bg-gray-100 text-gray-700' },
  REJECTED: { label: 'Odrzucony', color: 'bg-red-100 text-red-800' },
  CANCELLED: { label: 'Anulowany', color: 'bg-gray-100 text-gray-500' },
  OVERDUE: { label: 'Przeterminowane', color: 'bg-orange-100 text-orange-800' },
};

const TYPE_LABELS: Record<string, string> = {
  DESK: 'Biurko', CONFERENCE_ROOM: 'Sala konf.', LAPTOP: 'Laptop',
  MONITOR: 'Monitor', PROJECTOR: 'Projektor', VEHICLE: 'Pojazd', ACCESSORY: 'Akcesoria',
};

export default function MyRequestsPage() {
  const { user, loading: authLoading } = useAuth();
  const router = useRouter();
  const { toast } = useToast();
  const [requests, setRequests] = useState<AllocationRequest[]>([]);
  const [resourceMap, setResourceMap] = useState<Record<string, Resource>>({});
  const [loading, setLoading] = useState(true);

  const load = async () => {
    try {
```

```
      const [reqs, resources] = await Promise.all([requestsApi.my(), resourcesApi.list()]);
      setRequests(reqs);
      setResourceMap(Object.fromEntries(resources.map(r => [r.id, r])));
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    if (authLoading) return;

    if (!user) { router.push('/login'); return; }
    load();
  }, [user, authLoading, router]);

  const handleCancel = async (id: string) => {
    try {
      await requestsApi.cancel(id);
      toast({ title: 'Wniosek anulowany' });
      load();
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    }
  };

  return (
    <div className="min-h-screen bg-gray-50 p-8">
      <div className="max-w-4xl mx-auto">
        <div className="flex items-center gap-4 mb-6">
          <Link href="/"><Button variant="ghost" size="sm"><ArrowLeft className="h-4 w-4 mr-1"
          />Powrót</Button></Link>
          <div>
            <h1 className="text-2xl font-bold">Moje wnioski</h1>
            <p className="text-gray-600 text-sm">Historia rezerwacji zasobów</p>
          </div>
        </div>

        {loading ? (
          <p className="text-gray-500">Ładowanie...</p>
        ) : requests.length === 0 ? (
          <Card>
            <CardContent className="p-8 text-center text-gray-500">
              Brak wniosków. <Link href="/zasoby" className="text-blue-600 hover:underline">Złóż pierwszy
              wniosek</Link>.
            </CardContent>
          </Card>
        ) : (
          <div className="space-y-3">
            {requests.map(r => {
              const resource = resourceMap[r.resourceId];
              return (
                <Card key={r.id} className="hover:shadow-sm transition-shadow">
                  <CardContent className="p-4">
                    <div className="flex items-start justify-between gap-4">
                      <div className="flex-1 min-w-0">
                        <div className="flex items-center gap-2 mb-2 flex-wrap">
                          <span className={`text-xs px-2 py-0.5 rounded-full font-medium
                          ${STATUS_LABELS[r.status]?.color}`}>
                            {STATUS_LABELS[r.status]?.label}
                          </span>
                          <span className="text-xs text-gray-400">
                            złożony {new Date(r.submittedAt).toLocaleDateString('pl-PL')}
                          </span>
                        </div>

                        <div className="flex items-center gap-1.5 mb-1">
                          <Box className="h-4 w-4 text-gray-500 flex-shrink-0" />
                          <span className="font-semibold text-sm">
```

```
                          {resource ? resource.name : `Zasób #${r.resourceId.slice(0, 8)}`}
                        </span>
                        {resource && (
                          <span className="text-xs bg-blue-50 text-blue-700 px-1.5 py-0.5 rounded">
                            {TYPE_LABELS[resource.type] || resource.type}
                          </span>
                        )}
                      </div>

                      {resource && (
                        <div className="flex items-center gap-1 text-xs text-gray-500 mb-1">
                          <MapPin className="h-3 w-3" />
                          {resource.building}, piętro {resource.floor}, nr {resource.number}
                        </div>
                      )}

                      <div className="flex items-center gap-1 text-sm text-gray-700">
                        <Calendar className="h-3.5 w-3.5 text-gray-400" />
                        {r.dateFrom} - {r.dateTo}
                      </div>

                      {r.justification && (
                        <p className="text-xs text-gray-500 mt-1 italic">„{r.justification}"</p>
                      )}
                      {r.rejectionReason && (
                        <p className="text-xs text-red-600 mt-1">Powód odrzucenia: {r.rejectionReason}</p>
                      )}
                    </div>

                    {(r.status === 'SUBMITTED' || r.status === 'APPROVED') && (
                      <Button
                        variant="outline"
                        size="sm"
                        className="text-red-600 border-red-200 hover:bg-red-50 flex-shrink-0"
                        onClick={() => handleCancel(r.id)}
                      >
                        <XCircle className="h-4 w-4 mr-1" />
                        Anuluj
                      </Button>
                    )}
                  </div>
                </CardContent>
              </Card>
            );
          })}
        </div>
      )}
    </div>
  </div>
  );
}
```

Source code 6.208: deskflow-frontend/app/(dashboard)/sale/page.tsx

```
import { redirect } from 'next/navigation';
export default function RoomsRedirect() { redirect('/biurka'); }
```

Source code 6.209: deskflow-frontend/app/(dashboard)/sprzet/page.tsx

```
'use client';
import { useEffect } from 'react';
import { useRouter } from 'next/navigation';

export default function EquipmentRedirect() {
  const router = useRouter();
```

```
  useEffect(() => { router.replace('/biurka'); }, [router]);
  return null;
}
```

Source code 6.210: deskflow-frontend/app/(dashboard)/zasoby/page.tsx

```
import { redirect } from 'next/navigation';
export default function ResourcesRedirect() { redirect('/biurka'); }
```

Source code 6.211: deskflow-frontend/app/(dashboard)/zasoby-admin/page.tsx

```
import { redirect } from 'next/navigation';
export default function AdminITRedirect() { redirect('/admin/it'); }
```

Source code 6.212: deskflow-frontend/app/(dashboard)/wnioski/page.tsx

```
import { redirect } from 'next/navigation';
export default function RequestsRedirect() { redirect('/rezerwacje'); }
```

Source code 6.213: deskflow-frontend/app/(dashboard)/powiadomienia/page.tsx

```
'use client';
import { useEffect } from 'react';
import { useRouter } from 'next/navigation';

export default function NotificationsRedirect() {
  const router = useRouter();
  useEffect(() => { router.replace('/'); }, [router]);
  return null;
}
```

Source code 6.214: deskflow-frontend/app/(dashboard)/administracja/page.tsx

```
import { redirect } from 'next/navigation';
export default function AdminRedirect() { redirect('/admin'); }
```

Source code 6.215: deskflow-frontend/app/(dashboard)/admin/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { ArrowLeft, Check, X, Package, RotateCcw, Box, Calendar, MapPin, User } from 'lucide-react';
import { Card, CardContent } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dialog';
import { useAuth } from '@/lib/auth-context';
import { administrationApi, resourcesApi, AllocationRequest, Resource } from '@/lib/api';
import { useToast } from '@/hooks/use-toast';

const STATUS_LABELS: Record<string, { label: string; color: string }> = {
  SUBMITTED: { label: 'Złożony', color: 'bg-yellow-100 text-yellow-800' },
  APPROVED: { label: 'Zatwierdzony', color: 'bg-blue-100 text-blue-800' },
  ACTIVE: { label: 'Aktywny', color: 'bg-green-100 text-green-800' },
```

```
  COMPLETED: { label: 'Zakończony', color: 'bg-gray-100 text-gray-700' },
  REJECTED: { label: 'Odrzucony', color: 'bg-red-100 text-red-800' },
  CANCELLED: { label: 'Anulowany', color: 'bg-gray-100 text-gray-500' },
  OVERDUE: { label: 'Przeterminowane', color: 'bg-orange-100 text-orange-800' },
};

const TYPE_LABELS: Record<string, string> = {
  DESK: 'Biurko', CONFERENCE_ROOM: 'Sala konf.', LAPTOP: 'Laptop',
  MONITOR: 'Monitor', PROJECTOR: 'Projektor', VEHICLE: 'Pojazd', ACCESSORY: 'Akcesoria',
};

export default function AdminPanelPage() {
  const { user, loading: authLoading, isManager } = useAuth();
  const router = useRouter();
  const { toast } = useToast();
  const [requests, setRequests] = useState<AllocationRequest[]>([]);
  const [resourceMap, setResourceMap] = useState<Record<string, Resource>>({});
  const [filterStatus, setFilterStatus] = useState('');
  const [loading, setLoading] = useState(true);
  const [rejectDialog, setRejectDialog] = useState<string | null>(null);
  const [reason, setReason] = useState('');

  const load = async () => {
    try {
      const [reqs, resources] = await Promise.all([
        administrationApi.requests(filterStatus || undefined),
        resourcesApi.list(),
      ]);
      setRequests(reqs);
      setResourceMap(Object.fromEntries(resources.map(r => [r.id, r])));
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    if (authLoading) return;

    if (!user) { router.push('/login'); return; }
    if (!isManager()) { router.push('/'); return; }
    load();
  }, [user, authLoading, filterStatus, router]);

  const action = async (fn: () => Promise<unknown>, label: string) => {
    try {
      await fn();
      toast({ title: `${label} - sukces` });
      load();
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    }
  };

  const handleReject = async () => {
    if (!rejectDialog) return;
    await action(() => administrationApi.reject(rejectDialog, reason), 'Wniosek odrzucony');
    setRejectDialog(null);
    setReason('');
  };

  return (
    <div className="min-h-screen bg-gray-50 p-8">
      <div className="max-w-5xl mx-auto">
        <div className="flex items-center gap-4 mb-6">
          <Link href="/"><Button variant="ghost" size="sm"><ArrowLeft className="h-4 w-4 mr-1"
          />Powrót</Button></Link>
          <div>
            <h1 className="text-2xl font-bold">Panel administracji</h1>
```

```
            <p className="text-gray-600 text-sm">Zarządzanie wnioskami o zasoby</p>
          </div>
        </div>

        <div className="flex gap-2 mb-6 flex-wrap">
          {['', 'SUBMITTED', 'APPROVED', 'ACTIVE', 'COMPLETED', 'OVERDUE'].map(s => (
            <Button
              key={s}
              variant={filterStatus === s ? 'default' : 'outline'}
              size="sm"
              onClick={() => setFilterStatus(s)}
            >
              {s === '' ? 'Wszystkie' : STATUS_LABELS[s]?.label}
            </Button>
          ))}
        </div>

        {loading ? (
          <p className="text-gray-500">Ładowanie...</p>
        ) : requests.length === 0 ? (
          <Card><CardContent className="p-8 text-center text-gray-500">Brak wniosków.</CardContent></Card>
        ) : (
          <div className="space-y-3">
            {requests.map(r => {
              const resource = resourceMap[r.resourceId];
              return (
                <Card key={r.id} className="hover:shadow-sm transition-shadow">
                  <CardContent className="p-4">
                    <div className="flex items-start justify-between gap-4">
                      <div className="flex-1 min-w-0">
                        <div className="flex items-center gap-2 mb-2 flex-wrap">
                          <span className={`text-xs px-2 py-0.5 rounded-full font-medium
                          ${STATUS_LABELS[r.status]?.color}`}>
                            {STATUS_LABELS[r.status]?.label}
                          </span>
                          <span className="text-xs text-gray-400">
                            złożony {new Date(r.submittedAt).toLocaleDateString('pl-PL')}
                          </span>
                        </div>

                        <div className="flex items-center gap-1.5 mb-1">
                          <User className="h-3.5 w-3.5 text-gray-400 flex-shrink-0" />
                          <span className="text-sm text-gray-700">
                            {r.employeeFirstName && r.employeeLastName
                              ? <span className="font-medium">{r.employeeFirstName}
                              {r.employeeLastName}</span>
                              : <span className="font-mono text-xs text-gray-400">{r.employeeId}</span>
                            }
                          </span>
                        </div>

                        <div className="flex items-center gap-1.5 mb-1">
                          <Box className="h-3.5 w-3.5 text-gray-500 flex-shrink-0" />
                          <span className="font-semibold text-sm">
                            {resource ? resource.name : `Zasób #${r.resourceId.slice(0, 8)}`}
                          </span>
                          {resource && (
                            <span className="text-xs bg-blue-50 text-blue-700 px-1.5 py-0.5 rounded">
                              {TYPE_LABELS[resource.type] || resource.type}
                            </span>
                          )}
                        </div>

                        {resource && (
                          <div className="flex items-center gap-1 text-xs text-gray-500 mb-1">
                            <MapPin className="h-3 w-3" />
                            {resource.building}, piętro {resource.floor}, nr {resource.number}
                          </div>
                        )}

```

```
                        <div className="flex items-center gap-1 text-sm text-gray-700">
                          <Calendar className="h-3.5 w-3.5 text-gray-400" />
                          {r.dateFrom} - {r.dateTo}
                        </div>

                        {r.justification && <p className="text-xs text-gray-500 mt-1
                        italic">„{r.justification}"</p>}
                        {r.rejectionReason && <p className="text-xs text-red-500 mt-1">Powód:
                        {r.rejectionReason}</p>}
                      </div>

                      <div className="flex gap-2 flex-shrink-0 flex-wrap justify-end">
                        {r.status === 'SUBMITTED' && (
                          <>
                            <Button size="sm" className="gap-1 bg-green-600 hover:bg-green-700"
                              onClick={() => action(() => administrationApi.approve(r.id), 'Zatwierdzono')}>
                              <Check className="h-3 w-3" />Zatwierdź
                            </Button>
                            <Button size="sm" variant="destructive" className="gap-1"
                              onClick={() => setRejectDialog(r.id)}>
                              <X className="h-3 w-3" />Odrzuć
                            </Button>
                          </>
                        )}
                        {r.status === 'APPROVED' && (
                          <Button size="sm" className="gap-1"
                            onClick={() => action(() => administrationApi.issue(r.id), 'Zasób wydany')}>
                            <Package className="h-3 w-3" />Wydaj zasób
                          </Button>
                        )}
                        {(r.status === 'ACTIVE' || r.status === 'OVERDUE') && (
                          <Button size="sm" variant="outline" className="gap-1"
                            onClick={() => action(() => administrationApi.return(r.id), 'Zasób zwrócony')}>
                            <RotateCcw className="h-3 w-3" />Zwróć zasób
                          </Button>
                        )}
                      </div>
                    </div>
                  </CardContent>
                </Card>
              );
            })}
          </div>
        )}
      </div>

      <Dialog open={!!rejectDialog} onOpenChange={v => !v && setRejectDialog(null)}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Odrzuć wniosek</DialogTitle>
          </DialogHeader>
          <div className="space-y-4">
            <Input
              placeholder="Podaj powód odrzucenia..."
              value={reason}
              onChange={e => setReason(e.target.value)}
            />
            <div className="flex gap-2 justify-end">
              <Button variant="outline" onClick={() => setRejectDialog(null)}>Anuluj</Button>
              <Button variant="destructive" onClick={handleReject} disabled={!reason.trim()}>
                Odrzuć
              </Button>
            </div>
          </div>
        </DialogContent>
      </Dialog>
    </div>
  );
}
```

Source code 6.216: deskflow-frontend/app/(dashboard)/admin/layout.tsx

```
export default function AdminLayout({ children }: { children: React.ReactNode }) {
  return <>{children}</>;
}
```

Source code 6.217: deskflow-frontend/app/(dashboard)/admin/it/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import {
  ArrowLeft, Plus, Pencil, Trash2, CheckCircle, XCircle,
  Box, Users, History, Save, X,
} from 'lucide-react';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@/components/ui/select';
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dialog';
import { Tabs, TabsContent, TabsList, TabsTrigger } from '@/components/ui/tabs';
import { useAuth } from '@/lib/auth-context';
import {
  resourcesApi, employeeAdminApi, administrationApi,
  Resource, ResourceCreate, EmployeeWithLimits, AllocationRequest,
} from '@/lib/api';
import { useToast } from '@/hooks/use-toast';

const TYPE_LABELS: Record<string, string> = {
  DESK: 'Biurko', CONFERENCE_ROOM: 'Sala konf.', LAPTOP: 'Laptop',
  MONITOR: 'Monitor', PROJECTOR: 'Projektor', VEHICLE: 'Pojazd', ACCESSORY: 'Akcesoria',
};
const RESOURCE_TYPES = Object.keys(TYPE_LABELS);

const STATUS_LABELS: Record<string, { label: string; color: string }> = {
  SUBMITTED: { label: 'Złożony', color: 'bg-yellow-100 text-yellow-800' },
  APPROVED: { label: 'Zatwierdzony', color: 'bg-blue-100 text-blue-800' },
  ACTIVE: { label: 'Aktywny', color: 'bg-green-100 text-green-800' },
  COMPLETED: { label: 'Zakończony', color: 'bg-gray-100 text-gray-700' },
  REJECTED: { label: 'Odrzucony', color: 'bg-red-100 text-red-800' },
  CANCELLED: { label: 'Anulowany', color: 'bg-gray-100 text-gray-500' },
  OVERDUE: { label: 'Przeterminowane', color: 'bg-orange-100 text-orange-800' },
};

const EMPTY_FORM: ResourceCreate = { type: 'DESK', name: '', building: '', floor: 1, number: '' };

export default function ItAdminPage() {
  const { user, loading: authLoading, isItAdmin } = useAuth();
  const router = useRouter();
  const { toast } = useToast();

  // - Resources -
  const [resources, setResources] = useState<Resource[]>([]);
  const [resLoading, setResLoading] = useState(true);
  const [resDialog, setResDialog] = useState<'add' | 'edit' | null>(null);
  const [editTarget, setEditTarget] = useState<Resource | null>(null);
  const [form, setForm] = useState<ResourceCreate>(EMPTY_FORM);

  // - Limits -
  const [employees, setEmployees] = useState<EmployeeWithLimits[]>([]);
  const [empLoading, setEmpLoading] = useState(true);
  const [editingLimit, setEditingLimit] = useState<{ empId: string; type: string; value: number } |
  null>(null);

```

```
59    // - History -
60    const [history, setHistory] = useState<AllocationRequest[]>([]);
61    const [histLoading, setHistLoading] = useState(true);
62    const [histFilter, setHistFilter] = useState('');
63    const [resourceMap, setResourceMap] = useState<Record<string, Resource>>({});
64
65    useEffect(() => {
66      if (authLoading) return;
67
68      if (!user) { router.push('/login'); return; }
69      if (!isItAdmin()) { router.push('/'); return; }
70      loadResources();
71      loadEmployees();
72      loadHistory();
73    }, [user, authLoading, router]);
74
75    const loadResources = () =>
76      resourcesApi.list().then(setResources).catch(() => {}).finally(() => setResLoading(false));
77
78    const loadEmployees = () =>
79      employeeAdminApi.list().then(setEmployees).catch(() => {}).finally(() => setEmpLoading(false));
80
81    const loadHistory = () =>
82      Promise.all([administrationApi.requests(), resourcesApi.list()])
83        .then(([reqs, res]) => {
84          setHistory(reqs);
85          setResourceMap(Object.fromEntries(res.map(r => [r.id, r])));
86        })
87        .catch(() => {})
88        .finally(() => setHistLoading(false));
89
90    // - Resources actions -
91    const openAdd = () => { setForm(EMPTY_FORM); setEditTarget(null); setResDialog('add'); };
92    const openEdit = (r: Resource) => {
93      setForm({ type: r.type, name: r.name, building: r.building, floor: r.floor, number: r.number });
94      setEditTarget(r);
95      setResDialog('edit');
96    };
97
98    const handleSaveResource = async () => {
99      try {
100       if (resDialog === 'add') {
101         await resourcesApi.add(form);
102         toast({ title: 'Zasób dodany' });
103       } else if (editTarget) {
104         await resourcesApi.update(editTarget.id, form);
105         toast({ title: 'Zasób zaktualizowany' });
106       }
107       setResDialog(null);
108       loadResources();
109     } catch (e: unknown) {
110       toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
111     }
112   };
113
114   const handleToggleAvailability = async (r: Resource) => {
115     try {
116       if (r.available) await resourcesApi.markUnavailable(r.id);
117       else await resourcesApi.markAvailable(r.id);
118       loadResources();
119     } catch (e: unknown) {
120       toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
121     }
122   };
123
124   const handleDeactivate = async (id: string) => {
125     try {
126       await resourcesApi.deactivate(id);
127       toast({ title: 'Zasób dezaktywowany' });
128       loadResources();
```

```
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    }
  };

  // - Limits actions -
  const handleSaveLimit = async () => {
    if (!editingLimit) return;
    try {
      await employeeAdminApi.upsertLimit(editingLimit.empId, editingLimit.type, editingLimit.value);
      toast({ title: 'Limit zaktualizowany' });
      setEditingLimit(null);
      loadEmployees();
    } catch (e: unknown) {
      toast({ title: 'Błąd', description: e instanceof Error ? e.message : 'Błąd', variant: 'destructive' });
    }
  };

  const filteredHistory = histFilter
    ? history.filter(r => r.status === histFilter)
    : history;

  return (
    <div className="min-h-screen bg-gray-50 p-8">
      <div className="max-w-6xl mx-auto">
        <div className="flex items-center gap-4 mb-6">
          <Link href="/"><Button variant="ghost" size="sm"><ArrowLeft className="h-4 w-4 mr-1"
          />Powrót</Button></Link>
          <div>
            <h1 className="text-2xl font-bold">Panel IT Admin</h1>
            <p className="text-gray-600 text-sm">Zarządzanie zasobami, limitami i historia zdarzeń</p>
          </div>
        </div>

        <Tabs defaultValue="resources">
          <TabsList className="mb-6">
            <TabsTrigger value="resources" className="gap-2"><Box className="h-4 w-4" />Zasoby</TabsTrigger>
            <TabsTrigger value="limits" className="gap-2"><Users className="h-4 w-4" />Limity
            pracowników</TabsTrigger>
            <TabsTrigger value="history" className="gap-2"><History className="h-4 w-4" />Historia
            zdarzeń</TabsTrigger>
          </TabsList>

          {/* ===== RESOURCES ===== */}
          <TabsContent value="resources">
            <div className="flex justify-between items-center mb-4">
              <p className="text-sm text-gray-500">{resources.length} zasobów w systemie</p>
              <Button size="sm" className="gap-2" onClick={openAdd}>
                <Plus className="h-4 w-4" />Dodaj zasób
              </Button>
            </div>
            {resLoading ? (
              <p className="text-gray-500">Ładowanie...</p>
            ) : (
              <div className="space-y-2">
                {resources.map(r => (
                  <Card key={r.id} className={`${!r.available ? 'opacity-60' : ''}`}>
                    <CardContent className="p-4 flex items-center justify-between gap-4">
                      <div className="flex-1 min-w-0">
                        <div className="flex items-center gap-2 flex-wrap">
                          <span className="font-medium text-sm">{r.name}</span>
                          <span className="text-xs bg-blue-50 text-blue-700 px-1.5 py-0.5 rounded">
                            {TYPE_LABELS[r.type] || r.type}
                          </span>
                          {r.available
                            ? <span className="text-xs text-green-600 flex items-center gap-1"><CheckCircle
                            className="h-3 w-3" />Dostępny</span>
                            : <span className="text-xs text-red-500 flex items-center gap-1"><XCircle
                            className="h-3 w-3" />Niedostępny</span>
                          }
```

```
                    </div>
                    <p className="text-xs text-gray-500 mt-0.5">{r.building}, piętro {r.floor}, nr {r.number}</p>
                  </div>
                  <div className="flex gap-2 flex-shrink-0">
                    <Button size="sm" variant="outline" onClick={() => handleToggleAvailability(r)}>
                      {r.available ? 'Zablokuj' : 'Odblokuj'}
                    </Button>
                    <Button size="sm" variant="outline" onClick={() => openEdit(r)}>
                      <Pencil className="h-3.5 w-3.5" />
                    </Button>
                    <Button size="sm" variant="outline" className="text-red-600 border-red-200 hover:bg-red-50"
                      onClick={() => handleDeactivate(r.id)}>
                      <Trash2 className="h-3.5 w-3.5" />
                    </Button>
                  </div>
                </CardContent>
              </Card>
            ))}
          </div>
        )}
      </TabsContent>

      {/* ===== LIMITS ===== */}
      <TabsContent value="limits">
        {empLoading ? (
          <p className="text-gray-500">Ładowanie...</p>
        ) : (
          <div className="space-y-4">
            {employees.map(emp => (
              <Card key={emp.id}>
                <CardHeader className="pb-2">
                  <CardTitle className="text-base flex items-center gap-2">
                    {emp.firstName} {emp.lastName}
                    <span className="text-xs font-normal text-gray-500">{emp.email}</span>
                    <span className="text-xs bg-gray-100 text-gray-600 px-1.5 py-0.5 rounded">{emp.role}</span>
                  </CardTitle>
                </CardHeader>
                <CardContent className="pt-0">
                  <div className="flex flex-wrap gap-3">
                    {RESOURCE_TYPES.map(type => {
                      const current = emp.limits[type] ?? 0;
                      const isEditing = editingLimit?.empId === emp.id && editingLimit?.type === type;
                      return (
                        <div key={type} className="flex items-center gap-1.5 bg-gray-50 border rounded-lg px-3 py-2 text-sm">
                          <span className="text-gray-600 w-24 text-xs">{TYPE_LABELS[type]}</span>
                          {isEditing ? (
                            <>
                              <Input
                                type="number"
                                min={0}
                                className="w-16 h-7 text-center text-sm"
                                value={editingLimit.value}
                                onChange={e => setEditingLimit({ ...editingLimit, value: parseInt(e.target.value) || 0 })}
                                autoFocus
                              />
                              <Button size="sm" variant="ghost" className="h-7 w-7 p-0 text-green-600" onClick={handleSaveLimit}>
                                <Save className="h-3.5 w-3.5" />
                              </Button>
                              <Button size="sm" variant="ghost" className="h-7 w-7 p-0" onClick={() => setEditingLimit(null)}>
                                <X className="h-3.5 w-3.5" />
                              </Button>
                            </>
                          ) : (
```

```
                            <>
                              <span className="font-semibold w-6 text-center">{current}</span>
                              <Button
                                size="sm" variant="ghost" className="h-7 w-7 p-0 text-gray-400
                                hover:text-gray-700"
                                onClick={() => setEditingLimit({ empId: emp.id, type, value: current })}
                              >
                                <Pencil className="h-3 w-3" />
                              </Button>
                            </>
                          )}
                        </div>
                      );
                    })}
                  </div>
                </CardContent>
              </Card>
            ))}
          </div>
        )}
      </TabsContent>

      {/* ===== HISTORY ===== */}
      <TabsContent value="history">
        <div className="flex gap-2 mb-4 flex-wrap">
          {['', ...Object.keys(STATUS_LABELS)].map(s => (
            <Button key={s} variant={histFilter === s ? 'default' : 'outline'} size="sm"
              onClick={() => setHistFilter(s)}>
              {s === '' ? 'Wszystkie' : STATUS_LABELS[s]?.label}
            </Button>
          ))}
        </div>
        {histLoading ? (
          <p className="text-gray-500">Ładowanie...</p>
        ) : filteredHistory.length === 0 ? (
          <Card><CardContent className="p-8 text-center text-gray-500">Brak zdarzeń.</CardContent></Card>
        ) : (
          <div className="space-y-2">
            {filteredHistory.map(r => {
              const res = resourceMap[r.resourceId];
              return (
                <Card key={r.id}>
                  <CardContent className="p-3 flex items-center gap-4">
                    <div className="flex-1 min-w-0">
                      <div className="flex items-center gap-2 flex-wrap mb-1">
                        <span className={`text-xs px-2 py-0.5 rounded-full font-medium
                        ${STATUS_LABELS[r.status]?.color}`}>
                          {STATUS_LABELS[r.status]?.label}
                        </span>
                        <span className="text-xs text-gray-500">
                          {r.employeeFirstName && r.employeeLastName
                            ? `${r.employeeFirstName} ${r.employeeLastName}`
                            : r.employeeId.slice(0, 8)}
                        </span>
                        <span className="text-xs text-gray-400">→</span>
                        <span className="text-xs font-medium">{res ? res.name : r.resourceId.slice(0,
                        8)}</span>
                        {res && <span className="text-xs text-gray-400">{TYPE_LABELS[res.type] ||
                        res.type}</span>}
                      </div>
                      <div className="flex items-center gap-3 text-xs text-gray-500">
                        <span>{r.dateFrom} - {r.dateTo}</span>
                        <span>złożony: {new Date(r.submittedAt).toLocaleDateString('pl-PL')}</span>
                        {r.issuedAt && <span>wydany: {new
                        Date(r.issuedAt).toLocaleDateString('pl-PL')}</span>}
                        {r.returnedAt && <span>zwrócony: {new
                        Date(r.returnedAt).toLocaleDateString('pl-PL')}</span>}
                      </div>
                      {r.rejectionReason && <p className="text-xs text-red-500 mt-0.5">Powód:
                      {r.rejectionReason}</p>}
```

```
                      </div>
                    </CardContent>
                  </Card>
                );
              })}
            </div>
          )}
        </TabsContent>
      </Tabs>
    </div>

    {/* Resource add/edit dialog */}
    <Dialog open={!!resDialog} onOpenChange={v => !v && setResDialog(null)}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>{resDialog === 'add' ? 'Dodaj zasób' : 'Edytuj zasób'}</DialogTitle>
        </DialogHeader>
        <div className="space-y-3">
          {resDialog === 'add' && (
            <div>
              <Label>Typ</Label>
              <Select value={form.type} onValueChange={v => setForm(f => ({ ...f, type: v }))}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  {RESOURCE_TYPES.map(t => <SelectItem key={t} value={t}>{TYPE_LABELS[t]}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          <div>
            <Label>Nazwa</Label>
            <Input value={form.name} onChange={e => setForm(f => ({ ...f, name: e.target.value }))}
            placeholder="np. Biurko A-105" />
          </div>
          <div className="grid grid-cols-3 gap-2">
            <div>
              <Label>Budynek</Label>
              <Input value={form.building} onChange={e => setForm(f => ({ ...f, building: e.target.value
              }))} placeholder="A" />
            </div>
            <div>
              <Label>Piętro</Label>
              <Input type="number" value={form.floor} onChange={e => setForm(f => ({ ...f, floor:
              parseInt(e.target.value) || 0 }))} />
            </div>
            <div>
              <Label>Numer</Label>
              <Input value={form.number} onChange={e => setForm(f => ({ ...f, number: e.target.value }))}
              placeholder="105" />
            </div>
          </div>
          <div className="flex gap-2 justify-end pt-2">
            <Button variant="outline" onClick={() => setResDialog(null)}>Anuluj</Button>
            <Button onClick={handleSaveResource} disabled={!form.name || !form.building || !form.number}>
              {resDialog === 'add' ? 'Dodaj' : 'Zapisz'}
            </Button>
          </div>
        </div>
      </DialogContent>
    </Dialog>
  </div>
 );
}
```

Source code 6.218: deskflow-frontend/lib/api.ts

```
const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

```

```
function getToken(): string | null {
  if (typeof window === 'undefined') return null;
  return localStorage.getItem('deskflow_token');
}

async function request<T>(path: string, options: RequestInit = {}): Promise<T> {
  const token = getToken();
  const headers: Record<string, string> = {
    'Content-Type': 'application/json',
    ...(options.headers as Record<string, string>),
  };
  if (token) headers['Authorization'] = `Bearer ${token}`;

  const res = await fetch(`${API_URL}${path}`, { ...options, headers });

  if (res.status === 401) {
    localStorage.removeItem('deskflow_token');
    localStorage.removeItem('deskflow_user');
    window.location.href = '/login';
    throw new Error('Session expired');
  }

  if (!res.ok) {
    const err = await res.json().catch(() => ({ error: res.statusText }));
    throw new Error(err.error || 'Server error');
  }

  const text = await res.text();
  return text ? JSON.parse(text) : null;
}

export const api = {
  get: <T>(path: string) => request<T>(path, { method: 'GET' }),
  post: <T>(path: string, body: unknown) => request<T>(path, { method: 'POST', body: JSON.stringify(body) }),
  put: <T>(path: string, body?: unknown) => request<T>(path, { method: 'PUT', body: body ? JSON.stringify(body) : undefined }),
  delete: <T>(path: string) => request<T>(path, { method: 'DELETE' }),
};

// Auth
export const authApi = {
  login: (email: string, password: string) =>
    api.post<{ token: string; email: string; role: string; firstName: string; lastName: string; employeeId: string }>(
      '/api/auth/login', { email, password }
    ),
  me: () => api.get<{ email: string; role: string; firstName: string; lastName: string; employeeId: string }>('/api/auth/me'),
};

// Resources (UC-01)
export const resourcesApi = {
  list: () => api.get<Resource[]>('/api/resources'),
  add: (data: ResourceCreate) => api.post<{ resourceId: string }>('/api/resources', data),
  update: (id: string, data: ResourceCreate) => api.put(`/api/resources/${id}`, data),
  markUnavailable: (id: string) => api.put(`/api/resources/${id}/unavailable`),
  markAvailable: (id: string) => api.put(`/api/resources/${id}/available`),
  deactivate: (id: string) => api.delete(`/api/resources/${id}`),
};

// Allocation requests (UC-02, 03, 14)
export const requestsApi = {
  submit: (data: SubmitRequest) => api.post<{ requestId: string }>('/api/requests', data),
  cancel: (id: string) => api.put(`/api/requests/${id}/cancel`),
  my: () => api.get<AllocationRequest[]>('/api/requests/my'),
};

// Administration (UC-04-09, 15)
export const administrationApi = {
```

```
  requests: (status?: string) => api.get<AllocationRequest[]>(`/api/administration/requests${status ?
  `?status=${status}` : ''}`),
  approve: (id: string) => api.put(`/api/administration/requests/${id}/approve`),
  reject: (id: string, reason: string) => api.put(`/api/administration/requests/${id}/reject`, { reason }),
  issue: (id: string) => api.put(`/api/administration/requests/${id}/issue`),
  return: (id: string) => api.put(`/api/administration/requests/${id}/return`),
  allocations: () => api.get<AllocationRequest[]>('/api/administration/allocations'),
};

// Employee admin (IT_ADMIN - UC-10, UC-11, limits)
export const employeeAdminApi = {
  list: () => api.get<EmployeeWithLimits[]>('/api/administration/employees'),
  upsertLimit: (employeeId: string, resourceType: string, amount: number) =>
    api.put(`/api/administration/employees/${employeeId}/limits`, { resourceType, amount }),
};

// Types
export interface Resource {
  id: string;
  type: string;
  name: string;
  building: string;
  floor: number;
  number: string;
  available: boolean;
}

export interface ResourceCreate {
  type: string;
  name: string;
  building: string;
  floor: number;
  number: string;
}

export interface AllocationRequest {
  id: string;
  employeeId: string;
  employeeFirstName: string | null;
  employeeLastName: string | null;
  resourceId: string;
  status: string;
  dateFrom: string;
  dateTo: string;
  justification: string;
  submittedAt: string;
  issuedAt: string | null;
  returnedAt: string | null;
  rejectionReason: string | null;
}

export interface SubmitRequest {
  resourceId: string;
  dateFrom: string;
  dateTo: string;
  justification: string;
}

export interface EmployeeWithLimits {
  id: string;
  email: string;
  firstName: string;
  lastName: string;
  role: string;
  limits: Record<string, number>;
}

export interface CurrentUser {
  email: string;
  role: string;
```

```
  firstName: string;
  lastName: string;
  employeeId: string;
}
```

Source code 6.219: deskflow-frontend/lib/utils.ts

```
import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

Source code 6.220: deskflow-frontend/lib/auth-context.tsx

```
'use client';

import { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import { CurrentUser, authApi } from './api';

interface AuthContextType {
  user: CurrentUser | null;
  loading: boolean;
  login: (token: string, user: CurrentUser) => void;
  logout: () => void;
  isManager: () => boolean;
  isItAdmin: () => boolean;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<CurrentUser | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('deskflow_token');
    const stored = localStorage.getItem('deskflow_user');
    if (token && stored) {
      setUser(JSON.parse(stored));
    }
    setLoading(false);
  }, []);

  const login = (token: string, userData: CurrentUser) => {
    localStorage.setItem('deskflow_token', token);
    localStorage.setItem('deskflow_user', JSON.stringify(userData));
    setUser(userData);
  };

  const logout = () => {
    localStorage.removeItem('deskflow_token');
    localStorage.removeItem('deskflow_user');
    setUser(null);
    window.location.href = '/login';
  };

  const isManager = () => user?.role === 'MANAGER' || user?.role === 'IT_ADMIN';
  const isItAdmin = () => user?.role === 'IT_ADMIN';

  return (
    <AuthContext.Provider value={{ user, loading, login, logout, isManager, isItAdmin }}>
      {children}
    </AuthContext.Provider>
```

```
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used within AuthProvider');
  return ctx;
}
```

Source code 6.221: deskflow-frontend/hooks/use-toast.ts

```
'use client'

// Inspired by react-hot-toast library
import * as React from 'react'

import type { ToastActionElement, ToastProps } from '@/components/ui/toast'

const TOAST_LIMIT = 1
const TOAST_REMOVE_DELAY = 1000000

type ToasterToast = ToastProps & {
  id: string
  title?: React.ReactNode
  description?: React.ReactNode
  action?: ToastActionElement
}

const actionTypes = {
  ADD_TOAST: 'ADD_TOAST',
  UPDATE_TOAST: 'UPDATE_TOAST',
  DISMISS_TOAST: 'DISMISS_TOAST',
  REMOVE_TOAST: 'REMOVE_TOAST',
} as const

let count = 0

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER
  return count.toString()
}

type ActionType = typeof actionTypes

type Action =
  | {
      type: ActionType['ADD_TOAST']
      toast: ToasterToast
    }
  | {
      type: ActionType['UPDATE_TOAST']
      toast: Partial<ToasterToast>
    }
  | {
      type: ActionType['DISMISS_TOAST']
      toastId?: ToasterToast['id']
    }
  | {
      type: ActionType['REMOVE_TOAST']
      toastId?: ToasterToast['id']
    }

interface State {
  toasts: ToasterToast[]
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()

```

```
const addToRemoveQueue = (toastId: string) => {
  if (toastTimeouts.has(toastId)) {
    return
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId)
    dispatch({
      type: 'REMOVE_TOAST',
      toastId: toastId,
    })
  }, TOAST_REMOVE_DELAY)

  toastTimeouts.set(toastId, timeout)
}

export const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case 'ADD_TOAST':
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      }

    case 'UPDATE_TOAST':
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t,
        ),
      }

    case 'DISMISS_TOAST': {
      const { toastId } = action

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId)
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id)
        })
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t,
        ),
      }
    }
    case 'REMOVE_TOAST':
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        }
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      }
  }
}
```

```

const listeners: Array<(state: State) => void> = []

let memoryState: State = { toasts: [] }

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action)
  listeners.forEach((listener) => {
    listener(memoryState)
  })
}

type Toast = Omit<ToasterToast, 'id'>

function toast({ ...props }: Toast) {
  const id = genId()

  const update = (props: ToasterToast) =>
    dispatch({
      type: 'UPDATE_TOAST',
      toast: { ...props, id },
    })
  const dismiss = () => dispatch({ type: 'DISMISS_TOAST', toastId: id })

  dispatch({
    type: 'ADD_TOAST',
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss()
      },
    },
  })

  return {
    id: id,
    dismiss,
    update,
  }
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState)

  React.useEffect(() => {
    listeners.push(setState)
    return () => {
      const index = listeners.indexOf(setState)
      if (index > -1) {
        listeners.splice(index, 1)
      }
    }
  }, [state])

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: 'DISMISS_TOAST', toastId }),
  }
}

export { useToast, toast }
```

Source code 6.222: deskflow-frontend/hooks/use-mobile.ts

```
import * as React from 'react'
```

```
2
3  const MOBILE_BREAKPOINT = 768
4
5  export function useIsMobile() {
6    const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined)
7
8    React.useEffect(() => {
9      const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
10     const onChange = () => {
11       setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
12     }
13     mql.addEventListener('change', onChange)
14     setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
15     return () => mql.removeEventListener('change', onChange)
16   }, [])
17
18   return !!isMobile
19 }
```

Source code 6.223: deskflow-frontend/styles/globals.css

```
1  @import 'tailwindcss';
2  @import 'tw-animate-css';
3
4  @custom-variant dark (&:is(.dark *));
5
6  :root {
7    -background: oklch(1 0 0);
8    -foreground: oklch(0.145 0 0);
9    -card: oklch(1 0 0);
10   -card-foreground: oklch(0.145 0 0);
11   -popover: oklch(1 0 0);
12   -popover-foreground: oklch(0.145 0 0);
13   -primary: oklch(0.205 0 0);
14   -primary-foreground: oklch(0.985 0 0);
15   -secondary: oklch(0.97 0 0);
16   -secondary-foreground: oklch(0.205 0 0);
17   -muted: oklch(0.97 0 0);
18   -muted-foreground: oklch(0.556 0 0);
19   -accent: oklch(0.97 0 0);
20   -accent-foreground: oklch(0.205 0 0);
21   -destructive: oklch(0.577 0.245 27.325);
22   -destructive-foreground: oklch(0.577 0.245 27.325);
23   -border: oklch(0.922 0 0);
24   -input: oklch(0.922 0 0);
25   -ring: oklch(0.708 0 0);
26   -chart-1: oklch(0.646 0.222 41.116);
27   -chart-2: oklch(0.6 0.118 184.704);
28   -chart-3: oklch(0.398 0.07 227.392);
29   -chart-4: oklch(0.828 0.189 84.429);
30   -chart-5: oklch(0.769 0.188 70.08);
31   -radius: 0.625rem;
32   -sidebar: oklch(0.985 0 0);
33   -sidebar-foreground: oklch(0.145 0 0);
34   -sidebar-primary: oklch(0.205 0 0);
35   -sidebar-primary-foreground: oklch(0.985 0 0);
36   -sidebar-accent: oklch(0.97 0 0);
37   -sidebar-accent-foreground: oklch(0.205 0 0);
38   -sidebar-border: oklch(0.922 0 0);
39   -sidebar-ring: oklch(0.708 0 0);
40 }
41
42 .dark {
43   -background: oklch(0.145 0 0);
44   -foreground: oklch(0.985 0 0);
45   -card: oklch(0.145 0 0);
46   -card-foreground: oklch(0.985 0 0);
47   -popover: oklch(0.145 0 0);
```

```
  -popover-foreground: oklch(0.985 0 0);
  -primary: oklch(0.985 0 0);
  -primary-foreground: oklch(0.205 0 0);
  -secondary: oklch(0.269 0 0);
  -secondary-foreground: oklch(0.985 0 0);
  -muted: oklch(0.269 0 0);
  -muted-foreground: oklch(0.708 0 0);
  -accent: oklch(0.269 0 0);
  -accent-foreground: oklch(0.985 0 0);
  -destructive: oklch(0.396 0.141 25.723);
  -destructive-foreground: oklch(0.637 0.237 25.331);
  -border: oklch(0.269 0 0);
  -input: oklch(0.269 0 0);
  -ring: oklch(0.439 0 0);
  -chart-1: oklch(0.488 0.243 264.376);
  -chart-2: oklch(0.696 0.17 162.48);
  -chart-3: oklch(0.769 0.188 70.08);
  -chart-4: oklch(0.627 0.265 303.9);
  -chart-5: oklch(0.645 0.246 16.439);
  -sidebar: oklch(0.205 0 0);
  -sidebar-foreground: oklch(0.985 0 0);
  -sidebar-primary: oklch(0.488 0.243 264.376);
  -sidebar-primary-foreground: oklch(0.985 0 0);
  -sidebar-accent: oklch(0.269 0 0);
  -sidebar-accent-foreground: oklch(0.985 0 0);
  -sidebar-border: oklch(0.269 0 0);
  -sidebar-ring: oklch(0.439 0 0);
}

@theme inline {
  -font-sans: 'Geist', 'Geist Fallback';
  -font-mono: 'Geist Mono', 'Geist Mono Fallback';
  -color-background: var(-background);
  -color-foreground: var(-foreground);
  -color-card: var(-card);
  -color-card-foreground: var(-card-foreground);
  -color-popover: var(-popover);
  -color-popover-foreground: var(-popover-foreground);
  -color-primary: var(-primary);
  -color-primary-foreground: var(-primary-foreground);
  -color-secondary: var(-secondary);
  -color-secondary-foreground: var(-secondary-foreground);
  -color-muted: var(-muted);
  -color-muted-foreground: var(-muted-foreground);
  -color-accent: var(-accent);
  -color-accent-foreground: var(-accent-foreground);
  -color-destructive: var(-destructive);
  -color-destructive-foreground: var(-destructive-foreground);
  -color-border: var(-border);
  -color-input: var(-input);
  -color-ring: var(-ring);
  -color-chart-1: var(-chart-1);
  -color-chart-2: var(-chart-2);
  -color-chart-3: var(-chart-3);
  -color-chart-4: var(-chart-4);
  -color-chart-5: var(-chart-5);
  -radius-sm: calc(var(-radius) - 4px);
  -radius-md: calc(var(-radius) - 2px);
  -radius-lg: var(-radius);
  -radius-xl: calc(var(-radius) + 4px);
  -color-sidebar: var(-sidebar);
  -color-sidebar-foreground: var(-sidebar-foreground);
  -color-sidebar-primary: var(-sidebar-primary);
  -color-sidebar-primary-foreground: var(-sidebar-primary-foreground);
  -color-sidebar-accent: var(-sidebar-accent);
  -color-sidebar-accent-foreground: var(-sidebar-accent-foreground);
  -color-sidebar-border: var(-sidebar-border);
  -color-sidebar-ring: var(-sidebar-ring);
}

```

```
@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground;
  }
}
```

Source code 6.224: deskflow-frontend/components/auth-landing.tsx

```
"use client"

import { SignInButton, SignOutButton, SignUpButton } from "@clerk/nextjs"
import { Button } from "@/components/ui/button"

interface AuthLandingProps {
  title: string
  subtitle: string
  showAuthActions?: boolean
  showSignOutAction?: boolean
  statusMessage?: string
}

export function AuthLanding({
  title,
  subtitle,
  showAuthActions = true,
  showSignOutAction = false,
  statusMessage = "Brak przypisanej organizacji. Skontaktuj sie z administratorem firmy.",
}: AuthLandingProps) {
  return (
    <div className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-6 py-16 text-slate-100">
      <div className="pointer-events-none absolute inset-0">
        <div className="absolute -left-28 -top-28 h-80 w-80 rounded-full bg-cyan-500/20 blur-3xl" />
        <div className="absolute -right-24 bottom-0 h-96 w-96 rounded-full bg-emerald-500/20 blur-3xl" />
        <div className="absolute inset-0 bg-[radial-gradient(circle_at_20%_20%,rgba(34,211,238,0.12),transparent_35%),radial-gradient(circle_at_80%_80%,rgba(16,
        />
      </div>

      <div className="relative w-full max-w-3xl rounded-2xl border border-slate-700/60 bg-slate-900/70 p-8 shadow-2xl backdrop-blur md:p-12">
        <p className="inline-flex items-center rounded-full border border-cyan-400/30 bg-cyan-500/10 px-3 py-1 text-xs font-semibold tracking-wide text-cyan-300">
          DESKFLOW
        </p>

        <h1 className="mt-6 text-3xl font-bold leading-tight md:text-5xl">{title}</h1>
        <p className="mt-4 max-w-2xl text-sm text-slate-300 md:text-base">{subtitle}</p>

        {showAuthActions ? (
          <div className="mt-8 flex flex-wrap items-center gap-3">
            <SignInButton>
              <Button className="h-11 bg-cyan-500 px-6 text-slate-950 hover:bg-cyan-400">Zaloguj sie</Button>
            </SignInButton>
            <SignUpButton>
              <Button variant="outline" className="h-11 border-slate-300 bg-white px-6 text-black hover:bg-slate-100 hover:text-black">
                Zarejestruj sie
              </Button>
            </SignUpButton>
          </div>
        ) : (
          <div className="mt-8 space-y-3">
            <div className="rounded-lg border border-amber-300/40 bg-amber-500/10 p-3 text-sm text-amber-100">
```

```
              {statusMessage}
            </div>
            {showSignOutAction && (
              <SignOutButton>
                <Button variant="outline" className="h-11 border-slate-300 bg-white px-6 text-black
                hover:bg-slate-100 hover:text-black">
                  Wyloguj sie
                </Button>
              </SignOutButton>
            )}
          </div>
        )}

        <div className="mt-10 grid gap-3 text-xs text-slate-400 md:grid-cols-3">
          <div className="rounded-lg border border-slate-700/70 bg-slate-900/70 p-3">Rezerwacje biurek i
          sal</div>
          <div className="rounded-lg border border-slate-700/70 bg-slate-900/70 p-3">Wypozyczanie
          zasobow</div>
          <div className="rounded-lg border border-slate-700/70 bg-slate-900/70 p-3">Panel administracyjny
          firmy</div>
        </div>
      </div>
    </div>
  )
}
```

Source code 6.225: deskflow-frontend/components/theme-provider.tsx

```
'use client'

import * as React from 'react'
import {
  ThemeProvider as NextThemesProvider,
  type ThemeProviderProps,
} from 'next-themes'

export function ThemeProvider({ children, ...props }: ThemeProviderProps) {
  return <NextThemesProvider {...props}>{children}</NextThemesProvider>
}
```

Source code 6.226: deskflow-frontend/components/ui/accordion.tsx

```
'use client'

import * as React from 'react'
import * as AccordionPrimitive from '@radix-ui/react-accordion'
import { ChevronDownIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function Accordion({
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Root>) {
  return <AccordionPrimitive.Root data-slot="accordion" {...props} />
}

function AccordionItem({
  className,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Item>) {
  return (
    <AccordionPrimitive.Item
      data-slot="accordion-item"
      className={cn('border-b last:border-b-0', className)}
      {...props}
```

```
    />
  )
}

function AccordionTrigger({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Trigger>) {
  return (
    <AccordionPrimitive.Header className="flex">
      <AccordionPrimitive.Trigger
        data-slot="accordion-trigger"
        className={cn(
          'focus-visible:border-ring focus-visible:ring-ring/50 flex flex-1 items-start justify-between gap-4 rounded-md py-4 text-left text-sm font-medium transition-all outline-none hover:underline focus-visible:ring-[3px] disabled:pointer-events-none disabled:opacity-50 [&[data-state=open]>svg]:rotate-180',
          className,
        )}
        {...props}
      >
        {children}
        <ChevronDownIcon className="text-muted-foreground pointer-events-none size-4 shrink-0 translate-y-0.5 transition-transform duration-200" />
      </AccordionPrimitive.Trigger>
    </AccordionPrimitive.Header>
  )
}

function AccordionContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Content>) {
  return (
    <AccordionPrimitive.Content
      data-slot="accordion-content"
      className="data-[state=closed]:animate-accordion-up data-[state=open]:animate-accordion-down overflow-hidden text-sm"
      {...props}
    >
      <div className={cn('pt-0 pb-4', className)}>{children}</div>
    </AccordionPrimitive.Content>
  )
}

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }
```

Source code 6.227: deskflow-frontend/components/ui/alert-dialog.tsx

```
'use client'

import * as React from 'react'
import * as AlertDialogPrimitive from '@radix-ui/react-alert-dialog'

import { cn } from '@/lib/utils'
import { buttonVariants } from '@/components/ui/button'

function AlertDialog({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Root>) {
  return <AlertDialogPrimitive.Root data-slot="alert-dialog" {...props} />
}

function AlertDialogTrigger({
  ...props
```

```
}: React.ComponentProps<typeof AlertDialogPrimitive.Trigger>) {
  return (
    <AlertDialogPrimitive.Trigger data-slot="alert-dialog-trigger" {...props} />
  )
}

function AlertDialogPortal({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Portal>) {
  return (
    <AlertDialogPrimitive.Portal data-slot="alert-dialog-portal" {...props} />
  )
}

function AlertDialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Overlay>) {
  return (
    <AlertDialogPrimitive.Overlay
      data-slot="alert-dialog-overlay"
      className={cn(
        'data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
        data-[state=open]:fade-in-0 fixed inset-0 z-50 bg-black/50',
        className,
      )}
      {...props}
    />
  )
}

function AlertDialogContent({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Content>) {
  return (
    <AlertDialogPortal>
      <AlertDialogOverlay />
      <AlertDialogPrimitive.Content
        data-slot="alert-dialog-content"
        className={cn(
          'bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 fixed top-[50%] left-[50%] z-50 grid w-full max-w-[calc(100%-2rem)]
          translate-x-[-50%] translate-y-[-50%] gap-4 rounded-lg border p-6 shadow-lg duration-200
          sm:max-w-lg',
          className,
        )}
        {...props}
      />
    </AlertDialogPortal>
  )
}

function AlertDialogHeader({
  className,
  ...props
}: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="alert-dialog-header"
      className={cn('flex flex-col gap-2 text-center sm:text-left', className)}
      {...props}
    />
  )
}

function AlertDialogFooter({
  className,
  ...props
```

```
}: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="alert-dialog-footer"
      className={cn(
        'flex flex-col-reverse gap-2 sm:flex-row sm:justify-end',
        className,
      )}
      {...props}
    />
  )
}

function AlertDialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Title>) {
  return (
    <AlertDialogPrimitive.Title
      data-slot="alert-dialog-title"
      className={cn('text-lg font-semibold', className)}
      {...props}
    />
  )
}

function AlertDialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Description>) {
  return (
    <AlertDialogPrimitive.Description
      data-slot="alert-dialog-description"
      className={cn('text-muted-foreground text-sm', className)}
      {...props}
    />
  )
}

function AlertDialogAction({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Action>) {
  return (
    <AlertDialogPrimitive.Action
      className={cn(buttonVariants(), className)}
      {...props}
    />
  )
}

function AlertDialogCancel({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Cancel>) {
  return (
    <AlertDialogPrimitive.Cancel
      className={cn(buttonVariants({ variant: 'outline' }), className)}
      {...props}
    />
  )
}

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
```

```
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
  AlertDialogCancel,
}
```

Source code 6.228: deskflow-frontend/components/ui/alert.tsx

```
import * as React from 'react'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'

const alertVariants = cva(
  'relative w-full rounded-lg border px-4 py-3 text-sm grid has-[>svg]:grid-cols-[calc(var(-spacing)*4)_1fr] grid-cols-[0_1fr] has-[>svg]:gap-x-3 gap-y-0.5 items-start [&>svg]:size-4 [&>svg]:translate-y-0.5 [&>svg]:text-current',
  {
    variants: {
      variant: {
        default: 'bg-card text-card-foreground',
        destructive:
          'text-destructive bg-card [&>svg]:text-current *:data-[slot=alert-description]:text-destructive/90',
      },
    },
    defaultVariants: {
      variant: 'default',
    },
  },
)

function Alert({
  className,
  variant,
  ...props
}: React.ComponentProps<'div'> & VariantProps<typeof alertVariants>) {
  return (
    <div
      data-slot="alert"
      role="alert"
      className={cn(alertVariants({ variant }), className)}
      {...props}
    />
  )
}

function AlertTitle({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="alert-title"
      className={cn(
        'col-start-2 line-clamp-1 min-h-4 font-medium tracking-tight',
        className,
      )}
      {...props}
    />
  )
}

function AlertDescription({
  className,
  ...props
}: React.ComponentProps<'div'>) {
  return (
    <div
```

```
      data-slot="alert-description"
      className={cn(
        'text-muted-foreground col-start-2 grid justify-items-start gap-1 text-sm [&_p]:leading-relaxed',
        className,
      )}
      {...props}
    />
  )
}

export { Alert, AlertTitle, AlertDescription }
```

Source code 6.229: deskflow-frontend/components/ui/aspect-ratio.tsx

```
'use client'

import * as AspectRatioPrimitive from '@radix-ui/react-aspect-ratio'

function AspectRatio({
  ...props
}: React.ComponentProps<typeof AspectRatioPrimitive.Root>) {
  return <AspectRatioPrimitive.Root data-slot="aspect-ratio" {...props} />
}

export { AspectRatio }
```

Source code 6.230: deskflow-frontend/components/ui/avatar.tsx

```
'use client'

import * as React from 'react'
import * as AvatarPrimitive from '@radix-ui/react-avatar'

import { cn } from '@/lib/utils'

function Avatar({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Root>) {
  return (
    <AvatarPrimitive.Root
      data-slot="avatar"
      className={cn(
        'relative flex size-8 shrink-0 overflow-hidden rounded-full',
        className,
      )}
      {...props}
    />
  )
}

function AvatarImage({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Image>) {
  return (
    <AvatarPrimitive.Image
      data-slot="avatar-image"
      className={cn('aspect-square size-full', className)}
      {...props}
    />
  )
}

function AvatarFallback({
```

```
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Fallback>) {
  return (
    <AvatarPrimitive.Fallback
      data-slot="avatar-fallback"
      className={cn(
        'bg-muted flex size-full items-center justify-center rounded-full',
        className,
      )}
      {...props}
    />
  )
}

export { Avatar, AvatarImage, AvatarFallback }
```

Source code 6.231: deskflow-frontend/components/ui/badge.tsx

```
import * as React from 'react'
import { Slot } from '@radix-ui/react-slot'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'

const badgeVariants = cva(
  'inline-flex items-center justify-center rounded-md border px-2 py-0.5 text-xs font-medium w-fit
  whitespace-nowrap shrink-0 [&>svg]:size-3 gap-1 [&>svg]:pointer-events-none focus-visible:border-ring
  focus-visible:ring-ring/50 focus-visible:ring-[3px] aria-invalid:ring-destructive/20
  dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive transition-[color,box-shadow]
  overflow-hidden',
  {
    variants: {
      variant: {
        default:
          'border-transparent bg-primary text-primary-foreground [a&]:hover:bg-primary/90',
        secondary:
          'border-transparent bg-secondary text-secondary-foreground [a&]:hover:bg-secondary/90',
        destructive:
          'border-transparent bg-destructive text-white [a&]:hover:bg-destructive/90
          focus-visible:ring-destructive/20 dark:focus-visible:ring-destructive/40 dark:bg-destructive/60',
        outline:
          'text-foreground [a&]:hover:bg-accent [a&]:hover:text-accent-foreground',
      },
    },
    defaultVariants: {
      variant: 'default',
    },
  },
)

function Badge({
  className,
  variant,
  asChild = false,
  ...props
}: React.ComponentProps<'span'> &
  VariantProps<typeof badgeVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : 'span'

  return (
    <Comp
      data-slot="badge"
      className={cn(badgeVariants({ variant }), className)}
      {...props}
    />
  )
```

```
}

export { Badge, badgeVariants }
```

Source code 6.232: deskflow-frontend/components/ui/breadcrumb.tsx

```
import * as React from 'react'
import { Slot } from '@radix-ui/react-slot'
import { ChevronRight, MoreHorizontal } from 'lucide-react'

import { cn } from '@/lib/utils'

function Breadcrumb({ ...props }: React.ComponentProps<'nav'>) {
  return <nav aria-label="breadcrumb" data-slot="breadcrumb" {...props} />
}

function BreadcrumbList({ className, ...props }: React.ComponentProps<'ol'>) {
  return (
    <ol
      data-slot="breadcrumb-list"
      className={cn(
        'text-muted-foreground flex flex-wrap items-center gap-1.5 text-sm break-words sm:gap-2.5',
        className,
      )}
      {...props}
    />
  )
}

function BreadcrumbItem({ className, ...props }: React.ComponentProps<'li'>) {
  return (
    <li
      data-slot="breadcrumb-item"
      className={cn('inline-flex items-center gap-1.5', className)}
      {...props}
    />
  )
}

function BreadcrumbLink({
  asChild,
  className,
  ...props
}: React.ComponentProps<'a'> & {
  asChild?: boolean
}) {
  const Comp = asChild ? Slot : 'a'

  return (
    <Comp
      data-slot="breadcrumb-link"
      className={cn('hover:text-foreground transition-colors', className)}
      {...props}
    />
  )
}

function BreadcrumbPage({ className, ...props }: React.ComponentProps<'span'>) {
  return (
    <span
      data-slot="breadcrumb-page"
      role="link"
      aria-disabled="true"
      aria-current="page"
      className={cn('text-foreground font-normal', className)}
      {...props}
    />
  )
```

```
}

function BreadcrumbSeparator({
  children,
  className,
  ...props
}: React.ComponentProps<'li'>) {
  return (
    <li
      data-slot="breadcrumb-separator"
      role="presentation"
      aria-hidden="true"
      className={cn('[&>svg]:size-3.5', className)}
      {...props}
    >
      {children ?? <ChevronRight />}
    </li>
  )
}

function BreadcrumbEllipsis({
  className,
  ...props
}: React.ComponentProps<'span'>) {
  return (
    <span
      data-slot="breadcrumb-ellipsis"
      role="presentation"
      aria-hidden="true"
      className={cn('flex size-9 items-center justify-center', className)}
      {...props}
    >
      <MoreHorizontal className="size-4" />
      <span className="sr-only">More</span>
    </span>
  )
}

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,
  BreadcrumbSeparator,
  BreadcrumbEllipsis,
}
```

Source code 6.233: deskflow-frontend/components/ui/button-group.tsx

```
import { Slot } from '@radix-ui/react-slot'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'
import { Separator } from '@/components/ui/separator'

const buttonGroupVariants = cva(
  "flex w-fit items-stretch [&>*]:focus-visible:z-10 [&>*]:focus-visible:relative
  [&>[data-slot=select-trigger]:not([class*='w-'])]:w-fit [&>input]:flex-1
  has-[select[aria-hidden=true]:last-child]:[&>[data-slot=select-trigger]:last-of-type]:rounded-r-md
  has-[>[data-slot=button-group]]:gap-2",
  {
    variants: {
      orientation: {
        horizontal:
          '[&>*:not(:first-child)]:rounded-l-none [&>*:not(:first-child)]:border-l-0
          [&>*:not(:last-child)]:rounded-r-none',
```

```
        vertical:
          'flex-col [&>*:not(:first-child)]:rounded-t-none [&>*:not(:first-child)]:border-t-0
          [&>*:not(:last-child)]:rounded-b-none',
      },
    },
    defaultVariants: {
      orientation: 'horizontal',
    },
  },
)

function ButtonGroup({
  className,
  orientation,
  ...props
}: React.ComponentProps<'div'> & VariantProps<typeof buttonGroupVariants>) {
  return (
    <div
      role="group"
      data-slot="button-group"
      data-orientation={orientation}
      className={cn(buttonGroupVariants({ orientation }), className)}
      {...props}
    />
  )
}

function ButtonGroupText({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<'div'> & {
  asChild?: boolean
}) {
  const Comp = asChild ? Slot : 'div'

  return (
    <Comp
      className={cn(
        "bg-muted flex items-center gap-2 rounded-md border px-4 text-sm font-medium shadow-xs
        [&_svg]:pointer-events-none [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}

function ButtonGroupSeparator({
  className,
  orientation = 'vertical',
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="button-group-separator"
      orientation={orientation}
      className={cn(
        'bg-input relative !m-0 self-stretch data-[orientation=vertical]:h-auto',
        className,
      )}
      {...props}
    />
  )
}

export {
  ButtonGroup,
  ButtonGroupSeparator,
  ButtonGroupText,
```

```
  buttonGroupVariants,
}
```

Source code 6.234: deskflow-frontend/components/ui/button.tsx

```
import * as React from 'react'
import { Slot } from '@radix-ui/react-slot'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium
  transition-all disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-events-none
  [&_svg:not([class*='size-'])]:size-4 shrink-0 [&_svg]:shrink-0 outline-none focus-visible:border-ring
  focus-visible:ring-ring/50 focus-visible:ring-[3px] aria-invalid:ring-destructive/20
  dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive",
  {
    variants: {
      variant: {
        default: 'bg-primary text-primary-foreground hover:bg-primary/90',
        destructive:
          'bg-destructive text-white hover:bg-destructive/90 focus-visible:ring-destructive/20
          dark:focus-visible:ring-destructive/40 dark:bg-destructive/60',
        outline:
          'border bg-background shadow-xs hover:bg-accent hover:text-accent-foreground dark:bg-input/30
          dark:border-input dark:hover:bg-input/50',
        secondary:
          'bg-secondary text-secondary-foreground hover:bg-secondary/80',
        ghost:
          'hover:bg-accent hover:text-accent-foreground dark:hover:bg-accent/50',
        link: 'text-primary underline-offset-4 hover:underline',
      },
      size: {
        default: 'h-9 px-4 py-2 has-[>svg]:px-3',
        sm: 'h-8 rounded-md gap-1.5 px-3 has-[>svg]:px-2.5',
        lg: 'h-10 rounded-md px-6 has-[>svg]:px-4',
        icon: 'size-9',
        'icon-sm': 'size-8',
        'icon-lg': 'size-10',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  },
)

function Button({
  className,
  variant,
  size,
  asChild = false,
  ...props
}: React.ComponentProps<'button'> &
  VariantProps<typeof buttonVariants> & {
    asChild?: boolean
  }) {
  const Comp = asChild ? Slot : 'button'

  return (
    <Comp
      data-slot="button"
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    />
```

```
  )
}

export { Button, buttonVariants }
```

Source code 6.235: deskflow-frontend/components/ui/calendar.tsx

```
'use client'

import * as React from 'react'
import {
  ChevronDownIcon,
  ChevronLeftIcon,
  ChevronRightIcon,
} from 'lucide-react'
import { DayButton, DayPicker, getDefaultClassNames } from 'react-day-picker'

import { cn } from '@/lib/utils'
import { Button, buttonVariants } from '@/components/ui/button'

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  captionLayout = 'label',
  buttonVariant = 'ghost',
  formatters,
  components,
  ...props
}: React.ComponentProps<typeof DayPicker> & {
  buttonVariant?: React.ComponentProps<typeof Button>['variant']
}) {
  const defaultClassNames = getDefaultClassNames()

  return (
    <DayPicker
      showOutsideDays={showOutsideDays}
      className={cn(
        'bg-background group/calendar p-3 [-cell-size:-spacing(8)]
        [[data-slot=card-content]_&]:bg-transparent [[data-slot=popover-content]_&]:bg-transparent',
        String.raw`rtl:**:[.rdp-button\_next>svg]:rotate-180`,
        String.raw`rtl:**:[.rdp-button\_previous>svg]:rotate-180`,
        className,
      )}
      captionLayout={captionLayout}
      formatters={{
        formatMonthDropdown: (date) =>
          date.toLocaleString('default', { month: 'short' }),
        ...formatters,
      }}
      classNames={{
        root: cn('w-fit', defaultClassNames.root),
        months: cn(
          'flex gap-4 flex-col md:flex-row relative',
          defaultClassNames.months,
        ),
        month: cn('flex flex-col w-full gap-4', defaultClassNames.month),
        nav: cn(
          'flex items-center gap-1 w-full absolute top-0 inset-x-0 justify-between',
          defaultClassNames.nav,
        ),
        button_previous: cn(
          buttonVariants({ variant: buttonVariant }),
          'size-(--cell-size) aria-disabled:opacity-50 p-0 select-none',
          defaultClassNames.button_previous,
        ),
        button_next: cn(
          buttonVariants({ variant: buttonVariant }),
```

```
          'size-(--cell-size) aria-disabled:opacity-50 p-0 select-none',
          defaultClassNames.button_next,
        ),
        month_caption: cn(
          'flex items-center justify-center h-(--cell-size) w-full px-(--cell-size)',
          defaultClassNames.month_caption,
        ),
        dropdowns: cn(
          'w-full flex items-center text-sm font-medium justify-center h-(--cell-size) gap-1.5',
          defaultClassNames.dropdowns,
        ),
        dropdown_root: cn(
          'relative has-focus:border-ring border border-input shadow-xs has-focus:ring-ring/50 has-focus:ring-[3px] rounded-md',
          defaultClassNames.dropdown_root,
        ),
        dropdown: cn(
          'absolute bg-popover inset-0 opacity-0',
          defaultClassNames.dropdown,
        ),
        caption_label: cn(
          'select-none font-medium',
          captionLayout === 'label'
            ? 'text-sm'
            : 'rounded-md pl-2 pr-1 flex items-center gap-1 text-sm h-8 [&>svg]:text-muted-foreground [&>svg]:size-3.5',
          defaultClassNames.caption_label,
        ),
        table: 'w-full border-collapse',
        weekdays: cn('flex', defaultClassNames.weekdays),
        weekday: cn(
          'text-muted-foreground rounded-md flex-1 font-normal text-[0.8rem] select-none',
          defaultClassNames.weekday,
        ),
        week: cn('flex w-full mt-2', defaultClassNames.week),
        week_number_header: cn(
          'select-none w-(--cell-size)',
          defaultClassNames.week_number_header,
        ),
        week_number: cn(
          'text-[0.8rem] select-none text-muted-foreground',
          defaultClassNames.week_number,
        ),
        day: cn(
          'relative w-full h-full p-0 text-center [&:first-child[data-selected=true]_button]:rounded-l-md [&:last-child[data-selected=true]_button]:rounded-r-md group/day aspect-square select-none',
          defaultClassNames.day,
        ),
        range_start: cn(
          'rounded-l-md bg-accent',
          defaultClassNames.range_start,
        ),
        range_middle: cn('rounded-none', defaultClassNames.range_middle),
        range_end: cn('rounded-r-md bg-accent', defaultClassNames.range_end),
        today: cn(
          'bg-accent text-accent-foreground rounded-md data-[selected=true]:rounded-none',
          defaultClassNames.today,
        ),
        outside: cn(
          'text-muted-foreground aria-selected:text-muted-foreground',
          defaultClassNames.outside,
        ),
        disabled: cn(
          'text-muted-foreground opacity-50',
          defaultClassNames.disabled,
        ),
        hidden: cn('invisible', defaultClassNames.hidden),
        ...classNames,
      }}
      components={{
```

```
        Root: ({ className, rootRef, ...props }) => {
          return (
            <div
              data-slot="calendar"
              ref={rootRef}
              className={cn(className)}
              {...props}
            />
          )
        },
        Chevron: ({ className, orientation, ...props }) => {
          if (orientation === 'left') {
            return (
              <ChevronLeftIcon className={cn('size-4', className)} {...props} />
            )
          }

          if (orientation === 'right') {
            return (
              <ChevronRightIcon
                className={cn('size-4', className)}
                {...props}
              />
            )
          }

          return (
            <ChevronDownIcon className={cn('size-4', className)} {...props} />
          )
        },
        DayButton: CalendarDayButton,
        WeekNumber: ({ children, ...props }) => {
          return (
            <td {...props}>
              <div className="flex size-(--cell-size) items-center justify-center text-center">
                {children}
              </div>
            </td>
          )
        },
        ...components,
      }}
      {...props}
    />
  )
}

function CalendarDayButton({
  className,
  day,
  modifiers,
  ...props
}: React.ComponentProps<typeof DayButton>) {
  const defaultClassNames = getDefaultClassNames()

  const ref = React.useRef<HTMLButtonElement>(null)
  React.useEffect(() => {
    if (modifiers.focused) ref.current?.focus()
  }, [modifiers.focused])

  return (
    <Button
      ref={ref}
      variant="ghost"
      size="icon"
      data-day={day.date.toLocaleDateString()}
      data-selected-single={
        modifiers.selected &&
        !modifiers.range_start &&
        !modifiers.range_end &&
```

```
        !modifiers.range_middle
      }
      data-range-start={modifiers.range_start}
      data-range-end={modifiers.range_end}
      data-range-middle={modifiers.range_middle}
      className={cn(
        'data-[selected-single=true]:bg-primary data-[selected-single=true]:text-primary-foreground
        data-[range-middle=true]:bg-accent data-[range-middle=true]:text-accent-foreground
        data-[range-start=true]:bg-primary data-[range-start=true]:text-primary-foreground
        data-[range-end=true]:bg-primary data-[range-end=true]:text-primary-foreground
        group-data-[focused=true]/day:border-ring group-data-[focused=true]/day:ring-ring/50
        dark:hover:text-accent-foreground flex aspect-square size-auto w-full min-w-(--cell-size) flex-col
        gap-1 leading-none font-normal group-data-[focused=true]/day:relative
        group-data-[focused=true]/day:z-10 group-data-[focused=true]/day:ring-[3px]
        data-[range-end=true]:rounded-md data-[range-end=true]:rounded-r-md
        data-[range-middle=true]:rounded-none data-[range-start=true]:rounded-md
        data-[range-start=true]:rounded-l-md [&>span]:text-xs [&>span]:opacity-70',
        defaultClassNames.day,
        className,
      )}
      {...props}
    />
  )
}

export { Calendar, CalendarDayButton }
```

Source code 6.236: deskflow-frontend/components/ui/card.tsx

```
import * as React from 'react'

import { cn } from '@/lib/utils'

function Card({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card"
      className={cn(
        'bg-card text-card-foreground flex flex-col gap-6 rounded-xl border py-6 shadow-sm',
        className,
      )}
      {...props}
    />
  )
}

function CardHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card-header"
      className={cn(
        '@container/card-header grid auto-rows-min grid-rows-[auto_auto] items-start gap-2 px-6
        has-data-[slot=card-action]:grid-cols-[1fr_auto] [.border-b]:pb-6',
        className,
      )}
      {...props}
    />
  )
}

function CardTitle({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card-title"
      className={cn('leading-none font-semibold', className)}
      {...props}
    />
```

```
  )
}

function CardDescription({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card-description"
      className={cn('text-muted-foreground text-sm', className)}
      {...props}
    />
  )
}

function CardAction({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card-action"
      className={cn(
        'col-start-2 row-span-2 row-start-1 self-start justify-self-end',
        className,
      )}
      {...props}
    />
  )
}

function CardContent({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card-content"
      className={cn('px-6', className)}
      {...props}
    />
  )
}

function CardFooter({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="card-footer"
      className={cn('flex items-center px-6 [.border-t]:pt-6', className)}
      {...props}
    />
  )
}

export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardAction,
  CardDescription,
  CardContent,
}
```

Source code 6.237: deskflow-frontend/components/ui/carousel.tsx

```
'use client'

import * as React from 'react'
import useEmblaCarousel, {
  type UseEmblaCarouselType,
} from 'embla-carousel-react'
import { ArrowLeft, ArrowRight } from 'lucide-react'

import { cn } from '@/lib/utils'
import { Button } from '@/components/ui/button'
```

```
11
12 type CarouselApi = UseEmblaCarouselType[1]
13 type UseCarouselParameters = Parameters<typeof useEmblaCarousel>
14 type CarouselOptions = UseCarouselParameters[0]
15 type CarouselPlugin = UseCarouselParameters[1]
16
17 type CarouselProps = {
18   opts?: CarouselOptions
19   plugins?: CarouselPlugin
20   orientation?: 'horizontal' | 'vertical'
21   setApi?: (api: CarouselApi) => void
22 }
23
24 type CarouselContextProps = {
25   carouselRef: ReturnType<typeof useEmblaCarousel>[0]
26   api: ReturnType<typeof useEmblaCarousel>[1]
27   scrollPrev: () => void
28   scrollNext: () => void
29   canScrollPrev: boolean
30   canScrollNext: boolean
31 } & CarouselProps
32
33 const CarouselContext = React.createContext<CarouselContextProps | null>(null)
34
35 function useCarousel() {
36   const context = React.useContext(CarouselContext)
37
38   if (!context) {
39     throw new Error('useCarousel must be used within a <Carousel />')
40   }
41
42   return context
43 }
44
45 function Carousel({
46   orientation = 'horizontal',
47   opts,
48   setApi,
49   plugins,
50   className,
51   children,
52   ...props
53 }: React.ComponentProps<'div'> & CarouselProps) {
54   const [carouselRef, api] = useEmblaCarousel(
55     {
56       ...opts,
57       axis: orientation === 'horizontal' ? 'x' : 'y',
58     },
59     plugins,
60   )
61   const [canScrollPrev, setCanScrollPrev] = React.useState(false)
62   const [canScrollNext, setCanScrollNext] = React.useState(false)
63
64   const onSelect = React.useCallback((api: CarouselApi) => {
65     if (!api) return
66     setCanScrollPrev(api.canScrollPrev())
67     setCanScrollNext(api.canScrollNext())
68   }, [])
69
70   const scrollPrev = React.useCallback(() => {
71     api?.scrollPrev()
72   }, [api])
73
74   const scrollNext = React.useCallback(() => {
75     api?.scrollNext()
76   }, [api])
77
78   const handleKeyDown = React.useCallback(
79     (event: React.KeyboardEvent<HTMLDivElement>) => {
80       if (event.key === 'ArrowLeft') {
```

```
        event.preventDefault()
        scrollPrev()
      } else if (event.key === 'ArrowRight') {
        event.preventDefault()
        scrollNext()
      }
    },
    [scrollPrev, scrollNext],
  )

  React.useEffect(() => {
    if (!api || !setApi) return
    setApi(api)
  }, [api, setApi])

  React.useEffect(() => {
    if (!api) return
    onSelect(api)
    api.on('reInit', onSelect)
    api.on('select', onSelect)

    return () => {
      api?.off('select', onSelect)
    }
  }, [api, onSelect])

  return (
    <CarouselContext.Provider
      value={{
        carouselRef,
        api: api,
        opts,
        orientation:
          orientation || (opts?.axis === 'y' ? 'vertical' : 'horizontal'),
        scrollPrev,
        scrollNext,
        canScrollPrev,
        canScrollNext,
      }}
    >
      <div
        onKeyDownCapture={handleKeyDown}
        className={cn('relative', className)}
        role="region"
        aria-roledescription="carousel"
        data-slot="carousel"
        {...props}
      >
        {children}
      </div>
    </CarouselContext.Provider>
  )
}

function CarouselContent({ className, ...props }: React.ComponentProps<'div'>) {
  const { carouselRef, orientation } = useCarousel()

  return (
    <div
      ref={carouselRef}
      className="overflow-hidden"
      data-slot="carousel-content"
    >
      <div
        className={cn(
          'flex',
          orientation === 'horizontal' ? '-ml-4' : '-mt-4 flex-col',
          className,
        )}
        {...props}
```

```
      />
    </div>
  )
}

function CarouselItem({ className, ...props }: React.ComponentProps<'div'>) {
  const { orientation } = useCarousel()

  return (
    <div
      role="group"
      aria-roledescription="slide"
      data-slot="carousel-item"
      className={cn(
        'min-w-0 shrink-0 grow-0 basis-full',
        orientation === 'horizontal' ? 'pl-4' : 'pt-4',
        className,
      )}
      {...props}
    />
  )
}

function CarouselPrevious({
  className,
  variant = 'outline',
  size = 'icon',
  ...props
}: React.ComponentProps<typeof Button>) {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel()

  return (
    <Button
      data-slot="carousel-previous"
      variant={variant}
      size={size}
      className={cn(
        'absolute size-8 rounded-full',
        orientation === 'horizontal'
          ? 'top-1/2 -left-12 -translate-y-1/2'
          : '-top-12 left-1/2 -translate-x-1/2 rotate-90',
        className,
      )}
      disabled={!canScrollPrev}
      onClick={scrollPrev}
      {...props}
    >
      <ArrowLeft />
      <span className="sr-only">Previous slide</span>
    </Button>
  )
}

function CarouselNext({
  className,
  variant = 'outline',
  size = 'icon',
  ...props
}: React.ComponentProps<typeof Button>) {
  const { orientation, scrollNext, canScrollNext } = useCarousel()

  return (
    <Button
      data-slot="carousel-next"
      variant={variant}
      size={size}
      className={cn(
        'absolute size-8 rounded-full',
        orientation === 'horizontal'
          ? 'top-1/2 -right-12 -translate-y-1/2'
```

```
          : '-bottom-12 left-1/2 -translate-x-1/2 rotate-90',
        className,
      )}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}
    >
      <ArrowRight />
      <span className="sr-only">Next slide</span>
    </Button>
  )
}

export {
  type CarouselApi,
  Carousel,
  CarouselContent,
  CarouselItem,
  CarouselPrevious,
  CarouselNext,
}
```

Source code 6.238: deskflow-frontend/components/ui/chart.tsx

```
'use client'

import * as React from 'react'
import * as RechartsPrimitive from 'recharts'

import { cn } from '@/lib/utils'

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: '', dark: '.dark' } as const

export type ChartConfig = {
  [k in string]: {
    label?: React.ReactNode
    icon?: React.ComponentType
  } & (
    | { color?: string; theme?: never }
    | { color?: never; theme: Record<keyof typeof THEMES, string> }
  )
}

type ChartContextProps = {
  config: ChartConfig
}

const ChartContext = React.createContext<ChartContextProps | null>(null)

function useChart() {
  const context = React.useContext(ChartContext)

  if (!context) {
    throw new Error('useChart must be used within a <ChartContainer />')
  }

  return context
}

function ChartContainer({
  id,
  className,
  children,
  config,
  ...props
}: React.ComponentProps<'div'> & {
  config: ChartConfig
```

```
  children: React.ComponentProps<
    typeof RechartsPrimitive.ResponsiveContainer
  >['children']
}) {
  const uniqueId = React.useId()
  const chartId = `chart-${id || uniqueId.replace(/:/g, '')}`

  return (
    <ChartContext.Provider value={{ config }}>
      <div
        data-slot="chart"
        data-chart={chartId}
        className={cn(
          "[&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground
          [&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50
          [&_.recharts-curve.recharts-tooltip-cursor]:stroke-border
          [&_.recharts-polar-grid_[stroke='#ccc']]:stroke-border
          [&_.recharts-radial-bar-background-sector]:fill-muted
          [&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted
          [&_.recharts-reference-line_[stroke='#ccc']]:stroke-border flex aspect-video justify-center text-xs
          [&_.recharts-dot[stroke='#fff']]:stroke-transparent [&_.recharts-layer]:outline-hidden
          [&_.recharts-sector]:outline-hidden [&_.recharts-sector[stroke='#fff']]:stroke-transparent
          [&_.recharts-surface]:outline-hidden",
          className,
        )}
        {...props}
      >
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer>
          {children}
        </RechartsPrimitive.ResponsiveContainer>
      </div>
    </ChartContext.Provider>
  )
}

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
  const colorConfig = Object.entries(config).filter(
    ([, config]) => config.theme || config.color,
  )

  if (!colorConfig.length) {
    return null
  }

  return (
    <style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(
            ([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
  .map(([key, itemConfig]) => {
    const color =
      itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ||
      itemConfig.color
    return color ? `  --color-${key}: ${color};` : null
  })
  .join('\n')}
}
`,
          )
          .join('\n'),
      }}
    />
  )
}

const ChartTooltip = RechartsPrimitive.Tooltip
```

```

function ChartTooltipContent({
  active,
  payload,
  className,
  indicator = 'dot',
  hideLabel = false,
  hideIndicator = false,
  label,
  labelFormatter,
  labelClassName,
  formatter,
  color,
  nameKey,
  labelKey,
}: React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
  React.ComponentProps<'div'> & {
    hideLabel?: boolean
    hideIndicator?: boolean
    indicator?: 'line' | 'dot' | 'dashed'
    nameKey?: string
    labelKey?: string
  }) {
  const { config } = useChart()

  const tooltipLabel = React.useMemo(() => {
    if (hideLabel || !payload?.length) {
      return null
    }

    const [item] = payload
    const key = `${labelKey || item?.dataKey || item?.name || 'value'}`
    const itemConfig = getPayloadConfigFromPayload(config, item, key)
    const value =
      !labelKey && typeof label === 'string'
        ? config[label as keyof typeof config]?.label || label
        : itemConfig?.label

    if (labelFormatter) {
      return (
        <div className={cn('font-medium', labelClassName)}>
          {labelFormatter(value, payload)}
        </div>
      )
    }

    if (!value) {
      return null
    }

    return <div className={cn('font-medium', labelClassName)}>{value}</div>
  }, [
    label,
    labelFormatter,
    payload,
    hideLabel,
    labelClassName,
    config,
    labelKey,
  ])

  if (!active || !payload?.length) {
    return null
  }

  const nestLabel = payload.length === 1 && indicator !== 'dot'

  return (
    <div
      className={cn(
```

```
        'border-border/50 bg-background grid min-w-[8rem] items-start gap-1.5 rounded-lg border px-2.5 py-1.5 text-xs shadow-xl',
        className,
      )}
    >
      {!nestLabel ? tooltipLabel : null}
      <div className="grid gap-1.5">
        {payload.map((item, index) => {
          const key = `${nameKey || item.name || item.dataKey || 'value'}`
          const itemConfig = getPayloadConfigFromPayload(config, item, key)
          const indicatorColor = color || item.payload.fill || item.color

          return (
            <div
              key={item.dataKey}
              className={cn(
                '[&>svg]:text-muted-foreground flex w-full flex-wrap items-stretch gap-2 [&>svg]:h-2.5 [&>svg]:w-2.5',
                indicator === 'dot' && 'items-center',
              )}
            >
              {formatter && item?.value !== undefined && item.name ? (
                formatter(item.value, item.name, item, index, item.payload)
              ) : (
                <>
                  {itemConfig?.icon ? (
                    <itemConfig.icon />
                  ) : (
                    !hideIndicator && (
                      <div
                        className={cn(
                          'shrink-0 rounded-[2px] border-(-color-border) bg-(-color-bg)',
                          {
                            'h-2.5 w-2.5': indicator === 'dot',
                            'w-1': indicator === 'line',
                            'w-0 border-[1.5px] border-dashed bg-transparent':
                              indicator === 'dashed',
                            'my-0.5': nestLabel && indicator === 'dashed',
                          },
                        )}
                        style={
                          {
                            '-color-bg': indicatorColor,
                            '-color-border': indicatorColor,
                          } as React.CSSProperties
                        }
                      />
                    )
                  )}
                  <div
                    className={cn(
                      'flex flex-1 justify-between leading-none',
                      nestLabel ? 'items-end' : 'items-center',
                    )}
                  >
                    <div className="grid gap-1.5">
                      {nestLabel ? tooltipLabel : null}
                      <span className="text-muted-foreground">
                        {itemConfig?.label || item.name}
                      </span>
                    </div>
                    {item.value && (
                      <span className="text-foreground font-mono font-medium tabular-nums">
                        {item.value.toLocaleString()}
                      </span>
                    )}
                  </div>
                </>
              )}
            </div>
```

```
          )
        })}
      </div>
    </div>
  )
}

const ChartLegend = RechartsPrimitive.Legend

function ChartLegendContent({
  className,
  hideIcon = false,
  payload,
  verticalAlign = 'bottom',
  nameKey,
}: React.ComponentProps<'div'> &
  Pick<RechartsPrimitive.LegendProps, 'payload' | 'verticalAlign'> & {
    hideIcon?: boolean
    nameKey?: string
  }) {
  const { config } = useChart()

  if (!payload?.length) {
    return null
  }

  return (
    <div
      className={cn(
        'flex items-center justify-center gap-4',
        verticalAlign === 'top' ? 'pb-3' : 'pt-3',
        className,
      )}
    >
      {payload.map((item) => {
        const key = `${nameKey || item.dataKey || 'value'}`
        const itemConfig = getPayloadConfigFromPayload(config, item, key)

        return (
          <div
            key={item.value}
            className={
              '[&>svg]:text-muted-foreground flex items-center gap-1.5 [&>svg]:h-3 [&>svg]:w-3'
            }
          >
            {itemConfig?.icon && !hideIcon ? (
              <itemConfig.icon />
            ) : (
              <div
                className="h-2 w-2 shrink-0 rounded-[2px]"
                style={{
                  backgroundColor: item.color,
                }}
              />
            )}
            {itemConfig?.label}
          </div>
        )
      })}
    </div>
  )
}

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
  config: ChartConfig,
  payload: unknown,
  key: string,
) {
  if (typeof payload !== 'object' || payload === null) {
```

```
    return undefined
  }

  const payloadPayload =
    'payload' in payload &&
    typeof payload.payload === 'object' &&
    payload.payload !== null
      ? payload.payload
      : undefined

  let configLabelKey: string = key

  if (
    key in payload &&
    typeof payload[key as keyof typeof payload] === 'string'
  ) {
    configLabelKey = payload[key as keyof typeof payload] as string
  } else if (
    payloadPayload &&
    key in payloadPayload &&
    typeof payloadPayload[key as keyof typeof payloadPayload] === 'string'
  ) {
    configLabelKey = payloadPayload[
      key as keyof typeof payloadPayload
    ] as string
  }

  return configLabelKey in config
    ? config[configLabelKey]
    : config[key as keyof typeof config]
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
}
```

Source code 6.239: deskflow-frontend/components/ui/checkbox.tsx

```
'use client'

import * as React from 'react'
import * as CheckboxPrimitive from '@radix-ui/react-checkbox'
import { CheckIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function Checkbox({
  className,
  ...props
}: React.ComponentProps<typeof CheckboxPrimitive.Root>) {
  return (
    <CheckboxPrimitive.Root
      data-slot="checkbox"
      className={cn(
        'peer border-input dark:bg-input/30 data-[state=checked]:bg-primary
        data-[state=checked]:text-primary-foreground dark:data-[state=checked]:bg-primary
        data-[state=checked]:border-primary focus-visible:border-ring focus-visible:ring-ring/50
        aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive size-4 shrink-0 rounded-[4px] border shadow-xs transition-shadow
        outline-none focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50',
        className,
      )}
```

```
      {...props}
    >
      <CheckboxPrimitive.Indicator
        data-slot="checkbox-indicator"
        className="flex items-center justify-center text-current transition-none"
      >
        <CheckIcon className="size-3.5" />
      </CheckboxPrimitive.Indicator>
    </CheckboxPrimitive.Root>
  )
}

export { Checkbox }
```

Source code 6.240: deskflow-frontend/components/ui/collapsible.tsx

```
'use client'

import * as CollapsiblePrimitive from '@radix-ui/react-collapsible'

function Collapsible({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.Root>) {
  return <CollapsiblePrimitive.Root data-slot="collapsible" {...props} />
}

function CollapsibleTrigger({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleTrigger>) {
  return (
    <CollapsiblePrimitive.CollapsibleTrigger
      data-slot="collapsible-trigger"
      {...props}
    />
  )
}

function CollapsibleContent({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleContent>) {
  return (
    <CollapsiblePrimitive.CollapsibleContent
      data-slot="collapsible-content"
      {...props}
    />
  )
}

export { Collapsible, CollapsibleTrigger, CollapsibleContent }
```

Source code 6.241: deskflow-frontend/components/ui/command.tsx

```
'use client'

import * as React from 'react'
import { Command as CommandPrimitive } from 'cmdk'
import { SearchIcon } from 'lucide-react'

import { cn } from '@/lib/utils'
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
```

```
} from '@/components/ui/dialog'

function Command({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive>) {
  return (
    <CommandPrimitive
      data-slot="command"
      className={cn(
        'bg-popover text-popover-foreground flex h-full w-full flex-col overflow-hidden rounded-md',
        className,
      )}
      {...props}
    />
  )
}

function CommandDialog({
  title = 'Command Palette',
  description = 'Search for a command to run...',
  children,
  className,
  showCloseButton = true,
  ...props
}: React.ComponentProps<typeof Dialog> & {
  title?: string
  description?: string
  className?: string
  showCloseButton?: boolean
}) {
  return (
    <Dialog {...props}>
      <DialogHeader className="sr-only">
        <DialogTitle>{title}</DialogTitle>
        <DialogDescription>{description}</DialogDescription>
      </DialogHeader>
      <DialogContent
        className={cn('overflow-hidden p-0', className)}
        showCloseButton={showCloseButton}
      >
        <Command className="[&_[cmdk-group-heading]]:text-muted-foreground
        **:data-[slot=command-input-wrapper]:h-12 [&_[cmdk-group-heading]]:px-2
        [&_[cmdk-group-heading]]:font-medium [&_[cmdk-group]]:px-2
        [&_[cmdk-group]:not([hidden])_~[cmdk-group]]:pt-0 [&_[cmdk-input-wrapper]_svg]:h-5
        [&_[cmdk-input-wrapper]_svg]:w-5 [&_[cmdk-input]]:h-12 [&_[cmdk-item]]:px-2 [&_[cmdk-item]]:py-3
        [&_[cmdk-item]_svg]:h-5 [&_[cmdk-item]_svg]:w-5">
          {children}
        </Command>
      </DialogContent>
    </Dialog>
  )
}

function CommandInput({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Input>) {
  return (
    <div
      data-slot="command-input-wrapper"
      className="flex h-9 items-center gap-2 border-b px-3"
    >
      <SearchIcon className="size-4 shrink-0 opacity-50" />
      <CommandPrimitive.Input
        data-slot="command-input"
        className={cn(
          'placeholder:text-muted-foreground flex h-10 w-full rounded-md bg-transparent py-3 text-sm
          outline-hidden disabled:cursor-not-allowed disabled:opacity-50',
          className,
```

```
        )}
        {...props}
      />
    </div>
  )
}

function CommandList({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.List>) {
  return (
    <CommandPrimitive.List
      data-slot="command-list"
      className={cn(
        'max-h-[300px] scroll-py-1 overflow-x-hidden overflow-y-auto',
        className,
      )}
      {...props}
    />
  )
}

function CommandEmpty({
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Empty>) {
  return (
    <CommandPrimitive.Empty
      data-slot="command-empty"
      className="py-6 text-center text-sm"
      {...props}
    />
  )
}

function CommandGroup({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Group>) {
  return (
    <CommandPrimitive.Group
      data-slot="command-group"
      className={cn(
        'text-foreground [&_[cmdk-group-heading]]:text-muted-foreground overflow-hidden p-1
        [&_[cmdk-group-heading]]:px-2 [&_[cmdk-group-heading]]:py-1.5 [&_[cmdk-group-heading]]:text-xs
        [&_[cmdk-group-heading]]:font-medium',
        className,
      )}
      {...props}
    />
  )
}

function CommandSeparator({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Separator>) {
  return (
    <CommandPrimitive.Separator
      data-slot="command-separator"
      className={cn('bg-border -mx-1 h-px', className)}
      {...props}
    />
  )
}

function CommandItem({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Item>) {
```

```
  return (
    <CommandPrimitive.Item
      data-slot="command-item"
      className={cn(
        "data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground
        [&_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default items-center gap-2
        rounded-sm px-2 py-1.5 text-sm outline-hidden select-none data-[disabled=true]:pointer-events-none
        data-[disabled=true]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}

function CommandShortcut({
  className,
  ...props
}: React.ComponentProps<'span'>) {
  return (
    <span
      data-slot="command-shortcut"
      className={cn(
        'text-muted-foreground ml-auto text-xs tracking-widest',
        className,
      )}
      {...props}
    />
  )
}

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}
```

Source code 6.242: deskflow-frontend/components/ui/context-menu.tsx

```
'use client'

import * as React from 'react'
import * as ContextMenuPrimitive from '@radix-ui/react-context-menu'
import { CheckIcon, ChevronRightIcon, CircleIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function ContextMenu({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Root>) {
  return <ContextMenuPrimitive.Root data-slot="context-menu" {...props} />
}

function ContextMenuTrigger({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Trigger>) {
  return (
    <ContextMenuPrimitive.Trigger data-slot="context-menu-trigger" {...props} />
  )
}
```

```
22
23  function ContextMenuGroup({
24    ...props
25  }: React.ComponentProps<typeof ContextMenuPrimitive.Group>) {
26    return (
27      <ContextMenuPrimitive.Group data-slot="context-menu-group" {...props} />
28    )
29  }
30
31  function ContextMenuPortal({
32    ...props
33  }: React.ComponentProps<typeof ContextMenuPrimitive.Portal>) {
34    return (
35      <ContextMenuPrimitive.Portal data-slot="context-menu-portal" {...props} />
36    )
37  }
38
39  function ContextMenuSub({
40    ...props
41  }: React.ComponentProps<typeof ContextMenuPrimitive.Sub>) {
42    return <ContextMenuPrimitive.Sub data-slot="context-menu-sub" {...props} />
43  }
44
45  function ContextMenuRadioGroup({
46    ...props
47  }: React.ComponentProps<typeof ContextMenuPrimitive.RadioGroup>) {
48    return (
49      <ContextMenuPrimitive.RadioGroup
50        data-slot="context-menu-radio-group"
51        {...props}
52      />
53    )
54  }
55
56  function ContextMenuSubTrigger({
57    className,
58    inset,
59    children,
60    ...props
61  }: React.ComponentProps<typeof ContextMenuPrimitive.SubTrigger> & {
62    inset?: boolean
63  }) {
64    return (
65      <ContextMenuPrimitive.SubTrigger
66        data-slot="context-menu-sub-trigger"
67        data-inset={inset}
68        className={cn(
69          "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
            data-[state=open]:text-accent-foreground [&_svg:not([class*='text-'])]:text-muted-foreground flex
            cursor-default items-center rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
            data-[inset]:pl-8 [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
70          className,
71        )}
72        {...props}
73      >
74        {children}
75        <ChevronRightIcon className="ml-auto" />
76      </ContextMenuPrimitive.SubTrigger>
77    )
78  }
79
80  function ContextMenuSubContent({
81    className,
82    ...props
83  }: React.ComponentProps<typeof ContextMenuPrimitive.SubContent>) {
84    return (
85      <ContextMenuPrimitive.SubContent
86        data-slot="context-menu-sub-content"
87        className={cn(
```

```
        'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
        data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
        data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
        data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
        data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
        origin-(-radix-context-menu-content-transform-origin) overflow-hidden rounded-md border p-1
        shadow-lg',
        className,
      )}
      {...props}
    />
  )
}

function ContextMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Content>) {
  return (
    <ContextMenuPrimitive.Portal>
      <ContextMenuPrimitive.Content
        data-slot="context-menu-content"
        className={cn(
          'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50 max-h-(-radix-context-menu-content-available-height)
          min-w-[8rem] origin-(-radix-context-menu-content-transform-origin) overflow-x-hidden
          overflow-y-auto rounded-md border p-1 shadow-md',
          className,
        )}
        {...props}
      />
    </ContextMenuPrimitive.Portal>
  )
}

function ContextMenuItem({
  className,
  inset,
  variant = 'default',
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Item> & {
  inset?: boolean
  variant?: 'default' | 'destructive'
}) {
  return (
    <ContextMenuPrimitive.Item
      data-slot="context-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive
        data-[variant=destructive]:focus:bg-destructive/10
        dark:data-[variant=destructive]:focus:bg-destructive/20
        data-[variant=destructive]:focus:text-destructive
        data-[variant=destructive]:*:[svg]:!text-destructive
        [&_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default items-center gap-2
        rounded-sm px-2 py-1.5 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 data-[inset]:pl-8 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}

```

```
function ContextMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.CheckboxItem>) {
  return (
    <ContextMenuPrimitive.CheckboxItem
      data-slot="context-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2
        rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <ContextMenuPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.CheckboxItem>
  )
}

function ContextMenuRadioItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.RadioItem>) {
  return (
    <ContextMenuPrimitive.RadioItem
      data-slot="context-menu-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2
        rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <ContextMenuPrimitive.ItemIndicator>
          <CircleIcon className="size-2 fill-current" />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.RadioItem>
  )
}

function ContextMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.Label
      data-slot="context-menu-label"
      data-inset={inset}
      className={cn(
        'text-foreground px-2 py-1.5 text-sm font-medium data-[inset]:pl-8',
        className,
```

```
      )}
      {...props}
    />
  )
}

function ContextMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Separator>) {
  return (
    <ContextMenuPrimitive.Separator
      data-slot="context-menu-separator"
      className={cn('bg-border -mx-1 my-1 h-px', className)}
      {...props}
    />
  )
}

function ContextMenuShortcut({
  className,
  ...props
}: React.ComponentProps<'span'>) {
  return (
    <span
      data-slot="context-menu-shortcut"
      className={cn(
        'text-muted-foreground ml-auto text-xs tracking-widest',
        className,
      )}
      {...props}
    />
  )
}

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
}
```

Source code 6.243: deskflow-frontend/components/ui/dialog.tsx

```
'use client'

import * as React from 'react'
import * as DialogPrimitive from '@radix-ui/react-dialog'
import { XIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function Dialog({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Root>) {
  return <DialogPrimitive.Root data-slot="dialog" {...props} />
}
```

```
14
15  function DialogTrigger({
16    ...props
17  }: React.ComponentProps<typeof DialogPrimitive.Trigger>) {
18    return <DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} />
19  }
20
21  function DialogPortal({
22    ...props
23  }: React.ComponentProps<typeof DialogPrimitive.Portal>) {
24    return <DialogPrimitive.Portal data-slot="dialog-portal" {...props} />
25  }
26
27  function DialogClose({
28    ...props
29  }: React.ComponentProps<typeof DialogPrimitive.Close>) {
30    return <DialogPrimitive.Close data-slot="dialog-close" {...props} />
31  }
32
33  function DialogOverlay({
34    className,
35    ...props
36  }: React.ComponentProps<typeof DialogPrimitive.Overlay>) {
37    return (
38      <DialogPrimitive.Overlay
39        data-slot="dialog-overlay"
40        className={cn(
41          'data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
            data-[state=open]:fade-in-0 fixed inset-0 z-50 bg-black/50',
42          className,
43        )}
44        {...props}
45      />
46    )
47  }
48
49  function DialogContent({
50    className,
51    children,
52    showCloseButton = true,
53    ...props
54  }: React.ComponentProps<typeof DialogPrimitive.Content> & {
55    showCloseButton?: boolean
56  }) {
57    return (
58      <DialogPortal data-slot="dialog-portal">
59        <DialogOverlay />
60        <DialogPrimitive.Content
61          data-slot="dialog-content"
62          className={cn(
63            'bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
              data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
              data-[state=open]:zoom-in-95 fixed top-[50%] left-[50%] z-50 grid w-full max-w-[calc(100%-2rem)]
              translate-x-[-50%] translate-y-[-50%] gap-4 rounded-lg border p-6 shadow-lg duration-200
              sm:max-w-lg',
64            className,
65          )}
66          {...props}
67        >
68          {children}
69          {showCloseButton && (
70            <DialogPrimitive.Close
71              data-slot="dialog-close"
72              className="ring-offset-background focus:ring-ring data-[state=open]:bg-accent
                data-[state=open]:text-muted-foreground absolute top-4 right-4 rounded-xs opacity-70
                transition-opacity hover:opacity-100 focus:ring-2 focus:ring-offset-2 focus:outline-hidden
                disabled:pointer-events-none [&_svg]:pointer-events-none [&_svg]:shrink-0
                [&_svg:not([class*='size-'])]:size-4"
73            >
74              <XIcon />
```

```
            <span className="sr-only">Close</span>
          </DialogPrimitive.Close>
        )}
      </DialogPrimitive.Content>
    </DialogPortal>
  )
}

function DialogHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="dialog-header"
      className={cn('flex flex-col gap-2 text-center sm:text-left', className)}
      {...props}
    />
  )
}

function DialogFooter({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="dialog-footer"
      className={cn(
        'flex flex-col-reverse gap-2 sm:flex-row sm:justify-end',
        className,
      )}
      {...props}
    />
  )
}

function DialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Title>) {
  return (
    <DialogPrimitive.Title
      data-slot="dialog-title"
      className={cn('text-lg leading-none font-semibold', className)}
      {...props}
    />
  )
}

function DialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Description>) {
  return (
    <DialogPrimitive.Description
      data-slot="dialog-description"
      className={cn('text-muted-foreground text-sm', className)}
      {...props}
    />
  )
}

export {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogOverlay,
  DialogPortal,
  DialogTitle,
  DialogTrigger,
}
```

Source code 6.244: deskflow-frontend/components/ui/drawer.tsx

```
'use client'

import * as React from 'react'
import { Drawer as DrawerPrimitive } from 'vaul'

import { cn } from '@/lib/utils'

function Drawer({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Root>) {
  return <DrawerPrimitive.Root data-slot="drawer" {...props} />
}

function DrawerTrigger({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Trigger>) {
  return <DrawerPrimitive.Trigger data-slot="drawer-trigger" {...props} />
}

function DrawerPortal({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Portal>) {
  return <DrawerPrimitive.Portal data-slot="drawer-portal" {...props} />
}

function DrawerClose({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Close>) {
  return <DrawerPrimitive.Close data-slot="drawer-close" {...props} />
}

function DrawerOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Overlay>) {
  return (
    <DrawerPrimitive.Overlay
      data-slot="drawer-overlay"
      className={cn(
        'data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
        data-[state=open]:fade-in-0 fixed inset-0 z-50 bg-black/50',
        className,
      )}
      {...props}
    />
  )
}

function DrawerContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Content>) {
  return (
    <DrawerPortal data-slot="drawer-portal">
      <DrawerOverlay />
      <DrawerPrimitive.Content
        data-slot="drawer-content"
        className={cn(
          'group/drawer-content bg-background fixed z-50 flex h-auto flex-col',
          'data-[vaul-drawer-direction=top]:inset-x-0 data-[vaul-drawer-direction=top]:top-0
          data-[vaul-drawer-direction=top]:mb-24 data-[vaul-drawer-direction=top]:max-h-[80vh]
          data-[vaul-drawer-direction=top]:rounded-b-lg data-[vaul-drawer-direction=top]:border-b',
          'data-[vaul-drawer-direction=bottom]:inset-x-0 data-[vaul-drawer-direction=bottom]:bottom-0
          data-[vaul-drawer-direction=bottom]:mt-24 data-[vaul-drawer-direction=bottom]:max-h-[80vh]
          data-[vaul-drawer-direction=bottom]:rounded-t-lg data-[vaul-drawer-direction=bottom]:border-t',
```

```
          'data-[vaul-drawer-direction=right]:inset-y-0 data-[vaul-drawer-direction=right]:right-0
          data-[vaul-drawer-direction=right]:w-3/4 data-[vaul-drawer-direction=right]:border-l
          data-[vaul-drawer-direction=right]:sm:max-w-sm',
          'data-[vaul-drawer-direction=left]:inset-y-0 data-[vaul-drawer-direction=left]:left-0
          data-[vaul-drawer-direction=left]:w-3/4 data-[vaul-drawer-direction=left]:border-r
          data-[vaul-drawer-direction=left]:sm:max-w-sm',
          className,
        )}
        {...props}
      >
        <div className="bg-muted mx-auto mt-4 hidden h-2 w-[100px] shrink-0 rounded-full
        group-data-[vaul-drawer-direction=bottom]/drawer-content:block" />
        {children}
      </DrawerPrimitive.Content>
    </DrawerPortal>
  )
}

function DrawerHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="drawer-header"
      className={cn(
        'flex flex-col gap-0.5 p-4 group-data-[vaul-drawer-direction=bottom]/drawer-content:text-center
        group-data-[vaul-drawer-direction=top]/drawer-content:text-center md:gap-1.5 md:text-left',
        className,
      )}
      {...props}
    />
  )
}

function DrawerFooter({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="drawer-footer"
      className={cn('mt-auto flex flex-col gap-2 p-4', className)}
      {...props}
    />
  )
}

function DrawerTitle({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Title>) {
  return (
    <DrawerPrimitive.Title
      data-slot="drawer-title"
      className={cn('text-foreground font-semibold', className)}
      {...props}
    />
  )
}

function DrawerDescription({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Description>) {
  return (
    <DrawerPrimitive.Description
      data-slot="drawer-description"
      className={cn('text-muted-foreground text-sm', className)}
      {...props}
    />
  )
}

export {
  Drawer,
```

```
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
}
```

Source code 6.245: deskflow-frontend/components/ui/dropdown-menu.tsx

```
'use client'

import * as React from 'react'
import * as DropdownMenuPrimitive from '@radix-ui/react-dropdown-menu'
import { CheckIcon, ChevronRightIcon, CircleIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function DropdownMenu({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Root>) {
  return <DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} />
}

function DropdownMenuPortal({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Portal>) {
  return (
    <DropdownMenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} />
  )
}

function DropdownMenuTrigger({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Trigger>) {
  return (
    <DropdownMenuPrimitive.Trigger
      data-slot="dropdown-menu-trigger"
      {...props}
    />
  )
}

function DropdownMenuContent({
  className,
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Content>) {
  return (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        data-slot="dropdown-menu-content"
        sideOffset={sideOffset}
        className={cn(
          'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50 max-h-(-radix-dropdown-menu-content-available-height)
          min-w-[8rem] origin-(-radix-dropdown-menu-content-transform-origin) overflow-x-hidden
          overflow-y-auto rounded-md border p-1 shadow-md',
          className,
        )}
        {...props}
```

```
      />
    </DropdownMenuPrimitive.Portal>
  )
}

function DropdownMenuGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Group>) {
  return (
    <DropdownMenuPrimitive.Group data-slot="dropdown-menu-group" {...props} />
  )
}

function DropdownMenuItem({
  className,
  inset,
  variant = 'default',
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Item> & {
  inset?: boolean
  variant?: 'default' | 'destructive'
}) {
  return (
    <DropdownMenuPrimitive.Item
      data-slot="dropdown-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive
        data-[variant=destructive]:focus:bg-destructive/10
        dark:data-[variant=destructive]:focus:bg-destructive/20
        data-[variant=destructive]:focus:text-destructive
        data-[variant=destructive]:*:[svg]:!text-destructive
        [&_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default items-center gap-2
        rounded-sm px-2 py-1.5 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 data-[inset]:pl-8 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}

function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.CheckboxItem>) {
  return (
    <DropdownMenuPrimitive.CheckboxItem
      data-slot="dropdown-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2
        rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
```

```
    </DropdownMenuPrimitive.CheckboxItem>
  )
}

function DropdownMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioGroup>) {
  return (
    <DropdownMenuPrimitive.RadioGroup
      data-slot="dropdown-menu-radio-group"
      {...props}
    />
  )
}

function DropdownMenuRadioItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioItem>) {
  return (
    <DropdownMenuPrimitive.RadioItem
      data-slot="dropdown-menu-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2
        rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <CircleIcon className="size-2 fill-current" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.RadioItem>
  )
}

function DropdownMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.Label
      data-slot="dropdown-menu-label"
      data-inset={inset}
      className={cn(
        'px-2 py-1.5 text-sm font-medium data-[inset]:pl-8',
        className,
      )}
      {...props}
    />
  )
}

function DropdownMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Separator>) {
  return (
    <DropdownMenuPrimitive.Separator
      data-slot="dropdown-menu-separator"
      className={cn('bg-border -mx-1 my-1 h-px', className)}
```

```
      {...props}
    />
  )
}

function DropdownMenuShortcut({
  className,
  ...props
}: React.ComponentProps<'span'>) {
  return (
    <span
      data-slot="dropdown-menu-shortcut"
      className={cn(
        'text-muted-foreground ml-auto text-xs tracking-widest',
        className,
      )}
      {...props}
    />
  )
}

function DropdownMenuSub({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Sub>) {
  return <DropdownMenuPrimitive.Sub data-slot="dropdown-menu-sub" {...props} />
}

function DropdownMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.SubTrigger
      data-slot="dropdown-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground [&_svg:not([class*='text-'])]:text-muted-foreground flex
        cursor-default items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
        data-[inset]:pl-8 [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto size-4" />
    </DropdownMenuPrimitive.SubTrigger>
  )
}

function DropdownMenuSubContent({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubContent>) {
  return (
    <DropdownMenuPrimitive.SubContent
      data-slot="dropdown-menu-sub-content"
      className={cn(
        'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
        data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
        data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
        data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
        data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
        origin-(-radix-dropdown-menu-content-transform-origin) overflow-hidden rounded-md border p-1
        shadow-lg',
        className,
```

```
      )}
      {...props}
    />
  )
}

export {
  DropdownMenu,
  DropdownMenuPortal,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuGroup,
  DropdownMenuLabel,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioGroup,
  DropdownMenuRadioItem,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuSub,
  DropdownMenuSubTrigger,
  DropdownMenuSubContent,
}
```

Source code 6.246: deskflow-frontend/components/ui/empty.tsx

```
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'

function Empty({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="empty"
      className={cn(
        'flex min-w-0 flex-1 flex-col items-center justify-center gap-6 rounded-lg border-dashed p-6 text-center text-balance md:p-12',
        className,
      )}
      {...props}
    />
  )
}

function EmptyHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="empty-header"
      className={cn(
        'flex max-w-sm flex-col items-center gap-2 text-center',
        className,
      )}
      {...props}
    />
  )
}

const emptyMediaVariants = cva(
  'flex shrink-0 items-center justify-center mb-2 [&_svg]:pointer-events-none [&_svg]:shrink-0',
  {
    variants: {
      variant: {
        default: 'bg-transparent',
        icon: "bg-muted text-foreground flex size-10 shrink-0 items-center justify-center rounded-lg [&_svg:not([class*='size-'])]:size-6",
      },
    },
```

```
    defaultVariants: {
      variant: 'default',
    },
  },
)

function EmptyMedia({
  className,
  variant = 'default',
  ...props
}: React.ComponentProps<'div'> & VariantProps<typeof emptyMediaVariants>) {
  return (
    <div
      data-slot="empty-icon"
      data-variant={variant}
      className={cn(emptyMediaVariants({ variant, className }))}
      {...props}
    />
  )
}

function EmptyTitle({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="empty-title"
      className={cn('text-lg font-medium tracking-tight', className)}
      {...props}
    />
  )
}

function EmptyDescription({ className, ...props }: React.ComponentProps<'p'>) {
  return (
    <div
      data-slot="empty-description"
      className={cn(
        'text-muted-foreground [&>a:hover]:text-primary text-sm/relaxed [&>a]:underline
        [&>a]:underline-offset-4',
        className,
      )}
      {...props}
    />
  )
}

function EmptyContent({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="empty-content"
      className={cn(
        'flex w-full max-w-sm min-w-0 flex-col items-center gap-4 text-sm text-balance',
        className,
      )}
      {...props}
    />
  )
}

export {
  Empty,
  EmptyHeader,
  EmptyTitle,
  EmptyDescription,
  EmptyContent,
  EmptyMedia,
}
```

Source code 6.247: deskflow-frontend/components/ui/field.tsx

```
'use client'

import { useMemo } from 'react'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'
import { Label } from '@/components/ui/label'
import { Separator } from '@/components/ui/separator'

function FieldSet({ className, ...props }: React.ComponentProps<'fieldset'>) {
  return (
    <fieldset
      data-slot="field-set"
      className={cn(
        'flex flex-col gap-6',
        'has-[>[data-slot=checkbox-group]]:gap-3 has-[>[data-slot=radio-group]]:gap-3',
        className,
      )}
      {...props}
    />
  )
}

function FieldLegend({
  className,
  variant = 'legend',
  ...props
}: React.ComponentProps<'legend'> & { variant?: 'legend' | 'label' }) {
  return (
    <legend
      data-slot="field-legend"
      data-variant={variant}
      className={cn(
        'mb-3 font-medium',
        'data-[variant=legend]:text-base',
        'data-[variant=label]:text-sm',
        className,
      )}
      {...props}
    />
  )
}

function FieldGroup({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="field-group"
      className={cn(
        'group/field-group @container/field-group flex w-full flex-col gap-7 data-[slot=checkbox-group]:gap-3
        [&>[data-slot=field-group]]:gap-4',
        className,
      )}
      {...props}
    />
  )
}

const fieldVariants = cva(
  'group/field flex w-full gap-3 data-[invalid=true]:text-destructive',
  {
    variants: {
      orientation: {
        vertical: ['flex-col [&>*]:w-full [&>.sr-only]:w-auto'],
        horizontal: [
          'flex-row items-center',
          '[&>[data-slot=field-label]]:flex-auto',
          'has-[>[data-slot=field-content]]:items-start
          has-[>[data-slot=field-content]]:[&>[role=checkbox],[role=radio]]:mt-px',
        ],
```

```
        responsive: [
          'flex-col [&>*]:w-full [&>.sr-only]:w-auto @md/field-group:flex-row @md/field-group:items-center @md/field-group:[&>*]:w-auto',
          '@md/field-group:[&>[data-slot=field-label]]:flex-auto',
          '@md/field-group:has-[>[data-slot=field-content]]:items-start @md/field-group:has-[>[data-slot=field-content]]:[&>[role=checkbox],[role=radio]]:mt-px',
        ],
      },
    },
    defaultVariants: {
      orientation: 'vertical',
    },
  },
)

function Field({
  className,
  orientation = 'vertical',
  ...props
}: React.ComponentProps<'div'> & VariantProps<typeof fieldVariants>) {
  return (
    <div
      role="group"
      data-slot="field"
      data-orientation={orientation}
      className={cn(fieldVariants({ orientation }), className)}
      {...props}
    />
  )
}

function FieldContent({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="field-content"
      className={cn(
        'group/field-content flex flex-1 flex-col gap-1.5 leading-snug',
        className,
      )}
      {...props}
    />
  )
}

function FieldLabel({
  className,
  ...props
}: React.ComponentProps<typeof Label>) {
  return (
    <Label
      data-slot="field-label"
      className={cn(
        'group/field-label peer/field-label flex w-fit gap-2 leading-snug group-data-[disabled=true]/field:opacity-50',
        'has-[>[data-slot=field]]:w-full has-[>[data-slot=field]]:flex-col has-[>[data-slot=field]]:rounded-md has-[>[data-slot=field]]:border [&>*]:data-[slot=field]:p-4',
        'has-data-[state=checked]:bg-primary/5 has-data-[state=checked]:border-primary dark:has-data-[state=checked]:bg-primary/10',
        className,
      )}
      {...props}
    />
  )
}

function FieldTitle({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="field-label"
      className={cn(
```

```
        'flex w-fit items-center gap-2 text-sm leading-snug font-medium
        group-data-[disabled=true]/field:opacity-50',
        className,
      )}
      {...props}
    />
  )
}

function FieldDescription({ className, ...props }: React.ComponentProps<'p'>) {
  return (
    <p
      data-slot="field-description"
      className={cn(
        'text-muted-foreground text-sm leading-normal font-normal
        group-has-[[data-orientation=horizontal]]/field:text-balance',
        'last:mt-0 nth-last-2:-mt-1 [[data-variant=legend]+&]:-mt-1.5',
        '[&>a:hover]:text-primary [&>a]:underline [&>a]:underline-offset-4',
        className,
      )}
      {...props}
    />
  )
}

function FieldSeparator({
  children,
  className,
  ...props
}: React.ComponentProps<'div'> & {
  children?: React.ReactNode
}) {
  return (
    <div
      data-slot="field-separator"
      data-content={!!children}
      className={cn(
        'relative -my-2 h-5 text-sm group-data-[variant=outline]/field-group:-mb-2',
        className,
      )}
      {...props}
    >
      <Separator className="absolute inset-0 top-1/2" />
      {children && (
        <span
          className="bg-background text-muted-foreground relative mx-auto block w-fit px-2"
          data-slot="field-separator-content"
        >
          {children}
        </span>
      )}
    </div>
  )
}

function FieldError({
  className,
  children,
  errors,
  ...props
}: React.ComponentProps<'div'> & {
  errors?: Array<{ message?: string } | undefined>
}) {
  const content = useMemo(() => {
    if (children) {
      return children
    }

    if (!errors) {
      return null
```

```
    }

    if (errors.length === 1 && errors[0]?.message) {
      return errors[0].message
    }

    return (
      <ul className="ml-4 flex list-disc flex-col gap-1">
        {errors.map(
          (error, index) =>
            error?.message && <li key={index}>{error.message}</li>,
        )}
      </ul>
    )
  }, [children, errors])

  if (!content) {
    return null
  }

  return (
    <div
      role="alert"
      data-slot="field-error"
      className={cn('text-destructive text-sm font-normal', className)}
      {...props}
    >
      {content}
    </div>
  )
}

export {
  Field,
  FieldLabel,
  FieldDescription,
  FieldError,
  FieldGroup,
  FieldLegend,
  FieldSeparator,
  FieldSet,
  FieldContent,
  FieldTitle,
}
```

Source code 6.248: deskflow-frontend/components/ui/form.tsx

```
'use client'

import * as React from 'react'
import * as LabelPrimitive from '@radix-ui/react-label'
import { Slot } from '@radix-ui/react-slot'
import {
  Controller,
  FormProvider,
  useFormContext,
  useFormState,
  type ControllerProps,
  type FieldPath,
  type FieldValues,
} from 'react-hook-form'

import { cn } from '@/lib/utils'
import { Label } from '@/components/ui/label'

const Form = FormProvider

type FormFieldContextValue<
```

```
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>,
> = {
  name: TName
}

const FormFieldContext = React.createContext<FormFieldContextValue>(
  {} as FormFieldContextValue,
)

const FormField = <
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>,
>({
  ...props
}: ControllerProps<TFieldValues, TName>) => {
  return (
    <FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>
  )
}

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext)
  const itemContext = React.useContext(FormItemContext)
  const { getFieldState } = useFormContext()
  const formState = useFormState({ name: fieldContext.name })
  const fieldState = getFieldState(fieldContext.name, formState)

  if (!fieldContext) {
    throw new Error('useFormField should be used within <FormField>')
  }

  const { id } = itemContext

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  }
}

type FormItemContextValue = {
  id: string
}

const FormItemContext = React.createContext<FormItemContextValue>(
  {} as FormItemContextValue,
)

function FormItem({ className, ...props }: React.ComponentProps<'div'>) {
  const id = React.useId()

  return (
    <FormItemContext.Provider value={{ id }}>
      <div
        data-slot="form-item"
        className={cn('grid gap-2', className)}
        {...props}
      />
    </FormItemContext.Provider>
  )
}

function FormLabel({
  className,
```

```
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  const { error, formItemId } = useFormField()

  return (
    <Label
      data-slot="form-label"
      data-error={!!error}
      className={cn('data-[error=true]:text-destructive', className)}
      htmlFor={formItemId}
      {...props}
    />
  )
}

function FormControl({ ...props }: React.ComponentProps<typeof Slot>) {
  const { error, formItemId, formDescriptionId, formMessageId } = useFormField()

  return (
    <Slot
      data-slot="form-control"
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props}
    />
  )
}

function FormDescription({ className, ...props }: React.ComponentProps<'p'>) {
  const { formDescriptionId } = useFormField()

  return (
    <p
      data-slot="form-description"
      id={formDescriptionId}
      className={cn('text-muted-foreground text-sm', className)}
      {...props}
    />
  )
}

function FormMessage({ className, ...props }: React.ComponentProps<'p'>) {
  const { error, formMessageId } = useFormField()
  const body = error ? String(error?.message ?? '') : props.children

  if (!body) {
    return null
  }

  return (
    <p
      data-slot="form-message"
      id={formMessageId}
      className={cn('text-destructive text-sm', className)}
      {...props}
    >
      {body}
    </p>
  )
}

export {
  useFormField,
  Form,
  FormItem,
```

```
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
}
```

Source code 6.249: deskflow-frontend/components/ui/hover-card.tsx

```
'use client'

import * as React from 'react'
import * as HoverCardPrimitive from '@radix-ui/react-hover-card'

import { cn } from '@/lib/utils'

function HoverCard({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Root>) {
  return <HoverCardPrimitive.Root data-slot="hover-card" {...props} />
}

function HoverCardTrigger({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Trigger>) {
  return (
    <HoverCardPrimitive.Trigger data-slot="hover-card-trigger" {...props} />
  )
}

function HoverCardContent({
  className,
  align = 'center',
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Content>) {
  return (
    <HoverCardPrimitive.Portal data-slot="hover-card-portal">
      <HoverCardPrimitive.Content
        data-slot="hover-card-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50 w-64
          origin-(-radix-hover-card-content-transform-origin) rounded-md border p-4 shadow-md
          outline-hidden',
          className,
        )}
        {...props}
      />
    </HoverCardPrimitive.Portal>
  )
}

export { HoverCard, HoverCardTrigger, HoverCardContent }
```

Source code 6.250: deskflow-frontend/components/ui/input-group.tsx

```
'use client'

import { cva, type VariantProps } from 'class-variance-authority'
```

```

import { cn } from '@/lib/utils'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'

function InputGroup({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="input-group"
      role="group"
      className={cn(
        'group/input-group border-input dark:bg-input/30 relative flex w-full items-center rounded-md border shadow-xs transition-[color,box-shadow] outline-none',
        'h-9 has-[>textarea]:h-auto',

        // Variants based on alignment.
        'has-[>[data-align=inline-start]]:[&>input]:pl-2',
        'has-[>[data-align=inline-end]]:[&>input]:pr-2',
        'has-[>[data-align=block-start]]:h-auto has-[>[data-align=block-start]]:flex-col has-[>[data-align=block-start]]:[&>input]:pb-3',
        'has-[>[data-align=block-end]]:h-auto has-[>[data-align=block-end]]:flex-col has-[>[data-align=block-end]]:[&>input]:pt-3',

        // Focus state.
        'has-[[data-slot=input-group-control]:focus-visible]:border-ring has-[[data-slot=input-group-control]:focus-visible]:ring-ring/50 has-[[data-slot=input-group-control]:focus-visible]:ring-[3px]',

        // Error state.
        'has-[[data-slot][aria-invalid=true]]:ring-destructive/20 has-[[data-slot][aria-invalid=true]]:border-destructive dark:has-[[data-slot][aria-invalid=true]]:ring-destructive/40',

        className,
      )}
      {...props}
    />
  )
}

const inputGroupAddonVariants = cva(
  "text-muted-foreground flex h-auto cursor-text items-center justify-center gap-2 py-1.5 text-sm font-medium select-none [&>svg:not([class*='size-'])]:size-4 [&>kbd]:rounded-[calc(var(-radius)-5px)] group-data-[disabled=true]/input-group:opacity-50",
  {
    variants: {
      align: {
        'inline-start':
          'order-first pl-3 has-[>button]:ml-[-0.45rem] has-[>kbd]:ml-[-0.35rem]',
        'inline-end':
          'order-last pr-3 has-[>button]:mr-[-0.4rem] has-[>kbd]:mr-[-0.35rem]',
        'block-start':
          'order-first w-full justify-start px-3 pt-3 [.border-b]:pb-3 group-has-[>input]/input-group:pt-2.5',
        'block-end':
          'order-last w-full justify-start px-3 pb-3 [.border-t]:pt-3 group-has-[>input]/input-group:pb-2.5',
      },
    },
    defaultVariants: {
      align: 'inline-start',
    },
  },
)

function InputGroupAddon({
  className,
  align = 'inline-start',
  ...props
```

```
}: React.ComponentProps<'div'> & VariantProps<typeof inputGroupAddonVariants>) {
  return (
    <div
      role="group"
      data-slot="input-group-addon"
      data-align={align}
      className={cn(inputGroupAddonVariants({ align }), className)}
      onClick={(e) => {
        if ((e.target as HTMLElement).closest('button')) {
          return
        }
        e.currentTarget.parentElement?.querySelector('input')?.focus()
      }}
      {...props}
    />
  )
}

const inputGroupButtonVariants = cva(
  'text-sm shadow-none flex gap-2 items-center',
  {
    variants: {
      size: {
        xs: "h-6 gap-1 px-2 rounded-[calc(var(-radius)-5px)] [&>svg:not([class*='size-'])]:size-3.5
        has-[>svg]:px-2",
        sm: 'h-8 px-2.5 gap-1.5 rounded-md has-[>svg]:px-2.5',
        'icon-xs':
          'size-6 rounded-[calc(var(-radius)-5px)] p-0 has-[>svg]:p-0',
        'icon-sm': 'size-8 p-0 has-[>svg]:p-0',
      },
    },
    defaultVariants: {
      size: 'xs',
    },
  },
)

function InputGroupButton({
  className,
  type = 'button',
  variant = 'ghost',
  size = 'xs',
  ...props
}: Omit<React.ComponentProps<typeof Button>, 'size'> &
  VariantProps<typeof inputGroupButtonVariants>) {
  return (
    <Button
      type={type}
      data-size={size}
      variant={variant}
      className={cn(inputGroupButtonVariants({ size }), className)}
      {...props}
    />
  )
}

function InputGroupText({ className, ...props }: React.ComponentProps<'span'>) {
  return (
    <span
      className={cn(
        "text-muted-foreground flex items-center gap-2 text-sm [&_svg]:pointer-events-none
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}

function InputGroupInput({
```

```
  className,
  ...props
}: React.ComponentProps<'input'>) {
  return (
    <Input
      data-slot="input-group-control"
      className={cn(
        'flex-1 rounded-none border-0 bg-transparent shadow-none focus-visible:ring-0 dark:bg-transparent',
        className,
      )}
      {...props}
    />
  )
}

function InputGroupTextarea({
  className,
  ...props
}: React.ComponentProps<'textarea'>) {
  return (
    <Textarea
      data-slot="input-group-control"
      className={cn(
        'flex-1 resize-none rounded-none border-0 bg-transparent py-3 shadow-none focus-visible:ring-0
        dark:bg-transparent',
        className,
      )}
      {...props}
    />
  )
}

export {
  InputGroup,
  InputGroupAddon,
  InputGroupButton,
  InputGroupText,
  InputGroupInput,
  InputGroupTextarea,
}
```

Source code 6.251: deskflow-frontend/components/ui/input-otp.tsx

```
'use client'

import * as React from 'react'
import { OTPInput, OTPInputContext } from 'input-otp'
import { MinusIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function InputOTP({
  className,
  containerClassName,
  ...props
}: React.ComponentProps<typeof OTPInput> & {
  containerClassName?: string
}) {
  return (
    <OTPInput
      data-slot="input-otp"
      containerClassName={cn(
        'flex items-center gap-2 has-disabled:opacity-50',
        containerClassName,
      )}
      className={cn('disabled:cursor-not-allowed', className)}
      {...props}
    />
```

```
  )
}

function InputOTPGroup({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="input-otp-group"
      className={cn('flex items-center', className)}
      {...props}
    />
  )
}

function InputOTPSlot({
  index,
  className,
  ...props
}: React.ComponentProps<'div'> & {
  index: number
}) {
  const inputOTPContext = React.useContext(OTPInputContext)
  const { char, hasFakeCaret, isActive } = inputOTPContext?.slots[index] ?? {}

  return (
    <div
      data-slot="input-otp-slot"
      data-active={isActive}
      className={cn(
        'data-[active=true]:border-ring data-[active=true]:ring-ring/50
        data-[active=true]:aria-invalid:ring-destructive/20
        dark:data-[active=true]:aria-invalid:ring-destructive/40 aria-invalid:border-destructive
        data-[active=true]:aria-invalid:border-destructive dark:bg-input/30 border-input relative flex h-9
        w-9 items-center justify-center border-y border-r text-sm shadow-xs transition-all outline-none
        first:rounded-l-md first:border-l last:rounded-r-md data-[active=true]:z-10
        data-[active=true]:ring-[3px]',
        className,
      )}
      {...props}
    >
      {char}
      {hasFakeCaret && (
        <div className="pointer-events-none absolute inset-0 flex items-center justify-center">
          <div className="animate-caret-blink bg-foreground h-4 w-px duration-1000" />
        </div>
      )}
    </div>
  )
}

function InputOTPSeparator({ ...props }: React.ComponentProps<'div'>) {
  return (
    <div data-slot="input-otp-separator" role="separator" {...props}>
      <MinusIcon />
    </div>
  )
}

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }
```

Source code 6.252: deskflow-frontend/components/ui/input.tsx

```
import * as React from 'react'

import { cn } from '@/lib/utils'

function Input({ className, type, ...props }: React.ComponentProps<'input'>) {
  return (
```

```
    <input
      type={type}
      data-slot="input"
      className={cn(
        'file:text-foreground placeholder:text-muted-foreground selection:bg-primary
        selection:text-primary-foreground dark:bg-input/30 border-input h-9 w-full min-w-0 rounded-md border
        bg-transparent px-3 py-1 text-base shadow-xs transition-[color,box-shadow] outline-none
        file:inline-flex file:h-7 file:border-0 file:bg-transparent file:text-sm file:font-medium
        disabled:pointer-events-none disabled:cursor-not-allowed disabled:opacity-50 md:text-sm',
        'focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]',
        'aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive',
        className,
      )}
      {...props}
    />
  )
}

export { Input }
```

Source code 6.253: deskflow-frontend/components/ui/item.tsx

```
import * as React from 'react'
import { Slot } from '@radix-ui/react-slot'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'
import { Separator } from '@/components/ui/separator'

function ItemGroup({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      role="list"
      data-slot="item-group"
      className={cn('group/item-group flex flex-col', className)}
      {...props}
    />
  )
}

function ItemSeparator({
  className,
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="item-separator"
      orientation="horizontal"
      className={cn('my-0', className)}
      {...props}
    />
  )
}

const itemVariants = cva(
  'group/item flex items-center border border-transparent text-sm rounded-md transition-colors
  [a&]:hover:bg-accent/50 [a&]:transition-colors duration-100 flex-wrap outline-none
  focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]',
  {
    variants: {
      variant: {
        default: 'bg-transparent',
        outline: 'border-border',
        muted: 'bg-muted/50',
      },
      size: {
```

```
        default: 'p-4 gap-4 ',
        sm: 'py-3 px-4 gap-2.5',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  },
)

function Item({
  className,
  variant = 'default',
  size = 'default',
  asChild = false,
  ...props
}: React.ComponentProps<'div'> &
  VariantProps<typeof itemVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : 'div'
  return (
    <Comp
      data-slot="item"
      data-variant={variant}
      data-size={size}
      className={cn(itemVariants({ variant, size, className }))}
      {...props}
    />
  )
}

const itemMediaVariants = cva(
  'flex shrink-0 items-center justify-center gap-2 group-has-[[data-slot=item-description]]/item:self-start
  [&_svg]:pointer-events-none group-has-[[data-slot=item-description]]/item:translate-y-0.5',
  {
    variants: {
      variant: {
        default: 'bg-transparent',
        icon: "size-8 border rounded-sm bg-muted [&_svg:not([class*='size-'])]:size-4",
        image:
          'size-10 rounded-sm overflow-hidden [&_img]:size-full [&_img]:object-cover',
      },
    },
    defaultVariants: {
      variant: 'default',
    },
  },
)

function ItemMedia({
  className,
  variant = 'default',
  ...props
}: React.ComponentProps<'div'> & VariantProps<typeof itemMediaVariants>) {
  return (
    <div
      data-slot="item-media"
      data-variant={variant}
      className={cn(itemMediaVariants({ variant, className }))}
      {...props}
    />
  )
}

function ItemContent({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="item-content"
      className={cn(
        'flex flex-1 flex-col gap-1 [&+[data-slot=item-content]]:flex-none',
```

```
        className,
      )}
      {...props}
    />
  )
}

function ItemTitle({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="item-title"
      className={cn(
        'flex w-fit items-center gap-2 text-sm leading-snug font-medium',
        className,
      )}
      {...props}
    />
  )
}

function ItemDescription({ className, ...props }: React.ComponentProps<'p'>) {
  return (
    <p
      data-slot="item-description"
      className={cn(
        'text-muted-foreground line-clamp-2 text-sm leading-normal font-normal text-balance',
        '[&>a:hover]:text-primary [&>a]:underline [&>a]:underline-offset-4',
        className,
      )}
      {...props}
    />
  )
}

function ItemActions({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="item-actions"
      className={cn('flex items-center gap-2', className)}
      {...props}
    />
  )
}

function ItemHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="item-header"
      className={cn(
        'flex basis-full items-center justify-between gap-2',
        className,
      )}
      {...props}
    />
  )
}

function ItemFooter({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="item-footer"
      className={cn(
        'flex basis-full items-center justify-between gap-2',
        className,
      )}
      {...props}
    />
  )
}

```

```
export {
  Item,
  ItemMedia,
  ItemContent,
  ItemActions,
  ItemGroup,
  ItemSeparator,
  ItemTitle,
  ItemDescription,
  ItemHeader,
  ItemFooter,
}
```

Source code 6.254: deskflow-frontend/components/ui/kbd.tsx

```
import { cn } from '@/lib/utils'

function Kbd({ className, ...props }: React.ComponentProps<'kbd'>) {
  return (
    <kbd
      data-slot="kbd"
      className={cn(
        'bg-muted w-fit text-muted-foreground pointer-events-none inline-flex h-5 min-w-5 items-center justify-center gap-1 rounded-sm px-1 font-sans text-xs font-medium select-none',
        "[&_svg:not([class*='size-'])]:size-3",
        '[[data-slot=tooltip-content]_&]:bg-background/20 [[data-slot=tooltip-content]_&]:text-background dark:[[data-slot=tooltip-content]_&]:bg-background/10',
        className,
      )}
      {...props}
    />
  )
}

function KbdGroup({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <kbd
      data-slot="kbd-group"
      className={cn('inline-flex items-center gap-1', className)}
      {...props}
    />
  )
}

export { Kbd, KbdGroup }
```

Source code 6.255: deskflow-frontend/components/ui/label.tsx

```
'use client'

import * as React from 'react'
import * as LabelPrimitive from '@radix-ui/react-label'

import { cn } from '@/lib/utils'

function Label({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  return (
    <LabelPrimitive.Root
      data-slot="label"
      className={cn(
        'flex items-center gap-2 text-sm leading-none font-medium select-none group-data-[disabled=true]:pointer-events-none group-data-[disabled=true]:opacity-50 peer-disabled:cursor-not-allowed peer-disabled:opacity-50',
```

```
        className,
      )}
      {...props}
    />
  )
}

export { Label }
```

Source code 6.256: deskflow-frontend/components/ui/menubar.tsx

```
'use client'

import * as React from 'react'
import * as MenubarPrimitive from '@radix-ui/react-menubar'
import { CheckIcon, ChevronRightIcon, CircleIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function Menubar({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Root>) {
  return (
    <MenubarPrimitive.Root
      data-slot="menubar"
      className={cn(
        'bg-background flex h-9 items-center gap-1 rounded-md border p-1 shadow-xs',
        className,
      )}
      {...props}
    />
  )
}

function MenubarMenu({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Menu>) {
  return <MenubarPrimitive.Menu data-slot="menubar-menu" {...props} />
}

function MenubarGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Group>) {
  return <MenubarPrimitive.Group data-slot="menubar-group" {...props} />
}

function MenubarPortal({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Portal>) {
  return <MenubarPrimitive.Portal data-slot="menubar-portal" {...props} />
}

function MenubarRadioGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioGroup>) {
  return (
    <MenubarPrimitive.RadioGroup data-slot="menubar-radio-group" {...props} />
  )
}

function MenubarTrigger({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Trigger>) {
  return (
    <MenubarPrimitive.Trigger
      data-slot="menubar-trigger"
```

```
      className={cn(
        'focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground flex items-center rounded-sm px-2 py-1 text-sm font-medium
        outline-hidden select-none',
        className,
      )}
      {...props}
    />
  )
}

function MenubarContent({
  className,
  align = 'start',
  alignOffset = -4,
  sideOffset = 8,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Content>) {
  return (
    <MenubarPortal>
      <MenubarPrimitive.Content
        data-slot="menubar-content"
        align={align}
        alignOffset={alignOffset}
        sideOffset={sideOffset}
        className={cn(
          'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:fade-out-0
          data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
          data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
          data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[12rem]
          origin-(-radix-menubar-content-transform-origin) overflow-hidden rounded-md border p-1 shadow-md',
          className,
        )}
        {...props}
      />
    </MenubarPortal>
  )
}

function MenubarItem({
  className,
  inset,
  variant = 'default',
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Item> & {
  inset?: boolean
  variant?: 'default' | 'destructive'
}) {
  return (
    <MenubarPrimitive.Item
      data-slot="menubar-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive
        data-[variant=destructive]:focus:bg-destructive/10
        dark:data-[variant=destructive]:focus:bg-destructive/20
        data-[variant=destructive]:focus:text-destructive
        data-[variant=destructive]:*:[svg]:!text-destructive
        [&_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default items-center gap-2
        rounded-sm px-2 py-1.5 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
        data-[disabled]:opacity-50 data-[inset]:pl-8 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}
```

```
113
114 function MenubarCheckboxItem({
115   className,
116   children,
117   checked,
118   ...props
119 }: React.ComponentProps<typeof MenubarPrimitive.CheckboxItem>) {
120   return (
121     <MenubarPrimitive.CheckboxItem
122       data-slot="menubar-checkbox-item"
123       className={cn(
124         "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2
          rounded-xs py-1.5 pr-2 pl-8 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
          data-[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
          [&_svg:not([class*='size-'])]:size-4",
125         className,
126       )}
127       checked={checked}
128       {...props}
129     >
130       <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
131         <MenubarPrimitive.ItemIndicator>
132           <CheckIcon className="size-4" />
133         </MenubarPrimitive.ItemIndicator>
134       </span>
135       {children}
136     </MenubarPrimitive.CheckboxItem>
137   )
138 }
139
140 function MenubarRadioItem({
141   className,
142   children,
143   ...props
144 }: React.ComponentProps<typeof MenubarPrimitive.RadioItem>) {
145   return (
146     <MenubarPrimitive.RadioItem
147       data-slot="menubar-radio-item"
148       className={cn(
149         "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2
          rounded-xs py-1.5 pr-2 pl-8 text-sm outline-hidden select-none data-[disabled]:pointer-events-none
          data-[disabled]:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
          [&_svg:not([class*='size-'])]:size-4",
150         className,
151       )}
152       {...props}
153     >
154       <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
155         <MenubarPrimitive.ItemIndicator>
156           <CircleIcon className="size-2 fill-current" />
157         </MenubarPrimitive.ItemIndicator>
158       </span>
159       {children}
160     </MenubarPrimitive.RadioItem>
161   )
162 }
163
164 function MenubarLabel({
165   className,
166   inset,
167   ...props
168 }: React.ComponentProps<typeof MenubarPrimitive.Label> & {
169   inset?: boolean
170 }) {
171   return (
172     <MenubarPrimitive.Label
173       data-slot="menubar-label"
174       data-inset={inset}
175       className={cn(
176         'px-2 py-1.5 text-sm font-medium data-[inset]:pl-8',
```

```
        className,
      )}
      {...props}
    />
  )
}

function MenubarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Separator>) {
  return (
    <MenubarPrimitive.Separator
      data-slot="menubar-separator"
      className={cn('bg-border -mx-1 my-1 h-px', className)}
      {...props}
    />
  )
}

function MenubarShortcut({
  className,
  ...props
}: React.ComponentProps<'span'>) {
  return (
    <span
      data-slot="menubar-shortcut"
      className={cn(
        'text-muted-foreground ml-auto text-xs tracking-widest',
        className,
      )}
      {...props}
    />
  )
}

function MenubarSub({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Sub>) {
  return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />
}

function MenubarSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.SubTrigger
      data-slot="menubar-sub-trigger"
      data-inset={inset}
      className={cn(
        'focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm px-2 py-1.5
        text-sm outline-none select-none data-[inset]:pl-8',
        className,
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto h-4 w-4" />
    </MenubarPrimitive.SubTrigger>
  )
}

function MenubarSubContent({
  className,
```

```
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubContent>) {
  return (
    <MenubarPrimitive.SubContent
      data-slot="menubar-sub-content"
      className={cn(
        'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
        data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
        data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
        data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
        data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
        origin-(-radix-menubar-content-transform-origin) overflow-hidden rounded-md border p-1 shadow-lg',
        className,
      )}
      {...props}
    />
  )
}

export {
  Menubar,
  MenubarPortal,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarGroup,
  MenubarSeparator,
  MenubarLabel,
  MenubarItem,
  MenubarShortcut,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarSub,
  MenubarSubTrigger,
  MenubarSubContent,
}
```

Source code 6.257: deskflow-frontend/components/ui/navigation-menu.tsx

```
import * as React from 'react'
import * as NavigationMenuPrimitive from '@radix-ui/react-navigation-menu'
import { cva } from 'class-variance-authority'
import { ChevronDownIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function NavigationMenu({
  className,
  children,
  viewport = true,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Root> & {
  viewport?: boolean
}) {
  return (
    <NavigationMenuPrimitive.Root
      data-slot="navigation-menu"
      data-viewport={viewport}
      className={cn(
        'group/navigation-menu relative flex max-w-max flex-1 items-center justify-center',
        className,
      )}
      {...props}
    >
      {children}
      {viewport && <NavigationMenuViewport />}
```

```
    </NavigationMenuPrimitive.Root>
  )
}

function NavigationMenuList({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.List>) {
  return (
    <NavigationMenuPrimitive.List
      data-slot="navigation-menu-list"
      className={cn(
        'group flex flex-1 list-none items-center justify-center gap-1',
        className,
      )}
      {...props}
    />
  )
}

function NavigationMenuItem({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Item>) {
  return (
    <NavigationMenuPrimitive.Item
      data-slot="navigation-menu-item"
      className={cn('relative', className)}
      {...props}
    />
  )
}

const navigationMenuTriggerStyle = cva(
  'group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm
  font-medium hover:bg-accent hover:text-accent-foreground focus:bg-accent focus:text-accent-foreground
  disabled:pointer-events-none disabled:opacity-50 data-[state=open]:hover:bg-accent
  data-[state=open]:text-accent-foreground data-[state=open]:focus:bg-accent data-[state=open]:bg-accent/50
  focus-visible:ring-ring/50 outline-none transition-[color,box-shadow] focus-visible:ring-[3px]
  focus-visible:outline-1',
)

function NavigationMenuTrigger({
  className,
  children,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Trigger>) {
  return (
    <NavigationMenuPrimitive.Trigger
      data-slot="navigation-menu-trigger"
      className={cn(navigationMenuTriggerStyle(), 'group', className)}
      {...props}
    >
      {children}{' '}
      <ChevronDownIcon
        className="relative top-[1px] ml-1 size-3 transition duration-300 group-data-[state=open]:rotate-180"
        aria-hidden="true"
      />
    </NavigationMenuPrimitive.Trigger>
  )
}

function NavigationMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Content>) {
  return (
    <NavigationMenuPrimitive.Content
      data-slot="navigation-menu-content"
      className={cn(
```

```
        'data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out data-[motion^=from-]:fade-in
        data-[motion^=to-]:fade-out data-[motion=from-end]:slide-in-from-right-52
        data-[motion=from-start]:slide-in-from-left-52 data-[motion=to-end]:slide-out-to-right-52
        data-[motion=to-start]:slide-out-to-left-52 top-0 left-0 w-full p-2 pr-2.5 md:absolute md:w-auto',
        'group-data-[viewport=false]/navigation-menu:bg-popover
        group-data-[viewport=false]/navigation-menu:text-popover-foreground
        group-data-[viewport=false]/navigation-menu:data-[state=open]:animate-in
        group-data-[viewport=false]/navigation-menu:data-[state=closed]:animate-out
        group-data-[viewport=false]/navigation-menu:data-[state=closed]:zoom-out-95
        group-data-[viewport=false]/navigation-menu:data-[state=open]:zoom-in-95
        group-data-[viewport=false]/navigation-menu:data-[state=open]:fade-in-0
        group-data-[viewport=false]/navigation-menu:data-[state=closed]:fade-out-0
        group-data-[viewport=false]/navigation-menu:top-full
        group-data-[viewport=false]/navigation-menu:mt-1.5
        group-data-[viewport=false]/navigation-menu:overflow-hidden
        group-data-[viewport=false]/navigation-menu:rounded-md
        group-data-[viewport=false]/navigation-menu:border group-data-[viewport=false]/navigation-menu:shadow
        group-data-[viewport=false]/navigation-menu:duration-200
        **:data-[slot=navigation-menu-link]:focus:ring-0
        **:data-[slot=navigation-menu-link]:focus:outline-none',
        className,
      )}
      {...props}
    />
  )
}

function NavigationMenuViewport({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Viewport>) {
  return (
    <div
      className="absolute top-full left-0 isolate z-50 flex justify-center"
    >
      <NavigationMenuPrimitive.Viewport
        data-slot="navigation-menu-viewport"
        className={cn(
          'origin-top-center bg-popover text-popover-foreground data-[state=open]:animate-in
          data-[state=closed]:animate-out data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90
          relative mt-1.5 h-[var(-radix-navigation-menu-viewport-height)] w-full overflow-hidden rounded-md
          border shadow md:w-[var(-radix-navigation-menu-viewport-width)]',
          className,
        )}
        {...props}
      />
    </div>
  )
}

function NavigationMenuLink({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Link>) {
  return (
    <NavigationMenuPrimitive.Link
      data-slot="navigation-menu-link"
      className={cn(
        "data-[active=true]:focus:bg-accent data-[active=true]:hover:bg-accent
        data-[active=true]:bg-accent/50 data-[active=true]:text-accent-foreground hover:bg-accent
        hover:text-accent-foreground focus:bg-accent focus:text-accent-foreground focus-visible:ring-ring/50
        [&_svg:not([class*='text-'])]:text-muted-foreground flex flex-col gap-1 rounded-sm p-2 text-sm
        transition-all outline-none focus-visible:ring-[3px] focus-visible:outline-1
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
```

```
}

function NavigationMenuIndicator({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Indicator>) {
  return (
    <NavigationMenuPrimitive.Indicator
      data-slot="navigation-menu-indicator"
      className={cn(
        'data-[state=visible]:animate-in data-[state=hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:fade-in top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden',
        className,
      )}
      {...props}
    >
      <div className="bg-border relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm shadow-md" />
    </NavigationMenuPrimitive.Indicator>
  )
}

export {
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
  navigationMenuTriggerStyle,
}
```

Source code 6.258: deskflow-frontend/components/ui/pagination.tsx

```
import * as React from 'react'
import {
  ChevronLeftIcon,
  ChevronRightIcon,
  MoreHorizontalIcon,
} from 'lucide-react'

import { cn } from '@/lib/utils'
import { Button, buttonVariants } from '@/components/ui/button'

function Pagination({ className, ...props }: React.ComponentProps<'nav'>) {
  return (
    <nav
      role="navigation"
      aria-label="pagination"
      data-slot="pagination"
      className={cn('mx-auto flex w-full justify-center', className)}
      {...props}
    />
  )
}

function PaginationContent({
  className,
  ...props
}: React.ComponentProps<'ul'>) {
  return (
    <ul
      data-slot="pagination-content"
      className={cn('flex flex-row items-center gap-1', className)}
      {...props}
    />
  )
```

```
}

function PaginationItem({ ...props }: React.ComponentProps<'li'>) {
  return <li data-slot="pagination-item" {...props} />
}

type PaginationLinkProps = {
  isActive?: boolean
} & Pick<React.ComponentProps<typeof Button>, 'size'> &
  React.ComponentProps<'a'>

function PaginationLink({
  className,
  isActive,
  size = 'icon',
  ...props
}: PaginationLinkProps) {
  return (
    <a
      aria-current={isActive ? 'page' : undefined}
      data-slot="pagination-link"
      data-active={isActive}
      className={cn(
        buttonVariants({
          variant: isActive ? 'outline' : 'ghost',
          size,
        }),
        className,
      )}
      {...props}
    />
  )
}

function PaginationPrevious({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink>) {
  return (
    <PaginationLink
      aria-label="Go to previous page"
      size="default"
      className={cn('gap-1 px-2.5 sm:pl-2.5', className)}
      {...props}
    >
      <ChevronLeftIcon />
      <span className="hidden sm:block">Previous</span>
    </PaginationLink>
  )
}

function PaginationNext({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink>) {
  return (
    <PaginationLink
      aria-label="Go to next page"
      size="default"
      className={cn('gap-1 px-2.5 sm:pr-2.5', className)}
      {...props}
    >
      <span className="hidden sm:block">Next</span>
      <ChevronRightIcon />
    </PaginationLink>
  )
}

function PaginationEllipsis({
  className,
```

```
  ...props
}: React.ComponentProps<'span'>) {
  return (
    <span
      aria-hidden
      data-slot="pagination-ellipsis"
      className={cn('flex size-9 items-center justify-center', className)}
      {...props}
    >
      <MoreHorizontalIcon className="size-4" />
      <span className="sr-only">More pages</span>
    </span>
  )
}

export {
  Pagination,
  PaginationContent,
  PaginationLink,
  PaginationItem,
  PaginationPrevious,
  PaginationNext,
  PaginationEllipsis,
}
```

Source code 6.259: deskflow-frontend/components/ui/popover.tsx

```
'use client'

import * as React from 'react'
import * as PopoverPrimitive from '@radix-ui/react-popover'

import { cn } from '@/lib/utils'

function Popover({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Root>) {
  return <PopoverPrimitive.Root data-slot="popover" {...props} />
}

function PopoverTrigger({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Trigger>) {
  return <PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} />
}

function PopoverContent({
  className,
  align = 'center',
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Content>) {
  return (
    <PopoverPrimitive.Portal>
      <PopoverPrimitive.Content
        data-slot="popover-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50 w-72 origin-(-radix-popover-content-transform-origin)
          rounded-md border p-4 shadow-md outline-hidden',
          className,
        )}
```

```
        {...props}
      />
    </PopoverPrimitive.Portal>
  )
}

function PopoverAnchor({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Anchor>) {
  return <PopoverPrimitive.Anchor data-slot="popover-anchor" {...props} />
}

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor }
```

Source code 6.260: deskflow-frontend/components/ui/progress.tsx

```
'use client'

import * as React from 'react'
import * as ProgressPrimitive from '@radix-ui/react-progress'

import { cn } from '@/lib/utils'

function Progress({
  className,
  value,
  ...props
}: React.ComponentProps<typeof ProgressPrimitive.Root>) {
  return (
    <ProgressPrimitive.Root
      data-slot="progress"
      className={cn(
        'bg-primary/20 relative h-2 w-full overflow-hidden rounded-full',
        className,
      )}
      {...props}
    >
      <ProgressPrimitive.Indicator
        data-slot="progress-indicator"
        className="bg-primary h-full w-full flex-1 transition-all"
        style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
      />
    </ProgressPrimitive.Root>
  )
}

export { Progress }
```

Source code 6.261: deskflow-frontend/components/ui/radio-group.tsx

```
'use client'

import * as React from 'react'
import * as RadioGroupPrimitive from '@radix-ui/react-radio-group'
import { CircleIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function RadioGroup({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Root>) {
  return (
    <RadioGroupPrimitive.Root
      data-slot="radio-group"
```

```
      className={cn('grid gap-3', className)}
      {...props}
    />
  )
}

function RadioGroupItem({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Item>) {
  return (
    <RadioGroupPrimitive.Item
      data-slot="radio-group-item"
      className={cn(
        'border-input text-primary focus-visible:border-ring focus-visible:ring-ring/50
        aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive dark:bg-input/30 aspect-square size-4 shrink-0 rounded-full border
        shadow-xs transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
        disabled:cursor-not-allowed disabled:opacity-50',
        className,
      )}
      {...props}
    >
      <RadioGroupPrimitive.Indicator
        data-slot="radio-group-indicator"
        className="relative flex items-center justify-center"
      >
        <CircleIcon className="fill-primary absolute top-1/2 left-1/2 size-2 -translate-x-1/2
       -translate-y-1/2" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>
  )
}

export { RadioGroup, RadioGroupItem }
```

Source code 6.262: deskflow-frontend/components/ui/resizable.tsx

```
'use client'

import * as React from 'react'
import { GripVerticalIcon } from 'lucide-react'
import * as ResizablePrimitive from 'react-resizable-panels'

import { cn } from '@/lib/utils'

function ResizablePanelGroup({
  className,
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.PanelGroup>) {
  return (
    <ResizablePrimitive.PanelGroup
      data-slot="resizable-panel-group"
      className={cn(
        'flex h-full w-full data-[panel-group-direction=vertical]:flex-col',
        className,
      )}
      {...props}
    />
  )
}

function ResizablePanel({
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.Panel>) {
  return <ResizablePrimitive.Panel data-slot="resizable-panel" {...props} />
}
```

```
30
31  function ResizableHandle({
32    withHandle,
33    className,
34    ...props
35  }: React.ComponentProps<typeof ResizablePrimitive.PanelResizeHandle> & {
36    withHandle?: boolean
37  }) {
38    return (
39      <ResizablePrimitive.PanelResizeHandle
40        data-slot="resizable-handle"
41        className={cn(
42          'bg-border focus-visible:ring-ring relative flex w-px items-center justify-center after:absolute
          after:inset-y-0 after:left-1/2 after:w-1 after:-translate-x-1/2 focus-visible:ring-1
          focus-visible:ring-offset-1 focus-visible:outline-hidden data-[panel-group-direction=vertical]:h-px
          data-[panel-group-direction=vertical]:w-full data-[panel-group-direction=vertical]:after:left-0
          data-[panel-group-direction=vertical]:after:h-1 data-[panel-group-direction=vertical]:after:w-full
          data-[panel-group-direction=vertical]:after:translate-x-0
          data-[panel-group-direction=vertical]:after:-translate-y-1/2
          [&[data-panel-group-direction=vertical]>div]:rotate-90',
43          className,
44        )}
45        {...props}
46      >
47        {withHandle && (
48          <div className="bg-border z-10 flex h-4 w-3 items-center justify-center rounded-xs border">
49            <GripVerticalIcon className="size-2.5" />
50          </div>
51        )}
52      </ResizablePrimitive.PanelResizeHandle>
53    )
54  }
55
56  export { ResizablePanelGroup, ResizablePanel, ResizableHandle }
```

Source code 6.263: deskflow-frontend/components/ui/scroll-area.tsx

```
1   'use client'
2
3   import * as React from 'react'
4   import * as ScrollAreaPrimitive from '@radix-ui/react-scroll-area'
5
6   import { cn } from '@/lib/utils'
7
8   function ScrollArea({
9     className,
10    children,
11    ...props
12  }: React.ComponentProps<typeof ScrollAreaPrimitive.Root>) {
13    return (
14      <ScrollAreaPrimitive.Root
15        data-slot="scroll-area"
16        className={cn('relative', className)}
17        {...props}
18      >
19        <ScrollAreaPrimitive.Viewport
20          data-slot="scroll-area-viewport"
21          className="focus-visible:ring-ring/50 size-full rounded-[inherit] transition-[color,box-shadow]
            outline-none focus-visible:ring-[3px] focus-visible:outline-1"
22        >
23          {children}
24        </ScrollAreaPrimitive.Viewport>
25        <ScrollBar />
26        <ScrollAreaPrimitive.Corner />
27      </ScrollAreaPrimitive.Root>
28    )
29  }
```

```

function ScrollBar({
  className,
  orientation = 'vertical',
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.ScrollAreaScrollbar>) {
  return (
    <ScrollAreaPrimitive.ScrollAreaScrollbar
      data-slot="scroll-area-scrollbar"
      orientation={orientation}
      className={cn(
        'flex touch-none p-px transition-colors select-none',
        orientation === 'vertical' &&
          'h-full w-2.5 border-l border-l-transparent',
        orientation === 'horizontal' &&
          'h-2.5 flex-col border-t border-t-transparent',
        className,
      )}
      {...props}
    >
      <ScrollAreaPrimitive.ScrollAreaThumb
        data-slot="scroll-area-thumb"
        className="bg-border relative flex-1 rounded-full"
      />
    </ScrollAreaPrimitive.ScrollAreaScrollbar>
  )
}

export { ScrollArea, ScrollBar }
```

Source code 6.264: deskflow-frontend/components/ui/select.tsx

```
'use client'

import * as React from 'react'
import * as SelectPrimitive from '@radix-ui/react-select'
import { CheckIcon, ChevronDownIcon, ChevronUpIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function Select({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Root>) {
  return <SelectPrimitive.Root data-slot="select" {...props} />
}

function SelectGroup({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Group>) {
  return <SelectPrimitive.Group data-slot="select-group" {...props} />
}

function SelectValue({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Value>) {
  return <SelectPrimitive.Value data-slot="select-value" {...props} />
}

function SelectTrigger({
  className,
  size = 'default',
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Trigger> & {
  size?: 'sm' | 'default'
}) {
  return (
    <SelectPrimitive.Trigger
```

```
      data-slot="select-trigger"
      data-size={size}
      className={cn(
        "border-input data-[placeholder]:text-muted-foreground
        [&_svg:not([class*='text-'])]:text-muted-foreground focus-visible:border-ring
        focus-visible:ring-ring/50 aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive dark:bg-input/30 dark:hover:bg-input/50 flex w-fit items-center
        justify-between gap-2 rounded-md border bg-transparent px-3 py-2 text-sm whitespace-nowrap shadow-xs
        transition-[color,box-shadow] outline-none focus-visible:ring-[3px] disabled:cursor-not-allowed
        disabled:opacity-50 data-[size=default]:h-9 data-[size=sm]:h-8
        *:data-[slot=select-value]:line-clamp-1 *:data-[slot=select-value]:flex
        *:data-[slot=select-value]:items-center *:data-[slot=select-value]:gap-2 [&_svg]:pointer-events-none
        [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    >
      {children}
      <SelectPrimitive.Icon asChild>
        <ChevronDownIcon className="size-4 opacity-50" />
      </SelectPrimitive.Icon>
    </SelectPrimitive.Trigger>
  )
}

function SelectContent({
  className,
  children,
  position = 'popper',
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Content>) {
  return (
    <SelectPrimitive.Portal>
      <SelectPrimitive.Content
        data-slot="select-content"
        className={cn(
          'bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 relative z-50 max-h-(-radix-select-content-available-height)
          min-w-[8rem] origin-(-radix-select-content-transform-origin) overflow-x-hidden overflow-y-auto
          rounded-md border shadow-md',
          position === 'popper' &&
            'data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1
            data-[side=top]:-translate-y-1',
          className,
        )}
        position={position}
        {...props}
      >
        <SelectScrollUpButton />
        <SelectPrimitive.Viewport
          className={cn(
            'p-1',
            position === 'popper' &&
              'h-[var(-radix-select-trigger-height)] w-full min-w-[var(-radix-select-trigger-width)]
              scroll-my-1',
          )}
        >
          {children}
        </SelectPrimitive.Viewport>
        <SelectScrollDownButton />
      </SelectPrimitive.Content>
    </SelectPrimitive.Portal>
  )
}

function SelectLabel({
```

```
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Label>) {
  return (
    <SelectPrimitive.Label
      data-slot="select-label"
      className={cn('text-muted-foreground px-2 py-1.5 text-xs', className)}
      {...props}
    />
  )
}

function SelectItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Item>) {
  return (
    <SelectPrimitive.Item
      data-slot="select-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground [&_svg:not([class*='text-'])]:text-muted-foreground
        relative flex w-full cursor-default items-center gap-2 rounded-sm py-1.5 pr-8 pl-2 text-sm
        outline-hidden select-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4 *:[span]:last:flex
        *:[span]:last:items-center *:[span]:last:gap-2",
        className,
      )}
      {...props}
    >
      <span className="absolute right-2 flex size-3.5 items-center justify-center">
        <SelectPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </SelectPrimitive.ItemIndicator>
      </span>
      <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
    </SelectPrimitive.Item>
  )
}

function SelectSeparator({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Separator>) {
  return (
    <SelectPrimitive.Separator
      data-slot="select-separator"
      className={cn('bg-border pointer-events-none -mx-1 my-1 h-px', className)}
      {...props}
    />
  )
}

function SelectScrollUpButton({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.ScrollUpButton>) {
  return (
    <SelectPrimitive.ScrollUpButton
      data-slot="select-scroll-up-button"
      className={cn(
        'flex cursor-default items-center justify-center py-1',
        className,
      )}
      {...props}
    >
      <ChevronUpIcon className="size-4" />
    </SelectPrimitive.ScrollUpButton>
  )
}
```

```

function SelectScrollDownButton({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.ScrollDownButton>) {
  return (
    <SelectPrimitive.ScrollDownButton
      data-slot="select-scroll-down-button"
      className={cn(
        'flex cursor-default items-center justify-center py-1',
        className,
      )}
      {...props}
    >
      <ChevronDownIcon className="size-4" />
    </SelectPrimitive.ScrollDownButton>
  )
}

export {
  Select,
  SelectContent,
  SelectGroup,
  SelectItem,
  SelectLabel,
  SelectScrollDownButton,
  SelectScrollUpButton,
  SelectSeparator,
  SelectTrigger,
  SelectValue,
}
```

Source code 6.265: deskflow-frontend/components/ui/separator.tsx

```
'use client'

import * as React from 'react'
import * as SeparatorPrimitive from '@radix-ui/react-separator'

import { cn } from '@/lib/utils'

function Separator({
  className,
  orientation = 'horizontal',
  decorative = true,
  ...props
}: React.ComponentProps<typeof SeparatorPrimitive.Root>) {
  return (
    <SeparatorPrimitive.Root
      data-slot="separator"
      decorative={decorative}
      orientation={orientation}
      className={cn(
        'bg-border shrink-0 data-[orientation=horizontal]:h-px data-[orientation=horizontal]:w-full data-[orientation=vertical]:h-full data-[orientation=vertical]:w-px',
        className,
      )}
      {...props}
    />
  )
}

export { Separator }
```

Source code 6.266: deskflow-frontend/components/ui/sheet.tsx

```
'use client'

import * as React from 'react'
import * as SheetPrimitive from '@radix-ui/react-dialog'
import { XIcon } from 'lucide-react'

import { cn } from '@/lib/utils'

function Sheet({ ...props }: React.ComponentProps<typeof SheetPrimitive.Root>) {
  return <SheetPrimitive.Root data-slot="sheet" {...props} />
}

function SheetTrigger({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Trigger>) {
  return <SheetPrimitive.Trigger data-slot="sheet-trigger" {...props} />
}

function SheetClose({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Close>) {
  return <SheetPrimitive.Close data-slot="sheet-close" {...props} />
}

function SheetPortal({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Portal>) {
  return <SheetPrimitive.Portal data-slot="sheet-portal" {...props} />
}

function SheetOverlay({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Overlay>) {
  return (
    <SheetPrimitive.Overlay
      data-slot="sheet-overlay"
      className={cn(
        'data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
        data-[state=open]:fade-in-0 fixed inset-0 z-50 bg-black/50',
        className,
      )}
      {...props}
    />
  )
}

function SheetContent({
  className,
  children,
  side = 'right',
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Content> & {
  side?: 'top' | 'right' | 'bottom' | 'left'
}) {
  return (
    <SheetPortal>
      <SheetOverlay />
      <SheetPrimitive.Content
        data-slot="sheet-content"
        className={cn(
          'bg-background data-[state=open]:animate-in data-[state=closed]:animate-out fixed z-50 flex
          flex-col gap-4 shadow-lg transition ease-in-out data-[state=closed]:duration-300
          data-[state=open]:duration-500',
          side === 'right' &&
            'data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right inset-y-0 right-0
            h-full w-3/4 border-l sm:max-w-sm',
          side === 'left' &&
```

```
            'data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left inset-y-0 left-0
            h-full w-3/4 border-r sm:max-w-sm',
          side === 'top' &&
            'data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top inset-x-0 top-0 h-auto
            border-b',
          side === 'bottom' &&
            'data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-from-bottom inset-x-0
            bottom-0 h-auto border-t',
          className,
        )}
        {...props}
      >
        {children}
        <SheetPrimitive.Close className="ring-offset-background focus:ring-ring
        data-[state=open]:bg-secondary absolute top-4 right-4 rounded-xs opacity-70 transition-opacity
        hover:opacity-100 focus:ring-2 focus:ring-offset-2 focus:outline-hidden
        disabled:pointer-events-none">
          <XIcon className="size-4" />
          <span className="sr-only">Close</span>
        </SheetPrimitive.Close>
      </SheetPrimitive.Content>
    </SheetPortal>
  )
}

function SheetHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="sheet-header"
      className={cn('flex flex-col gap-1.5 p-4', className)}
      {...props}
    />
  )
}

function SheetFooter({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="sheet-footer"
      className={cn('mt-auto flex flex-col gap-2 p-4', className)}
      {...props}
    />
  )
}

function SheetTitle({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Title>) {
  return (
    <SheetPrimitive.Title
      data-slot="sheet-title"
      className={cn('text-foreground font-semibold', className)}
      {...props}
    />
  )
}

function SheetDescription({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Description>) {
  return (
    <SheetPrimitive.Description
      data-slot="sheet-description"
      className={cn('text-muted-foreground text-sm', className)}
      {...props}
    />
  )
}
```

```

export {
  Sheet,
  SheetTrigger,
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
}
```

Source code 6.267: deskflow-frontend/components/ui/sidebar.tsx

```
'use client'

import * as React from 'react'
import { Slot } from '@radix-ui/react-slot'
import { cva, VariantProps } from 'class-variance-authority'
import { PanelLeftIcon } from 'lucide-react'

import { useIsMobile } from '@/hooks/use-mobile'
import { cn } from '@/lib/utils'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Separator } from '@/components/ui/separator'
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
} from '@/components/ui/sheet'
import { Skeleton } from '@/components/ui/skeleton'
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from '@/components/ui/tooltip'

const SIDEBAR_COOKIE_NAME = 'sidebar_state'
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7
const SIDEBAR_WIDTH = '16rem'
const SIDEBAR_WIDTH_MOBILE = '18rem'
const SIDEBAR_WIDTH_ICON = '3rem'
const SIDEBAR_KEYBOARD_SHORTCUT = 'b'

type SidebarContextProps = {
  state: 'expanded' | 'collapsed'
  open: boolean
  setOpen: (open: boolean) => void
  openMobile: boolean
  setOpenMobile: (open: boolean) => void
  isMobile: boolean
  toggleSidebar: () => void
}

const SidebarContext = React.createContext<SidebarContextProps | null>(null)

function useSidebar() {
  const context = React.useContext(SidebarContext)
  if (!context) {
    throw new Error('useSidebar must be used within a SidebarProvider.')
  }

  return context
}
```

```
55
56  function SidebarProvider({
57    defaultOpen = true,
58    open: openProp,
59    onOpenChange: setOpenProp,
60    className,
61    style,
62    children,
63    ...props
64  }: React.ComponentProps<'div'> & {
65    defaultOpen?: boolean
66    open?: boolean
67    onOpenChange?: (open: boolean) => void
68  }) {
69    const isMobile = useIsMobile()
70    const [openMobile, setOpenMobile] = React.useState(false)
71
72    // This is the internal state of the sidebar.
73    // We use openProp and setOpenProp for control from outside the component.
74    const [_open, _setOpen] = React.useState(defaultOpen)
75    const open = openProp ?? _open
76    const setOpen = React.useCallback(
77      (value: boolean | ((value: boolean) => boolean)) => {
78        const openState = typeof value === 'function' ? value(open) : value
79        if (setOpenProp) {
80          setOpenProp(openState)
81        } else {
82          _setOpen(openState)
83        }
84
85        // This sets the cookie to keep the sidebar state.
86        document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`
87      },
88      [setOpenProp, open],
89    )
90
91    // Helper to toggle the sidebar.
92    const toggleSidebar = React.useCallback(() => {
93      return isMobile ? setOpenMobile((open) => !open) : setOpen((open) => !open)
94    }, [isMobile, setOpen, setOpenMobile])
95
96    // Adds a keyboard shortcut to toggle the sidebar.
97    React.useEffect(() => {
98      const handleKeyDown = (event: KeyboardEvent) => {
99        if (
100         event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
101         (event.metaKey || event.ctrlKey)
102       ) {
103         event.preventDefault()
104         toggleSidebar()
105       }
106     }
107
108     window.addEventListener('keydown', handleKeyDown)
109     return () => window.removeEventListener('keydown', handleKeyDown)
110   }, [toggleSidebar])
111
112   // We add a state so that we can do data-state="expanded" or "collapsed".
113   // This makes it easier to style the sidebar with Tailwind classes.
114   const state = open ? 'expanded' : 'collapsed'
115
116   const contextValue = React.useMemo<SidebarContextProps>(
117     () => ({
118       state,
119       open,
120       setOpen,
121       isMobile,
122       openMobile,
123       setOpenMobile,
124       toggleSidebar,
```

```
    }),
    [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar],
  )

  return (
    <SidebarContext.Provider value={contextValue}>
      <TooltipProvider delayDuration={0}>
        <div
          data-slot="sidebar-wrapper"
          style={
            {
              '--sidebar-width': SIDEBAR_WIDTH,
              '--sidebar-width-icon': SIDEBAR_WIDTH_ICON,
              ...style,
            } as React.CSSProperties
          }
          className={cn(
            'group/sidebar-wrapper has-data-[variant=inset]:bg-sidebar flex min-h-svh w-full',
            className,
          )}
          {...props}
        >
          {children}
        </div>
      </TooltipProvider>
    </SidebarContext.Provider>
  )
}

function Sidebar({
  side = 'left',
  variant = 'sidebar',
  collapsible = 'offcanvas',
  className,
  children,
  ...props
}: React.ComponentProps<'div'> & {
  side?: 'left' | 'right'
  variant?: 'sidebar' | 'floating' | 'inset'
  collapsible?: 'offcanvas' | 'icon' | 'none'
}) {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar()

  if (collapsible === 'none') {
    return (
      <div
        data-slot="sidebar"
        className={cn(
          'bg-sidebar text-sidebar-foreground flex h-full w-(--sidebar-width) flex-col',
          className,
        )}
        {...props}
      >
        {children}
      </div>
    )
  }

  if (isMobile) {
    return (
      <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
        <SheetContent
          data-sidebar="sidebar"
          data-slot="sidebar"
          data-mobile="true"
          className="bg-sidebar text-sidebar-foreground w-(--sidebar-width) p-0 [&>button]:hidden"
          style={
            {
              '--sidebar-width': SIDEBAR_WIDTH_MOBILE,
            } as React.CSSProperties
```

```
          }
          side={side}
        >
          <SheetHeader className="sr-only">
            <SheetTitle>Sidebar</SheetTitle>
            <SheetDescription>Displays the mobile sidebar.</SheetDescription>
          </SheetHeader>
          <div className="flex h-full w-full flex-col">{children}</div>
        </SheetContent>
      </Sheet>
    )
  }

  return (
    <div
      className="group peer text-sidebar-foreground hidden md:block"
      data-state={state}
      data-collapsible={state === 'collapsed' ? collapsible : ''}
      data-variant={variant}
      data-side={side}
      data-slot="sidebar"
    >
      {/* This is what handles the sidebar gap on desktop */}
      <div
        data-slot="sidebar-gap"
        className={cn(
          'relative w-(-sidebar-width) bg-transparent transition-[width] duration-200 ease-linear',
          'group-data-[collapsible=offcanvas]:w-0',
          'group-data-[side=right]:rotate-180',
          variant === 'floating' || variant === 'inset'
            ? 'group-data-[collapsible=icon]:w-[calc(var(-sidebar-width-icon)+(-spacing(4)))]'
            : 'group-data-[collapsible=icon]:w-(-sidebar-width-icon)',
        )}
      />
      <div
        data-slot="sidebar-container"
        className={cn(
          'fixed inset-y-0 z-10 hidden h-svh w-(-sidebar-width) transition-[left,right,width] duration-200
          ease-linear md:flex',
          side === 'left'
            ? 'left-0 group-data-[collapsible=offcanvas]:left-[calc(var(-sidebar-width)*-1)]'
            : 'right-0 group-data-[collapsible=offcanvas]:right-[calc(var(-sidebar-width)*-1)]',
          // Adjust the padding for floating and inset variants.
          variant === 'floating' || variant === 'inset'
            ? 'p-2 group-data-[collapsible=icon]:w-[calc(var(-sidebar-width-icon)+(-spacing(4))+2px)]'
            : 'group-data-[collapsible=icon]:w-(-sidebar-width-icon) group-data-[side=left]:border-r
            group-data-[side=right]:border-l',
          className,
        )}
        {...props}
      >
        <div
          data-sidebar="sidebar"
          data-slot="sidebar-inner"
          className="bg-sidebar group-data-[variant=floating]:border-sidebar-border flex h-full w-full
          flex-col group-data-[variant=floating]:rounded-lg group-data-[variant=floating]:border
          group-data-[variant=floating]:shadow-sm"
        >
          {children}
        </div>
      </div>
    </div>
  )
}

function SidebarTrigger({
  className,
  onClick,
  ...props
}: React.ComponentProps<typeof Button>) {
```

```
  const { toggleSidebar } = useSidebar()

  return (
    <Button
      data-sidebar="trigger"
      data-slot="sidebar-trigger"
      variant="ghost"
      size="icon"
      className={cn('size-7', className)}
      onClick={(event) => {
        onClick?.(event)
        toggleSidebar()
      }}
      {...props}
    >
      <PanelLeftIcon />
      <span className="sr-only">Toggle Sidebar</span>
    </Button>
  )
}

function SidebarRail({ className, ...props }: React.ComponentProps<'button'>) {
  const { toggleSidebar } = useSidebar()

  return (
    <button
      data-sidebar="rail"
      data-slot="sidebar-rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        'hover:after:bg-sidebar-border absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear group-data-[side=left]:-right-4 group-data-[side=right]:left-0 after:absolute after:inset-y-0 after:left-1/2 after:w-[2px] sm:flex',
        'in-data-[side=left]:cursor-w-resize in-data-[side=right]:cursor-e-resize',
        '[[data-side=left][data-state=collapsed]_&]:cursor-e-resize [[data-side=right][data-state=collapsed]_&]:cursor-w-resize',
        'hover:group-data-[collapsible=offcanvas]:bg-sidebar group-data-[collapsible=offcanvas]:translate-x-0 group-data-[collapsible=offcanvas]:after:left-full',
        '[[data-side=left][data-collapsible=offcanvas]_&]:-right-2',
        '[[data-side=right][data-collapsible=offcanvas]_&]:-left-2',
        className,
      )}
      {...props}
    />
  )
}

function SidebarInset({ className, ...props }: React.ComponentProps<'main'>) {
  return (
    <main
      data-slot="sidebar-inset"
      className={cn(
        'bg-background relative flex w-full flex-1 flex-col',
        'md:peer-data-[variant=inset]:m-2 md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shadow-sm md:peer-data-[variant=inset]:peer-data-[state=collapsed]:ml-2',
        className,
      )}
      {...props}
    />
  )
}

function SidebarInput({
  className,
  ...props
}: React.ComponentProps<typeof Input>) {
```

```
  return (
    <Input
      data-slot="sidebar-input"
      data-sidebar="input"
      className={cn('bg-background h-8 w-full shadow-none', className)}
      {...props}
    />
  )
}

function SidebarHeader({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="sidebar-header"
      data-sidebar="header"
      className={cn('flex flex-col gap-2 p-2', className)}
      {...props}
    />
  )
}

function SidebarFooter({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="sidebar-footer"
      data-sidebar="footer"
      className={cn('flex flex-col gap-2 p-2', className)}
      {...props}
    />
  )
}

function SidebarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="sidebar-separator"
      data-sidebar="separator"
      className={cn('bg-sidebar-border mx-2 w-auto', className)}
      {...props}
    />
  )
}

function SidebarContent({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="sidebar-content"
      data-sidebar="content"
      className={cn(
        'flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden',
        className,
      )}
      {...props}
    />
  )
}

function SidebarGroup({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="sidebar-group"
      data-sidebar="group"
      className={cn('relative flex w-full min-w-0 flex-col p-2', className)}
      {...props}
    />
  )
}
```

```
395
396 function SidebarGroupLabel({
397   className,
398   asChild = false,
399   ...props
400 }: React.ComponentProps<'div'> & { asChild?: boolean }) {
401   const Comp = asChild ? Slot : 'div'
402
403   return (
404     <Comp
405       data-slot="sidebar-group-label"
406       data-sidebar="group-label"
407       className={cn(
408         'text-sidebar-foreground/70 ring-sidebar-ring flex h-8 shrink-0 items-center rounded-md px-2 text-xs
          font-medium outline-hidden transition-[margin,opacity] duration-200 ease-linear focus-visible:ring-2
          [&>svg]:size-4 [&>svg]:shrink-0',
409         'group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0',
410         className,
411       )}
412       {...props}
413     />
414   )
415 }
416
417 function SidebarGroupAction({
418   className,
419   asChild = false,
420   ...props
421 }: React.ComponentProps<'button'> & { asChild?: boolean }) {
422   const Comp = asChild ? Slot : 'button'
423
424   return (
425     <Comp
426       data-slot="sidebar-group-action"
427       data-sidebar="group-action"
428       className={cn(
429         'text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
          hover:text-sidebar-accent-foreground absolute top-3.5 right-3 flex aspect-square w-5 items-center
          justify-center rounded-md p-0 outline-hidden transition-transform focus-visible:ring-2 [&>svg]:size-4
          [&>svg]:shrink-0',
430         // Increases the hit area of the button on mobile.
431         'after:absolute after:-inset-2 md:after:hidden',
432         'group-data-[collapsible=icon]:hidden',
433         className,
434       )}
435       {...props}
436     />
437   )
438 }
439
440 function SidebarGroupContent({
441   className,
442   ...props
443 }: React.ComponentProps<'div'>) {
444   return (
445     <div
446       data-slot="sidebar-group-content"
447       data-sidebar="group-content"
448       className={cn('w-full text-sm', className)}
449       {...props}
450     />
451   )
452 }
453
454 function SidebarMenu({ className, ...props }: React.ComponentProps<'ul'>) {
455   return (
456     <ul
457       data-slot="sidebar-menu"
458       data-sidebar="menu"
459       className={cn('flex w-full min-w-0 flex-col gap-1', className)}
```

```
      {...props}
    />
  )
}

function SidebarMenuItem({ className, ...props }: React.ComponentProps<'li'>) {
  return (
    <li
      data-slot="sidebar-menu-item"
      data-sidebar="menu-item"
      className={cn('group/menu-item relative', className)}
      {...props}
    />
  )
}

const sidebarMenuButtonVariants = cva(
  'peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm
  outline-hidden ring-sidebar-ring transition-[width,height,padding] hover:bg-sidebar-accent
  hover:text-sidebar-accent-foreground focus-visible:ring-2 active:bg-sidebar-accent
  active:text-sidebar-accent-foreground disabled:pointer-events-none disabled:opacity-50
  group-has-data-[sidebar=menu-action]/menu-item:pr-8 aria-disabled:pointer-events-none
  aria-disabled:opacity-50 data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium
  data-[active=true]:text-sidebar-accent-foreground data-[state=open]:hover:bg-sidebar-accent
  data-[state=open]:hover:text-sidebar-accent-foreground group-data-[collapsible=icon]:size-8!
  group-data-[collapsible=icon]:p-2! [&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0',
  {
    variants: {
      variant: {
        default: 'hover:bg-sidebar-accent hover:text-sidebar-accent-foreground',
        outline:
          'bg-background shadow-[0_0_0_1px_hsl(var(-sidebar-border))] hover:bg-sidebar-accent
          hover:text-sidebar-accent-foreground hover:shadow-[0_0_0_1px_hsl(var(-sidebar-accent))]',
      },
      size: {
        default: 'h-8 text-sm',
        sm: 'h-7 text-xs',
        lg: 'h-12 text-sm group-data-[collapsible=icon]:p-0!',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  },
)

function SidebarMenuButton({
  asChild = false,
  isActive = false,
  variant = 'default',
  size = 'default',
  tooltip,
  className,
  ...props
}: React.ComponentProps<'button'> & {
  asChild?: boolean
  isActive?: boolean
  tooltip?: string | React.ComponentProps<typeof TooltipContent>
} & VariantProps<typeof sidebarMenuButtonVariants>) {
  const Comp = asChild ? Slot : 'button'
  const { isMobile, state } = useSidebar()

  const button = (
    <Comp
      data-slot="sidebar-menu-button"
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
```

```
      {...props}
    />
  )

  if (!tooltip) {
    return button
  }

  if (typeof tooltip === 'string') {
    tooltip = {
      children: tooltip,
    }
  }

  return (
    <Tooltip>
      <TooltipTrigger asChild>{button}</TooltipTrigger>
      <TooltipContent
        side="right"
        align="center"
        hidden={state !== 'collapsed' || isMobile}
        {...tooltip}
      />
    </Tooltip>
  )
}

function SidebarMenuAction({
  className,
  asChild = false,
  showOnHover = false,
  ...props
}: React.ComponentProps<'button'> & {
  asChild?: boolean
  showOnHover?: boolean
}) {
  const Comp = asChild ? Slot : 'button'

  return (
    <Comp
      data-slot="sidebar-menu-action"
      data-sidebar="menu-action"
      className={cn(
        'text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
        hover:text-sidebar-accent-foreground peer-hover/menu-button:text-sidebar-accent-foreground absolute
        top-1.5 right-1 flex aspect-square w-5 items-center justify-center rounded-md p-0 outline-hidden
        transition-transform focus-visible:ring-2 [&>svg]:size-4 [&>svg]:shrink-0',
        // Increases the hit area of the button on mobile.
        'after:absolute after:-inset-2 md:after:hidden',
        'peer-data-[size=sm]/menu-button:top-1',
        'peer-data-[size=default]/menu-button:top-1.5',
        'peer-data-[size=lg]/menu-button:top-2.5',
        'group-data-[collapsible=icon]:hidden',
        showOnHover &&
          'peer-data-[active=true]/menu-button:text-sidebar-accent-foreground
          group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100
          data-[state=open]:opacity-100 md:opacity-0',
        className,
      )}
      {...props}
    />
  )
}

function SidebarMenuBadge({
  className,
  ...props
}: React.ComponentProps<'div'>) {
  return (
    <div
```

```
      data-slot="sidebar-menu-badge"
      data-sidebar="menu-badge"
      className={cn(
        'text-sidebar-foreground pointer-events-none absolute right-1 flex h-5 min-w-5 items-center justify-center rounded-md px-1 text-xs font-medium tabular-nums select-none',
        'peer-hover/menu-button:text-sidebar-accent-foreground peer-data-[active=true]/menu-button:text-sidebar-accent-foreground',
        'peer-data-[size=sm]/menu-button:top-1',
        'peer-data-[size=default]/menu-button:top-1.5',
        'peer-data-[size=lg]/menu-button:top-2.5',
        'group-data-[collapsible=icon]:hidden',
        className,
      )}
      {...props}
    />
  )
}

function SidebarMenuSkeleton({
  className,
  showIcon = false,
  ...props
}: React.ComponentProps<'div'> & {
  showIcon?: boolean
}) {
  // Random width between 50 to 90%.
  const width = React.useMemo(() => {
    return `${Math.floor(Math.random() * 40) + 50}%`
  }, [])

  return (
    <div
      data-slot="sidebar-menu-skeleton"
      data-sidebar="menu-skeleton"
      className={cn('flex h-8 items-center gap-2 rounded-md px-2', className)}
      {...props}
    >
      {showIcon && (
        <Skeleton
          className="size-4 rounded-md"
          data-sidebar="menu-skeleton-icon"
        />
      )}
      <Skeleton
        className="h-4 max-w-(-skeleton-width) flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            '-skeleton-width': width,
          } as React.CSSProperties
        }
      />
    </div>
  )
}

function SidebarMenuSub({ className, ...props }: React.ComponentProps<'ul'>) {
  return (
    <ul
      data-slot="sidebar-menu-sub"
      data-sidebar="menu-sub"
      className={cn(
        'border-sidebar-border mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l px-2.5 py-0.5',
        'group-data-[collapsible=icon]:hidden',
        className,
      )}
      {...props}
    />
  )
}
```

```
654
655 function SidebarMenuSubItem({
656   className,
657   ...props
658 }: React.ComponentProps<'li'>) {
659   return (
660     <li
661       data-slot="sidebar-menu-sub-item"
662       data-sidebar="menu-sub-item"
663       className={cn('group/menu-sub-item relative', className)}
664       {...props}
665     />
666   )
667 }
668
669 function SidebarMenuSubButton({
670   asChild = false,
671   size = 'md',
672   isActive = false,
673   className,
674   ...props
675 }: React.ComponentProps<'a'> & {
676   asChild?: boolean
677   size?: 'sm' | 'md'
678   isActive?: boolean
679 }) {
680   const Comp = asChild ? Slot : 'a'
681
682   return (
683     <Comp
684       data-slot="sidebar-menu-sub-button"
685       data-sidebar="menu-sub-button"
686       data-size={size}
687       data-active={isActive}
688       className={cn(
689         'text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
          hover:text-sidebar-accent-foreground active:bg-sidebar-accent active:text-sidebar-accent-foreground
          [&>svg]:text-sidebar-accent-foreground flex h-7 min-w-0 -translate-x-px items-center gap-2
          overflow-hidden rounded-md px-2 outline-hidden focus-visible:ring-2 disabled:pointer-events-none
          disabled:opacity-50 aria-disabled:pointer-events-none aria-disabled:opacity-50
          [&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0',
690         'data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-foreground',
691         size === 'sm' && 'text-xs',
692         size === 'md' && 'text-sm',
693         'group-data-[collapsible=icon]:hidden',
694         className,
695       )}
696       {...props}
697     />
698   )
699 }
700
701 export {
702   Sidebar,
703   SidebarContent,
704   SidebarFooter,
705   SidebarGroup,
706   SidebarGroupAction,
707   SidebarGroupContent,
708   SidebarGroupLabel,
709   SidebarHeader,
710   SidebarInput,
711   SidebarInset,
712   SidebarMenu,
713   SidebarMenuAction,
714   SidebarMenuBadge,
715   SidebarMenuButton,
716   SidebarMenuItem,
717   SidebarMenuSkeleton,
718   SidebarMenuSub,
```

```
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
}
```

Source code 6.268: deskflow-frontend/components/ui/skeleton.tsx

```
import { cn } from '@/lib/utils'

function Skeleton({ className, ...props }: React.ComponentProps<'div'>) {
  return (
    <div
      data-slot="skeleton"
      className={cn('bg-accent animate-pulse rounded-md', className)}
      {...props}
    />
  )
}

export { Skeleton }
```

Source code 6.269: deskflow-frontend/components/ui/slider.tsx

```
'use client'

import * as React from 'react'
import * as SliderPrimitive from '@radix-ui/react-slider'

import { cn } from '@/lib/utils'

function Slider({
  className,
  defaultValue,
  value,
  min = 0,
  max = 100,
  ...props
}: React.ComponentProps<typeof SliderPrimitive.Root>) {
  const _values = React.useMemo(
    () =>
      Array.isArray(value)
        ? value
        : Array.isArray(defaultValue)
          ? defaultValue
          : [min, max],
    [value, defaultValue, min, max],
  )

  return (
    <SliderPrimitive.Root
      data-slot="slider"
      defaultValue={defaultValue}
      value={value}
      min={min}
      max={max}
      className={cn(
        'relative flex w-full touch-none items-center select-none data-[disabled]:opacity-50
        data-[orientation=vertical]:h-full data-[orientation=vertical]:min-h-44
        data-[orientation=vertical]:w-auto data-[orientation=vertical]:flex-col',
        className,
      )}
```

```
      {...props}
    >
      <SliderPrimitive.Track
        data-slot="slider-track"
        className={
          'bg-muted relative grow overflow-hidden rounded-full data-[orientation=horizontal]:h-1.5
          data-[orientation=horizontal]:w-full data-[orientation=vertical]:h-full
          data-[orientation=vertical]:w-1.5'
        }
      >
        <SliderPrimitive.Range
          data-slot="slider-range"
          className={
            'bg-primary absolute data-[orientation=horizontal]:h-full data-[orientation=vertical]:w-full'
          }
        />
      </SliderPrimitive.Track>
      {Array.from({ length: _values.length }, (_, index) => (
        <SliderPrimitive.Thumb
          data-slot="slider-thumb"
          key={index}
          className="border-primary ring-ring/50 block size-4 shrink-0 rounded-full border bg-white shadow-sm
          transition-[color,box-shadow] hover:ring-4 focus-visible:ring-4 focus-visible:outline-hidden
          disabled:pointer-events-none disabled:opacity-50"
        />
      ))}
    </SliderPrimitive.Root>
  )
}

export { Slider }
```

Source code 6.270: deskflow-frontend/components/ui/sonner.tsx

```
'use client'

import { useTheme } from 'next-themes'
import { Toaster as Sonner, ToasterProps } from 'sonner'

const Toaster = ({ ...props }: ToasterProps) => {
  const { theme = 'system' } = useTheme()

  return (
    <Sonner
      theme={theme as ToasterProps['theme']}
      className="toaster group"
      style={
        {
          '--normal-bg': 'var(--popover)',
          '--normal-text': 'var(--popover-foreground)',
          '--normal-border': 'var(--border)',
        } as React.CSSProperties
      }
      {...props}
    />
  )
}

export { Toaster }
```

Source code 6.271: deskflow-frontend/components/ui/spinner.tsx

```
import { Loader2Icon } from 'lucide-react'

import { cn } from '@/lib/utils'
```

```
4
5  function Spinner({ className, ...props }: React.ComponentProps<'svg'>) {
6    return (
7      <Loader2Icon
8        role="status"
9        aria-label="Loading"
10       className={cn('size-4 animate-spin', className)}
11       {...props}
12     />
13   )
14 }
15
16 export { Spinner }
```

Source code 6.272: deskflow-frontend/components/ui/switch.tsx

```
1  'use client'
2
3  import * as React from 'react'
4  import * as SwitchPrimitive from '@radix-ui/react-switch'
5
6  import { cn } from '@/lib/utils'
7
8  function Switch({
9    className,
10   ...props
11 }: React.ComponentProps<typeof SwitchPrimitive.Root>) {
12   return (
13     <SwitchPrimitive.Root
14       data-slot="switch"
15       className={cn(
16         'peer data-[state=checked]:bg-primary data-[state=unchecked]:bg-input focus-visible:border-ring
          focus-visible:ring-ring/50 dark:data-[state=unchecked]:bg-input/80 inline-flex h-[1.15rem] w-8
          shrink-0 items-center rounded-full border border-transparent shadow-xs transition-all outline-none
          focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50',
17         className,
18       )}
19       {...props}
20     >
21       <SwitchPrimitive.Thumb
22         data-slot="switch-thumb"
23         className={
24           'bg-background dark:data-[state=unchecked]:bg-foreground
            dark:data-[state=checked]:bg-primary-foreground pointer-events-none block size-4 rounded-full
            ring-0 transition-transform data-[state=checked]:translate-x-[calc(100%-2px)]
            data-[state=unchecked]:translate-x-0'
25         }
26       />
27     </SwitchPrimitive.Root>
28   )
29 }
30
31 export { Switch }
```

Source code 6.273: deskflow-frontend/components/ui/table.tsx

```
1  'use client'
2
3  import * as React from 'react'
4
5  import { cn } from '@/lib/utils'
6
7  function Table({ className, ...props }: React.ComponentProps<'table'>) {
8    return (
9      <div
```

```
      data-slot="table-container"
      className="relative w-full overflow-x-auto"
    >
      <table
        data-slot="table"
        className={cn('w-full caption-bottom text-sm', className)}
        {...props}
      />
    </div>
  )
}

function TableHeader({ className, ...props }: React.ComponentProps<'thead'>) {
  return (
    <thead
      data-slot="table-header"
      className={cn('[&_tr]:border-b', className)}
      {...props}
    />
  )
}

function TableBody({ className, ...props }: React.ComponentProps<'tbody'>) {
  return (
    <tbody
      data-slot="table-body"
      className={cn('[&_tr:last-child]:border-0', className)}
      {...props}
    />
  )
}

function TableFooter({ className, ...props }: React.ComponentProps<'tfoot'>) {
  return (
    <tfoot
      data-slot="table-footer"
      className={cn(
        'bg-muted/50 border-t font-medium [&>tr]:last:border-b-0',
        className,
      )}
      {...props}
    />
  )
}

function TableRow({ className, ...props }: React.ComponentProps<'tr'>) {
  return (
    <tr
      data-slot="table-row"
      className={cn(
        'hover:bg-muted/50 data-[state=selected]:bg-muted border-b transition-colors',
        className,
      )}
      {...props}
    />
  )
}

function TableHead({ className, ...props }: React.ComponentProps<'th'>) {
  return (
    <th
      data-slot="table-head"
      className={cn(
        'text-foreground h-10 px-2 text-left align-middle font-medium whitespace-nowrap
        [&:has([role=checkbox])]:pr-0 [&>[role=checkbox]]:translate-y-[2px]',
        className,
      )}
      {...props}
    />
  )
```

```
}

function TableCell({ className, ...props }: React.ComponentProps<'td'>) {
  return (
    <td
      data-slot="table-cell"
      className={cn(
        'p-2 align-middle whitespace-nowrap [&:has([role=checkbox])]:pr-0
        [&>[role=checkbox]]:translate-y-[2px]',
        className,
      )}
      {...props}
    />
  )
}

function TableCaption({
  className,
  ...props
}: React.ComponentProps<'caption'>) {
  return (
    <caption
      data-slot="table-caption"
      className={cn('text-muted-foreground mt-4 text-sm', className)}
      {...props}
    />
  )
}

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}
```

Source code 6.274: deskflow-frontend/components/ui/tabs.tsx

```
'use client'

import * as React from 'react'
import * as TabsPrimitive from '@radix-ui/react-tabs'

import { cn } from '@/lib/utils'

function Tabs({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Root>) {
  return (
    <TabsPrimitive.Root
      data-slot="tabs"
      className={cn('flex flex-col gap-2', className)}
      {...props}
    />
  )
}

function TabsList({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.List>) {
  return (
    <TabsPrimitive.List
```

```
      data-slot="tabs-list"
      className={cn(
        'bg-muted text-muted-foreground inline-flex h-9 w-fit items-center justify-center rounded-lg p-[3px]',
        className,
      )}
      {...props}
    />
  )
}

function TabsTrigger({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Trigger>) {
  return (
    <TabsPrimitive.Trigger
      data-slot="tabs-trigger"
      className={cn(
        "data-[state=active]:bg-background dark:data-[state=active]:text-foreground focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:outline-ring dark:data-[state=active]:border-input dark:data-[state=active]:bg-input/30 text-foreground dark:text-muted-foreground inline-flex h-[calc(100%-1px)] flex-1 items-center justify-center gap-1.5 rounded-md border border-transparent px-2 py-1 text-sm font-medium whitespace-nowrap transition-[color,box-shadow] focus-visible:ring-[3px] focus-visible:outline-1 disabled:pointer-events-none disabled:opacity-50 data-[state=active]:shadow-sm [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  )
}

function TabsContent({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Content>) {
  return (
    <TabsPrimitive.Content
      data-slot="tabs-content"
      className={cn('flex-1 outline-none', className)}
      {...props}
    />
  )
}

export { Tabs, TabsList, TabsTrigger, TabsContent }
```

Source code 6.275: deskflow-frontend/components/ui/textarea.tsx

```
import * as React from 'react'

import { cn } from '@/lib/utils'

function Textarea({ className, ...props }: React.ComponentProps<'textarea'>) {
  return (
    <textarea
      data-slot="textarea"
      className={cn(
        'border-input placeholder:text-muted-foreground focus-visible:border-ring focus-visible:ring-ring/50 aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive dark:bg-input/30 flex field-sizing-content min-h-16 w-full rounded-md border bg-transparent px-3 py-2 text-base shadow-xs transition-[color,box-shadow] outline-none focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50 md:text-sm',
        className,
      )}
```

```
      {...props}
    />
  )
}

export { Textarea }
```

Source code 6.276: deskflow-frontend/components/ui/toast.tsx

```
'use client'

import * as React from 'react'
import * as ToastPrimitives from '@radix-ui/react-toast'
import { cva, type VariantProps } from 'class-variance-authority'
import { X } from 'lucide-react'

import { cn } from '@/lib/utils'

const ToastProvider = ToastPrimitives.Provider

const ToastViewport = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Viewport>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Viewport>
>(({ className, ...props }, ref) => (
  <ToastPrimitives.Viewport
    ref={ref}
    className={cn(
      'fixed top-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4 sm:bottom-0 sm:right-0 sm:top-auto
      sm:flex-col md:max-w-[420px]',
      className,
    )}
    {...props}
  />
))
ToastViewport.displayName = ToastPrimitives.Viewport.displayName

const toastVariants = cva(
  'group pointer-events-auto relative flex w-full items-center justify-between space-x-4 overflow-hidden
  rounded-md border p-6 pr-8 shadow-lg transition-all data-[swipe=cancel]:translate-x-0
  data-[swipe=end]:translate-x-[var(-radix-toast-swipe-end-x)]
  data-[swipe=move]:translate-x-[var(-radix-toast-swipe-move-x)] data-[swipe=move]:transition-none
  data-[state=open]:animate-in data-[state=closed]:animate-out data-[swipe=end]:animate-out
  data-[state=closed]:fade-out-80 data-[state=closed]:slide-out-to-right-full
  data-[state=open]:slide-in-from-top-full data-[state=open]:sm:slide-in-from-bottom-full',
  {
    variants: {
      variant: {
        default: 'border bg-background text-foreground',
        destructive:
          'destructive group border-destructive bg-destructive text-destructive-foreground',
      },
    },
    defaultVariants: {
      variant: 'default',
    },
  },
)

const Toast = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Root>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Root> &
    VariantProps<typeof toastVariants>
>(({ className, variant, ...props }, ref) => {
  return (
    <ToastPrimitives.Root
      ref={ref}
      className={cn(toastVariants({ variant }), className)}
```

```
      {...props}
    />
  )
})
Toast.displayName = ToastPrimitives.Root.displayName

const ToastAction = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Action>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Action>
>(({ className, ...props }, ref) => (
  <ToastPrimitives.Action
    ref={ref}
    className={cn(
      'inline-flex h-8 shrink-0 items-center justify-center rounded-md border bg-transparent px-3 text-sm
      font-medium ring-offset-background transition-colors hover:bg-secondary focus:outline-none focus:ring-2
      focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none disabled:opacity-50
      group-[.destructive]:border-muted/40 group-[.destructive]:hover:border-destructive/30
      group-[.destructive]:hover:bg-destructive group-[.destructive]:hover:text-destructive-foreground
      group-[.destructive]:focus:ring-destructive',
      className,
    )}
    {...props}
  />
))
ToastAction.displayName = ToastPrimitives.Action.displayName

const ToastClose = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Close>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Close>
>(({ className, ...props }, ref) => (
  <ToastPrimitives.Close
    ref={ref}
    className={cn(
      'absolute right-2 top-2 rounded-md p-1 text-foreground/50 opacity-0 transition-opacity
      hover:text-foreground focus:opacity-100 focus:outline-none focus:ring-2 group-hover:opacity-100
      group-[.destructive]:text-red-300 group-[.destructive]:hover:text-red-50
      group-[.destructive]:focus:ring-red-400 group-[.destructive]:focus:ring-offset-red-600',
      className,
    )}
    toast-close=""
    {...props}
  >
    <X className="h-4 w-4" />
  </ToastPrimitives.Close>
))
ToastClose.displayName = ToastPrimitives.Close.displayName

const ToastTitle = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Title>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Title>
>(({ className, ...props }, ref) => (
  <ToastPrimitives.Title
    ref={ref}
    className={cn('text-sm font-semibold', className)}
    {...props}
  />
))
ToastTitle.displayName = ToastPrimitives.Title.displayName

const ToastDescription = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Description>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Description>
>(({ className, ...props }, ref) => (
  <ToastPrimitives.Description
    ref={ref}
    className={cn('text-sm opacity-90', className)}
    {...props}
  />
))
ToastDescription.displayName = ToastPrimitives.Description.displayName
```

```

type ToastProps = React.ComponentPropsWithoutRef<typeof Toast>

type ToastActionElement = React.ReactElement<typeof ToastAction>

export {
  type ToastProps,
  type ToastActionElement,
  ToastProvider,
  ToastViewport,
  Toast,
  ToastTitle,
  ToastDescription,
  ToastClose,
  ToastAction,
}
```

Source code 6.277: deskflow-frontend/components/ui/toaster.tsx

```
'use client'

import { useToast } from '@/hooks/use-toast'
import {
  Toast,
  ToastClose,
  ToastDescription,
  ToastProvider,
  ToastTitle,
  ToastViewport,
} from '@/components/ui/toast'

export function Toaster() {
  const { toasts } = useToast()

  return (
    <ToastProvider>
      {toasts.map(function ({ id, title, description, action, ...props }) {
        return (
          <Toast key={id} {...props}>
            <div className="grid gap-1">
              {title && <ToastTitle>{title}</ToastTitle>}
              {description && (
                <ToastDescription>{description}</ToastDescription>
              )}
            </div>
            {action}
            <ToastClose />
          </Toast>
        )
      })}
      <ToastViewport />
    </ToastProvider>
  )
}
```

Source code 6.278: deskflow-frontend/components/ui/toggle-group.tsx

```
'use client'

import * as React from 'react'
import * as ToggleGroupPrimitive from '@radix-ui/react-toggle-group'
import { type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'
import { toggleVariants } from '@/components/ui/toggle'
```

```
9
10  const ToggleGroupContext = React.createContext<
11    VariantProps<typeof toggleVariants>
12  >({
13    size: 'default',
14    variant: 'default',
15  })
16
17  function ToggleGroup({
18    className,
19    variant,
20    size,
21    children,
22    ...props
23  }: React.ComponentProps<typeof ToggleGroupPrimitive.Root> &
24    VariantProps<typeof toggleVariants>) {
25    return (
26      <ToggleGroupPrimitive.Root
27        data-slot="toggle-group"
28        data-variant={variant}
29        data-size={size}
30        className={cn(
31          'group/toggle-group flex w-fit items-center rounded-md data-[variant=outline]:shadow-xs',
32          className,
33        )}
34        {...props}
35      >
36        <ToggleGroupContext.Provider value={{ variant, size }}>
37          {children}
38        </ToggleGroupContext.Provider>
39      </ToggleGroupPrimitive.Root>
40    )
41  }
42
43  function ToggleGroupItem({
44    className,
45    children,
46    variant,
47    size,
48    ...props
49  }: React.ComponentProps<typeof ToggleGroupPrimitive.Item> &
50    VariantProps<typeof toggleVariants>) {
51    const context = React.useContext(ToggleGroupContext)
52
53    return (
54      <ToggleGroupPrimitive.Item
55        data-slot="toggle-group-item"
56        data-variant={context.variant || variant}
57        data-size={context.size || size}
58        className={cn(
59          toggleVariants({
60            variant: context.variant || variant,
61            size: context.size || size,
62          }),
63          'min-w-0 flex-1 shrink-0 rounded-none shadow-none first:rounded-l-md last:rounded-r-md focus:z-10
            focus-visible:z-10 data-[variant=outline]:border-l-0 data-[variant=outline]:first:border-l',
64          className,
65        )}
66        {...props}
67      >
68        {children}
69      </ToggleGroupPrimitive.Item>
70    )
71  }
72
73  export { ToggleGroup, ToggleGroupItem }
```

Source code 6.279: deskflow-frontend/components/ui/toggle.tsx

```
'use client'

import * as React from 'react'
import * as TogglePrimitive from '@radix-ui/react-toggle'
import { cva, type VariantProps } from 'class-variance-authority'

import { cn } from '@/lib/utils'

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium hover:bg-muted
  hover:text-muted-foreground disabled:pointer-events-none disabled:opacity-50 data-[state=on]:bg-accent
  data-[state=on]:text-accent-foreground [&_svg]:pointer-events-none [&_svg:not([class*='size-'])]:size-4
  [&_svg]:shrink-0 focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px] outline-none
  transition-[color,box-shadow] aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
  aria-invalid:border-destructive whitespace-nowrap",
  {
    variants: {
      variant: {
        default: 'bg-transparent',
        outline:
          'border border-input bg-transparent shadow-xs hover:bg-accent hover:text-accent-foreground',
      },
      size: {
        default: 'h-9 px-2 min-w-9',
        sm: 'h-8 px-1.5 min-w-8',
        lg: 'h-10 px-2.5 min-w-10',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  },
)

function Toggle({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps<typeof TogglePrimitive.Root> &
  VariantProps<typeof toggleVariants>) {
  return (
    <TogglePrimitive.Root
      data-slot="toggle"
      className={cn(toggleVariants({ variant, size, className }))}
      {...props}
    />
  )
}

export { Toggle, toggleVariants }
```

Source code 6.280: deskflow-frontend/components/ui/tooltip.tsx

```
'use client'

import * as React from 'react'
import * as TooltipPrimitive from '@radix-ui/react-tooltip'

import { cn } from '@/lib/utils'

function TooltipProvider({
  delayDuration = 0,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Provider>) {
```

```
  return (
    <TooltipPrimitive.Provider
      data-slot="tooltip-provider"
      delayDuration={delayDuration}
      {...props}
    />
  )
}

function Tooltip({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Root>) {
  return (
    <TooltipProvider>
      <TooltipPrimitive.Root data-slot="tooltip" {...props} />
    </TooltipProvider>
  )
}

function TooltipTrigger({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Trigger>) {
  return <TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} />
}

function TooltipContent({
  className,
  sideOffset = 0,
  children,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Content>) {
  return (
    <TooltipPrimitive.Portal>
      <TooltipPrimitive.Content
        data-slot="tooltip-content"
        sideOffset={sideOffset}
        className={cn(
          'bg-foreground text-background animate-in fade-in-0 zoom-in-95 data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=closed]:zoom-out-95
          data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
          data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 z-50 w-fit
          origin-(-radix-tooltip-content-transform-origin) rounded-md px-3 py-1.5 text-xs text-balance',
          className,
        )}
        {...props}
      >
        {children}
        <TooltipPrimitive.Arrow className="bg-foreground fill-foreground z-50 size-2.5
        translate-y-[calc(-50%_-_2px)] rotate-45 rounded-[2px]" />
      </TooltipPrimitive.Content>
    </TooltipPrimitive.Portal>
  )
}

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider }
```

Source code 6.281: deskflow-frontend/components/ui/use-mobile.tsx

```
import * as React from 'react'

const MOBILE_BREAKPOINT = 768

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined)

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
```

```
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    }
    mql.addEventListener('change', onChange)
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    return () => mql.removeEventListener('change', onChange)
  }, [])

  return !!isMobile
}
```

Source code 6.282: deskflow-frontend/components/ui/use-toast.ts

```
'use client'

// Inspired by react-hot-toast library
import * as React from 'react'

import type { ToastActionElement, ToastProps } from '@/components/ui/toast'

const TOAST_LIMIT = 1
const TOAST_REMOVE_DELAY = 1000000

type ToasterToast = ToastProps & {
  id: string
  title?: React.ReactNode
  description?: React.ReactNode
  action?: ToastActionElement
}

const actionTypes = {
  ADD_TOAST: 'ADD_TOAST',
  UPDATE_TOAST: 'UPDATE_TOAST',
  DISMISS_TOAST: 'DISMISS_TOAST',
  REMOVE_TOAST: 'REMOVE_TOAST',
} as const

let count = 0

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER
  return count.toString()
}

type ActionType = typeof actionTypes

type Action =
  | {
      type: ActionType['ADD_TOAST']
      toast: ToasterToast
    }
  | {
      type: ActionType['UPDATE_TOAST']
      toast: Partial<ToasterToast>
    }
  | {
      type: ActionType['DISMISS_TOAST']
      toastId?: ToasterToast['id']
    }
  | {
      type: ActionType['REMOVE_TOAST']
      toastId?: ToasterToast['id']
    }

interface State {
  toasts: ToasterToast[]
}

```

```
const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()

const addToRemoveQueue = (toastId: string) => {
  if (toastTimeouts.has(toastId)) {
    return
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId)
    dispatch({
      type: 'REMOVE_TOAST',
      toastId: toastId,
    })
  }, TOAST_REMOVE_DELAY)

  toastTimeouts.set(toastId, timeout)
}

export const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case 'ADD_TOAST':
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      }

    case 'UPDATE_TOAST':
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t,
        ),
      }

    case 'DISMISS_TOAST': {
      const { toastId } = action

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId)
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id)
        })
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t,
        ),
      }
    }
    case 'REMOVE_TOAST':
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        }
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      }
```

```
  }
}

const listeners: Array<(state: State) => void> = []

let memoryState: State = { toasts: [] }

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action)
  listeners.forEach((listener) => {
    listener(memoryState)
  })
}

type Toast = Omit<ToasterToast, 'id'>

function toast({ ...props }: Toast) {
  const id = genId()

  const update = (props: ToasterToast) =>
    dispatch({
      type: 'UPDATE_TOAST',
      toast: { ...props, id },
    })
  const dismiss = () => dispatch({ type: 'DISMISS_TOAST', toastId: id })

  dispatch({
    type: 'ADD_TOAST',
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss()
      },
    },
  })

  return {
    id: id,
    dismiss,
    update,
  }
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState)

  React.useEffect(() => {
    listeners.push(setState)
    return () => {
      const index = listeners.indexOf(setState)
      if (index > -1) {
        listeners.splice(index, 1)
      }
    }
  }, [state])

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: 'DISMISS_TOAST', toastId }),
  }
}

export { useToast, toast }
```

Bibliography

List of Figures

2.1 Model dziedziny. 6
2.2 Bounded Context Canvas: Udostępnianie zasobów 10
2.3 Bounded Context Canvas: Zarządzanie zasobami 11
2.4 Bounded Context Canvas: Administracja wnioskami 12
2.5 Diagram przypadków użycia. 13

3.1 Architektura heksagonalna — dziedzina główna (kontekst `allocation`) 23
3.2 Architektura heksagonalna — dziedzina pomocnicza (kontekst `resources`) . . 24
3.3 Architektura heksagonalna — dziedzina pomocnicza (kontekst `administration`) 25
3.4 Diagram dziedzin i kontekstów ograniczonych systemu DeskFlow 26
3.5 Diagram komponentów — backend systemu DeskFlow 31
3.6 Diagram wdrożenia — system DeskFlow 32
3.7 Diagram pakietów — backend systemu DeskFlow (`com.deskflow`) 34
3.8 Diagram klas — kontekst `allocation` (agregat WniosekOUdostępnienie) . . . 35
3.9 Diagram stanów — agregat WniosekOUdostępnienie 37
3.10 Diagram klas — kontekst `resources` (agregat ZasóbBiurowy) 40
3.11 Diagram stanów — agregat ZasóbBiurowy 42
3.12 Diagram klas — kontekst `administration` 45
3.13 Diagram sekwencji — PU-02: Składanie wniosku o udostępnienie zasobu biurowego 48
3.14 Diagram sekwencji — PU-03: Anulowanie wniosku o udostępnienie zasobu biurowego 49
3.15 Diagram sekwencji — PU-05/PU-06/PU-07: Analiza, zatwierdzenie i odrzucenie wniosku 51
3.16 Diagram sekwencji — PU-08: Wydanie zasobu biurowego 53
3.17 Diagram sekwencji — PU-09: Zwrot zasobu biurowego 54
3.18 Diagram sekwencji — PU-10: Obsługa przeterminowanego udostępnienia . . . 55
3.19 Diagram sekwencji — PU-12: Zarządzanie dostępnością zasobów biurowych . . 56
3.20 Diagram sekwencji — PU-13 (wariant: dezaktywacja zasobu biurowego) 58
3.21 Diagram sekwencji — PU-14: Konfiguracja limitu udostępnień 59

List of Tables

2.1 Mapowanie dziedzin na konteksty ograniczone 7
2.2 Język Wszechobecny — kontekst Udostępnianie zasobów biurowych 8
2.3 Język Wszechobecny — kontekst Zarządzanie zasobami biurowymi 8
2.4 Język Wszechobecny — kontekst Administracja wnioskami 9

3.1 Mapowanie kontekstów ograniczonych systemu DeskFlow 27
3.2 Stany agregatu WniosekOUdostępnienie 37
3.3 Komendy agregatu WniosekOUdostępnienie 39
3.4 Przepustowość agregatu WniosekOUdostępnienie 39
3.5 Stany agregatu ZasóbBiurowy 43
3.6 Komendy agregatu ZasóbBiurowy 44
3.7 Przepustowość agregatu ZasóbBiurowy 44

5.1 Porównanie Dockerfile backendu w środowisku deweloperskim i produkcyjnym 78
5.2 Porównanie Dockerfile frontendu w środowisku deweloperskim i produkcyjnym 79